Introducción a la Ciencia de Datos

Universidad de Montevideo, FCEE

??ndice

1 Introduccion a Git **1**

2 Primeros pasos en github **6**

3 Lenguajes, intérpretes y entornos de trabajo **13**

4 Archivos, formatos y rutas **27**

5 Generación de datos **39**

6 Diseño de encuestas **44**

7 Cuando el dato falla **63**

8 Fundamentos: objetos, tipos y errores **76**

9 Anexo: interpretación de errores en R y Python **83**

10 SeudoCodigo **96**

11 Cargar datos: de la fuente al DataFrame **98**

12 Carga de datos **118**

13 Flujo de Análisis de Datos y PostgreSQL **127**

14 Carga y transformacion **133**

15 Las partes de un gráfico **147**

16 Series de tiempo: una primera mirada **166**

17 Tránsito y calidad del aire **174**

18 Preguntas, gráficos y una primera mirada a las series de tiempo **188**

19 Inteligencia artificial para leer documentos **199**

1. Introduccion a Git

Introducción a la Ciencia de Datos
Universidad de Montevideo — FCEE

¿Por qué versionar?

El problema que todos conocen

```
TrabajoFinal.xlsx
TrabajoFinal_v2.xlsx
TrabajoFinal_v2_FINAL.xlsx
TrabajoFinal_v2_FINAL_corregido.xlsx
TrabajoFinal_v2_FINAL_corregido_AHORA_SI.xlsx
copia de TrabajoFinal_v2_FINAL (1).xlsx
```

Seis archivos, ninguna respuesta a las preguntas que importan:

- ¿cuál es la versión buena?
- ¿qué cambió entre una y otra?
- ¿quién lo cambió, y por qué?

Qué es Git

Un programa que lleva **el historial completo de una carpeta**:

- cada versión guardada queda registrada **para siempre**
- cada versión dice **quién**, **cuándo** y **por qué**
- se puede **volver** a cualquier versión anterior
- varias personas trabajan **a la vez** sin pisarse

En este curso Git es **el método de trabajo**.

Los conceptos

El ciclo básico: tres lugares

```
carpeta de trabajo  →  área de preparación  →  historial
  (tus archivos)         (git add)              (git commit)
```

1. **Editás** archivos en tu carpeta, como siempre
2. **Elegís** qué cambios entran en la próxima "foto": `git add`
3. **Sacás la foto** con un mensaje que explica el porqué: `git commit`

La foto se llama **confirmación** (*commit*). Es la unidad de trabajo del curso: chica, frecuente, con mensaje que se entiende.

Anatomía de una confirmación

```
git commit -m "obtención: agrega descarga del IPC desde el INE"
```

Parte	Qué registra
Autor	quién hizo el trabajo (nombre y mail de **tu** cuenta)

Parte	Qué registra
Fecha	cuándo, con hora
Mensaje	**por qué** se hizo el cambio
Contenido	la diferencia exacta, línea por línea

El mensaje importa. “cambios” no dice nada; “limpieza: elimina duplicados de negocio en ventas” sí.

Leer un historial

```
git log --oneline
```

```
a3f21c9 informe: agrega intervalo de confianza al pronóstico
8e07d44 limpieza: trata los precios en cero como faltantes
51b9a02 obtención: agrega descarga del IPC desde el INE
c112f30 estructura inicial del proyecto
```

Se lee de abajo hacia arriba: **es la historia del análisis**. Un tercero puede reconstruir qué se decidió y en qué orden.

Ramas: trabajar sin romper

Una **rama** (*branch*) es una línea de trabajo paralela.

```
git switch -c limpieza-duplicados   # crear rama y pasarse a ella
# ... trabajar y confirmar acá, sin tocar lo estable ...
git switch main                     # volver a la rama principal
git merge limpieza-duplicados       # incorporar el trabajo
```

- `main` siempre queda **funcionando**
- lo experimental vive en ramas hasta que está listo
- en el equipo: **una rama por persona o por tarea**

Conflictos: qué son y por qué no asustan

Dos personas tocaron **las mismas líneas** del mismo archivo. Git no adivina: te muestra las dos versiones y te pide decidir.

```
<<<<<<< HEAD
umbral = 90   # definición de cliente activo: 90 días
=======
umbral = 60   # definición de cliente activo: 60 días
>>>>>>> rama-de-ana
```

Un conflicto no es un error: es una **decisión de método** que el equipo tenía pendiente. Se conversa, se elige, se confirma. (Y se anota en `decisiones.md`.)

Remoto: GitHub

Hasta acá, todo pasó en tu máquina. El **remoto** es la copia compartida en GitHub.

```
git push    # subir mis confirmaciones
git pull    # traer las de los demás
```

- el equipo se sincroniza contra el remoto
- el docente ve el trabajo **cuando ocurre**, no en la entrega
- si tu máquina muere, el trabajo no

Solicitud de incorporación (*pull request*)

Proponer cambios **sin fusionarlos** todavía: otro los revisa primero.

```
rama cap/git-ramas-perez  →  solicitud  →  dos revisiones
       →  correcciones  →  fusión por el docente
```

Es la forma normal de trabajar en cualquier equipo profesional de datos o software.

No todo es terminal

Git es el motor; el volante lo elegís vos

Todas estas herramientas hacen **exactamente el mismo Git**: crean las mismas confirmaciones, el mismo historial, los mismos remotos.

Herramienta	Para quién
Terminal	quien quiere ver qué pasa por dentro
VS Code (panel de control de código fuente)	quien ya edita ahí — la opción más cómoda del curso
RStudio (pestaña Git)	quien trabaja en R
GitHub Desktop	quien prefiere una aplicación dedicada, simple y visual
GitHub web	revisar solicitudes y leer historial desde el navegador

Elegí una, la que te resulte natural. El historial que producen es **idéntico**, y en la evaluación no se distingue cuál usaste.

VS Code: el ciclo completo sin escribir comandos

1. Editás tus archivos, como siempre
2. Panel **Control de código fuente** (icono de ramas, `Ctrl+Shift+G`)
3. **+** junto al archivo = `git add`
4. Escribís el mensaje y ✓ **Confirmar** = `git commit`
5. **Sincronizar** = `git push` + `git pull`

[captura: panel de control de código fuente con un cambio preparado y el mensaje escrito — ver `imagenes/README.md`]

GitHub Desktop: la vista más visual

1. Muestra **la diferencia** de cada archivo, línea por línea
2. Tildás los archivos que entran en la confirmación
3. Mensaje + botón **Commit**
4. **Push origin** para publicar
5. Cambiar de rama es un menú desplegable

[captura: diferencia de un archivo y el formulario de commit — ver `imagenes/README.md`]

RStudio: la pestaña Git

1. Pestaña **Git** (arriba a la derecha, junto a Environment)
2. Casilla **Staged** = `git add`

3. Botón **Commit** abre la ventana con la diferencia y el mensaje
4. **Push** / **Pull** con las flechas verdes

[captura: pestaña Git con archivos preparados — ver `imagenes/README.md`]

Entonces, ¿para qué aprender los comandos?

- La interfaz alcanza para el **90 %** de los días
- Los comandos aparecen cuando algo se complica: un conflicto enredado, un historial que hay que entender, una máquina ajena
- Y son el **vocabulario común**: la documentación, los foros y los asistentes de IA hablan en comandos.

Las reglas del curso

Qué se versiona y qué no

Sí	No
Código (`.py`, `.R`, `.qmd`)	Datos crudos (`datos/crudo/`)
El script que **obtiene** los datos	Salidas: figuras y tablas (`salidas/`)
`MANIFIESTO.md`, `decisiones.md`	Datos personales — **nunca**
`requirements.txt` / `renv.lock`	Claves y contraseñas — **nunca**

El archivo **`.gitignore`** le dice a Git qué ignorar. Regla del curso: *lo que se puede regenerar, no se versiona.*

La política de autoría

Cada estudiante confirma su propio trabajo, desde su propia cuenta.

- el historial es la evidencia individual de participación
- el integrador del equipo **fusiona**, no es autor de todo

```
git commit --author="Ana Pérez <aperez@um.edu.uy>" -m "..."
```

Un historial donde una sola persona firma todo ⇒ participación mínima **para todo el equipo**.

Los repositorios de cada estudiante

Repositorio	Patrón de Git
`lab-L<n>-<tema>` (equipo)	ramas y fusión
`bitacora-<apellido>` (personal)	repositorio propio, commit semanal

Manos al teclado

Hoy, antes de irse: la primera confirmación

```
# 1. presentarse ante Git (una sola vez por máquina)
git config --global user.name  "Nombre Apellido"
git config --global user.email "usuario@um.edu.uy"
```

```
# 2. clonar el repositorio de su equipo
git clone https://github.com/um-icd-2026/lab-L1-macro.git
cd lab-L1-macro

# 3. crear su rama, trabajar, confirmar
git switch -c inicio-apellido
echo "# Notas de <nombre>" > notas_apellido.md
git add notas_apellido.md
git commit -m "estructura: agrega archivo de notas de <nombre>"

# 4. publicar
git push -u origin inicio-apellido
```

O por interfaz, mismo resultado: clonar desde VS Code o GitHub Desktop (paso 2), crear el archivo, preparar → mensaje → confirmar → sincronizar (pasos 3 y 4). El paso 1 sí conviene hacerlo una vez en la terminal.

Los diez comandos del semestre

Comando	Para qué
`git status`	¿en qué estado está todo? (el más usado)
`git add <archivo>`	preparar un cambio
`git commit -m "..."`	confirmar con mensaje
`git log --oneline`	leer la historia
`git diff`	¿qué cambié exactamente?
`git switch -c <rama>`	crear una rama y pasarse
`git merge <rama>`	incorporar una rama
`git push` / `git pull`	sincronizar con GitHub
`git clone <url>`	copiar un repositorio
`git tag hito-1`	etiquetar una entrega

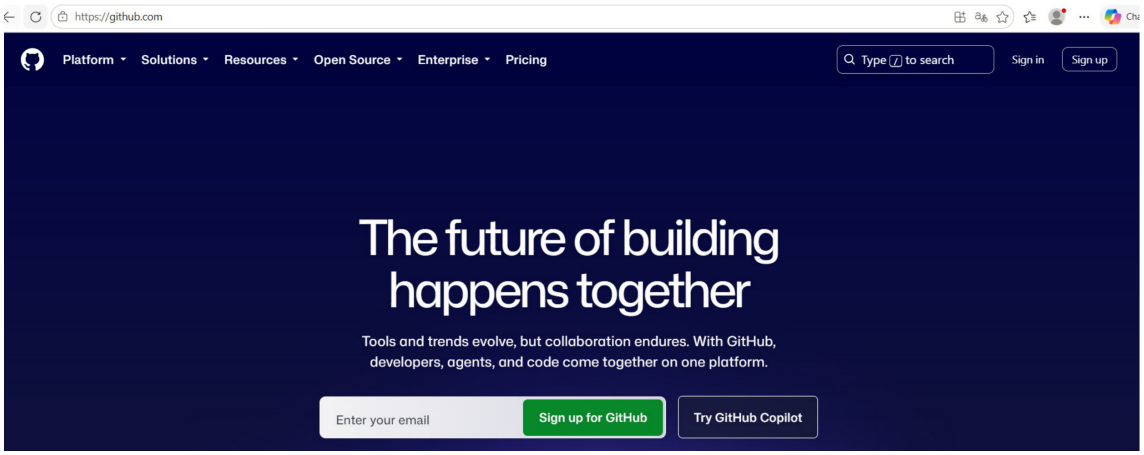
Para profundizar

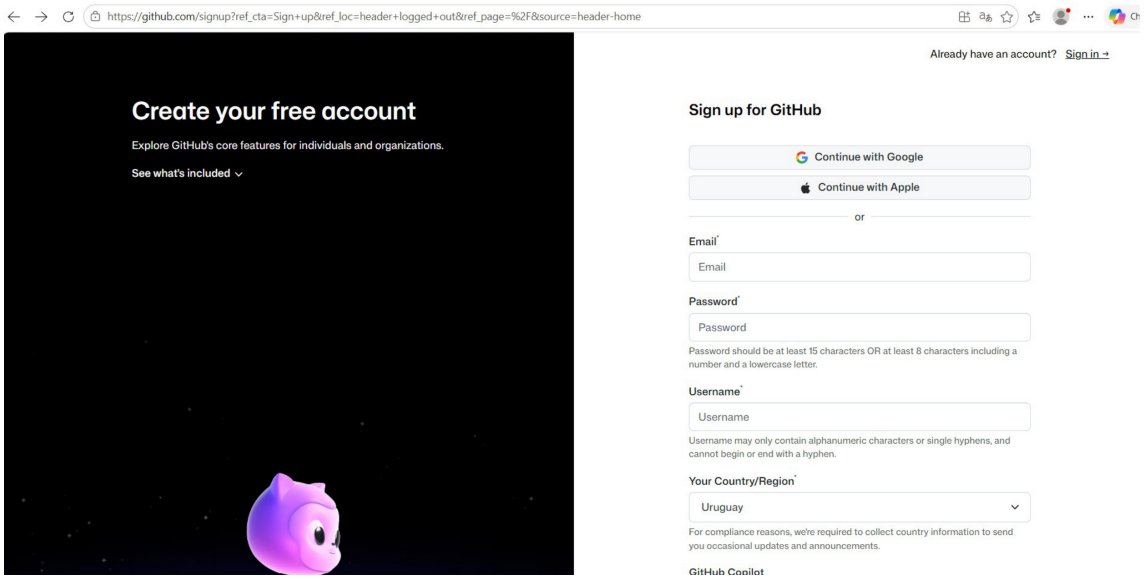
- Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Gratuito en línea: https://git-scm.com/book/es — **en español**.
- La documentación oficial: https://git-scm.com/docs
- Capítulos del *book* de la clase: *Git: ramas, fusión y conflictos* y *Solicitudes de incorporación y revisión de código*.

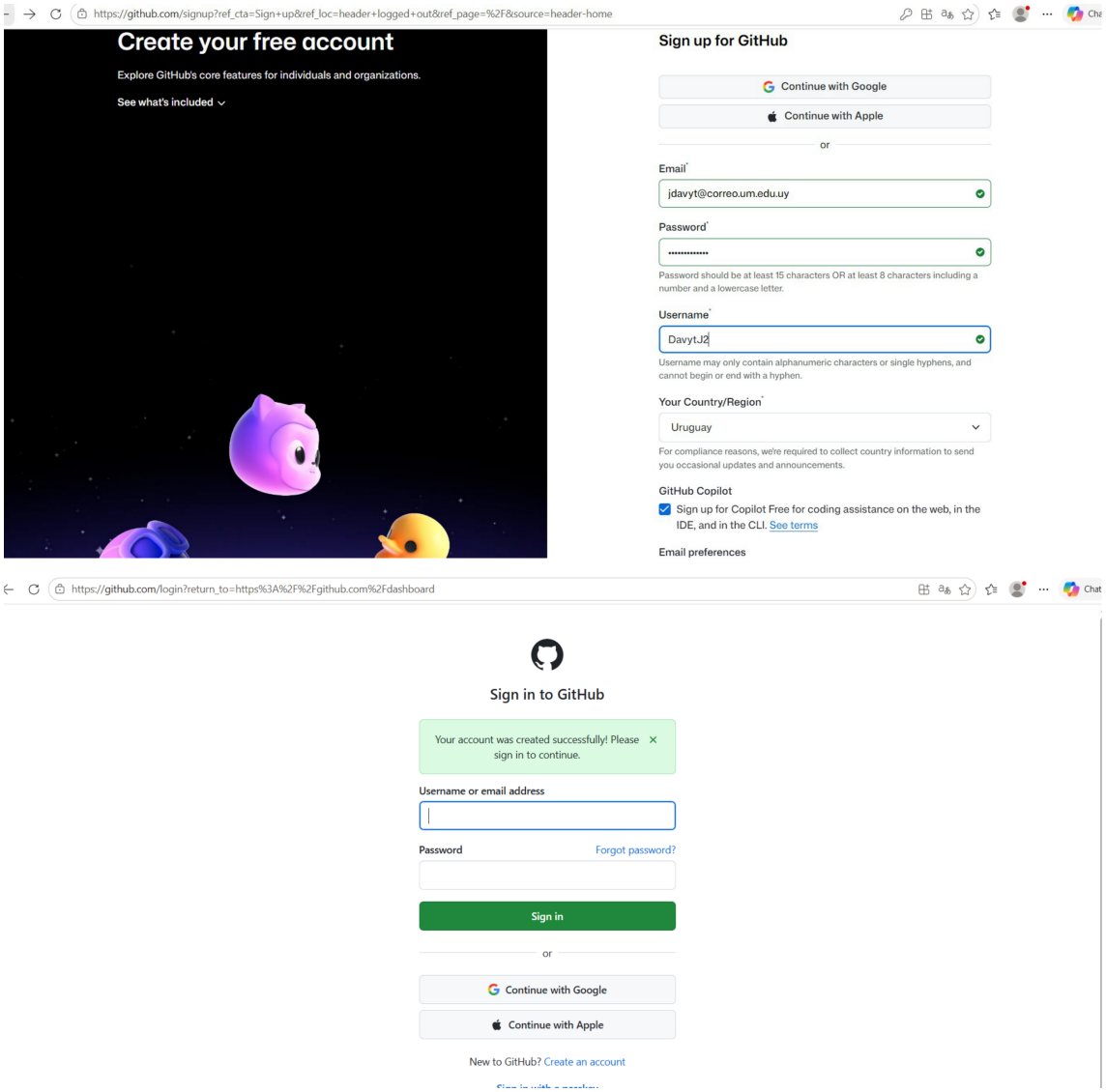
2. Primeros pasos en github

Reproducibilidad, versionado y trabajo en equipo

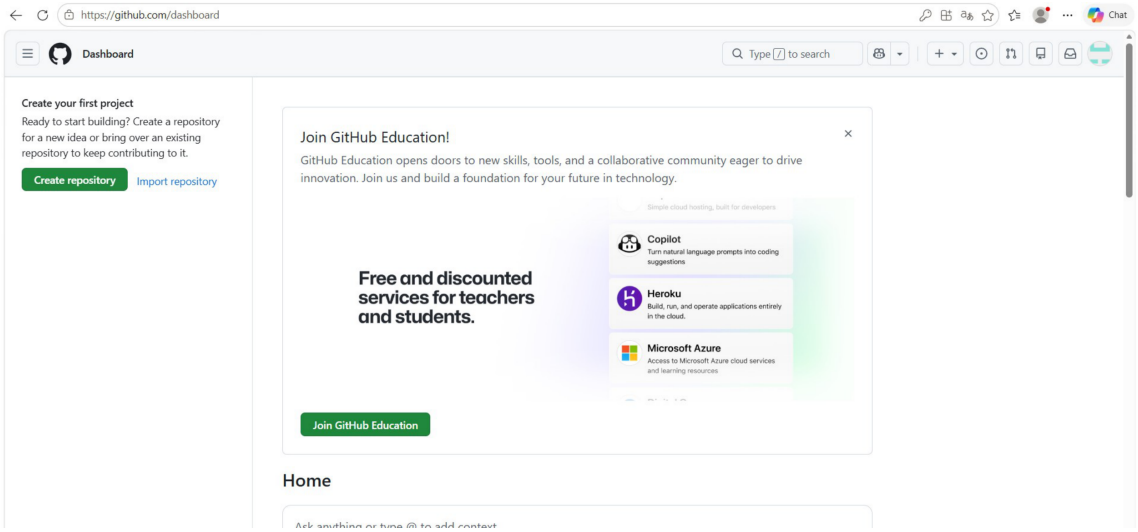
1-Creación de usuario github

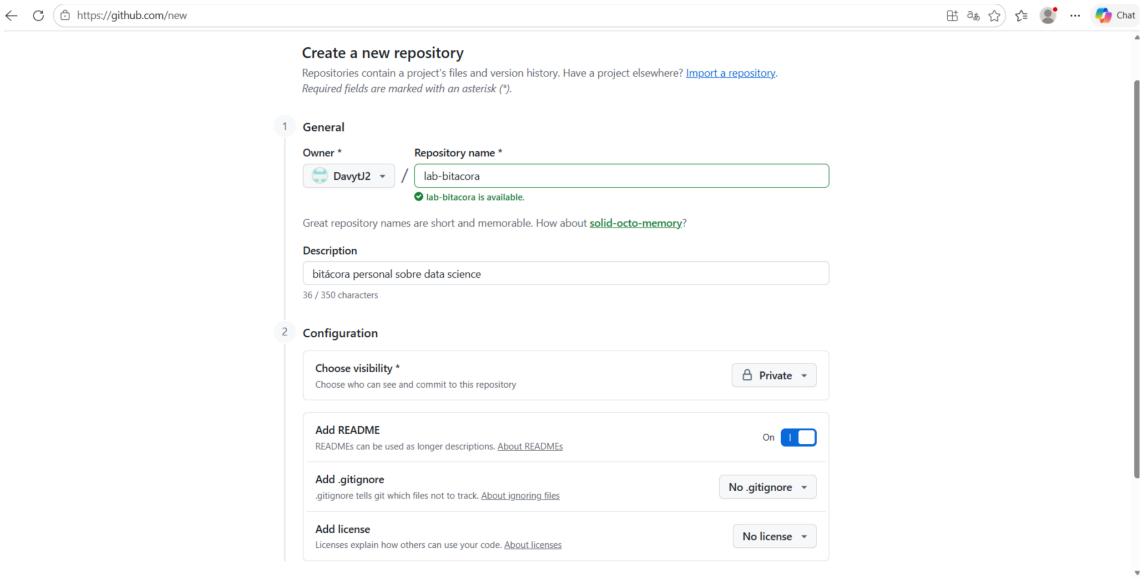


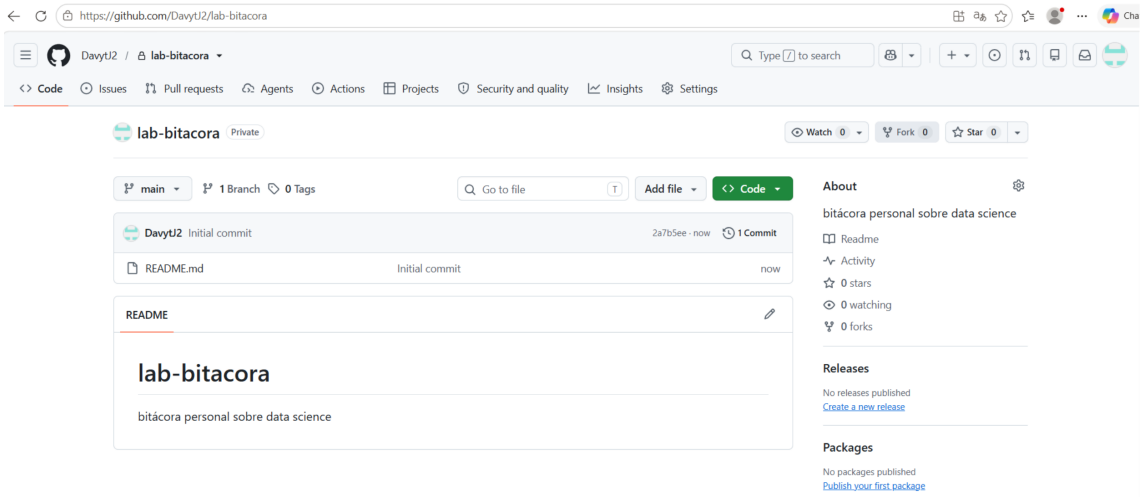




2-Creación de repositorio personal

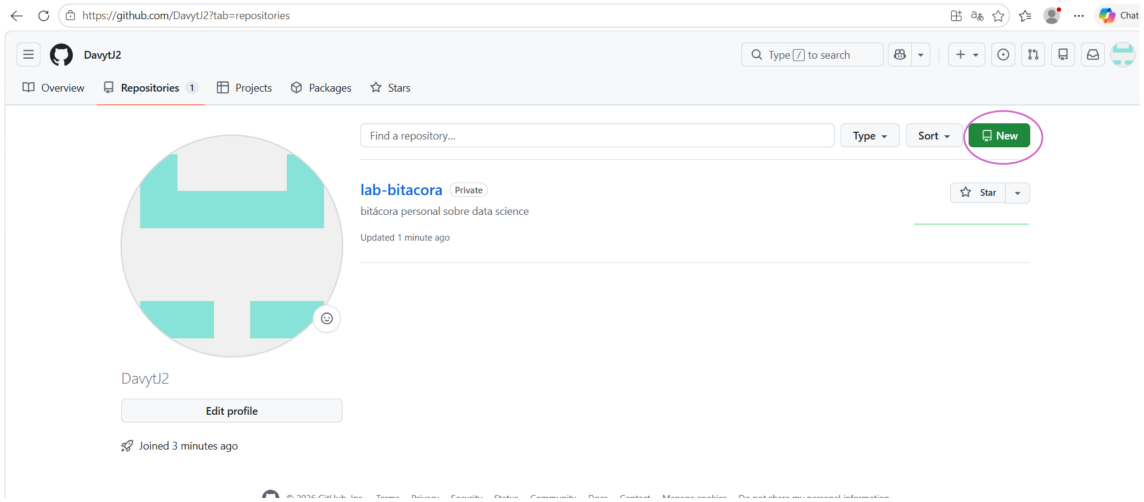


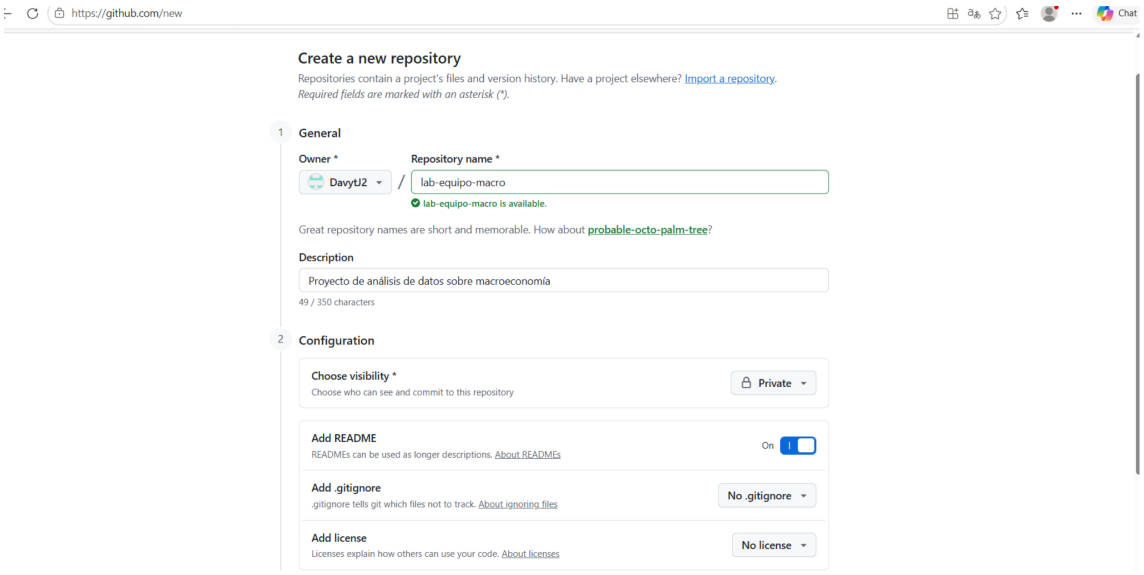


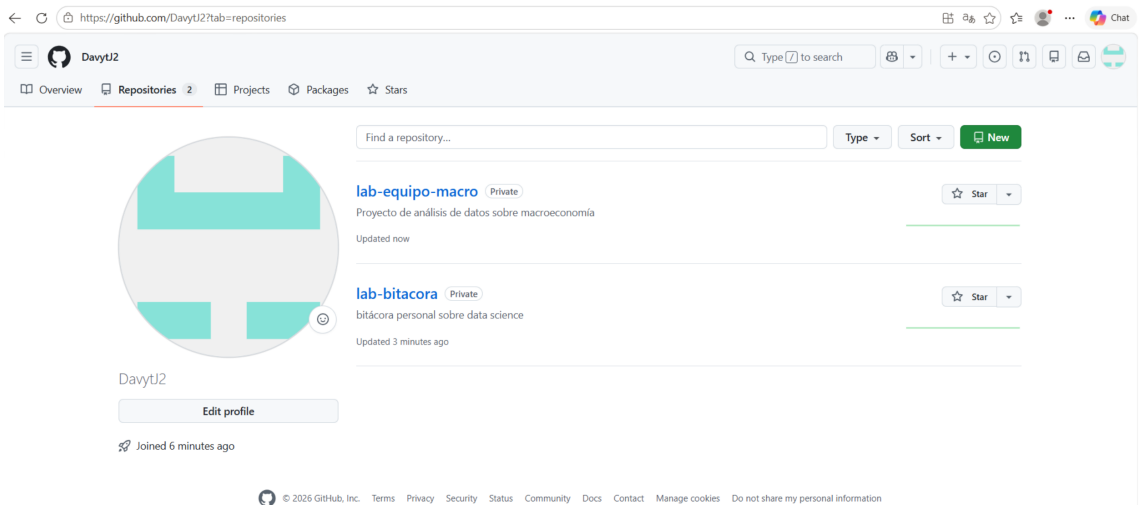


3-Repositorio equipo

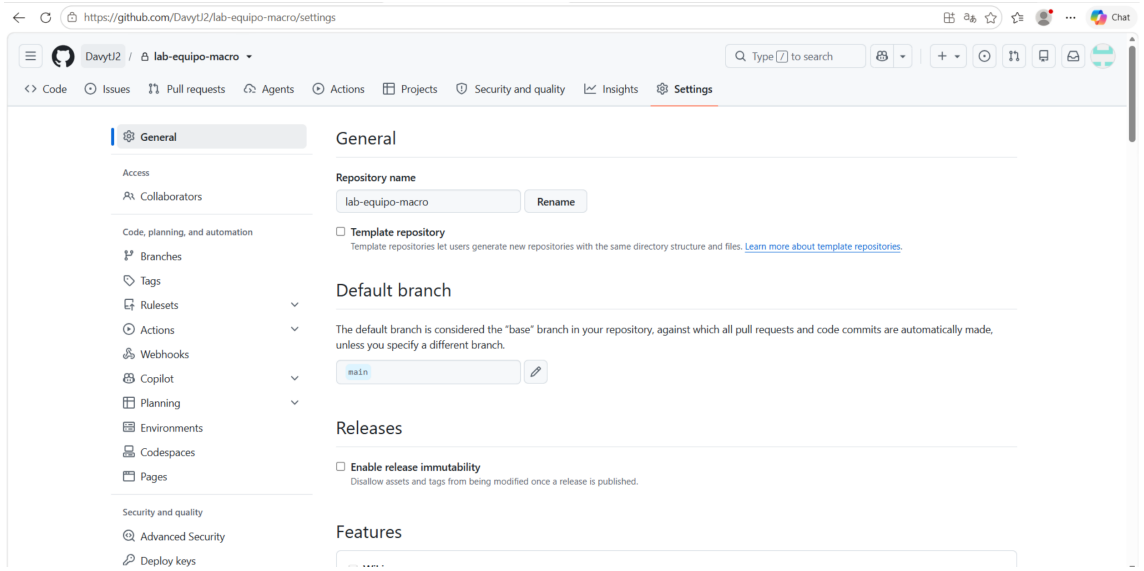
Si soy coordinador:

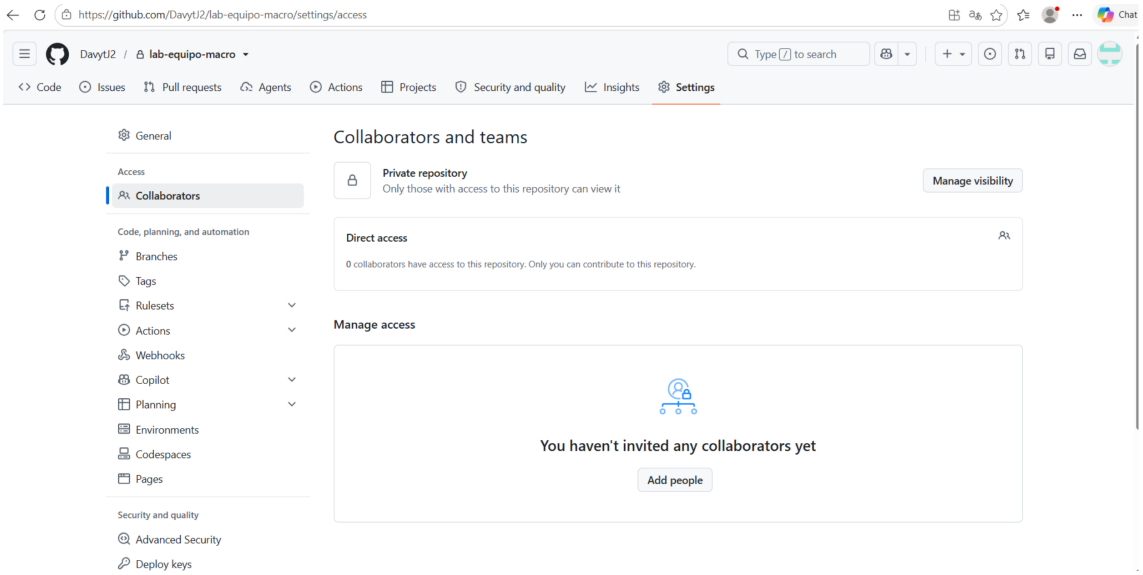




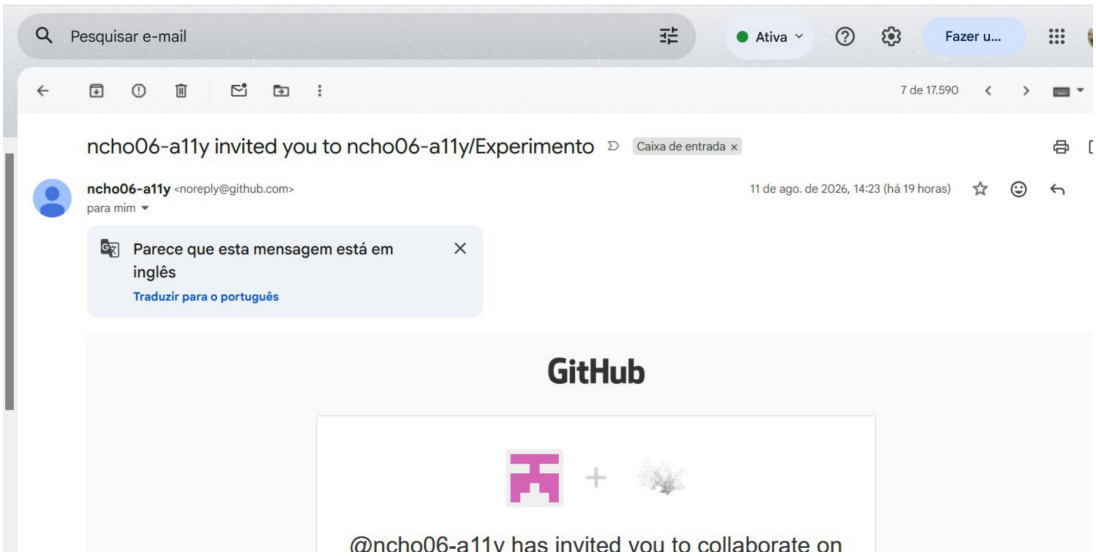


Agregar colaboradores



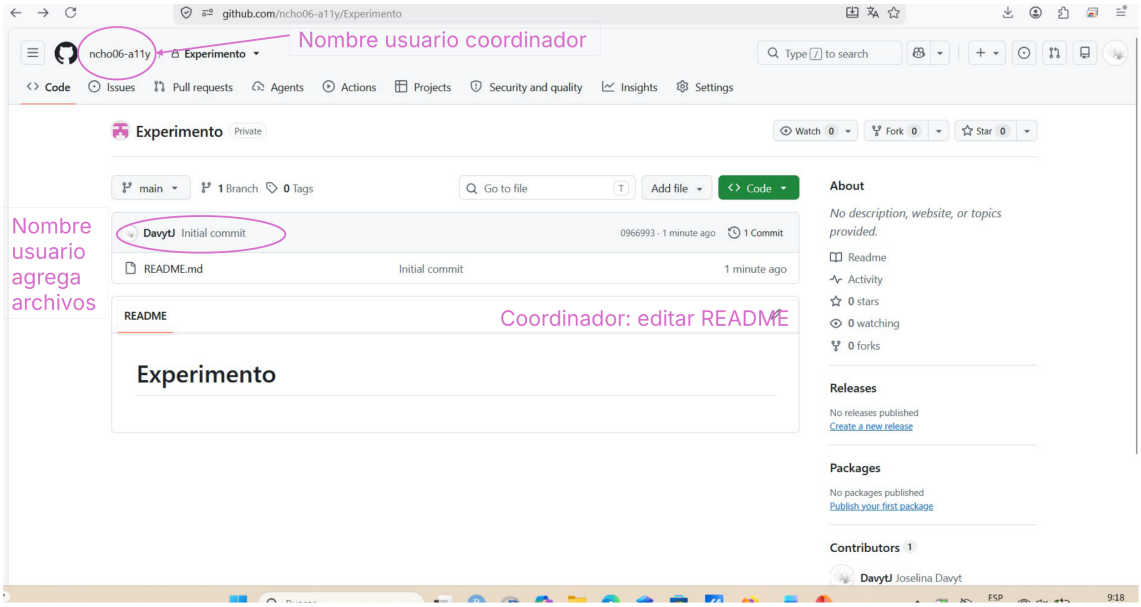


Si no soy coordinador:



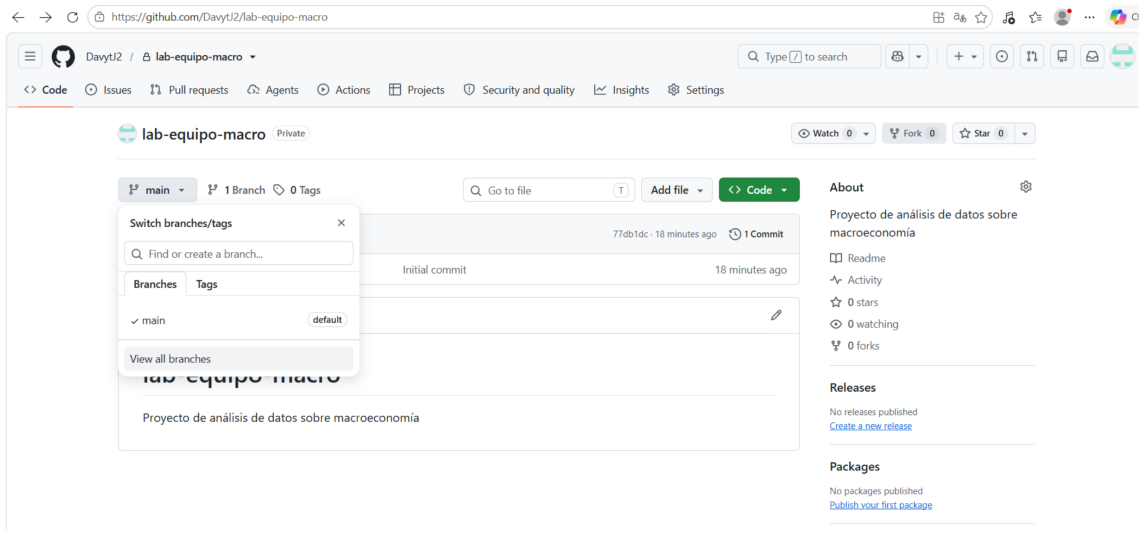
3-Como es el repositorio equipo

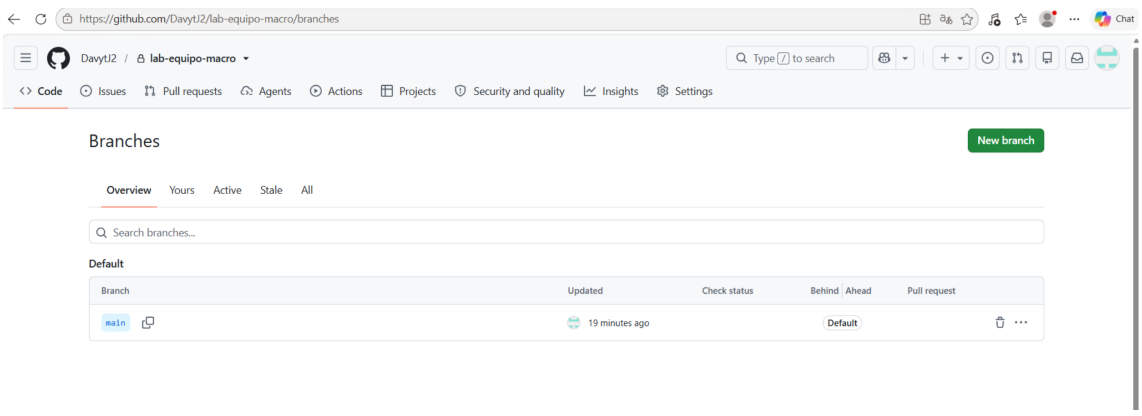
Nombre usuario coordinador



Crear una rama:

Ramas - Documentación de GitHub

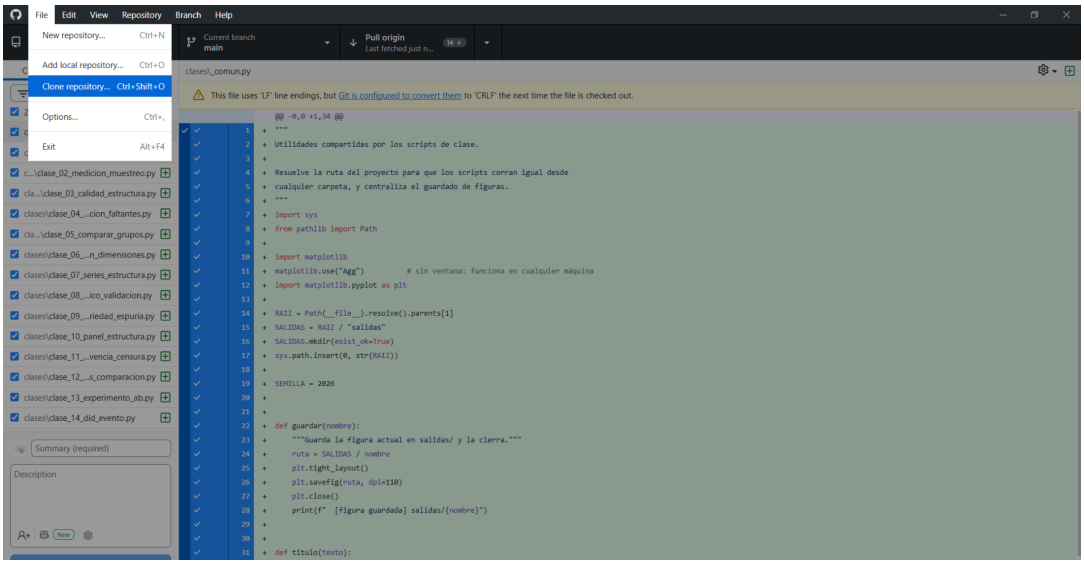


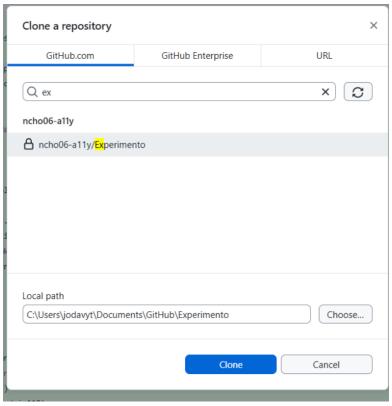


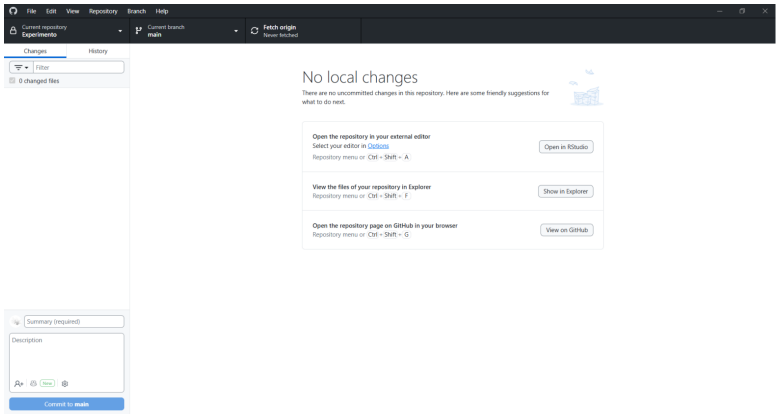
Clonar repositorio

= tener el trabajo en mi computadora personal

Opción 1:







3. Lenguajes, intérpretes y entornos de trabajo

RStudio, Spyder, Anaconda y Visual Studio Code
Introducción a la Ciencia de Datos · Universidad de Montevideo

Tres piezas distintas que solemos confundir

01 Qué es un lenguaje de programación

Lenguaje de programación: sintaxis y semántica

Definición. Un lenguaje de programación es un sistema formal de notación para escribir instrucciones que una computadora puede ejecutar.

- **Sintaxis:** qué combinaciones de símbolos son válidas.
- **Semántica:** qué hace la computadora con cada instrucción válida.
- Un lenguaje es una especificación. Para que algo ocurra hace falta un programa que lo ejecute.

La misma idea, dos sintaxis

R

```
if (x > 0) {
  print("positivo")
}
```

Python: la sangría es obligatoria

```
if x > 0:
    print("positivo")
```

Compilador e intérprete

Compilador

Traduce todo el programa a código de máquina y después se ejecuta. Ejemplos: C, Fortran.

Intérprete

Lee y ejecuta las instrucciones una a una. Ejemplos: R, Python.

```
> x <- c(3, 5, 10)
> mean(x)
[1] 6
```

Consecuencia práctica. Cuando decimos «instalar R» o «instalar Python» estamos instalando el intérprete: el programa que entiende y ejecuta ese lenguaje.

Script, paquete y biblioteca

- `.R` instrucciones en R
- `.py` instrucciones en Python
- `.ipynb` notebook de Jupyter (JSON con código, texto y resultados)

Un script es un archivo de texto plano: se puede abrir con el Bloc de notas.

Paquete (R) o biblioteca (Python): funciones escritas por otros que se agregan al lenguaje base. Se instalan en un intérprete concreto.

Dos instalaciones de Python en la misma computadora: lo instalado en una no existe en la otra.

02 R y Python: características y usos

R: un lenguaje creado para la estadística

- La tabla de datos (`data.frame`) es nativa.
- Los faltantes (NA) son nativos.
- Vectorizado: x + 1 suma 1 a cada elemento.
- Los índices empiezan en 1.
- Fórmulas para modelos: `y ~ x1 + x2`
- Fuerte en inferencia, gráficos (ggplot2) e informes reproducibles.

Python: un lenguaje de propósito general

- Propósito general: web, automatización, sistemas, datos.
- La sangría es parte de la sintaxis.
- Los índices empiezan en 0.
- Las tablas no son nativas: vienen de pandas.
- Orientado a objetos: `df.head()` aplica el método head al objeto df.
- PyPI: cientos de miles de bibliotecas, de calidad heterogénea.

La misma tarea en los dos lenguajes

Leer un archivo, quitar faltantes y calcular el promedio de una columna.

R

Py

```
datos <- read.csv("ventas.csv")

mean(datos$monto, na.rm = TRUE)

nrow(datos)     # filas
```

```
import pandas as pd
datos = pd.read_csv("ventas.csv")
datos["monto"].mean()

len(datos)      # filas
```

En R la tabla y los faltantes son parte del lenguaje. En Python hay que importar pandas, y mean() ignora los faltantes por defecto.

Comparación entre R y Python

Aspecto	R	Python
Tipo de lenguaje	Específico para estadística	Propósito general
Tabla de datos	Nativa (data.frame)	Biblioteca (pandas)
Faltantes	Nativos (NA)	Según la biblioteca (NaN, None, pd.NA)
Primer índice	1	0
Repositorio de paquetes	CRAN	PyPI
IDE más usado	RStudio	VS Code, Spyder, Jupyter
Comunidad de origen	Estadística	Programación e ingeniería

Cuándo conviene uno u otro

R

Es preferible R cuando

Py

Es preferible Python cuando

- El trabajo es sobre todo análisis estadístico: modelos, pruebas, panel, supervivencia, encuestas.
- Se necesita un método específico o reciente, que suele aparecer antes en CRAN.
- El producto final es un informe o un artículo.
- El entorno es académico, de salud o de economía.
- El análisis es parte de un sistema mayor (una aplicación, una base de datos).
- El foco es la predicción con aprendizaje automático.
- Los datos son texto, imágenes o vienen de APIs.
- El equipo ya programa en Python.

Otros lenguajes y programas para analizar datos

Lenguaje	Origen	Licencia	Uso típico
MATLAB	1984, MathWorks	Comercial	Ingeniería: cálculo con matrices, señales, control
GNU Octave	Década de 1990, proyecto GNU	Libre	Alternativa compatible con buena parte de MATLAB
Julia	2012	Libre	Cálculo científico de alto rendimiento
SAS	1976, SAS Institute	Comercial	Industria farmacéutica, banca, seguros
Stata	1985, StataCorp	Comercial	Econometría, epidemiología
SPSS	1968	Comercial	Ciencias sociales; se usa mucho por menús
SQL	1974, IBM	Estándar	Consultar y combinar tablas en bases de datos

Las hojas de cálculo (Excel) también sirven para analizar datos, pero no dejan un registro de los pasos que permita repetir el análisis.

Lenguajes de matrices

MATLAB (MATrix LABoratory) nació para el cálculo con matrices: en MATLAB todo objeto es una matriz. R trabaja con vectores y matrices de forma nativa, y Python incorporó esta forma de trabajo con NumPy.

Producto de una matriz por sí misma:

MATLAB / Octave

```
A = [1 2; 3 4];
A * A
```

R

```
A <- matrix(c(1, 2, 3, 4),
  nrow = 2, byrow = TRUE)
A %*% A
```

Python (NumPy)

```
import numpy as np
A = np.array([[1, 2],
    [3, 4]])
A @ A
```

En los tres casos el resultado es la matriz [7 10; 15 22]. Cambia la notación del producto: *, %*% y @.

03 Cuando los ingenieros entraron en la estadística

Dos tradiciones en el análisis de datos

Objetivo de la tradición estadística: explicar el mecanismo que generó los datos y cuantificar la incertidumbre.

Objetivo de la tradición ingenieril: predecir bien en datos nuevos, aunque el mecanismo quede sin explicar.

Las dos culturas según Breiman (2001)

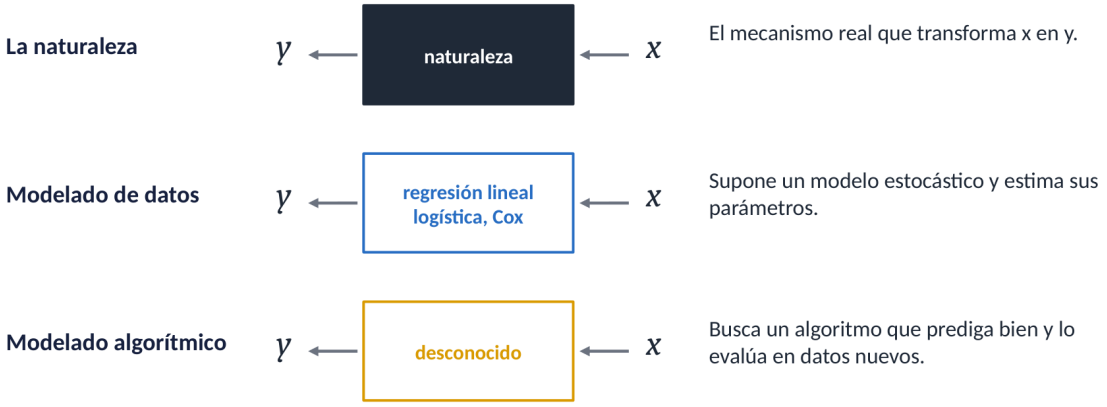


Diagrama adaptado de Breiman, L. (2001). Statistical modeling: The two cultures. Statistical Science, 16(3).

Qué cambió y qué quedó en tensión

Lo que aportó la ingeniería

- Escala: millones de filas y miles de variables.
- Prácticas de software: versiones, pruebas, código reutilizable.
- Validación por predicción en datos de prueba.
- Python como lenguaje del análisis (NumPy, pandas, scikit-learn).

Lo que la estadística sigue señalando

- Explicar no es predecir (Shmueli, 2010).
- Más datos no compensan un sesgo de selección (Meng, 2018).
- Una predicción sin margen de error está incompleta.
- Predecir quién compra no dice qué pasa si cambia el precio.

04 Qué es un IDE

IDE: entorno de desarrollo integrado

Definición. Un IDE (Integrated Development Environment) es un programa que reúne en una sola ventana las herramientas para escribir, ejecutar y revisar código.

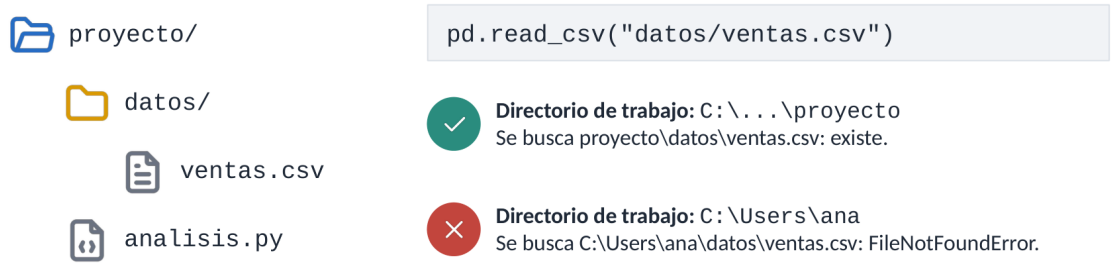
El IDE no es el lenguaje ni el intérprete

Pieza	En R	En Python
Lenguaje	R	Python
Intérprete (lo que ejecuta)	R de CRAN	Python de python.org, de Anaconda o del instalador de Spyder
IDE (donde se escribe)	RStudio	Spyder, VS Code

- RStudio sin R instalado abre, pero no tiene qué ejecutar.
- VS Code sin Python permite escribir un .py, pero no ejecutarlo.
- Un mismo script funciona igual en Spyder, VS Code o la terminal si el intérprete y los paquetes son los mismos.

Directorio de trabajo y rutas relativas

Una ruta relativa se completa a partir del directorio de trabajo del intérprete.



Cada IDE fija el directorio de trabajo de un modo distinto. Para cada uno veremos cómo hacerlo.

05 Las herramientas: instalación y configuración

Anaconda: una distribución, no un IDE

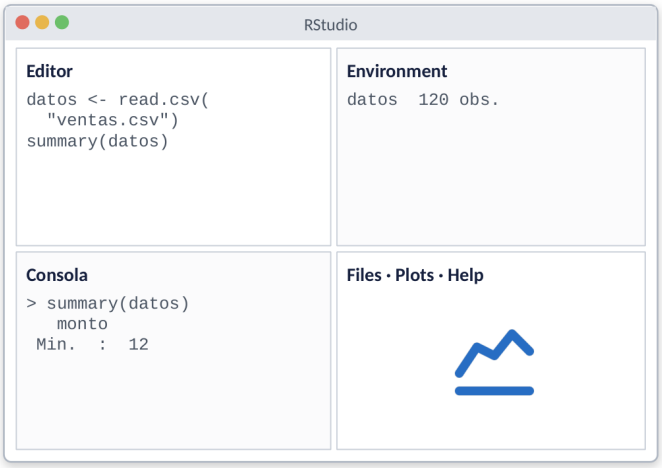
Todo queda instalado junto: al abrir Spyder o Jupyter desde Anaconda, las bibliotecas ya están disponibles.

Instalar Anaconda en Windows

1. **Descargar**
   anaconda.com/download. Se puede omitir el registro (Skip registration).
2. **Ejecutar**
   Abrir el instalador y aceptar la licencia.
3. **Just Me**
   Instalar solo para el usuario: no requiere permisos de administrador.
4. **Ruta**
   Si el usuario tiene espacios, tildes o ñ, instalar en C:\anaconda3.
5. **PATH**
   No marcar «Add Anaconda to my PATH».
6. **Verificar**
   Abrir Spyder y ejecutar en la consola: import pandas.

Requiere al menos 5 GB. Desde junio de 2026 Anaconda no publica paquetes nuevos para Windows 10. Gratis para uso individual y organizaciones de menos de 200 empleados.

RStudio: el IDE de R



Desarrollado por Posit (antes RStudio, PBC). Gratuito y de código abierto.

- Arriba a la izquierda: el script.
- Abajo a la izquierda: R ejecuta y responde.
- Arriba a la derecha: los objetos en memoria.
- Abajo a la derecha: archivos, gráficos, paquetes y ayuda.

Instalar y configurar RStudio

1. Instalar R

cran.r-project.org → Download R for Windows → base. En Mac: arm64 (Apple M) o x86_64 (Intel).

2. Instalar RStudio

posit.co/download/rstudio-desktop. Detecta R automáticamente.

3. Configurar

Tools → Global Options: no restaurar .RData y no guardarlo nunca.

4. Crear proyecto

File → New Project → Existing Directory. Fija el directorio de trabajo.

El orden importa: RStudio no incluye R.

```
getwd()  # carpeta del proyecto
```

No guardar .RData hace que cada sesión empiece limpia: el resultado depende solo del script.

Spyder: un IDE científico para Python

Su diseño se parece al de RStudio y al del entorno de MATLAB: editor, consola y variables visibles al mismo tiempo.

Dos formas de instalarlo

- **Con Anaconda:** ya viene incluido.
- **Instalador propio:** spyder-ide.org. Trae Python y las bibliotecas principales (unos 640 MB).

Configurar y usar Spyder

Herramientas → Preferencias

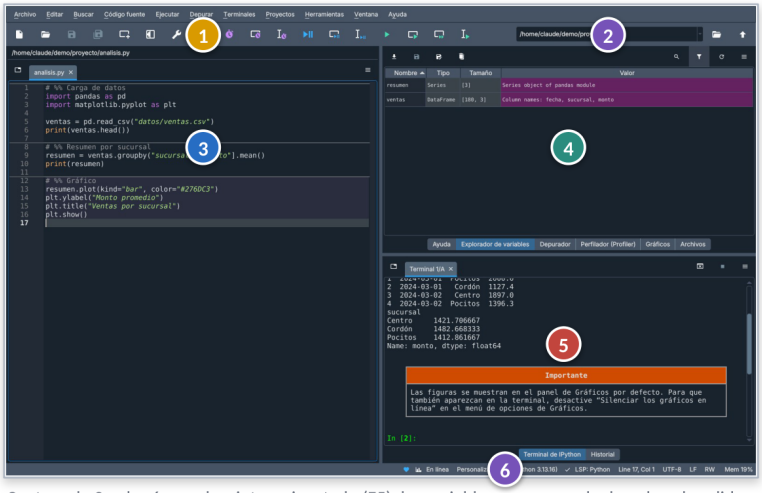
- **Intérprete de Python:** cuál usa Spyder. Por defecto, el que trae su instalador o el de Anaconda.
- **Directorio de trabajo:** que la consola arranque en el directorio del archivo que se ejecuta.

```
import sys
print(sys.executable)
```

F5	ejecutar el archivo completo
F9	ejecutar la línea o la selección
Ctrl + Enter	ejecutar la celda actual

Una celda es un bloque que empieza con # %%. Permite ejecutar el script por partes sin convertirlo en notebook.

Spyder en pantalla

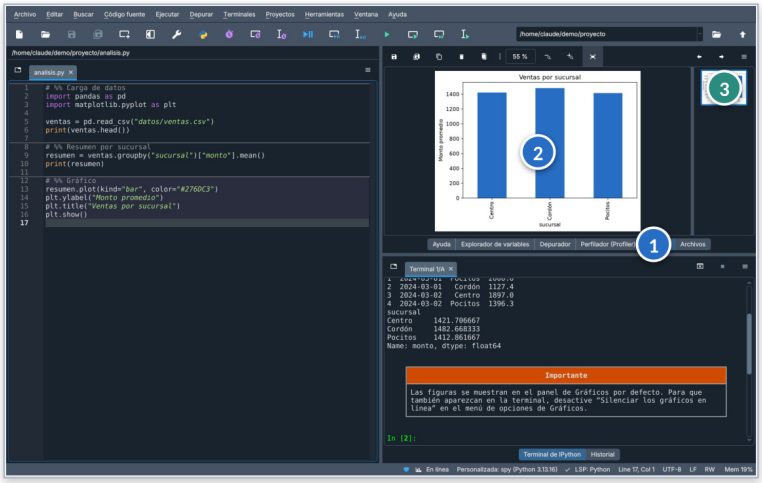


Las partes de la ventana

1. Barra de herramientas: ejecutar, depurar
2. Directorio de trabajo
3. Editor: el script analisis.py
4. Explorador de variables; al lado, las pestañas Ayuda, Gráficos y Archivos
5. Consola de IPython: la salida
6. Barra de estado: qué Python se usa

Captura de Spyder 6 con el script ya ejecutado (F5): las variables aparecen a la derecha y la salida, abajo.

Spyder: ver el gráfico y abrir los datos

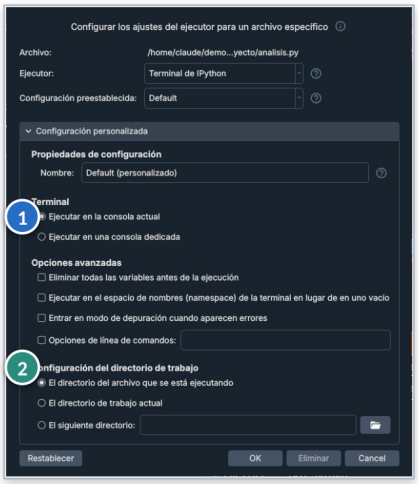


El panel de gráficos

1. La pestaña Gráficos muestra las figuras que genera el script
2. El gráfico de barras de analisis.py
3. Miniaturas: una por figura

Con doble clic sobre una variable del explorador se abre la tabla completa, como una planilla de solo lectura.

Spyder: configurar cómo se ejecuta un archivo



Ctrl + F6

1. Ejecutar en la consola actual o en una dedicada
2. Directorio de trabajo: elegir el del archivo que se ejecuta, para que las rutas relativas funcionen

Ejecutar → Configuración por archivo (Ctrl+F6). La segunda opción resuelve los errores de rutas relativas.

Visual Studio Code: un editor que se vuelve IDE

Editor gratuito de Microsoft. Se convierte en IDE con extensiones. Trae Git integrado.

No confundir con Visual Studio (sin «Code»), que es otro programa.

Instalación

- code.visualstudio.com
- En Windows, marcar «Agregar a PATH».
- No trae Python ni R: hay que instalarlos aparte.

Configurar VS Code para Python y R

Para	Extensión	Autor
Python	Python	Microsoft
Notebooks	Jupyter	Microsoft
R	R	REditorSupport

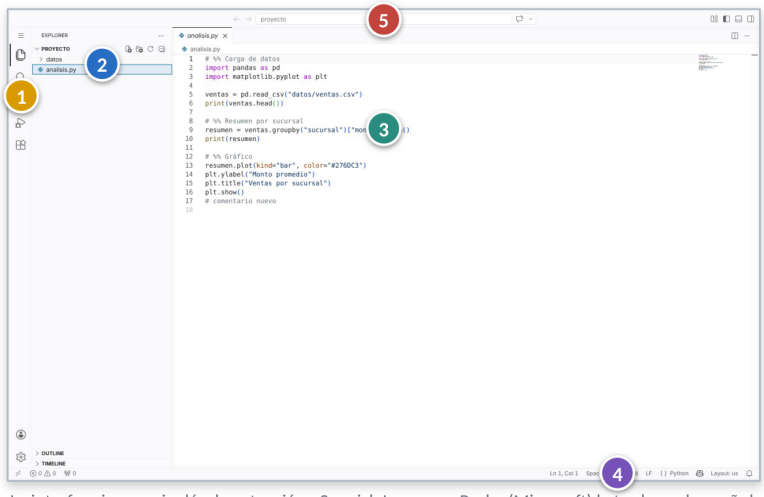
```
install.packages("languageserver")  # para R
```

```
> Python: Select Interpreter
Python 3.12 ('base')  ~\anaconda3
Python 3.12  ~\AppData\...\python.exe
Python 3.11  Spyder\python.exe
```

Ctrl + Shift + P abre la paleta de comandos.

1. **Abrir la carpeta**
   Archivo → Abrir carpeta. Es el equivalente al proyecto de RStudio.
2. **Elegir el intérprete**
   Queda indicado en la barra inferior.
3. **Ejecutar**
   Botón Run arriba a la derecha, o Shift + Enter para la selección o la celda.

VS Code en pantalla

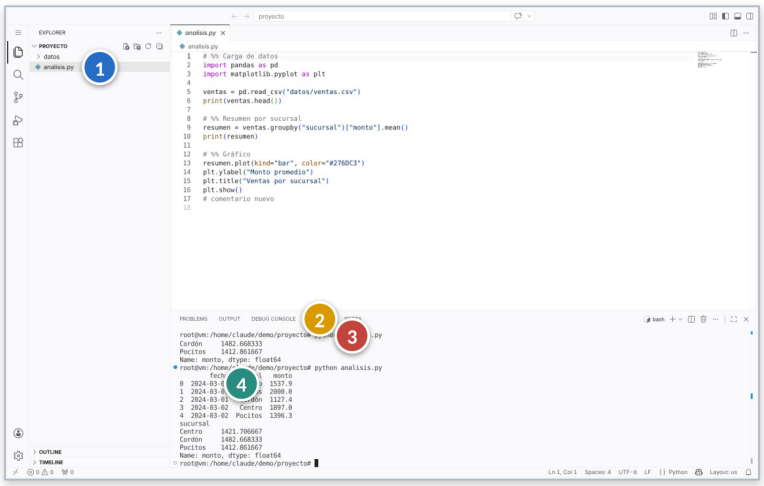


La interfaz viene en inglés; la extensión «Spanish Language Pack» (Microsoft) la traduce al español.

Las partes de la ventana

1. Barra de actividades: explorador, búsqueda, Git, ejecutar y depurar, extensiones
2. Explorador: la carpeta abierta (el «proyecto»)
3. Editor
4. Barra de estado: lenguaje e intérprete elegido
5. Barra superior: buscar archivos y comandos

VS Code: ejecutar un script en la terminal

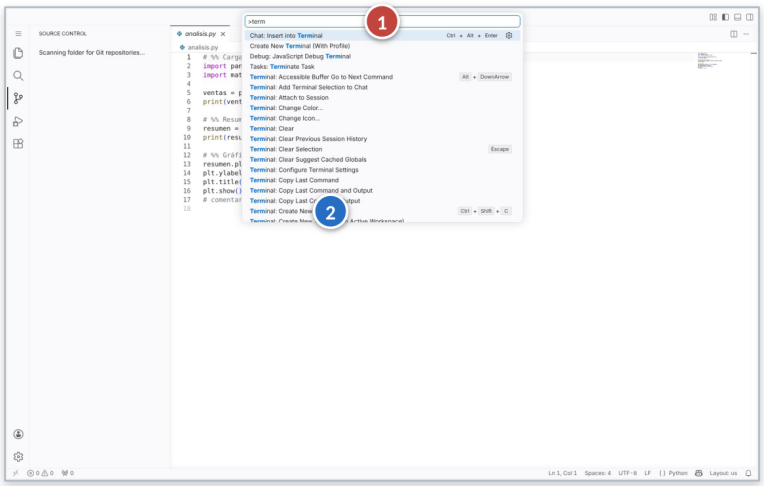


Con la extensión Python instalada también se puede usar el botón Run, que hace esto mismo.

Paso a paso

1. Abrir analisis.py desde el explorador
2. Abrir la terminal integrada: Ctrl + ` (o menú Terminal)
3. Escribir python analisis.py
4. La salida aparece en la misma terminal

VS Code: la paleta de comandos



Desde aquí se elige el intérprete (Python: Select Interpreter), se abre la terminal o se cambia la configuración, sin memorizar menús.

Notebooks de Jupyter

Un notebook (.ipynb, formato Jupyter) alterna celdas de código y de texto, y guarda el resultado debajo de cada celda. El código lo ejecuta un kernel: un intérprete que corre en segundo plano.

```
In [2]:

datos = pd.read_csv("ventas.csv")
```

El orden de ejecución no es el orden del documento

```
In [5]:

datos = datos[datos["monto"] > 0]
```

La celda 3 se ejecutó antes que la 2: el promedio se calculó sin filtrar. Si se ejecuta de arriba abajo, el número cambia.

```
In [3]:

datos["monto"].mean()

Out[3]: 1520.4
```

Dónde se usan: Jupyter y JupyterLab (incluidos en Anaconda), VS Code con la extensión Jupyter, Google Colab en el navegador.

Notebook o script con celdas

Notebook (.ipynb)

Script con celdas (.py)

- Junta explicación, código y resultados.
- Útil para explorar y para mostrar un análisis paso a paso.
- Las celdas se pueden ejecutar en cualquier orden.
- Si el kernel no tiene los paquetes, falla aunque el código esté bien.
- Guarda imágenes dentro del archivo: es difícil comparar versiones en Git.

```
# %% Carga
datos = pd.read_csv("ventas.csv")

# %% Resumen
datos["monto"].mean()
```

- Se ejecuta por partes, como un notebook.
- Sigue siendo texto plano: F5 lo corre de arriba abajo.
- Se compara sin problemas en Git.

Otros entornos: ¿hacen falta?

Positron	De Posit, sobre la misma base que VS Code. Trata a R y Python por igual. Versión estable desde 2025; RStudio se sigue manteniendo.
PyCharm	De JetBrains. Muy completo, orientado a desarrollo de software. Tiene versión gratuita.
JupyterLab	Entorno de notebooks en el navegador. Viene con Anaconda.
Google Colab	Notebooks en la nube con una cuenta de Google. No requiere instalar nada.

Para este curso no hace falta ninguno: RStudio cubre R; Spyder o VS Code cubren Python. Positron sirve a quien quiera un solo programa para ambos; Colab, a quien no pueda instalar.

Qué conviene no instalar (o no usar)

Dos instalaciones de Python a la vez	python.org + Anaconda + Microsoft Store: los paquetes quedan en un intérprete y el IDE usa otro («No module named…»). Elegir una sola.
La ventana interactiva (Jupyter) de VS Code para todo	Cada archivo abre un kernel en segundo plano y la memoria se consume enseguida. Para el curso alcanza con la terminal; si se usan celdas, un solo kernel.
Extensiones «por las dudas»	Cada extensión carga al arrancar VS Code. Con Python y Jupyter alcanza; las demás, solo cuando hagan falta.
Actualizar todo a mitad de semestre	Actualizar Anaconda o el IDE puede cambiar versiones que los scripts de clase usan. Actualizar al terminar el curso.

Comparación de las herramientas del curso

	RStudio	Spyder	VS Code	Anaconda
¿Es un IDE?	Sí	Sí	Sí, con extensiones	No: distrib
Lenguaje	R	Python	Cualquiera	Python
¿Trae intérprete?	No: instalar R antes	Sí, en su instalador	No	Sí
Explorador de variables	Sí	Sí	Limitado	—
Carpeta de trabajo	Proyecto .Rproj	Preferencias	Abrir carpeta	—

	RStudio	Spyder	VS Code	Anaconda
Git integrado	Sí	No	Sí	—

Problemas frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
FileNotFoundError / cannot open file	El directorio de trabajo no es la carpeta del proyecto	Proyecto, abrir carpeta o preferencia de directorio. Verificar con getwd() u os.getcwd()
No module named ‘pandas’	El paquete está en otro intérprete	Revisar sys.executable y elegir el intérprete correcto
RStudio pide ubicar R	R no está instalado	Instalar R desde CRAN y reiniciar RStudio
«python» no se reconoce como comando	Python no está en el PATH	Usar Anaconda Prompt o elegir el intérprete en el IDE
Errores al instalar Anaconda	Ruta con espacios o tildes	Reinstalar en C:\anaconda3

Fuentes

Breiman, L. (2001). Statistical modeling: The two cultures. Statistical Science, 16(3), 199–231.

Donoho, D. (2017). 50 years of data science. Journal of Computational and Graphical Statistics, 26(4), 745–766.

McKinney, W. (2022). Python for Data Analysis (3.ª ed.). O’Reilly.

Meng, X.-L. (2018). Statistical paradises and paradoxes in big data (I). The Annals of Applied Statistics, 12(2), 685–726.

Shmueli, G. (2010). To explain or to predict? Statistical Science, 25(3), 289–310.

Tukey, J. W. (1962). The future of data analysis. The Annals of Mathematical Statistics, 33(1), 1–67.

Wickham, H., Çetinkaya-Rundel, M. y Grolemund, G. (2023). R for Data Science (2.ª ed.). O’Reilly.

Yu, B. y Barter, R. L. (2024). Veridical Data Science. MIT Press.

Documentación oficial: anaconda.com/docs, posit.co, spyder-ide.org, code.visualstudio.com.

4. Archivos, formatos y rutas

Introducción a la Ciencia de Datos — Universidad de Montevideo

Lo que vimos hasta ahora en el curso

	Tema	Qué hicimos
📋	Medición y encuestas	operacionalización, niveles de medición, diseño de un cuestionario
📂	Carga de datos	leer archivos en distintos formatos; rutas y directorio de trabajo
🔎	Perfilado y fechas	filas, columnas, faltantes, fechas y agregación con los datos abiertos de aire y tránsito
📊	Gráficos	las partes de un gráfico, más de una serie, colores por grupo
🔗	Unión de tablas	merge de ozono con PM2.5 por la hora
⏱️	Series de tiempo	grilla regular, rezago y autocorrelación
🧾	IA y OCR	leer tickets: la imagen se manda como bytes a la API de Gemini
📈	Variables y modelo	variables nuevas, correlación y la recta, con la tabla de tránsito y aire

Hoy volvemos a la base: **qué hay dentro de un archivo, y cómo lo encuentra el programa.**

Qué es un archivo

El bit y el byte 💡

Bit (*binary digit*). La unidad mínima de información: vale 0 o 1. En el hardware es algo que está en uno de dos estados, como un interruptor apagado o encendido.

Byte. Un grupo de 8 bits. Con 8 lugares que valen 0 o 1 hay $2^8 = 256$ combinaciones distintas: de `00000000` a `11111111`, es decir, de 0 a 255.

Cada lugar vale el doble que el de su derecha, como en el sistema decimal cada lugar vale diez veces más:

Valor del lugar	128	64	32	16	8	4	2	1
Bits	0	1	0	0	0	0	0	1

`01000001` = 64 + 1 = **65**. En la tabla ASCII, 65 es la letra **A**. 🅰️

El hexadecimal: un byte en dos cifras

Escribir `01000001` es largo. El **sistema hexadecimal** usa 16 cifras: `0 1 2 3 4 5 6 7 8 9 A B C D E F`, donde A vale 10 y F vale 15.

Cada cifra hexadecimal reemplaza exactamente 4 bits, así que un byte siempre se escribe con dos cifras:

Bits	Se parte en	Hexadecimal	Decimal
`01000001`	`0100` · `0001`	`41`	65
`11000011`	`1100` · `0011`	`C3`	195
`11111111`	`1111` · `1111`	`FF`	255

Por eso en esta clase los bytes aparecen como `41`, `C3` o `FF`. En Python, un byte que no es un carácter se muestra como `\xc3`: la `\x` avisa que lo que sigue es hexadecimal.

Cuánto ocupa un archivo 📏

Unidad	Bytes	Un ejemplo del curso
byte	1	una letra sin tilde en un CSV
kilobyte (KB)	1000	`ventas.csv`: 70 bytes; `transito_aire_junio2024.csv`: 131 KB
megabyte (MB)	1 000 000	una foto de 1000 × 1000 píxeles sin comprimir: 3 MB
gigabyte (GB)	1 000 000 000	una película, o un año de datos minutales de varias estaciones

Windows cuenta de a 1024 en lugar de 1000 (1 KB = 1024 bytes), así que los tamaños que muestra pueden diferir un poco de estos.

Un archivo es una secuencia de bytes con un nombre

Archivo. Secuencia finita de bytes guardada en un disco, con un nombre y una ubicación (la carpeta donde está). El archivo no sabe qué significan sus bytes: eso lo decide el programa que lo lee.

Los primeros 12 bytes de `ventas.csv`:

```
66 65 63 68 61 2C 6C 6F 63 61 6C 2C
```

Así lo ve la computadora: solo números. 🤖 Para saber si son letras, píxeles o celdas de una planilla, hace falta una regla que diga cómo leerlos.

Todo archivo es una fila de bytes: cambia la regla para leerlos

	Archivo	Primeros bytes	Qué son
📄	`ventas.csv`	`66 65 63 68 61 2C`	las letras «fecha,»
🐍	`saludo.py`	`70 72 69 6E 74 28`	las letras «print(»
📊	`ventas.xlsx`	`50 4B 03 04`	«PK»: la firma de un archivo ZIP
📕	`ejercicios.pdf`	`25 50 44 46`	«%PDF»: la firma de un PDF
🖼️	`foto.jpg`	`FF D8 FF E0`	la firma de una imagen JPG
🔊	`tono.wav`	`52 49 46 46`	«RIFF»: la firma de un audio WAV

Los primeros bytes de muchos formatos son una **firma**: le dicen al programa qué tipo de archivo es. Después vienen letras, píxeles, celdas o sonido, según el formato.

Texto plano: cada byte es un carácter

Archivo de texto plano. Archivo cuyos bytes representan solo caracteres, según una codificación (una tabla que asigna un número a cada carácter). No guarda formato: ni negrita, ni colores, ni imágenes.

Los bytes de ventas.csv, leídos con la tabla ASCII

66	65	63	68	61	2C	6C	6F	63	61	6C	2C
f	e	c	h	a	,	l	o	c	a	l	,

2C → «,»

Son de texto plano: `.txt` `.csv` `.py` `.R` `.qmd` `.md` `.json` `.html`. Se pueden abrir y leer con cualquier editor de texto.

Archivo binario: los bytes no son caracteres

Archivo binario. Archivo cuyos bytes siguen una estructura propia del formato: números, imágenes comprimidas, partes de otros archivos. Solo lo lee bien el programa que conoce esa estructura.

`ventas.xlsx` abierto con el Bloc de notas: 🗑️

```
PK   ? a]I:O  [Content_Types].xmlËnÂ0 E÷ý
ÈÛ*1tQUEË©ô\{B,ÛòþŸðP[Q M¬dîÜsÇ<--
```

Un `.xlsx` es una carpeta de archivos comprimida (por eso empieza con PK, como un ZIP). El resto no son caracteres, y el editor los muestra como basura.

Los mismos datos, 3 columnas y 2 filas:

70 bytes en `ventas.csv`

5360 bytes en `ventas.xlsx`

Texto o binario: cómo se guarda el número 1250,5

En un archivo de texto se guardan los caracteres que se ven: «1», «2», «5», «0», «.», «5». Son 6 bytes, uno por carácter:

1	2	5	0	.	5
31	32	35	30	2E	35

En un archivo binario se guarda el número mismo, con la regla que usa la computadora para hacer cuentas: un número decimal ocupa siempre 8 bytes.

@	“	Š					
40	93	8A	00	00	00	00	00

Arriba de cada byte: lo que muestra el Bloc de notas si lo lee como texto. 🗑️

- En texto, al leer, el programa tiene que **traducir caracteres a números**. Ahí aparecen los problemas de separador, decimal y codificación.
- En binario el número ya está listo, pero hay que **saber el formato**: dónde empieza cada dato y cuántos bytes ocupa.

Un ejemplo de R: el formato .rds 📦

```
ventas <- read.csv("ventas.csv")
saveRDS(ventas, "ventas.rds")
ventas2 <- readRDS("ventas.rds")
identical(ventas, ventas2)   # TRUE
```

`ventas.rds` empieza con `1F 8B`: la firma de un archivo comprimido con gzip. 🗜️

Sin comprimir (`compress = FALSE`), adentro está la columna `monto`:

vector de números decimales (tipo 14)	`00 00 00 0E`
con 2 valores	`00 00 00 02`
1250,5	`40 93 8A 00 00 00 00 00`
980	`40 8E A0 00 00 00 00 00`

Gana: se lee de nuevo idéntica, con los tipos de cada columna, sin separador ni decimal. **Pierde:** solo la lee R (Python necesita un paquete como `pyreadr`), no se puede abrir con el Bloc de notas y Git no muestra qué cambió.

Una imagen también es una secuencia de bytes

Cada píxel: tres bytes

rojo, verde y azul, de 00 a FF

La taza entera: 16 × 16 píxeles = 256 píxeles × 3 = 768 bytes

Una foto de 1000 × 1000: 3 000 000 de bytes sin comprimir

Píxel. Cada punto de una imagen. Su color se guarda con tres bytes: cuánto rojo, cuánto verde y cuánto azul tiene, de `00` a `FF`.

Un JPG comprime los píxeles: abierto con el Bloc de notas, es basura.

En la clase de OCR, la foto del ticket se abrió así: 🧾

```
archivo = open(RUTA + FOTO, "rb")
```

`"rb"`: *read binary*, leer los bytes sin tratarlos como texto.

Texto plano y binario, lado a lado

	Texto plano 📄	Binario 📦
Qué guardan los bytes	Caracteres, según una codificación	Una estructura propia del formato
Para leerlo hace falta	Saber la codificación	Un programa que conozca el formato
Con el Bloc de notas	Se lee y se puede editar	Se ve basura; si se guarda, se rompe
En Git (GitHub Desktop)	Muestra qué líneas cambiaron	Solo dice que el archivo cambió
En Python	`open(ruta, "r", encoding="utf-8")`	`open(ruta, "rb")`
Ejemplos	`.txt .csv .py .R .qmd .json`	`.xlsx .docx .pdf .jpg .zip`

Criterio: un archivo es de texto plano si, leídos sus bytes con una codificación, todo lo que aparece son caracteres y el texto tiene sentido.

Actividad: seis archivos con el Bloc de notas

📄 `nota.txt`	📄 `ventas.csv`	🐍 `saludo.py`	📊 `ventas.xlsx`	📝 `informe.docx`	🖼️ `foto.jpg`
Se lee	Se lee	Se lee	Basura	Basura	Basura

Abrir cada archivo con el Bloc de notas: clic derecho → *Abrir con* → *Bloc de notas*. En Mac, con TextEdit.

Un script de Python es un archivo de texto: se puede escribir y corregir con cualquier editor, y Python lo lee y lo ejecuta. 🐍

La extensión

La extensión es parte del nombre, no del contenido

Extensión. Las letras que siguen al último punto del nombre: en `ventas.csv`, la extensión es `csv`. El sistema operativo la usa para elegir con qué programa abrir el archivo. Cambiarla no cambia ningún byte.

Antes	Después	Resultado
`ventas.csv` (70 bytes de texto)	✏️ ➡ | `ventas.txt` (los mismos 70 bytes)	Sigue siendo el mismo CSV ✅ |
`ventas.xlsx` (5360 bytes binarios)	✏️ ➡ | `falso.csv` (los mismos 5360 bytes)	No es un CSV: `UnicodeDecodeError` ❌

Windows oculta las extensiones 🙈. Para verlas: Explorador de archivos → *Vista* → *Mostrar* → *Extensiones de nombre de archivo*. En Mac: Finder → *Ajustes* → *Avanzado* → *Mostrar todas las extensiones*.

El CSV

Un CSV es texto plano con tres convenciones

CSV (*comma-separated values*). Archivo de texto plano que representa una tabla: una fila por línea, la primera con los nombres de las columnas. Para leerlo hay que saber tres cosas:

- **Separador:** el carácter que separa las columnas (coma o punto y coma).
- **Decimal:** el carácter que marca los decimales (punto o coma).
- **Codificación:** la tabla que pasa de caracteres a bytes.

```
fecha,local,monto
03/01/2024,Pocitos,1250.50
04/01/2024,Unión,980.00
```

Coma y punto: la convención de la norma

```
fecha;local;monto
03/01/2024;Pocitos;1250,50
04/01/2024;Unión;980,00
```

Punto y coma y coma decimal: Excel en Uruguay UY

La norma (RFC 4180) fija la coma como separador. Donde la coma es el signo decimal, Excel guarda con punto y coma. El archivo no dice cuál usa: hay que abrirlo y mirar. 👀

La codificación

ASCII: la primera tabla común, sin tildes

Codificación. Regla que asigna a cada carácter un número, y dice cómo se escribe ese número en bytes. Sin una regla compartida, dos computadoras no pueden intercambiar texto.

ASCII (*American Standard Code for Information Interchange*, 1963) usa 7 bits: 128 números, del 0 al 127. Alcanza para el inglés: letras sin tilde, cifras, signos de puntuación y caracteres de control, como el salto de línea. US

Carácter	Número	Byte
A	65	`41`
a	97	`61`
0	48	`30`
,	44	`2C`
salto de línea	10	`0A`
ó, ñ, á	no está	—

El número de cada carácter se escribe en un byte. Como un byte tiene 8 bits, quedan libres los números del 128 al 255.

El mismo byte, tres letras distintas

Cada región usó los números del 128 al 255 para sus propias letras. Así nacieron las **páginas de códigos**: una tabla distinta por idioma.

Byte	Letra	Página de códigos
`F3`	➡ | ó |	**Latin-1** (ISO 8859-1, 1987): Europa occidental. Windows usa una variante, Windows-
`F3`	➡ | y |	**Windows-1251**: cirílico (ruso, ucraniano, búlga
`F3`	➡ | σ |	**ISO 8859-7**: griego. |

Un archivo no dice con qué página de códigos se escribió. Abierto con otra, muestra otras letras: es el origen de los textos con caracteres raros. 🤯

Unicode: un número para cada carácter de cada idioma

Unicode es una sola lista de caracteres para todos los sistemas de escritura. La publica el Consorcio Unicode desde 1991 (versión 1.0) y hoy tiene más de 150 000 caracteres: alfabetos, ideogramas, símbolos matemáticos y emojis. 🌍

Punto de código. El número que Unicode le asigna a un carácter. Se escribe `U+` seguido del número en hexadecimal. Los primeros 128 coinciden con ASCII.

Carácter	Punto de código
A	`U+0041`
ó	`U+00F3`
ñ	`U+00F1`
€	`U+20AC`
中	`U+4E2D`
😀	`U+1F600`

Unicode dice qué número tiene cada carácter, pero no cómo se guarda ese número en bytes. Eso lo resuelve una codificación de Unicode, y la más usada es UTF-8.

UTF-8: entre uno y cuatro bytes por carácter

UTF-8 (*Unicode Transformation Format, 8 bits*) escribe cada punto de código con 1, 2, 3 o 4 bytes. La diseñaron Ken Thompson y Rob Pike en septiembre de 1992, en los laboratorios Bell, para el sistema Plan 9 🍽️. Se normalizó en 2003 (RFC 3629).

Puntos de código	Bytes	Ejemplo	Bytes del ejemplo
`U+0000` a `U+007F` (ASCII)	1	A	`41`
`U+0080` a `U+07FF`	2	ó	`C3 B3`
`U+0800` a `U+FFFF`	3	€	`E2 82 AC`
`U+10000` en adelante	4	😀	`F0 9F 98 80`

Por qué se impuso. Un texto ASCII ya es UTF-8 válido, byte por byte, así que los archivos viejos en inglés no hubo que convertirlos. Y puede escribir cualquier carácter de Unicode. Hoy lo usa el 99 % de los sitios web, y es la codificación por defecto de Python y de pandas.

Cómo la ó se convierte en C3 B3

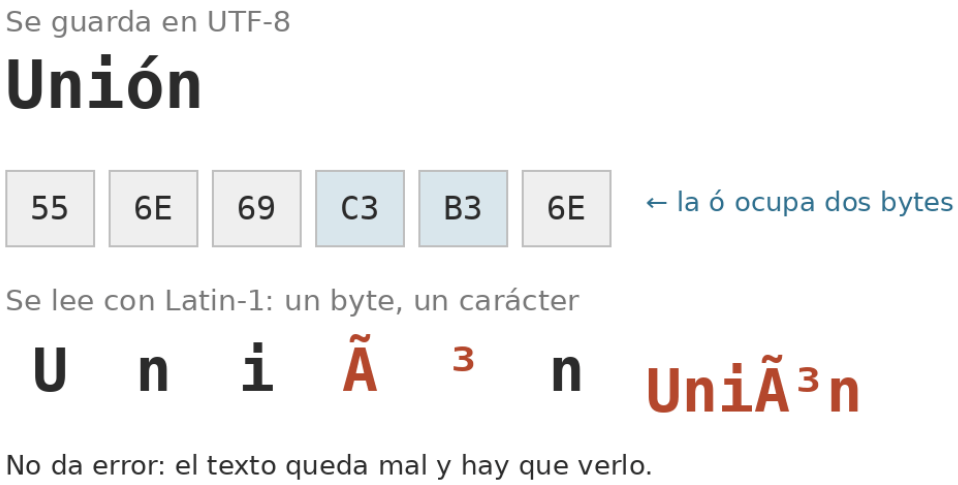
La ó es el punto de código `U+00F3`: el número 243, que en binario es `11110011`. Para números de este tamaño, UTF-8 usa dos bytes con un molde fijo.

	Primer byte	Segundo byte
Molde	110xxxxx	10xxxxxx
243 en 11 bits	00000011	00110011
Resultado	11000011 = **`C3`**	10110011 = **`B3`**

Gris: bits fijos del molde. Azul: los bits del número.

Los bits fijos permiten reconocer los bytes: uno que empieza con `110` abre un carácter de dos bytes, y uno que empieza con `10` lo continúa. Cuando Python encuentra un byte que no encaja en el molde, como el `F3` de Latin-1, da *invalid continuation byte*.

UniÃ³n: bytes de UTF-8 leídos con Latin-1



Al revés, un archivo en Latin-1 leído como UTF-8 sí da error 💥:

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf3 in position 59: invalid
↪  continuation byte
```

La solución es decirle a pandas la codificación: `pd.read_csv(ruta, encoding="latin-1")`.

Regla para la codificación: primero UTF-8 🧭

Paso	Qué se hace	Qué se ve	Qué quiere decir
1	`pd.read_csv(ruta)` (UTF-8, por defecto)	Las tildes y la ñ se ven bien	✅ El archivo está en UTF-8. Listo.
2	lo mismo que en el paso 1	`UnicodeDecodeError` 💥	El archivo **no** está en UTF-8. Leerlo con `encoding="latin-1"`.
3	`pd.read_csv(ruta, encoding="latin-1")`	Aparece `Ã³`, `Ã±`, `Ã¡` 👽	El archivo **está** en UTF-8 y se leyó mal. Sacar el `encoding`.

Por qué primero UTF-8. UTF-8 tiene un molde: si los bytes no encajan, da error y avisa. Latin-1 no avisa nunca, porque los 256 bytes posibles son letras válidas. Si se empieza por Latin-1, el texto puede quedar mal sin que nada lo indique.

Para ver en qué está guardado: el Bloc de notas y VS Code lo muestran abajo a la derecha (*UTF-8* o *ANSI*). **Para pasarlo a UTF-8:** Bloc de notas → *Guardar como* → Codificación *UTF-8*.

Cuatro síntomas al leer un CSV, y su causa

Síntoma	Causa	Se arregla con
🧱 La tabla queda con una sola columna	Separador	`sep=";"`
🔤 La columna monto queda como texto	Decimal	`decimal=","`
👽 Aparece `UniÃ³n` en lugar de `Unión`	Codificación	`encoding="latin-1"`
🔍 `FileNotFoundError`	Ubicación del archivo	corregir la ruta

Los tres primeros se resuelven con argumentos de `pd.read_csv`, sin modificar el archivo. El cuarto no tiene que ver con el contenido: el programa no encontró el archivo.

```
pd.read_csv("ventas_pc.csv", sep=";", decimal=",", encoding="utf-8")
```

Actividad: escribir un CSV a mano y romperlo 🔨

1. Abrir el Bloc de notas y escribir una tabla de cinco filas: fecha, local y monto.
2. Guardarla como `mis_ventas.csv`, en la carpeta de la clase, con codificación UTF-8.
3. Leerla con pandas y comprobar que tiene 5 filas y 3 columnas.
4. Romperla de a una cosa por vez: separador, decimal, codificación, nombre. Anotar el síntoma que aparece.
5. Arreglar cada síntoma con un argumento de `pd.read_csv`, sin volver a tocar el archivo.

Los pasos están en `practico_archivos.py`.

Carpetas y rutas

El sistema de archivos es un árbol 🌳

```
C:/
└── Users/
    └── ana/
        └── curso/
            └── clases/
                ├── clase10/
                │   └── transito_aire_junio2024.csv
                └── clase12/
                    └── clase12_ejercicios.py
```

Carpeta (directorio). Un lugar con nombre que contiene archivos y otras carpetas. 📁

Raíz. La carpeta que contiene a todas las demás: `C:/` en Windows, `/` en Mac y Linux.

Ruta. La lista de carpetas que hay que recorrer para llegar a un archivo, separadas por `/`.

Ruta absoluta y ruta relativa

Ruta absoluta. Empieza en la raíz. Solo sirve en la computadora donde existe esa carpeta. 🏠

```
C:/Users/ana/curso/clases/clase10/transito_aire_junio2024.csv
```

Ruta relativa. Empieza en el directorio de trabajo. Desde `curso/`: 🧭

```
clases/clase10/transito_aire_junio2024.csv
```

La ruta relativa funciona en la computadora de cualquier compañero que tenga la carpeta `curso/`, esté donde esté.

El directorio de trabajo decide dónde se busca

Directorio de trabajo. La carpeta desde la que el programa resuelve las rutas relativas. En Python se ve con `Path.cwd()`; en R, con `getwd()`. Dos puntos (`..`) significan «subir una carpeta».

```
RUTA =  ../clase10/transito_aire_junio2024.csv
```

✓ Con .. sube a clases/ y lo encuentra

En VS Code, cada forma de correr usa otra carpeta

▶ Run Python File

Corre el script en la terminal. El directorio de trabajo es la carpeta abierta en VS Code (*Archivo → Abrir carpeta*).

```
curso/
```

🧩 **Por celdas `#%%`** (Shift + Enter)

Corre en la ventana interactiva. El directorio de trabajo es, por defecto, la carpeta donde está el script.

```
curso/clases/clase12/
```

Regla del curso: se abre siempre la carpeta del repositorio, nunca un archivo suelto. El repositorio trae el archivo `.vscode/settings.json`, que hace que las celdas también usen la carpeta del repositorio. Así las dos formas de correr dan el mismo directorio de trabajo.

Dos líneas al comienzo de cada script ✅

```python
from pathlib import Path

RUTA = "clases/clase10/transito_aire_junio2024.csv"

print(Path.cwd())            # dónde está parado el programa
print(Path(RUTA).exists())   # True si el archivo está ahí
```

- **Si dice `True`** ✅, la ruta está bien y se puede leer el archivo.
- **Si dice `False`** ❌, el problema es la ruta. Se compara lo que imprimió `Path.cwd()` con el árbol de carpetas, y se corrige la ruta o la carpeta abierta.

Con estas dos líneas, un `FileNotFoundError` deja de ser un mensaje largo y se vuelve una pregunta con respuesta: ¿desde dónde corre el programa, y dónde está el archivo?

Fuentes

McKinney, W. (2022). *Python for Data Analysis* (3.ª ed.). O'Reilly. Cap. 6: Data Loading, Storage, and File Formats.

Wickham, H., Çetinkaya-Rundel, M. y Grolemund, G. (2023). *R for Data Science* (2.ª ed.). O'Reilly. Cap. 6: Workflow: scripts and projects.

Shafranovich, Y. (2005). *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180, IETF.

Yergeau, F. (2003). *UTF-8, a transformation format of ISO 10646*. RFC 3629, IETF.

The Unicode Consortium (2024). *The Unicode Standard*, versión 16.0.

Pike, R. (2003). *UTF-8 history*. Correo electrónico reproducido en doc.cat-v.org/bell_labs/utf-8_history.

W3Techs (2026). *Usage statistics of character encodings for websites*.

Python Software Foundation. *pathlib — Object-oriented filesystem paths*. Documentación de Python 3.

pandas development team. *pandas.read_csv*. Documentación de pandas.

5. Generación de datos

Joselina Davyt-Colo

Clase 2: El Proceso Generador de Datos (DGP)

Subtítulo: De la caja negra al dataset: medición, incertidumbre y aumento de datos
Institución: Facultad de Ciencias Empresariales y Economía — Universidad de Montevideo

1. Marco Teórico: El Proceso Generador de Datos (DGP)

1.1 ¿De dónde vienen los datos?

En la práctica profesional, solemos recibir un archivo `.csv` o una tabla SQL y asumimos que esa es la "realidad". Sin embargo, **los datos no existen en la naturaleza; se construyen mediante un acto de medición.**

Cualquier conjunto de datos observable es el resultado final de un **Proceso Generador de Datos** (DGP, por sus siglas en inglés: *Data Generating Process*). El DGP es el sistema o mecanismo subyacente (físico, biológico, social o económico) que produce los eventos que intentamos registrar.

1.2 Una caja negra

El estadístico Leo Breiman (2001) planteó que la naturaleza es una **caja negra** inaccesible en su totalidad. Dentro de la caja negra ocurren interacciones complejas, ruido e incertidumbre. Lo único que nosotros podemos observar son las salidas (Y) generadas a partir de ciertos insumos (X).

$$X \longrightarrow \boxed{\text{Caja Negra (DGP): } f(X) + \epsilon} \longrightarrow Y$$

¿Por qué un modelo "no sabe" cómo se generaron los datos?

> Un algoritmo de Machine Learning o un modelo estadístico de regresión solo procesa **matrices numéricas**. El modelo no tiene sentido común ni contexto ontológico: - No sabe si la variable `peso_g` se midió con una balanza digital o con una estimación a ojo. - No sabe si hubo un sesgo de selección en quién decidió responder la encuesta (Meng, 2018). - No comprende la relación causal subyacente; solo optimiza una función de pérdida sobre la muestra que se le entrega.
>
> Por eso, si el mecanismo de medición o el muestreo está roto, el modelo simplemente aprenderá y magnificará ese error (*"Garbage In, Garbage Out"*).

1.2 La Metáfora de la Caja Negra: Dinámica de Aula

Para vivenciar este concepto, realizaremos un experimento táctil en clase:

1. **El Objeto Oculto:** SE colocará una serie de objetos diversos dentro de una caja cerrada (la "Caja Negra" o DGP).
2. **La Medición Indirecta:** Cada estudiante tocará un objeto sin verlo durante 15 segundos.

Tipología de Variables a registrar:

- **Nominales:** Textura percibida (`suave`, `rugoso`, `poroso`).
- **Ordinales:** Nivel de rigidez (`blando`, `medio`, `rígido`).
- **Numéricas Discretas:** Cantidad de bordes o aristas detectadas ($k \in \mathbb{N}$).
- **Numéricas Continuas:** Estimación del peso en gramos (peso $\in \mathbb{R}^+$).
- **Metadatos:** Identificador del observador, tiempo de inspección y nivel de certeza asignado (1 al 5).

2. Construcción del Dataset en Código

A continuación, traducimos la experiencia táctil en una estructura de datos real en Python y R.

2.1 Crear dataframe en Python

En visual studio code o en python (PowerShell en Windows o Terminal en Mac):

pip install pandas numpy matplotlib

En RStudio:

```r
#install.packages(c("reticulate", "png", "jsonlite", "knitr", "rmarkdown"))

reticulate::py_require(c("pandas", "numpy", "matplotlib"))
```

Diccionarios en python

```python
import pandas as pd
import numpy as np

# Definición del diccionaro crudo recabado en la dinámica
datos_caja = {
  "objeto_id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
  "textura": ["suave", "rugoso", "poroso", "suave", "rugoso", "suave", "poroso",
  ↪  "rugoso", "suave", "poroso"],
  "rigidez": ["blando", "rígido", "medio", "blando", "rígido", "rígido", "blando",
  ↪  "medio", "medio", "blando"],
  "num_aristas": [0, 8, 0, 4, 12, 0, 0, 6, 4, 0],
    "peso_estimado_g": [45.0, 120.5, 15.0, 85.0, 210.0, 35.0, 18.5, 95.0, 70.0, 22.0],
    "certeza_observador": [5, 4, 3, 5, 4, 2, 4, 3, 4, 3]
}

# transformacion a dataframe
df_python = pd.DataFrame(datos_caja)

# Convertir tipos categóricos y ordinales
df_python["textura"] = df_python["textura"].astype("category")
df_python["rigidez"] = pd.Categorical(
    df_python["rigidez"],
    categories=["blando", "medio", "rígido"],
```

```
    ordered=True
)

print("Estructura del DataFrame en Python:")
```

```
Estructura del DataFrame en Python:
```

```
print(df_python.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   objeto_id           10 non-null     int64
 1   textura             10 non-null     category
 2   rigidez             10 non-null     category
 3   num_aristas         10 non-null     int64
 4   peso_estimado_g     10 non-null     float64
 5   certeza_observador  10 non-null     int64
dtypes: category(2), float64(1), int64(3)
memory usage: 628.0 bytes
None
```

```
print(df_python.head())
```

```
   objeto_id textura rigidez  num_aristas  peso_estimado_g  certeza_observador
0          1   suave  blando            0             45.0                   5
1          2  rugoso  rígido            8            120.5                   4
2          3  poroso   medio            0             15.0                   3
3          4   suave  blando            4             85.0                   5
4          5  rugoso  rígido           12            210.0                   4
```

Datos simulados vs datos sintéticos

Datos simulados

Los datos simulados se generan mediante un proceso que implica imitar la operación, comportamiento o evolución de un sistema o proceso del mundo real a lo largo del tiempo, utilizando un modelo simplificado (matemático, físico o computacional).

Se ejecuta un modelo para imitar la dinámica de un sistema a lo largo del tiempo o bajo distintas condiciones (por ejemplo, simular una partida de ajedrez, el tráfico aéreo o la trayectoria de un huracán).

Define reglas, ecuaciones o mecánicas de comportamiento

3 Datos sintéticos

Crear datos sintéticos implica construir un ‘modelo’ de lo que creemos que hay dentro de la caja. Si nuestro modelo es simple, los datos sintéticos serán engañosos. Pero si lo hacemos bien, nos permite compartir datos para investigar sin mostrar el objeto real ni comprometer la privacidad de nadie.

Toma una muestra real de datos y usa algoritmos (Bootstrap, distribuciones ajustadas, redes GANs) para generar nuevas filas que se parezcan estadísticamente a la tabla original.

Métodos para crear datos sintéticos

Generación paramétrica

Generación Paramétrica (Distribución Normal): Para variables cuantitativas continuas (peso_estimado_g). Calculas la media (μ) y desviación estándar (σ) de los 10 alumnos e "inventas" 500 pesos nuevos siguiendo esa campana de Gauss.

Re-muestreo

Re-muestreo Bootstrap:

El Bootstrap (Efron, 1979) asume que la muestra observada es la mejor aproximación disponible de la población desconocida.

```
graph LR
    subgraph Original ["Muestra (N=10)"]
        O1[Objeto 1]
        O2[Objeto 2]
        O3[Objeto 3]
    end

    subgraph Proceso ["Muestreo con Reemplazo"]
        P((Extraer + Devolver))
    end

    subgraph Bootstrap ["Bootstrap (N=100)"]
        B1[Objeto 1]
        B2[Objeto 1]
        B3[Objeto 3]
        B4[Objeto 2]
        B5[...]
    end

    Original --> Proceso
    Proceso --> Bootstrap
```

Datos sintéticos en python

```python
# ------------------------------------------------------------------------
# METODO 1: BOOTSTRAP (Muestreo con reemplazo para expandir a N = 100)
# ------------------------------------------------------------------------
np.random.seed(42)  # Semilla para reproducibilidad
df_bootstrap = df_python.sample(n=100, replace=True).reset_index(drop=True)

# ------------------------------------------------------------------------
# METODO 2: SINTETICO PARAMETRICO (Normal) para Peso (N = 500)
# ------------------------------------------------------------------------
# A. Calculamos los parámetros del modelo probabilístico (DGP estimado)
mu = df_python['peso_estimado_g'].mean()  # ~71.6 g
sigma = df_python['peso_estimado_g'].std()  # ~58.3 g

# B. Generamos 500 observaciones sintéticas
pesos_sinteticos = np.random.normal(loc=mu, scale=sigma, size=500)

# C. Aplicamos restricciones del mundo real (un peso no puede ser <= 0)
```

```
pesos_sinteticos = np.clip(pesos_sinteticos, a_min=1.0, a_max=None)

df_sintetico_pesos = pd.DataFrame(
    {'peso_estimado_g': pesos_sinteticos, 'tipo': 'Sintético'}
)
```

Limitaciones:

Reality gap

Representación limitada del mundo real: Los conjuntos de datos sintéticos pueden no capturar la complejidad y variabilidad completas de los datos reales.

Lectura recomendada: técnicas de aleatorización de dominios.

Tremblay, J., Prakash, A., Acuna, D., Brophy, M., Jampani, V., Anil, C., ... & Birchfield, S. (2018, June). Training deep networks with synthetic data: Bridging the reality gap by domain randomization. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW) (pp. 1082-10828). IEEE.

Kim, M., Liu, F., Jain, A., & Liu, X. (2023, June). Dcface: Synthetic face generation with dual condition diffusion model. In 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 12715-12725). IEEE.

Otras limitaciones

El peligro de los datos sintéticos: Generar datos sintéticos asumiendo una distribución Normal cuando el verdadero DGP es multimodal (varios objetos distintos) crea datos “fantasmas” que nunca existirían en la caja real.

Posible sesgo: El proceso de generación puede introducir sesgos si no refleja con precisión las características reales de la población.

Falta de contexto: Los conjuntos de datos sintéticos suelen carecer de la información contextual presente en los datos reales, lo que afecta su utilidad para el análisis. Generalización limitada: Los modelos entrenados con datos sintéticos pueden no tener un buen rendimiento con datos reales debido a diferencias en la distribución o los patrones subyacentes. Retos de validación: Puede resultar difícil validar modelos entrenados con datos sintéticos sin oportunidades de prueba en el mundo real.

Entonces ¿que es lo que hace crear un dataset de entrenamiento bueno?

- Alta Variación Inter-Clase (Cobertura del Dominio):
- Alta Variación Intra-Clase (Invariancia al Entorno):
- Mínimo Ruido en las Etiquetas (Fidelidad del Target):

Crear datos sintéticos o entrenar modelos complejos no compensa un dataset base con baja diversidad o etiquetas ruidosas. El límite de la capacidad de aprendizaje de un modelo no lo define la complejidad de su algoritmo, sino la riqueza estadística y la fidelidad del Proceso Generador de Datos (DGP) que se utilizó para entrenarlo.

6. Diseño de encuestas

Del cuestionario a la tabla de datos

Las encuestas, que son

Las encuestas son herramientas descriptivas que emplean preguntas para recopilar información estadística sobre algún aspecto de una población.

¿Conviene hacer una encuesta?

Tres preguntas antes de empezar (Bulut):

1. **¿Hay tiempo?** Diseñar, probar y aplicar una encuesta de calidad lleva semanas. Si el plazo es menor que esa estimación, no es el instrumento adecuado.
2. **¿Hay recursos?** Tiempo de las personas, plataforma, incentivos, procesamiento. Sin eso, la encuesta se hace mal o no se termina.
3. **¿Ya existen los datos?** Antes de preguntar, buscar si otra fuente ya responde: registros administrativos, encuestas anteriores, datos transaccionales de la diapositiva anterior.

Una encuesta se justifica cuando lo que se necesita no está en ninguna otra parte: percepciones, intenciones, razones, o hechos que ningún sistema registra.

¿Es la encuesta la herramienta adecuada?

Las encuestas son más adecuadas para recopilar datos sobre fenómenos humanos no observables. Estos incluyen actitudes, creencias y opiniones, pero también pueden incluir informes de comportamientos y acciones que de otro modo serían inmensurables (o muy difíciles de medir).

El uso de una encuesta debe alinearse lógicamente con las preguntas subyacentes que se formulan y las variables o resultados que se estudian.

Además, los investigadores deben describir claramente en sus artículos la justificación de esas medidas y el uso (o usos) posterior de una encuesta. Así como seguir los protocolos éticos y las leyes o reglamentaciones de uso y/o extracción de datos.

Objetivo del bloque

Al terminar, cada pregunta que se escriba en un cuestionario debería poder responder tres cosas:

1. Qué columna genera en la tabla de datos.
2. De qué tipo es esa columna.
3. Cómo se codifican las respuestas y los faltantes.

No se discute el objeto de estudio ni el marco teórico. Solo el armado.

La regla básica

Diseñar la tabla antes que el formulario.

Filas: una por respondente.

Columnas: una por variable. Si no se puede dibujar la columna que produce una pregunta, la pregunta no va.

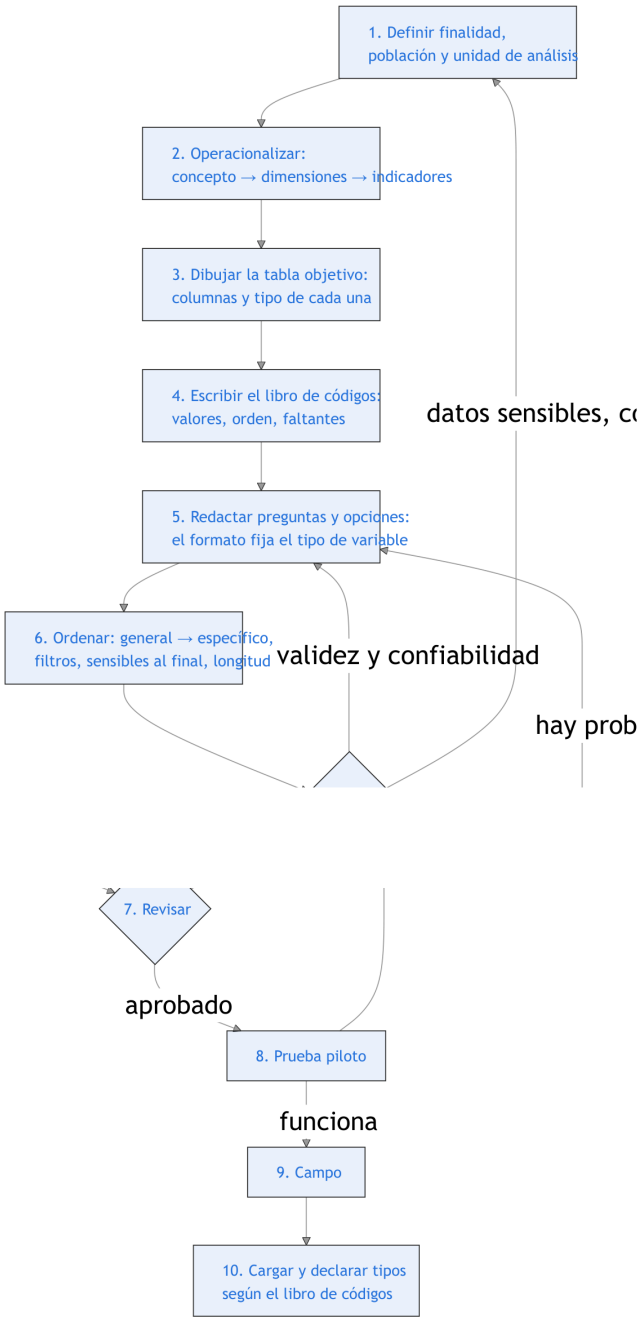
Ponerse en el lugar del respondente

Dos objetivos del proceso de redacción (Bulut):

1. Leer cada pregunta como la leería alguien de la población, no como la lee quien la escribió. Lo que es obvio para el diseñador rara vez lo es para el respondente.
2. Escribir preguntas claras y breves que **distingan** a los respondentes. Una pregunta a la que todos contestan lo mismo no aporta información, aunque esté bien escrita.

Legibilidad: revisar ortografía y gramática, y medir la dificultad de lectura. Para textos en español existen adaptaciones del índice de Flesch (Fernández-Huerta, INFLESZ); un cuestionario para población general debería leerse sin esfuerzo a nivel de enseñanza media.

Pasos del diseño



Elementos

Un cuestionario completo tiene seis partes, no solo las preguntas (Bulut):

1. Título.
2. Texto introductorio.
3. Consentimiento o carta de información.
4. Instrucciones para completarlo.
5. Las preguntas.
6. Cierre y agradecimiento.

Ejemplo de trabajo

Encuesta de gasto mensual a estudiantes universitarios. Once preguntas en tres bloques: **A. Perfil** 1. ¿En qué año de la carrera está? 2. ¿Vive en la casa familiar? 3. ¿Cuánto paga de alquiler por mes? 4. ¿Trabaja actualmente? **B. Gastos en los últimos 30 días** 5. ¿Cuánto gastó en comida fuera de casa? 6. ¿Cuánto gastó en transporte? 7. ¿En cuáles de estos rubros tuvo algún gasto? (marque todas) 8. ¿Lleva registro de sus gastos? **C. Ingresos y percepción** 9. ¿Cuál es su principal fuente de ingresos? 10. ¿Cómo describiría su situación económica? 11. ¿Qué gasto le resulta más difícil de controlar? Antes de seguir: ¿cuántas columnas tiene la tabla resultante?

Fundamentos

Fuente de datos

De dónde salen los datos sobre personas

Una encuesta es una de varias formas de generar datos. Se distinguen por quién decide qué se registra.

Fuente	Qué se registra	Quién decide qué se registra	Qué se obtiene	Qué se pierde
Preferencias declaradas (encuesta)	Lo que la persona dice	El diseñador del cuestionario	Percepciones, intenciones, razones; cualquier cosa que se pueda preguntar	Precisión: memoria, deseabilidad social, no respuesta
Preferencias reveladas (transacciones, registros administrativos)	Lo que la persona hace, inferido de sus elecciones	El sistema que opera la transacción	Comportamiento efectivo sin depender de la memoria ni de la honestidad	El “por qué”; todo lo que el sistema no registra

Huella digital (logs, clics, ubicación, tiempo en pantalla) | Comportamiento como subproducto de usar un sistema | Nadie, en el sentido de medición: es un residuo técnico | Volumen y granularidad | Definición: cada campo hay que reinterpretarlo como si fuera una pregunta |

En el ejemplo, “cuánto gastó en comida fuera de casa” se puede obtener de tres maneras: preguntándolo (P5), leyendo los cobros con tarjeta de estudiante en la cafetería, o leyendo los registros de una app de

`gasto_comida`

La idea de preferencia revelada viene de Samuelson (1938): la elección observada informa sobre la preferencia mejor que la declaración. Lo que se pierde es la posibilidad de preguntar por lo que no ocurrió.

Datos de comportamiento sin preguntas

Cuando el dato no sale de un cuestionario, el libro de códigos igual hace falta. La diferencia es que hay que reconstruirlo:

Qué mide cada campo. Un registro `monto` en una tabla de cobros: ¿incluye propina? ¿descuentos? ¿en

```
monto
```

qué moneda?

Qué unidad de análisis tiene. Un log de la app registra sesiones, no personas. Pasar de sesión a persona es una decisión.

Qué faltantes hay y por qué. Quien paga en efectivo no aparece en los cobros con tarjeta. Es un faltante estructural que nadie diseñó.

Quién queda afuera. Solo se observa a quien usa el sistema. La cobertura la define la tecnología, no el investigador. Los equipos que trabajan con datos transaccionales y con registros de uso de IA están en esta situación: tienen la tabla, y tienen que escribir el cuestionario que la habría producido.

Datos personales: marco legal en Uruguay

Ley 18.331 (2008), Protección de Datos Personales y Acción de Habeas Data. Reglamentada por el Decreto 414/009; modificada por la Ley 19.670 (2018), el Decreto 64/020 y la Ley 19.924 (2020). Órgano de control: URCDP.

Alcanza a toda base de datos con información de personas identificadas o identificables, cualquiera sea su soporte y su origen: lo que se preguntó y lo que se registró sin preguntar. Principios con consecuencia directa en el diseño:

Principio	Qué exige	En el cuestionario
Finalidad	Declarar para qué se recogen los datos y usarlos solo para eso	El propósito se escribe antes de la primera pregunta y no se cambia después
Consentimiento previo informado	Manifestación libre, expresa e inequívoca; no se infiere del silencio	Se registra que fue otorgado
Pertinencia	No recoger más de lo necesario para la finalidad	El criterio de longitud tiene base legal

Principio	Qué exige	En el cuestionario
Seguridad y reserva	Proteger los datos y no divulgarlos	Separar identificadores de respuestas
Derechos del titular	Acceso, rectificación, supresión	Hay que poder encontrar a una persona en la base y borrarla

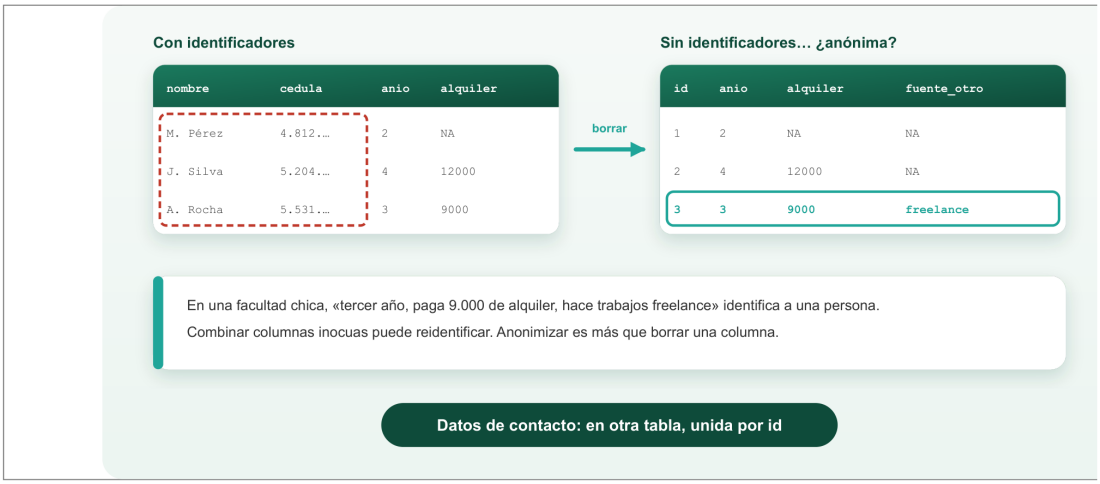
Datos sensibles e identificación

Datos sensibles (art. 18): origen racial o étnico, preferencias políticas, convicciones religiosas o morales, afiliación sindical, salud, vida sexual. Solo se recogen con consentimiento expreso y por escrito, y con justificación. Si una pregunta cae en esa lista, la pregunta que corresponde es si hace falta. **Identificación**:

`id` es un número correlativo, no la cédula ni el correo.

Si hay que volver a contactar a la persona, los datos de contacto van en una tabla separada, vinculada por `id` , con acceso restringido.

Una tabla sin nombres puede seguir identificando: año de carrera, alquiler exacto y fuente de ingresos “Otro: trabajos freelance” pueden bastar para reconocer a alguien en una facultad chica. Anonimizar es más que borrar una columna. La aplicación a un caso concreto requiere asesoramiento jurídico; lo que corresponde al diseñador de la encuesta es saber que estas obligaciones existen antes de salir a campo.



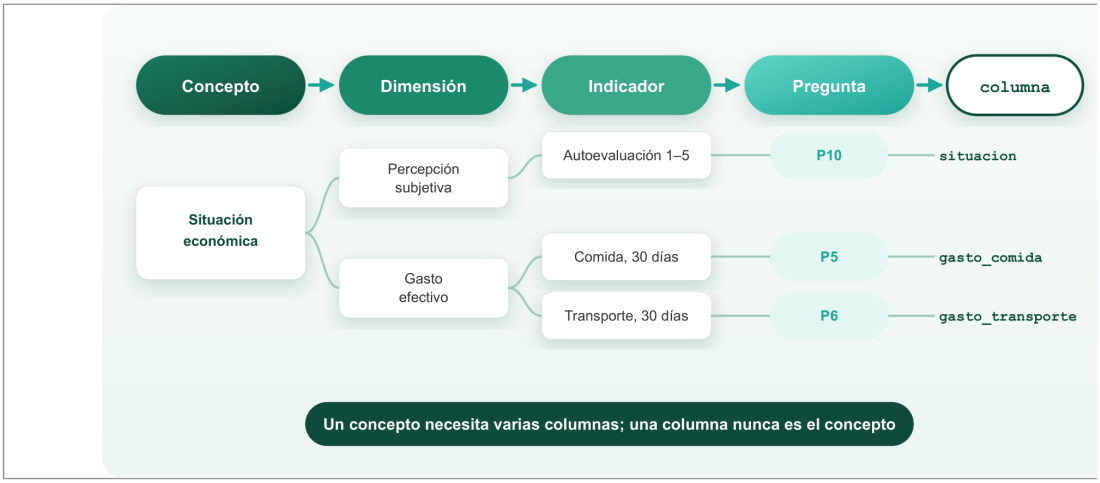
Una pregunta es una medición

Operacionalización: el camino desde lo que se quiere saber hasta la columna. concepto → dimensión → indicador → pregunta → variable En el ejemplo:

Concepto	Dimensión	Indicador	Pregunta	Variable
Situación económica	Percepción subjetiva	Autoevaluación 1–5	P10	`situacion`
Situación económica	Gasto efectivo	Monto en comida, 30 días	P5	`gasto_comida`
Situación económica	Gasto efectivo	Monto en transporte, 30	P6	`gasto_transporte`

días

Un concepto suele necesitar varias columnas. Una columna nunca es el concepto: es un indicador de una de sus dimensiones. “Diseñar la tabla antes que el formulario” es esto: decidir qué indicadores se necesitan antes de escribir cómo se preguntan.



Niveles de medición

Stevens (1946) distingue cuatro niveles según qué operaciones tienen sentido sobre los valores:

Nivel	Qué se puede hacer	Ejemplo	En la tabla
Nominal	Distinguir categorías	fuente de ingresos	`fuente_ingreso`
Ordinal	Ordenarlas	año de carrera, escala 1–5	`anio` , `situacion`
Intervalo	Medir distancias; el cero es convencional	temperatura en °C	(no hay en el ejemplo)
Razón	Distancias y cocientes; el cero es real	pesos gastados	`gasto_comida`

Cada nivel permite lo del anterior y algo más.

El diseño de la pregunta fija el nivel. Después no se puede subir: de tramos no se recupera el monto.

Sí se puede bajar: de `gasto_comida` se construyen tramos cuando haga falta.

`gasto_comida`

Promediar una escala 1–5 es tratar una ordinal como de intervalo. Se hace con frecuencia; lo que no se hace es sin declararlo.

Validez y confiabilidad

Dos propiedades de una pregunta como instrumento de medición:

Validez: mide lo que dice medir. ¿Está conforme con lo que gasta en comida y en transporte? no mide ninguna de las dos cosas por separado.

Confiabilidad: mide lo mismo cada vez que se aplica. ¿Cuánto gasta en transporte? sin período de referencia recibe respuestas distintas de la misma persona según el día. La sección de redacción que sigue es una lista de formas conocidas de perder validez o confiabilidad.

Fuentes de error

Marco de error total de encuesta (Groves et al. 2009). Cuatro fuentes, en el orden en que aparecen:

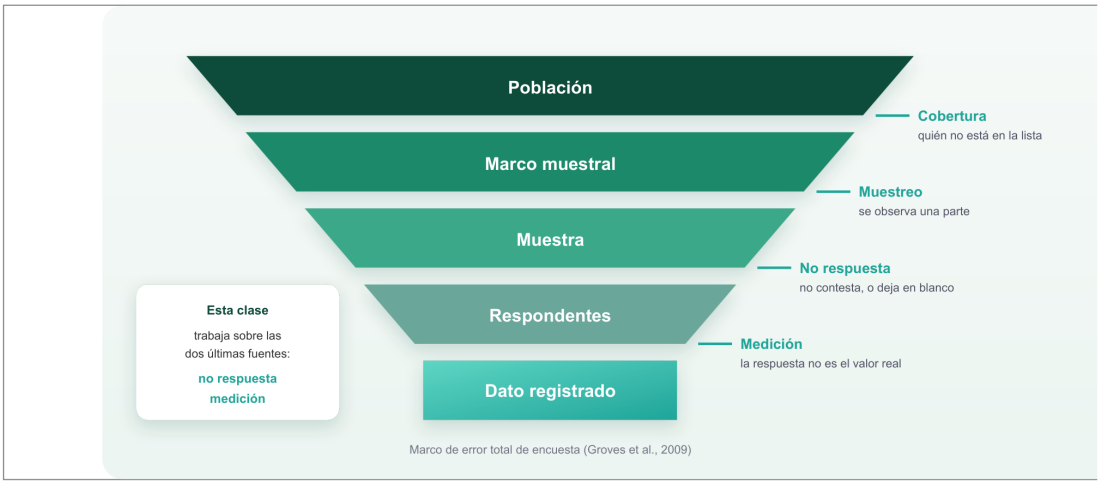
Fuente Qué es Dónde se controla

Cobertura La lista de la que se saca la Marco muestral muestra no contiene a toda la población

Muestreo	Se observa una parte, no el todo	Diseño muestral
No respuesta	Personas seleccionadas que no contestan, o ítems que quedan en blanco	Longitud, orden, redacción

Medición La respuesta registrada no Redacción, opciones, escalas coincide con el valor verdadero

Este bloque trabaja sobre las dos últimas. Cobertura y muestreo quedan para otra clase.



Unidades

Población: a quién se quiere describir.

Unidad de análisis: sobre qué se hacen afirmaciones (una fila de la tabla).

Unidad de respuesta: quién contesta el cuestionario. En el ejemplo las tres coinciden: estudiantes de la facultad, un estudiante por fila, contesta el propio estudiante. No siempre es así. En una encuesta de gasto de hogares la unidad de análisis es el hogar y responde un informante; en una encuesta a empresas responde una persona por la organización. Cuando no coinciden, el cuestionario tiene que decir explícitamente por quién se está respondiendo.

Tipos de pregunta y variable que generan

Opción única, sin orden

> P9. ¿Cuál es su principal fuente de ingresos? ☐ Familia ☐ Trabajo ☐ Beca ☐ Ahorros ☐ Otro

Una columna.

Variable **categórica nominal**: las categorías no se ordenan.

Se guarda como texto o como código numérico con etiqueta.

Opción única, con orden

> P1. ¿En qué año de la carrera está? ☐ 1.º ☐ 2.º ☐ 3.º ☐ 4.º ☐ 5.º o más

Una columna.

Variable **ordinal**: hay orden, pero la distancia entre categorías no está definida.

El orden debe quedar registrado explícitamente (no se infiere del texto).

Sí / No

> P2. ¿Vive en la casa familiar? ☐ Sí ☐ No

Una columna.

Variable **binaria**. Es una nominal de dos categorías; conviene almacenarla como lógica (`TRUE` / `FALSE`) o

`TRUE` / `FALSE`

como 0/1.

Escala de valoración

> P10. ¿Cómo describiría su situación económica en los últimos 30 días? 1 Muy ajustada · 2 · 3 · 4 · 5 Muy holgada

Una columna.

Variable **ordinal**. Que los valores sean números no la vuelve cuantitativa.

Tratarla como numérica (promediar, por ejemplo) es una decisión analítica que se toma después y se declara. No es una propiedad del dato.

Numérica directa

> P5. ¿Cuánto gastó en comida fuera de casa en los últimos 30 días? $ ____

Una columna.

Variable **cuantitativa** (continua; sería discreta para una cantidad de veces).

Es la que más información conserva. El costo: preguntas difíciles de recordar o sensibles bajan la tasa de respuesta.

Numérica por tramos

> P6. ¿Cuánto gastó en transporte en los últimos 30 días? ☐ Nada ☐ Hasta $1.000 ☐ $1.001 a $3.000 ☐ $3.001 a $6.000 ☐ Más de $6.000

Una columna, **ordinal**.

Se pierde la posibilidad de calcular promedios o de recortar tramos distintos después.

Se gana tasa de respuesta. Es un intercambio que se decide antes de salir a campo, porque no se puede deshacer. P5 y P6 preguntan lo mismo (un monto) con dos diseños distintos. Ninguno es correcto en general.

Selección múltiple

> P7. ¿En cuáles de estos rubros tuvo algún gasto en los últimos 30 días? ☐ Materiales de estudio ☐ Salidas ☐ Ropa ☐ Suscripciones digitales ☐ Salud

No genera una variable. Genera una columna binaria por opción:

id	rubro_materiales	rubro_salidas	rubro_ropa	rubro_suscripciones	rubro_salud
1	1	1	0	1	0
2	0	1	1	1	0

Guardar la respuesta como un solo texto (`"Salidas; Ropa; Suscripciones"`) es

en encuestas armadas sin pensar la tabla.

Pregunta abierta

> P11. ¿Qué gasto le resulta más difícil de controlar? _____

Una columna de **texto**.

No se analiza directamente: requiere codificación posterior (leer, agrupar, asignar categorías). Es la pregunta más cara de procesar.

Se justifica cuando no se conocen las categorías de antemano o cuando se quiere el lenguaje del respondente.

Resumen

Formato de pregunta	Columnas	Tipo de variable	En el ejemplo
Opción única sin orden	1	Nominal	P9
Opción única con orden	1	Ordinal	P1, P4, P8
Sí / No	1	Binaria	P2
Escala 1–5	1	Ordinal	P10
Número	1	Cuantitativa	P3, P5
Tramos	1	Ordinal	P6
Selección múltiple con k opciones	k	Binarias	P7
Abierta	1	Texto (a codificar)	P11

Once preguntas, diecisiete columnas.

Redacción de preguntas

Una idea por pregunta

> ¿Cree que la comida de la facultad es cara y de mala calidad?

Dos preguntas en una. Quien la considera cara pero de buena calidad no tiene respuesta. Cada pregunta mide una sola cosa. Si aparece una “y”, revisar.

Lenguaje neutro

> ¿No cree que debería ahorrar más?

Induce la respuesta y contiene una negación (responder “no” es ambiguo). Formulación neutra: ¿Lleva registro de sus gastos? o ¿Cómo describiría su situación económica?

Período de referencia explícito

> ¿Cuánto gasta en transporte?

¿Por semana, por mes, en un mes típico, en el último mes? Cada persona responderá algo distinto. ¿Cuánto gastó en transporte en los últimos 30 días? “Los últimos 30 días”

y “un mes típico” no son equivalentes: uno pide recordar, el otro pide estimar. Se elige uno y se sostiene en todo el cuestionario.

Categorías exhaustivas y excluyentes

> Alquiler mensual: ☐ Hasta $8.000 ☐ $8.000 a $12.000 ☐ $12.000 a $20.000

No excluyentes: 8.000 y 12.000 caen en dos tramos.

No exhaustivas: más de 20.000 no tiene opción. Toda respuesta posible debe tener exactamente una casilla.

Selección múltiple disfrazada

> ¿De dónde recibe ingresos? ☐ Familia ☐ Trabajo ☐ Ambos ☐ Otro

“Ambos” delata que la pregunta admite más de una respuesta. Hay dos salidas:

Selección múltiple: una columna binaria por fuente.

Opción única con la palabra principal: ¿Cuál es su principal fuente de ingresos? (es lo que hace P9).

Opciones de respuesta

Escalas

Cantidad de puntos: 4 obliga a tomar posición; 5 permite un punto neutro. Ninguna es correcta en general; se decide y se mantiene igual en todo el cuestionario.

Etiquetas: etiquetar solo los extremos (1 = Muy ajustada, 5 = Muy holgada) o todos los puntos. Etiquetar todos reduce interpretaciones distintas.

Dirección: mantener siempre el mismo sentido (mayor valor = mejor situación) para no confundir al respondente ni al analista.

“Otro (especifique)” y “No sabe / No contesta”

Otro

`fuente_ingreso`

Otro (especifique) genera dos columnas: la categoría `Otro` en `fuente_ingreso` y un campo de texto `fuente_otro` . Si muchas respuestas caen en “Otro”, faltó una categoría.

`fuente_otro`

No sabe / No contesta es una decisión de diseño. Si se ofrece, hay que definir cómo se codifica y distinguirla de la no respuesta (dejar en blanco).

id	vive_fam	alquiler	gasto_comida	fuente_otro
1	TRUE	NA	4500	NA
2	FALSE	12000	8200	NA
3	TRUE	NA	NA	NA
4	FALSE	9000	6000	freelance

Orden, filtros y longitud

Orden

De lo general a lo específico.

Preguntas sensibles (montos de ingreso, situación económica) al final: si la persona abandona, ya respondió el resto.

Las primeras preguntas deben ser fáciles y claramente relacionadas con el tema anunciado. En el ejemplo: el perfil va primero, los montos de gasto en el medio, ingresos y percepción al final.

Preguntas filtro

> P2. ¿Vive en la casa familiar? ☐ Sí ☐ No → si Sí, pasar a P4 P3. ¿Cuánto paga de alquiler por mes? $ ____

Quien responde “Sí” en P2 no tiene P3. Esa celda queda vacía **por diseño**. Son **faltantes estructurales**. No son lo mismo que la **no respuesta** (la persona debía contestar y no lo hizo). El libro de códigos debe distinguirlos.

Longitud

Cada pregunta tiene un costo: tiempo del respondente y tasa de abandono. Criterio para conservar una pregunta: la columna que genera se usa en algún análisis previsto.

Si no se sabe para qué se va a usar, se elimina.

Libro de códigos

Qué es

El diccionario de variables se escribe **antes** de salir a campo, no después. Es el mismo diccionario del Práctico A y lo que SPSS guarda como metadatos.

Variable	Etiqueta	Tipo	Valores	Faltante
`id`	Identificador	entero	1, 2, …	no admite
`anio`	Año de la carrera	ordinal	1 < 2 < 3 < 4 < 5+	NA
`vive_familia`	Vive en la casa familiar	binaria	TRUE, FALSE	NA
`alquiler`	Alquiler mensual (pesos)	cuantitativa	≥ 0	NA si `vive_familia` = TRUE
`trabajo`	Situación laboral	ordinal	No < Menos de 20 h < 20 h o más	NA

Variable	Etiqueta	Tipo	Valores	Faltante
`gasto_comida`	Comida fuera de casa, 30 días (pesos)	cuantitativa	≥ 0	NA
`gasto_transporte`	Transporte, 30 días	ordinal	Nada < Hasta 1.000 < 1.001–3.000 < 3.001–6.000 < Más de 6.000	NA
`rubro_*` (5)	Tuvo gasto en el rubro	binaria	0, 1	NA
`registro`	Lleva registro de gastos	ordinal	Nunca < A veces < Siempre	NA
`fuente_ingreso`	Principal fuente de ingresos	nominal	Familia, Trabajo, Beca, Ahorros, Otro	NA
`fuente_otro`	Otra fuente (texto)	texto	libre	vacío si `fuente_ingreso` ≠ Otro

Variable	Etiqueta	Tipo	Valores	Faltante
`situacion`	Situación económica, 30 días	ordinal	1 < 2 < 3 < 4 < 5	NA
`gasto_dificil`	Gasto difícil de controlar (abierta)	texto	libre	vacío

Convenciones de nombres

`gasto_comida`

`Gasto en comida`

Prefijo común para las binarias de una misma selección múltiple: `rubro_materiales`, `rubro_salidas`,

`rubro_materiales`

`rubro_salidas`

`rubro_ropa`

El nombre describe el contenido, no la posición: `alquiler`, no `p3`.

`alquiler`

Cómo queda la tabla

Lectura y tipos

R Python

```r
library(tidyverse)

encuesta <- read_csv("encuesta_gasto.csv")
glimpse(encuesta)
```

```
Rows: 4
Columns: 17
$ id                  <dbl> 1, 2, 3, 4
$ anio                <dbl> 2, 4, 1, 3
$ vive_familia        <chr> "Si", "No", "Si", "No"
$ alquiler            <dbl> NA, 12000, NA, 9000
$ trabajo             <chr> "No", "Si, 20 h o más", "No", "Si, menos de 20 h"
$ gasto_comida        <dbl> 4500, 8200, NA, 6000
$ gasto_transporte    <chr> "1.001 a 3.000", "Más de 6.000", "Hasta 1.000", "Nada"
$ rubro_materiales    <dbl> 1, 0, 1, 0
$ rubro_salidas       <dbl> 1, 1, 0, 1
$ rubro_ropa          <dbl> 0, 1, 0, 0
$ rubro_suscripciones <dbl> 1, 1, 0, 1
$ rubro_salud         <dbl> 0, 0, 0, 1
$ registro            <chr> "A veces", "Siempre", "Nunca", "A veces"
$ fuente_ingreso      <chr> "Familia", "Trabajo", "Familia", "Otro"
$ fuente_otro         <chr> NA, NA, NA, "trabajos freelance"
$ situacion           <dbl> 3, 2, 4, 3
$ gasto_dificil       <chr> "salidas", "delivery", NA, "suscripciones"
```

```python
import pandas as pd

encuesta = pd.read_csv("encuesta_gasto.csv")
encuesta.dtypes
```

```
id                    int64
anio                  int64
vive_familia         object
alquiler            float64
```

```
trabajo               object
gasto_comida         float64
gasto_transporte      object
rubro_materiales       int64
rubro_salidas          int64
rubro_ropa             int64
rubro_suscripciones    int64
rubro_salud            int64
registro              object
fuente_ingreso        object
fuente_otro           object
situacion              int64
gasto_dificil         object
dtype: object
```

Tres faltantes de naturaleza distinta: `alquiler` en los respondentes 1 y 3 (filtro), `gasto_comida` en el 3 (no respuesta), `fuente_otro` en 1, 2 y 3 (no aplica). La tabla no los distingue; el libro de códigos sí.

Declarar los tipos

R Python

```r
encuesta <- encuesta |>
  mutate(
    anio = factor(anio, levels = 1:5, ordered = TRUE),
    vive_familia = vive_familia == "Sí",
    trabajo = factor(trabajo,
      levels = c("No", "Sí, menos de 20 h", "Sí, 20 h o más"), ordered = TRUE),
    gasto_transporte = factor(gasto_transporte,
      levels = c("Nada", "Hasta 1.000", "1.001 a 3.000",
                 "3.001 a 6.000", "Más de 6.000"), ordered = TRUE),
    registro = factor(registro,
      levels = c("Nunca", "A veces", "Siempre"), ordered = TRUE),
    fuente_ingreso = factor(fuente_ingreso),
    situacion = factor(situacion, levels = 1:5, ordered = TRUE)
  )
```

```
$ anio             <ord> 2, 4, 1, 3
$ vive_familia     <lgl> TRUE, FALSE, TRUE, FALSE
$ trabajo          <ord> No, Sí, 20 h o más, No, Sí, menos de 20 h
$ gasto_transporte <ord> 1.001 a 3.000, Más de 6.000, Hasta 1.000, Nada
$ registro         <ord> A veces, Siempre, Nunca, A veces
$ fuente_ingreso   <fct> Familia, Trabajo, Familia, Otro
$ situacion        <ord> 3, 2, 4, 3
```

```python
encuesta["anio"] = pd.Categorical(encuesta["anio"],
    categories=[1, 2, 3, 4, 5], ordered=True)
encuesta["vive_familia"] = encuesta["vive_familia"] == "Sí"
encuesta["trabajo"] = pd.Categorical(encuesta["trabajo"],
    categories=["No", "Sí, menos de 20 h", "Sí, 20 h o más"], ordered=True)
encuesta["gasto_transporte"] = pd.Categorical(encuesta["gasto_transporte"],
    categories=["Nada", "Hasta 1.000", "1.001 a 3.000",
                "3.001 a 6.000", "Más de 6.000"], ordered=True)
encuesta["registro"] = pd.Categorical(encuesta["registro"],
    categories=["Nunca", "A veces", "Siempre"], ordered=True)
encuesta["fuente_ingreso"] = encuesta["fuente_ingreso"].astype("category")
```

```python
encuesta["situacion"] = pd.Categorical(encuesta["situacion"],
    categories=[1, 2, 3, 4, 5], ordered=True)
```

```
anio               category
vive_familia           bool
trabajo            category
gasto_transporte   category
registro           category
fuente_ingreso     category
situacion          category
```

El orden de las categorías se declara en el código porque no está en el archivo. Esa declaración sale del libro de códigos.

Ejercicio

De la tabla al cuestionario

Se quiere obtener esta tabla a partir de una encuesta a clientes de la cafetería de la facultad. Escribir el cuestionario que la produce: texto de cada pregunta, opciones de respuesta, orden y filtros.

Variable	Tipo	Valores	Faltante
`id`	entero	1, 2, …	no admite
`frecuencia`	ordinal	Nunca < 1 a 3 veces < 4 a 8 veces < Más de 8 veces	NA
`prod_cafe` , `prod_comida` , `prod_frias` , `prod_otros`	binaria (×4)	0, 1	NA si `frecuencia` = Nunca
`atencion`	ordinal	1 < 2 < 3 < 4 < 5	NA si `frecuencia` = Nunca
`pago_efectivo` , `pago_tarjeta` , `pago_transferencia`	binaria (×3)	0, 1	NA si `frecuencia` = Nunca

Variable	Tipo	Valores	Faltante
`estudiante`	binaria	TRUE, FALSE	NA
`edad`	cuantitativa	años	NA
`mejora`	texto	libre	vacío

Qué se verifica

Toda pregunta tiene período de referencia.

`frecuencia`

`prod_*` y `pago_*` salen de preguntas de selección múltiple, no de opción única con “ambos”.

`prod_*`

`pago_*`

`atencion`

`atencion` tiene los extremos etiquetados y la misma dirección que el resto del cuestionario.

`edad` está al final.

Cuántas preguntas se necesitan para producir trece columnas.

Para la bitácora

Seis preguntas con problemas. Para cada una: identificar el problema, reescribirla y decir qué tipo de variable produce (y cuántas columnas).

1. ¿Está conforme con lo que gasta en comida y en transporte?
2. ¿No le parece que gasta demasiado en salidas?

3. ¿Cuánto gastó últimamente en ropa?
4. Gasto en transporte: ☐ $0 a $1.000 ☐ $1.000 a $3.000 ☐ $3.000 a $6.000
5. ¿Cómo paga sus gastos? ☐ Efectivo ☐ Tarjeta ☐ Ambos ☐ Transferencia
6. (Primera pregunta del cuestionario) ¿Cuál es el ingreso mensual exacto de su hogar?

Fuentes

Bulut, O. Survey Design 101, Part I: Introduction to Survey Design. Centre for Research in Applied Measurement and Evaluation, University of Alberta. Base de las secciones sobre plan, escalas, estructura y piloto.

Dillman, D. A., Smyth, J. D. y Christian, L. M. (2014). Internet, Phone, Mail, and Mixed-Mode Surveys: The Tailored Design Method (4.ª ed.). Wiley.

Groves, R. M. et al. (2009). Survey Methodology (2.ª ed.). Wiley. Marco de error total.

Johnson, R. L. y Morgan, G. B. (2016). Survey Scales: A Guide to Development, Analysis, and Reporting. Guilford. Escalas equilibradas y ordenadas.

Lozano, L. M., García-Cueto, E. y Muñiz, J. (2008). Effect of the number of response categories on the reliability and validity of rating scales. Methodology, 4(2), 73–79.

Samuelson, P. A. (1938). A note on the pure theory of consumer’s behaviour. Economica, 5(17), 61–71.

Stevens, S. S. (1946). On the theory of scales of measurement. Science, 103(2684), 677–680.

Ley 18.331 (2008) y normativa complementaria. URCDP, Uruguay.

Speaker notes

7. Cuando el dato falla

Casos reales de errores en datos, en investigación e instituciones
— Introducción a la Ciencia de Datos, Universidad de Montevideo

Por qué esta clase

- Un error de datos casi nunca avisa: el programa corre y el resultado

parece razonable.

- Casos de hoy: una planilla, una conversión de fechas, una unidad,

un archivo demasiado grande, un promedio, un algoritmo.

- Consecuencias: decisiones de política económica, una misión

espacial perdida, personas acusadas sin base, datos de salud expuestos.

- Pregunta guía en cada caso: **¿qué control lo habría detectado?**

Parte 1 — En la investigación

Reinhart y Rogoff (2010)

- Growth in a Time of Debt (American Economic Review): con deuda

pública mayor al 90 % del PIB, el crecimiento promedio era de **−0,1 %**. Fue muy citado en debates de política fiscal.

- En 2013 Herndon, Ash y Pollin intentaron replicarlo y hallaron tres

problemas:

1. Se dejaron afuera años de Australia, Canadá y Nueva Zelanda

(1946–50).

2. **Error de planilla**: el rango de una fórmula cortaba la lista y

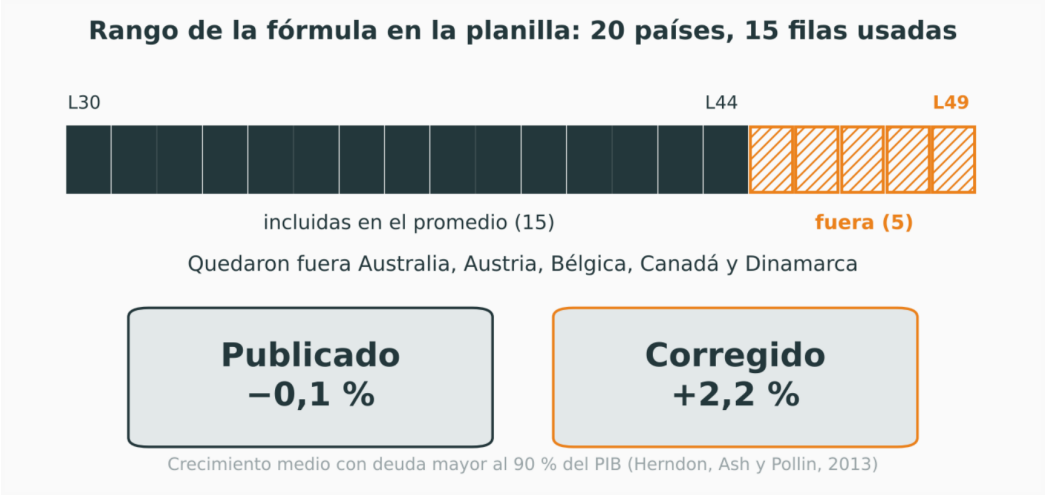
quedaban **fuera cinco países** (Australia, Austria, Bélgica, Canadá, Dinamarca): `L30:L44` en lugar de `L30:L49`.

3. Se promediaron promedios por país, dando igual peso a un país con

un año que a uno con muchos.

- Resultado con los datos completos y mejor ponderación: **+2,2 %**.

Reinhart y Rogoff en una imagen



Reinhart y Rogoff: la lección

- Lo detectó una réplica: alguien volvió a hacer la cuenta con los

datos originales.

- Controles posibles: contar cuántas observaciones entran al cálculo;

comparar con un cálculo hecho en código; publicar datos y código.

- Conexión con el curso: unidad de observación, conteo de filas,

reproducibilidad (clases 4, 7 y 1).

Nombres de genes convertidos en fechas

- Las planillas convierten por su cuenta lo que parece una fecha: el

gen `SEPT2` pasa a «2-sep» y `MARCH1` a «1-mar». El nombre original se pierde.

- Ziemann y colaboradores (2016) revisaron 18 revistas de genómica:

704 de 3.597 artículos (19,6 %) con listas de genes en Excel tenían nombres corruptos.

- Un estudio de 2021 (Abeysooriya y colaboradores), sobre artículos

de 2014 a 2020, halló errores en **3.436 de 11.117 (30,9 %)**: conocer el problema no alcanzó.

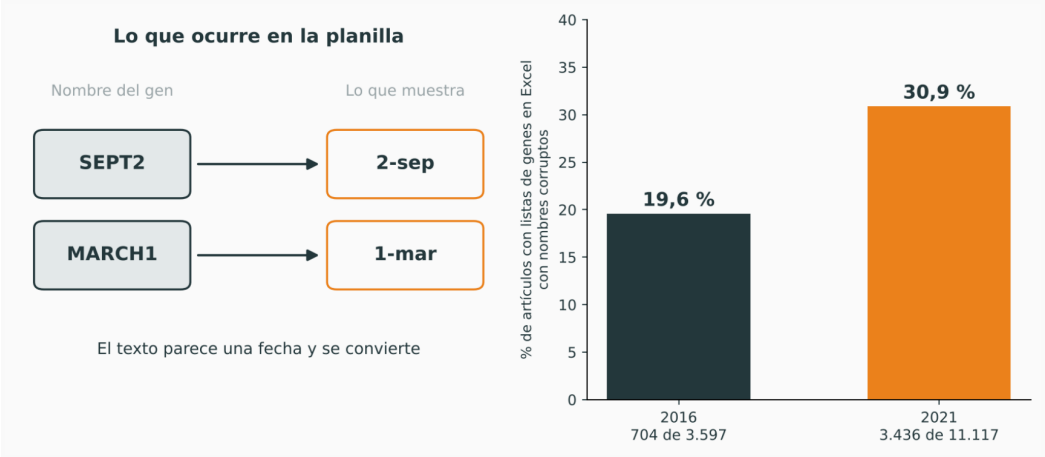
- En 2020 el comité de nomenclatura de genes (HGNC) renombró los

genes afectados: `SEPT1` pasó a `SEPTIN1`, `MARCH1` a `MARCHF1`. Se cambió el dato para adaptarlo a la herramienta.

- Control: declarar el tipo de cada columna (texto), conservar el

archivo crudo intacto y verificar contra una lista oficial.

Genes y fechas en una imagen



Parte 2 — En las instituciones

Salud pública, Inglaterra (2020)

- Entre el 25 de septiembre y el 2 de octubre de 2020 quedaron sin

informar **15.841 casos positivos** de COVID-19; más del 75 % (11.968) correspondía a los últimos tres días.

- Según el organismo (Public Health England), algunos archivos con

resultados positivos superaron el tamaño máximo que admitía el sistema de carga. La prensa lo atribuyó al límite de filas del formato `.xls` de Excel.

- Los casos se cargaron al sistema de rastreo de contactos recién a la

1 de la madrugada del 3 de octubre.

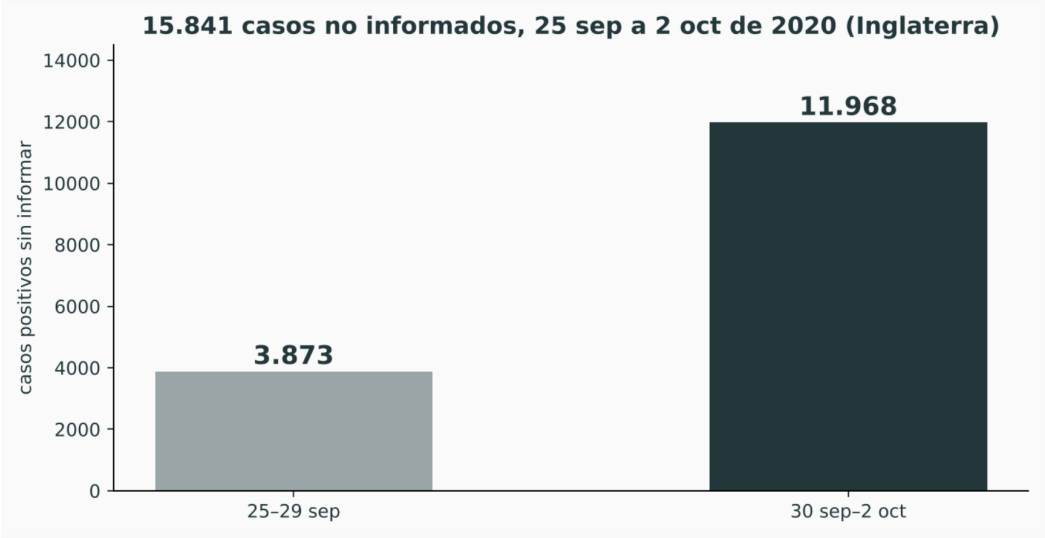
- Nadie recibió mal su resultado; el error estuvo en lo que llegó a

quienes debían rastrear los contactos.

- Control: **conciliar filas de entrada y de salida** (la suma de `n` de

la clase 7), alertas cuando un archivo crece.

Salud pública, Inglaterra, en una imagen



NASA: Mars Climate Orbiter (1999)

- La sonda se perdió el 23 de septiembre de 1999 al llegar a Marte.
- Causa raíz según la junta de investigación: un archivo de software

en tierra (`SM_FORCES`) entregaba el impulso de los propulsores en **libras-fuerza por segundo** y el sistema de navegación esperaba **newton por segundo**. Factor: 4,45.

- La altura prevista del primer paso era de 226 km; la real fue de unos

57 km, por debajo del mínimo de 80 km para sobrevivir.

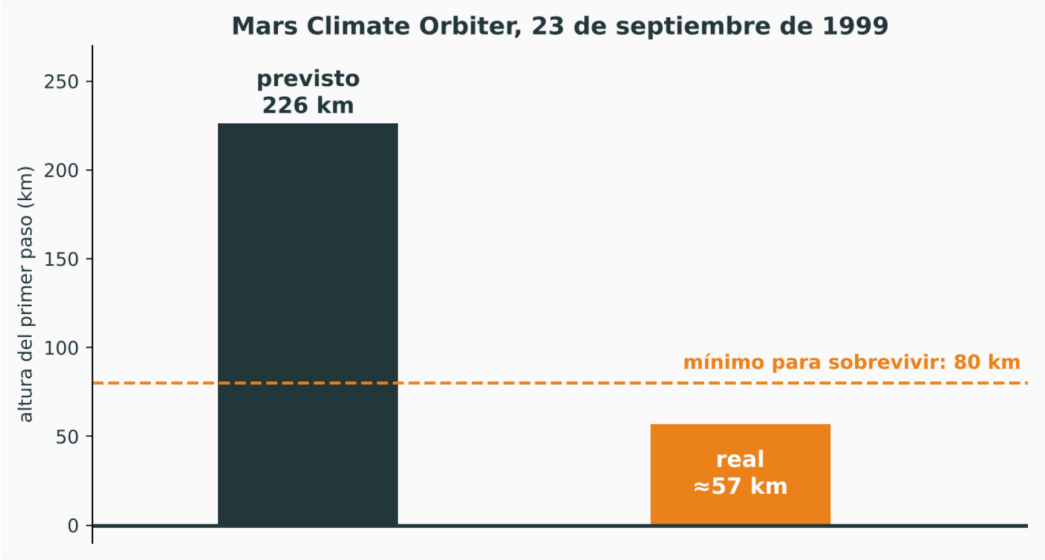
- Durante el vuelo se habían visto discrepancias entre soluciones de

navegación, pero solo se informaron de manera informal.

- Control: la **unidad en el diccionario de variables**, pruebas entre

sistemas, y que las alertas se registren formalmente.

Mars Climate Orbiter en una imagen



Robodebt, Australia (2016–2020)

- El sistema comparaba el ingreso anual declarado al organismo

tributario con los pagos de asistencia social, que se calculan por quincena. **Repartía el ingreso anual en partes iguales** entre las quincenas, como si la persona hubiera trabajado todo el año.

- Generó **470.000 deudas emitidas por error**. En 2019 el gobierno

aceptó ante la justicia que el método era ilegal; el acuerdo colectivo fue de **1.872 millones de dólares australianos** (2021).

- La Comisión Real (informe del 7 de julio de 2023) lo calificó de

«costoso fracaso de la administración pública».

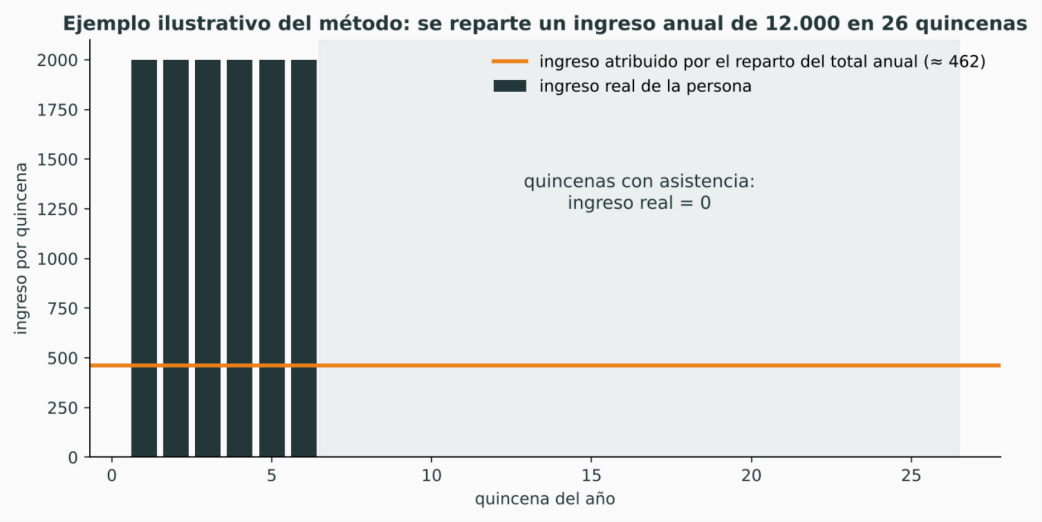
- Error de datos: un supuesto (ingreso constante) aplicado a datos

que no lo cumplen, **sin revisión de personas**.

- Control: probar el método con casos reales, supuestos escritos y

revisados, revisión humana antes de actuar sobre personas.

Robodebt en una imagen



Subsidios de cuidado infantil, Países Bajos

- La administración tributaria usó un sistema de clasificación de

riesgo de fraude en el que la **doble nacionalidad** pesaba como indicador.

- Unos **26.000 padres** fueron acusados injustamente entre 2005 y

2019; muchas familias quedaron con deudas de decenas de miles de euros.

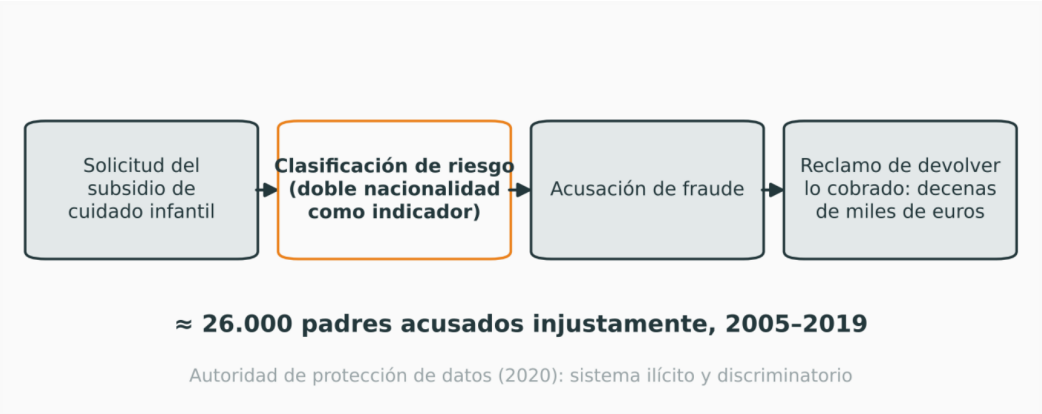
- En julio de 2020 la autoridad de protección de datos concluyó que el

sistema era ilícito y discriminatorio; el informe parlamentario «Ongekend onrecht» (diciembre de 2020) y la renuncia del gobierno (15 de enero de 2021) siguieron.

- Control: auditar qué variables entran a un modelo, medir efectos

por grupo antes de usarlo, y supervisión humana con posibilidad real de apelar.

Cuidado infantil, Países Bajos, en una imagen



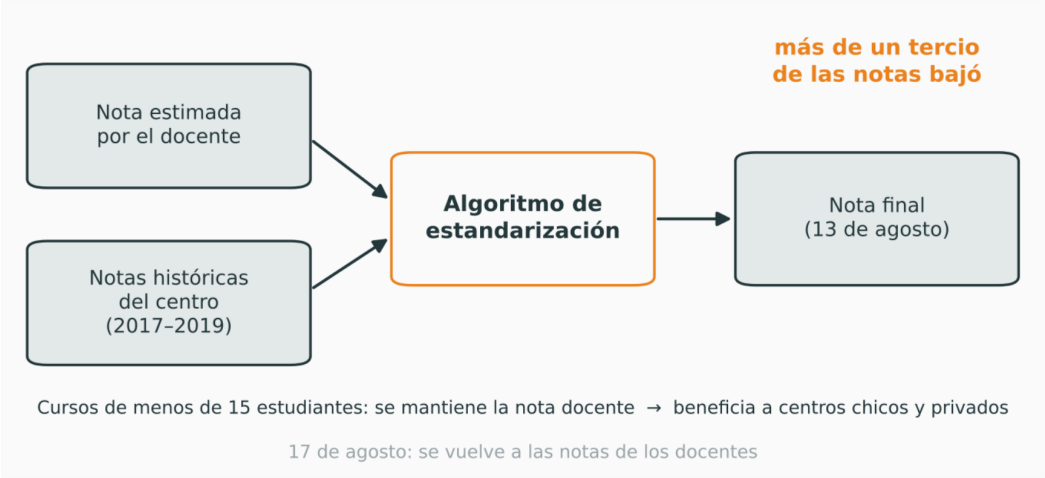
Notas de bachillerato, Reino Unido (2020)

- Sin exámenes por la pandemia, un algoritmo ajustó las notas

estimadas por los docentes usando la **distribución histórica de cada centro** (2017–2019).

- El 13 de agosto se publicaron resultados con **más de un tercio de las notas rebajadas** respecto de la estimación docente. En cursos de menos de 15 estudiantes se mantuvo la nota docente.
- El efecto cayó sobre estudiantes de centros públicos grandes; los centros privados, con cursos chicos, salieron beneficiados.
- El 17 de agosto se volvió a las notas de los docentes.
- Control: simular antes de aplicar, comparar efectos por tipo de centro, y un camino de apelación.

Notas de bachillerato, Reino Unido, en una imagen



Parte 3 — Datos personales y sensibles

Qué dice la ley uruguaya (18.331)

- **Dato personal**: «información de cualquier tipo referida a personas físicas o jurídicas determinadas o determinables» (art. 4). Determinables: no hace falta el nombre.
- **Datos sensibles**: los que revelan «origen racial y étnico, preferencias políticas, convicciones religiosas o morales, afiliación sindical e informaciones referentes a la salud o a la vida sexual» (art. 4).
- El control está a cargo de la Unidad Reguladora y de Control de Datos Personales (URCDP).
- Los trabajos de los equipos con **salud mental, presión arterial e imágenes de rodilla** son datos de salud: sensibles.

Anonimizar no es quitar el nombre

- **Weld (1997).** Latanya Sweeney identificó al gobernador de

Massachusetts en una base de salud «anonimizada» cruzándola con el padrón electoral por código postal, fecha de nacimiento y sexo.

- Sus estudios hallaron que cerca del **87 % de la población de**

Estados Unidos queda identificada de manera única por esos tres datos.

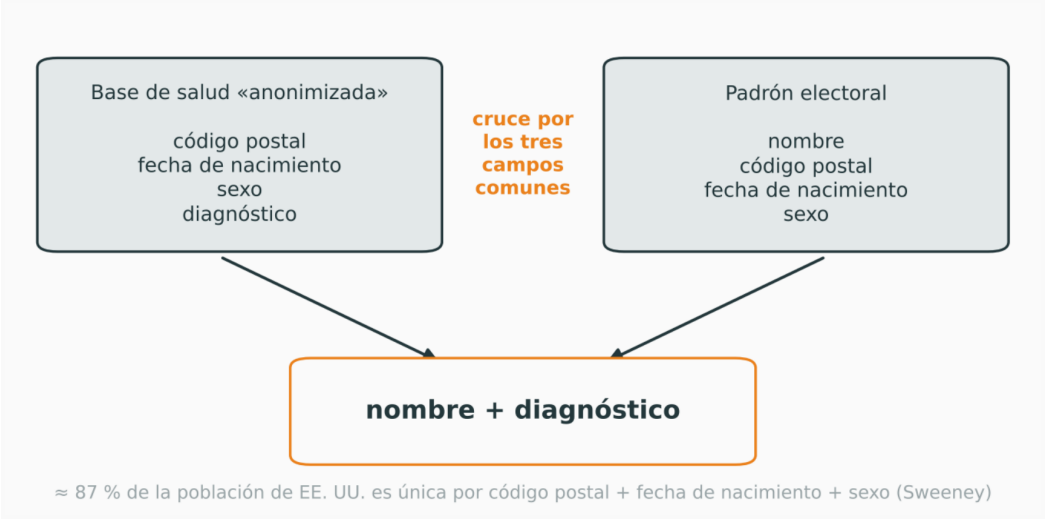
- **Netflix (2006).** Se publicaron 100 millones de valoraciones de

480.000 usuarios, «anonimizadas». Investigadores las cruzaron con IMDb para identificar personas. Hubo una demanda y Netflix canceló la segunda competencia en 2010.

- Lección: las columnas inofensivas por separado identifican

combinadas (cuasi-identificadores).

El caso Weld en una imagen



Strava (2017–2018)

- La empresa publicó un mapa global de calor con las rutas de sus

usuarios, con datos «anónimos».

- En enero de 2018 un estudiante mostró que el mapa revelaba

bases militares y sus rutinas, incluidas bases de Estados Unidos en Siria y Afganistán y una base naval británica.

- Ningún dato individual tenía nombre: el problema estaba en **la**

agregación y el contexto.

- Control: evaluar qué revela el conjunto, no solo cada registro;

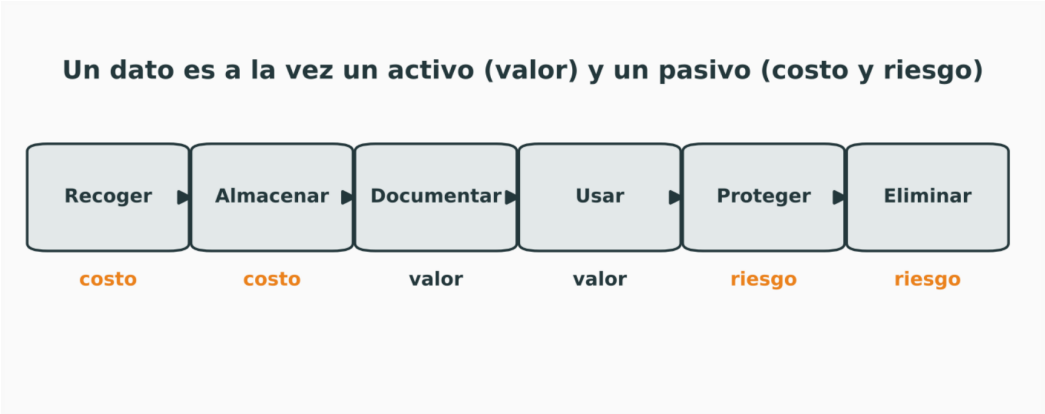
minimizar lo que se recoge; defaults de privacidad que protejan.

Parte 4 — Los datos como activos

Los datos como activos

- Un activo es algo que **tiene valor, cuesta mantener y hay que gestionar**. Los datos cumplen las tres condiciones.
- **Valor**: sirven para decidir, para operar, para entrenar modelos y para ofrecer servicios.
- **Costo**: recoger, almacenar, limpiar, documentar y proteger.
- **Riesgo**: un dato mal hecho, perdido o filtrado genera daños, reclamos y sanciones.
- Por eso un dato es a la vez un activo y un pasivo, y alguien tiene que hacerse cargo.

Ciclo de vida de un dato



Qué cuesta que los datos sean malos

- Redman (Harvard Business Review, 2016) cita una estimación de IBM de **3 billones de dólares por año** para la economía de Estados Unidos por datos de mala calidad. Es una estimación gruesa, no una medición.
- Costos directos: tiempo de personas rehaciendo trabajo, decisiones equivocadas, reclamos y sanciones.
- Los casos de hoy lo ilustran: Robodebt terminó en un acuerdo de 1.872 millones de dólares australianos, el error de Inglaterra retrasó el rastreo de contactos y el de unidades costó una misión.

Cuando el activo se filtra

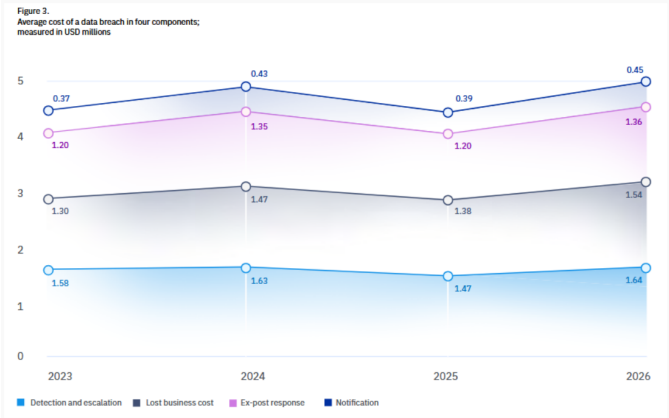
- **Equifax (2017):** datos personales de **147 millones de personas** expuestos; acuerdo de hasta **700 millones de dólares** con la FTC, la CFPB y los estados (julio de 2019).
- **IBM, Cost of a Data Breach 2026:** costo promedio global de una

filtración de **4,99 millones de dólares**, un récord (+12 % sobre el año anterior).

- Cuanto más dato se acumula sin necesidad, más hay para perder.

Principio de **minimización**: recoger solo lo necesario para la finalidad declarada, y eliminarlo cuando deja de servir.

Cuánto cuesta una filtración: el gráfico de IBM



Costo promedio de una filtración de datos en cuatro componentes, en millones de dólares. Fuente: IBM, Cost of a Data Breach Report 2026, figura 3.

Cómo se lee el gráfico

- Cada línea es un **componente del costo** de una filtración, en

millones de dólares, para los años 2023 a 2026. Los números junto a cada punto son el valor de ese componente.

- Las líneas están **apiladas**: la altura de cada una es la suma de ese

componente y de los que están debajo. La línea de arriba es el **total**.

- Control: 1,64 + 1,54 + 1,36 + 0,45 = 4,99 millones en 2026. Los

totales de 2023, 2024 y 2025 dan 4,45, 4,88 y 4,44.

- El total subió cerca de 12 % entre 2025 y 2026 (de 4,44 a 4,99).
- En cada año el componente mayor es la detección y el

escalamiento; el menor, la notificación.

Los cuatro componentes

Componente	Qué incluye (según la metodología de IBM)	2026	%
Detección y escalamiento	Investigación forense, auditorías, gestión de la crisis, comunicación a la dirección	1,64	33
Negocio perdido	Interrupción de la operación, ingresos perdidos, clientes que se van, deterioro de la reputación	1,54	31
Respuesta posterior	Atención a afectados, monitoreo de crédito o identidad, costos legales, multas	1,36	27
Notificación	Avisar a las personas afectadas, a los reguladores y a terceros	0,45	9
Total		**4,99**	**100**

Valores en millones de dólares. Lo que más pesa no es la multa sino **encontrar el problema y perder negocio**: la confianza de quienes entregaron sus datos es parte del activo.

Gestionar un activo de datos

Práctica	Qué resuelve	Dónde la vimos
Diccionario de variables y libro de códigos	Significado, unidad, tipo	Clases 3 y 6
Perfilado y controles de cierre	Calidad	Clases 4 y 7
Código reproducible y Git	Trazabilidad	Clase 1
Dato crudo intacto, versiones	Poder volver atrás	Clases 4 y 5
Responsable y control de acceso	Quién puede usar qué	Ley 18.331
Finalidad, minimización y retención	Riesgo y cumplimiento	Ley 18.331

El caso uruguayo

- La Ley 18.331 (2008) y la URCDP regulan el uso de datos personales.
- Según la Comisión Europea, Uruguay tiene una **decisión de**

adecuación (2012), mantenida en la revisión de enero de 2024: los datos personales pueden circular desde la Unión Europea hacia Uruguay sin garantías adicionales.

- Cumplir la ley es también un activo comercial: abre mercados y da

confianza a quienes entregan sus datos.

Parte 5 — Qué hace la profesión

Qué controles habrían evitado cada caso

Caso	Qué falló	Control

Reinhart y Rogoff Rango de planilla excluyó 5 Contar *n* usado; replicar en países código; compartir datos Genes en Excel Tipo inferido por la herramienta Declarar tipo; guardar crudo intacto; verificar

Salud pública, Inglaterra	Archivos mayores al límite, sin aviso	Conciliar filas de entrada y salida
Mars Climate Orbiter	Unidades distintas entre sistemas	Unidad en el diccionario; pruebas entre sistemas
Robodebt	Promedio anual aplicado a datos quincenales	Probar con casos; revisión humana
Cuidado infantil, Países Bajos	Nacionalidad como indicador de riesgo	Auditar variables; efectos por grupo
Notas, Reino Unido	Historia del centro aplicada a cada estudiante	Simular; efectos por grupo; apelación
Weld, Netflix, Strava	Cuasi-identificadores y agregados	Generalizar; evaluar riesgo; minimizar datos

Por qué importa la profesión

- Detrás de cada una de esas filas había una persona con la tarea de

verificar el dato, y no la hizo, no pudo hacerla o nadie la escuchó.

- Quien trabaja con datos decide qué se cuenta, cómo se une, qué se

descarta y qué se publica, y esas decisiones afectan a otras personas.

- Las defensas son las que ya practicamos: perfilar antes y después,

conteos de cierre, unidad y tipo en el diccionario, código reproducible, documentar los supuestos.

- Y una más: preguntar **sobre quién** son los datos, qué revelan

juntos y quién puede resultar dañado.

Fuentes

- Herndon, T., Ash, M. y Pollin, R. (2013, working paper PERI;

Cambridge Journal of Economics, 2014). Does high public debt consistently stifle economic growth? A critique of Reinhart and Rogoff.

- Reinhart, C. y Rogoff, K. (2010). Growth in a Time of Debt. American

Economic Review 100(2).

- Ziemann, M., Eren, Y. y El-Osta, A. (2016). Gene name errors are

widespread in the scientific literature. Genome Biology 17:177. Abeysooriya, M. et al. (2021). PLOS Computational Biology 17(6): e1008984.

- Mars Climate Orbiter Mishap Investigation Board (1999). Phase I

Report, 10 de noviembre de 1999.

- Public Health England (2020). PHE statement on delayed reporting

of COVID-19 cases. gov.uk.

- Royal Commission into the Robodebt Scheme (2023). Report.

Australia.

- Tweede Kamer (2020). Ongekend onrecht. Autoriteit

Persoonsgegevens (2020), informe sobre la administración tributaria.

- Ofqual (2020). Informes sobre las calificaciones de verano de 2020

(Inglaterra).

- Sweeney, L. (2000). Simple demographics often identify people

uniquely. Carnegie Mellon University.

- Narayanan, A. y Shmatikov, V. (2008). Robust de-anonymization of

large sparse datasets. IEEE Symposium on Security and Privacy.

- Uruguay. Ley 18.331 de Protección de Datos Personales (2008), art.

4.

- Redman, T. (2016). Bad Data Costs the U.S. $3 Trillion Per Year.

Harvard Business Review, 22 de septiembre de 2016.

- Federal Trade Commission. Equifax Data Breach Settlement (2019).
- IBM. Cost of a Data Breach Report 2026.
- Comisión Europea (2024). Informe de revisión de las decisiones de

adecuación, COM(2024) 7 final; decisión sobre Uruguay (2012).

- Los gráficos de los casos son esquemas propios, hechos con las

cifras de las fuentes citadas; el ejemplo de Robodebt es ilustrativo, con datos inventados.

Cifras contrastadas con las fuentes citadas; confirmar en la fuente primaria antes de proyectar.

8. Fundamentos: objetos, tipos y errores

Joselina Davyt-Colo

Clase 3: Fundamentos de programación para datos

Subtítulo: Qué es cada cosa antes de escribir código
Institución: Facultad de Ciencias Empresariales y Economía — Universidad de Montevideo

Por qué esta clase

En la clase 2 escribimos un diccionario y lo convertimos en un DataFrame. Funcionó porque copiamos el código.

Esta clase responde una pregunta anterior: **¿qué es cada uno de esos objetos?**

La mayoría de los bloqueos en la primera práctica no son de sintaxis. Son de tipo: el alumno cree que tiene un número y tiene un texto, o cree que tiene una tabla y tiene una lista.

Punto 1. Ejecutar y leer el error

Hay tres maneras de ejecutar código y conviene distinguirlas:

- **Línea por línea** en la consola: para probar.
- **Archivo completo** (`.py`, `.R`): de arriba hacia abajo, siempre en orden.
- **Celda por celda** (notebook, `.qmd`): en el orden en que uno las corre, que no siempre es de arriba hacia abajo.

> **Nota**
> La tercera es la que genera más confusión: si se ejecuta la celda 5 antes que la 2, el programa no sabe nada de lo que había en la 2.

El error no es un castigo, es información

```
NameError: name 'df' is not defined
```

Se lee de abajo hacia arriba. La última línea dice **qué** pasó; las de arriba dicen **dónde**.

Este mensaje no significa que el código esté mal escrito. Significa que el nombre `df` nunca fue creado: probablemente la celda que lo creaba no se ejecutó.

Los tres errores del primer día

Python	R	Qué significa
`NameError: name 'x' is not defined`	`Error: object 'x' not found`	El nombre no existe todavía
`TypeError`	`Error in ... : non-numeric argument`	La operación no corresponde a ese tipo

Python	R	Qué significa
`FileNotFoundError`	`cannot open file`	El archivo no está donde se lo buscó

El primero es de orden de ejecución. El segundo es de tipo. El tercero es de ubicación.

Ninguno de los tres se resuelve reescribiendo la línea al azar.

Punto 2. Asignación: el = no es una igualdad

```
# Python
edad = 35

# R
edad <- 35
```

Se lee: *"guardá el valor 35 bajo el nombre* `edad`*"*.

No es una ecuación. `edad = edad + 1` es perfectamente válido y significa "tomá lo que hay en `edad`, sumale 1 y volvé a guardarlo ahí".

Nombre y valor son cosas distintas

```
peso = 45.0        # el nombre 'peso' apunta al valor 45.0
peso = 120.5       # ahora apunta a otro valor; el 45.0 se perdió
```

Una consecuencia práctica: si se ejecuta dos veces una celda que dice `df = df.dropna()`, la segunda vez trabaja sobre el resultado de la primera. El nombre es el mismo, el contenido no.

Punto 3. Tipos elementales

Concepto	Python	R	Ejemplo
Entero	`int`	`integer`	`8`
Decimal	`float`	`numeric/double`	`45.0`
Texto	`str`	`character`	`"suave"`
Lógico	`bool`	`logical`	`True` / `TRUE`
Fecha	`datetime.date`	`Date`	`2025-03-12`
Categórica	`category` (pandas)	`factor`	`"blando"` de tres valores posibles
Ausente	`None` / `NaN`	`NA`	celda vacía

Los dos últimos no son simétricos

`factor` es un tipo de R base: el lenguaje fue diseñado para datos y ya sabe qué es una variable categórica. En Python no hay equivalente en el lenguaje: la categórica la aporta pandas. Lo mismo con `NA`, que en R es un valor del lenguaje y en Python son dos cosas distintas según el contexto.

Las comillas cambian el tipo

```
peso = 45.0        # float: se le puede calcular un promedio
peso = "45.0"      # str: es la cadena de caracteres "45.0"

"45.0" + "1"       # devuelve '45.01'  (concatena)
45.0 + 1           # devuelve 46.0     (suma)
```

Este es exactamente el error del práctico: un CSV mal leído deja los pesos como texto, y las operaciones siguen funcionando pero dan resultados sin sentido.

Punto 4. El vector en R y la lista en Python

Acá los dos lenguajes se separan, y la diferencia importa.

```
# R
c(1, 2, 3)
# [1] 1 2 3
```

`c()` combina valores en un **vector**: una secuencia ordenada de elementos **del mismo tipo**.

En R no existen los números sueltos

```
x <- 1
length(x)     # [1] 1
is.vector(x)  # [1] TRUE
```

`1` **es** un vector numérico de longitud 1. R no tiene un tipo "escalar" separado.

Por eso las operaciones se aplican a todo el vector de una vez:

```
c(1, 2, 3) * 10
# [1] 10 20 30
```

No hace falta un bucle. Esto se llama *vectorización* y es el rasgo central de R.

El vector de R homogeneiza los tipos

```
c(1, "a", TRUE)
# [1] "1"    "a"    "TRUE"
```

No da error: convierte **todo** a texto, porque es el único tipo que puede contener a los tres.

> **Advertencia**
> Esta conversión es silenciosa. Un solo valor de texto en una columna que debería ser numérica convierte la columna entera en texto.

En Python: lista, y recién después array

```
[1, 2, 3]              # lista: admite tipos mezclados
[1, "a", True]         # válido, no convierte nada

[1, 2, 3] * 10         # NO multiplica: repite la lista 10 veces
```

Para el comportamiento vectorizado de R hay que usar otra estructura:

```python
import numpy as np
np.array([1, 2, 3]) * 10
# array([10, 20, 30])
```

Resumen del punto 4

	R	Python
Estructura básica	vector (`c()`)	lista (`[ ]`)
¿Tipos mezclados?	No: convierte todo	Sí: los conserva
¿Un valor suelto?	Es un vector de largo 1	Es un escalar
Vectorización	Por defecto	Con `numpy`
Primer elemento	`x[1]`	`x[0]`

Punto 5. Diccionario y data frame

Un **diccionario** en Python asocia nombres a valores:

```python
{"textura": ["suave", "rugoso"], "peso": [45.0, 120.5]}
```

Cada clave es el nombre de una columna; cada valor es la columna entera.

Un **data frame** es exactamente eso, con la exigencia de que todas las columnas tengan el mismo largo.

El DataFrame es una colección de columnas

```python
import pandas as pd
df = pd.DataFrame({"textura": ["suave", "rugoso"],
                   "peso": [45.0, 120.5]})
```

```r
df <- data.frame(textura = c("suave", "rugoso"),
                 peso    = c(45.0, 120.5))
```

Cada columna tiene **su propio tipo**. Una fila, en cambio, mezcla tipos: por eso se trabaja por columnas y no por filas.

Punto 6. Función y método

Una **función** se aplica a un objeto:

```python
len(x)          # Python
length(x)       # R
```

Un **método** pertenece al objeto y se invoca con punto:

```python
df.head()       # Python
```

R casi no usa métodos; Python los usa todo el tiempo. Por eso en Python el punto aparece por todos lados.

Los argumentos con nombre son instrucciones

```python
pd.read_csv("datos.csv", sep=";", decimal=",", encoding="latin-1")
```

```r
read.csv("datos.csv", sep = ";", dec = ",", fileEncoding = "latin1")
```

Solo el primer argumento es un **dato** (qué archivo leer). Los otros tres son **instrucciones sobre cómo interpretarlo**.

> **Importante**
> Cambiar `sep` no cambia el archivo: cambia la lectura. El mismo archivo leído con dos separadores distintos produce dos tablas distintas.

Punto 7. La comparación devuelve muchos valores

```
df["peso"] > 50
```

No devuelve un `True`. Devuelve **un `True` o `False` por cada fila**:

```
0    False
1     True
2    False
```

Esa secuencia se llama **máscara booleana** y sirve para dos cosas:

```
df[df["peso"] > 50]           # filtrar: quedarse con las filas True
(df["peso"] > 50).sum()       # contar: True vale 1, False vale 0
```

Punto 8. Dónde está el archivo

Todo programa se ejecuta **desde algún lugar**: el directorio de trabajo.

```
from pathlib import Path
Path.cwd()
```

```
getwd()
```

Una ruta **relativa** (`datos/caja_negra.csv`) se busca a partir de ahí. Una ruta **absoluta** (`C:/Users/jodavyt/...`) contiene el nombre de usuario de quien la escribió y falla en cualquier otra máquina.

Ejercicio: opción múltiple

Una sola respuesta correcta por pregunta. Justificar la elección en la bitácora cuando la consigna lo pida.

Preguntas 1 a 4

1. En R, `c(1, 2, 3)` es:

a. Una función que suma 1 + 2 + 3
b. Un vector numérico de longitud 3
c. Tres objetos independientes guardados juntos
d. Una matriz de tres filas

2. En R, ¿qué es el valor `1` escrito solo?

a. Un número, que no es lo mismo que un vector
b. Un vector numérico de longitud 1
c. Un entero, incompatible con los decimales
d. Depende de si fue asignado a un nombre

3. `c(1, "a", TRUE)` en R devuelve:

a. Un error por mezclar tipos
b. Un vector con un número, un texto y un lógico
c. Un vector de texto: `"1" "a" "TRUE"`
d. Un vector lógico de longitud 3

4. En Python, `[1, 2, 3] * 2` devuelve:

a. `[2, 4, 6]`
b. `[1, 2, 3, 1, 2, 3]`
c. Un error de tipo
d. `[1, 2, 3, 2]`

Preguntas 5 a 8

5. `edad = "35"` guarda:

a. El número 35
b. El texto `"35"`
c. El número 35, con las comillas ignoradas
d. Un error, porque falta declarar el tipo

6. `NameError: name 'df' is not defined` significa que:

a. El archivo `df.csv` no existe
b. El nombre `df` nunca fue creado en esta sesión
c. `df` tiene el tipo equivocado
d. Falta importar pandas

7. En `pd.read_csv("datos.csv", sep=";")`, el `sep=";"` es:

a. Una segunda columna que se quiere leer
b. Una instrucción sobre cómo interpretar el archivo
c. El carácter que se va a eliminar del archivo
d. Un comentario para el lector del código

8. Un mismo archivo leído con `sep=","` y con `sep=";"` produce:

a. Siempre la misma tabla
b. Dos tablas distintas: la lectura es una decisión
c. Un error en uno de los dos casos, siempre
d. La misma tabla, con las columnas en otro orden

Preguntas 9 a 12

9. `df["peso"] > 50` devuelve:

a. La cantidad de pesos mayores a 50
b. Los valores de peso mayores a 50
c. Un `True` o `False` por cada fila
d. Un único `True` si algún peso supera 50

10. La diferencia entre `len(x)` y `x.upper()` es que:

a. La primera es una función aplicada al objeto; la segunda, un método del objeto
b. Ninguna: son dos formas de escribir lo mismo
c. La segunda solo funciona en R
d. La primera es más rápida

11. Un compañero comparte un script que empieza con `pd.read_csv("C:/Users/jodavyt/datos.csv")`. En tu máquina:

a. Funciona igual, porque la ruta está completa
b. Falla, porque esa carpeta no existe en tu computadora
c. Funciona si tenés la misma versión de Windows
d. Funciona si copiás el archivo a tu escritorio

12. En un data frame, ¿qué comparte el mismo tipo de dato?

a. Cada fila
b. Cada columna
c. Toda la tabla
d. Ninguno: cada celda es independiente

Clave de corrección

	Correcta	El error que detecta
1	b	—
2	**b**	Creer que existe un escalar en R
3	**c**	Esperar un error donde hay conversión silenciosa
4	**b**	Suponer que la lista de Python vectoriza como R
5	b	No ver que las comillas definen el tipo
6	b	Confundir nombre inexistente con archivo faltante
7	b	Leer el argumento como dato y no como instrucción
8	b	Creer que el archivo tiene una única lectura posible
9	c	Esperar un valor único donde hay una secuencia
10	a	—
11	b	No distinguir ruta absoluta de relativa
12	b	Pensar la tabla por filas y no por columnas

Las cuatro en negrita son las que suelen fallar más; conviene discutirlas en clase antes de seguir.

Lo que sigue

Con estos ocho puntos ya es posible hacer el práctico de creación y carga de datasets:

- Escribir un diccionario y convertirlo en DataFrame
- Guardarlo en un archivo y volver a leerlo
- Diagnosticar un archivo mal formado
- Distinguir un faltante de un cero

9. Anexo: interpretación de errores en R y Python

Joselina Davyt-Colo

Anexo: interpretación de errores

Complemento del Punto 1 de la Clase 3
Institución: Facultad de Ciencias Empresariales y Economía — Universidad de Montevideo

Qué es este anexo

El Punto 1 de la clase presentó tres mensajes de error. Este anexo los extiende a un catálogo de consulta.

No está pensado para dictarse completo: se usa **durante el práctico**, cuando aparece un mensaje que no se entiende.

Cubre R y Python en paralelo, porque los dos lenguajes fallan de maneras distintas y esa diferencia es en sí misma un contenido de la materia.

Parte I. Cómo se lee un mensaje

Los tres desenlaces posibles

Cuando una instrucción no hace lo que se esperaba, hay tres finales distintos:

Desenlace	Qué hace el programa	Cómo se detecta
Error	Se detiene y no devuelve nada	Solo: la pantalla lo muestra
Advertencia	Sigue, devuelve algo y avisa	Leyendo la salida completa
Resultado sin sentido	Sigue, devuelve algo y no avisa	Únicamente si uno lo controla

> **Importante**
> El primero interrumpe el trabajo. El tercero es el único que llega al informe final.

Cada lenguaje calla en lugares distintos

Situación	Python	R
Convertir `"45,0"` a número	Se detiene (`ValueError`)	Devuelve `NA` con advertencia
Pedir el elemento 5 de un vector de 3	Se detiene (`IndexError`)	Devuelve `NA`, sin aviso
Pedir una columna que no existe	Se detiene (`KeyError`)	Devuelve `NULL`, sin aviso

Situación	Python	R
Sumar una columna que no existe	Se detiene	Devuelve `0`, sin aviso
Archivo con una fila de más	Se detiene (`ParserError`)	Lo lee mal y sigue
Promediar con datos faltantes	**Los ignora y devuelve un número**	**Devuelve `NA`**

> **Advertencia**
> La regla no es “R es permisivo y Python es estricto”. La última fila va en sentido contrario: ante datos faltantes, R se detiene en `NA` y Python devuelve un promedio calculado sobre las filas que quedaron. Cada lenguaje tiene su propia zona de silencio.

Anatomía del mensaje en Python

```
Traceback (most recent call last):
  File "analisis.py", line 12, in <module>
    promedio = df["peso"].mean()
  File "analisis.py", line 4, in cargar
    return pd.read_csv(ruta)
ValueError: could not convert string to float: '45,0'
```

Parte	Qué aporta
`ValueError`	La categoría del problema
`could not convert...`	Qué pasó, en una frase
`File ..., line 12`	Dónde pasó
`promedio = df[...]`	La línea exacta

Se lee de abajo hacia arriba. La última línea dice *qué*; las de arriba, *por dónde llegó*.

Anatomía del mensaje en R

R no arma un traceback: escribe una sola línea con dos piezas separadas por `:`.

```
Error in edad + 1 : non-numeric argument to binary operator
```

Parte	Qué aporta
`Error in edad + 1`	**Dónde**: la operación que falló
`non-numeric argument...`	**Qué**: la descripción del problema

Cuando el problema no está en ninguna operación en particular, la primera pieza desaparece:

```
Error: object 'datos' not found
```

Las advertencias de R se leen aparte

Las advertencias no interrumpen nada y **pueden aparecer al final**, lejos de la línea que las produjo:

```r
warnings()          # lista las advertencias acumuladas
traceback()         # muestra el camino del último error
```

> **Nota**
> Un script de R puede terminar "bien" y haber dejado veinte advertencias por el camino. Correr `warnings()` después de cada bloque largo es parte del oficio.

Las tres preguntas

Frente a cualquier mensaje, en cualquiera de los dos lenguajes:

1. **¿De qué tipo es el problema?** ¿Un nombre, un tipo, un valor, una posición, un archivo?
2. **¿Sobre qué objeto ocurrió?** El mensaje casi siempre lo nombra.
3. **¿Qué esperaba encontrar el programa y qué encontró en su lugar?**

Las tres se responden leyendo. Ninguna se responde reescribiendo la línea al azar.

Parte II. Catálogo por causa

Seis causas cubren casi todo lo que aparece en el primer mes.

A. El nombre no existe

Python	R
`NameError: name 'datos' is not defined`	`Error: object 'datos' not found`
`ModuleNotFoundError: No module named 'pandas'`	`Error in library(tidyverse) : there is no package called 'tidyverse'`
`AttributeError: 'DataFrame' object has no attribute 'colums'`	`Error in meanx(x) : could not find function "meanx"`

Qué revisar, en este orden:

1. ¿La celda que creaba el objeto se ejecutó en esta sesión?
2. ¿Está escrito exactamente igual? `Peso`, `peso` y `PESO` son tres nombres distintos en los dos lenguajes.
3. ¿El paquete está instalado, o está instalado pero sin cargar? Son dos problemas distintos con soluciones distintas.

El caso de `df` en R

```r
df$peso
# Error in df$peso : object of type 'closure' is not subsettable
```

`closure` significa **función**. El mensaje dice: "le pediste una columna a algo que es una función".

En R, `df` ya existe: es la densidad de la distribución F. Si el objeto propio nunca se creó, el nombre no queda vacío, queda apuntando a la función de R.

> **Advertencia**
> Nombres a evitar en R porque ya están ocupados: `df`, `data`, `c`, `mean`, `T`, `F`.
> En R conviene llamar a los datos `datos` o `encuesta`, no `df`.

B. El código no se puede leer

Acá el programa ni siquiera llegó a ejecutar: no entendió la escritura.

```
# Python
print("hola"
# SyntaxError: '(' was never closed

# R, en un script
mean(x
# Error: unexpected end of input

# R, un símbolo donde no corresponde
x <- c(1 2)
# Error: unexpected numeric constant
```

La consola colgada

Si falta cerrar un paréntesis o una comilla, la consola **no da error**: queda esperando el resto.

Lenguaje	Señal
R	El prompt cambia de `>` a `+`
Python	El prompt cambia de `>>>` a `...`

No es un error: es una pregunta. Se sale con `Esc` (R) o `Ctrl + C` (Python) y se vuelve a escribir la línea completa.

La indentación en Python

```
if peso > 50:
print(peso)
# IndentationError: expected an indented block after 'if' statement on line 1
```

Python usa el **espacio en blanco** para delimitar bloques; R usa llaves `{ }`. Un error de sangría en Python es un error real; en R, apenas un problema estético.

> **Nota**
> Configurar el editor para que inserte cuatro espacios en lugar de tabulaciones elimina toda una familia de errores.

C. El tipo no corresponde

```
edad = "35"
edad + 1
# TypeError: can only concatenate str (not "int") to str

edad <- "35"
edad + 1
# Error in edad + 1 : non-numeric argument to binary operator
```

Los dos mensajes dicen lo mismo: **hay un texto donde el programa esperaba un número**.

El origen casi nunca está en esa línea. Está antes: en una lectura de archivo que dejó la columna como texto.

El método no existe para ese tipo

```
peso = 45.0
peso.upper()
# AttributeError: 'float' object has no attribute 'upper'
```

Dos lecturas posibles, y conviene descartarlas en este orden:

1. El nombre del método está mal escrito (`.uper()` en lugar de `.upper()`).
2. El objeto no es lo que se creía. Un `AttributeError` sobre un objeto que debería ser un DataFrame suele significar que se recibió otra cosa.

D. El valor no se puede convertir

Este es el punto donde los dos lenguajes se separan de verdad.

```
float("45,0")
# ValueError: could not convert string to float: '45,0'
```

```
as.numeric("45,0")
# [1] NA
# Warning message: NAs introduced by coercion
```

El tipo era el correcto (los dos recibieron texto, que es lo que esperaban). El **contenido** no se puede interpretar como número, porque la coma decimal no es el separador que el lenguaje espera.

Por qué esta diferencia importa

Python **se detiene** y obliga a resolverlo. R **devuelve `NA`** y sigue.

Un archivo con quinientas filas y decimales con coma, leído sin corregir:

- En Python: el proceso se interrumpe en la primera conversión.
- En R: se obtienen quinientos `NA`, un promedio que es `NA` y una advertencia de una línea que es fácil pasar por alto.

> **Importante**
> `NAs introduced by coercion` es la advertencia más importante de R para quien trabaja con datos: significa que se perdieron valores en silencio.

E. La posición o la columna no existe

Operación	Python	R
Elemento fuera de rango	`IndexError: list index out of range`	Devuelve `NA`
Columna inexistente	`KeyError: 'pesoo'`	`d$pesoo` devuelve `NULL`

Operación	Python	R
Selección explícita	_____	`d[, "pesoo"]` sí da error: `undefined columns selected`

```
x <- c(10, 20, 30)
x[5]
# [1] NA        <- no hay error
```

Recordar además que **el primer elemento es `x[1]` en R y `x[0]` en Python**: pedir `x[0]` en R devuelve un vector vacío, no el primer valor.

El `$` de R hace coincidencias parciales

```
datos <- data.frame(peso = c(45, 120.5))
datos$pes
# [1] 45.0 120.5     <- devolvió la columna 'peso'
```

R completa el nombre si no hay ambigüedad. Es cómodo y es peligroso: un error de tipeo puede devolver la columna equivocada sin ningún aviso.

> **Advertencia**
> Usar `datos[["peso"]]` en lugar de `datos$peso` evita la coincidencia parcial. En Python el problema no existe: `df["pes"]` da `KeyError`.

F. El archivo: cinco fallas distintas

Qué pasó	Python	R
No está donde se lo buscó	`FileNotFoundError: [Errno 2] No such file or directory`	`cannot open file 'datos.csv': No such file or directory`
Separador equivocado	Una sola columna con todo adentro	Una sola columna, y a veces filas renombradas
Decimal equivocado	Columna de texto (`object`)	Columna `chr`
Codificación equivocada	`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1`	Advertencia y texto deformado: `Se\361or`
Fila con un campo de más	`ParserError: Expected 3 fields in line 3, saw 4`	Lee mal y sigue

Las cinco se resuelven con argumentos de lectura, no reescribiendo el archivo.

El archivo no está donde se lo buscó

```
from pathlib import Path
Path.cwd()                          # dónde estoy parado
list(Path.cwd().iterdir())          # qué hay acá

getwd()                             # dónde estoy parado
list.files()                        # qué hay acá
```

Si el archivo aparece en el listado, el problema es el nombre. Si no aparece, el problema es la carpeta.

Separador, decimal y codificación

```
pd.read_csv("datos.csv", sep=";", decimal=",", encoding="latin-1")
```

```
read.csv("datos.csv", sep = ";", dec = ",", fileEncoding = "latin1")
```

Tres argumentos, tres problemas independientes:

- `sep` decide **dónde termina cada columna**.
- `decimal` / `dec` decide **si la coma separa decimales o campos**.
- `encoding` / `fileEncoding` decide **cómo se traducen las eñes y las tildes**.

El caso más insidioso: la fila de más

```
a,b,c
1,2,3
4,5,6,7      <- una coma de más
```

```
pd.read_csv("malo.csv")
# ParserError: Error tokenizing data. C error: Expected 3 fields in line 3, saw 4
```

```
read.csv("malo.csv")
#   a b  c
# 1 2 3 NA
# 4 5 6  7
```

R no protesta: usa la primera columna como nombres de fila y devuelve una tabla que parece correcta. Los datos están corridos un lugar.

> **Importante**
> Este es el argumento central del anexo: el mensaje de error de Python es una molestia de treinta segundos; el silencio de R puede sobrevivir hasta la presentación final.

Parte III. Los errores que no dan error

Siete fallas silenciosas

Situación	Qué se obtiene	Cómo se detecta
Columna numérica leída como texto	`.sum()` concatena: `"45,0120,5"`	Revisar los tipos antes de calcular
Columna inexistente en R	`sum(d$pesoo)` devuelve `0`	Comparar contra `names(d)`
Coincidencia parcial en R	Devuelve otra columna	Usar `[["nombre"]]`
Faltantes al promediar	Python los ignora; R devuelve `NA`	Contar los faltantes primero
Reciclaje de vectores en R	Repite valores para completar	Comparar `length()` de los dos vectores
Resultado no asignado	Nada cambia	Verificar `shape` / `dim` después
Celdas ejecutadas fuera de orden	Un objeto viejo con nombre nuevo	Reiniciar y correr todo de arriba abajo

La suma que concatena

```python
pd.read_csv("pesos.csv", sep=";")["peso"].sum()
# '45,0120,5'
```

No hay error. Hay una cadena de texto que pegó los dos valores uno detrás del otro.

```python
pd.read_csv("pesos.csv", sep=";")["peso"].mean()
# TypeError: Cannot perform reduction 'mean' with string dtype
```

El promedio sí protesta. La suma no. **La misma columna mal leída produce un error o un disparate según qué se le pida.**

La columna que no existe suma cero

```r
datos <- data.frame(peso = c(45, 120.5))
sum(datos$pesoo)
# [1] 0
```

`datos$pesoo` devuelve `NULL`, y la suma de nada es cero. Un cero es un resultado perfectamente creíble en un informe.

Verificación de un segundo:

```r
names(datos)
```

El promedio con faltantes

```python
df["peso"].mean()     # 82.75  -> ignoró el faltante
```

```r
mean(datos$peso)      # NA     -> se detuvo en el faltante
mean(datos$peso, na.rm = TRUE)   # 82.75
```

Los dos resultados son defendibles, pero **son decisiones distintas sobre los mismos datos**. Python la toma por defecto; R obliga a escribirla.

> **Importante**
> Ninguna de las dos es "la correcta". Lo incorrecto es no saber cuál se tomó. Esto se registra en la bitácora.

El resultado que no se asignó

```python
df.dropna()           # devuelve una tabla nueva; df queda igual
df = df.dropna()      # ahora sí
```

```r
subset(datos, !is.na(peso))          # se imprime y se pierde
datos <- subset(datos, !is.na(peso)) # ahora sí
```

Síntoma típico: "corrí la línea y no cambió nada". No falló: el resultado se calculó, se mostró y se descartó.

El reciclaje de R

```r
c(1, 2, 3, 4) + c(1, 2)
# [1] 2 4 4 6      <- repitió el vector corto, sin aviso

c(1, 2, 3) + c(1, 2)
```

```
# [1] 2 4 4
# Warning: longer object length is not a multiple of shorter object length
```

R avisa solo cuando los largos no son múltiplos. Cuando sí lo son —el caso más frecuente en la práctica— la operación se completa en silencio.

Comparar contra un faltante

```
x <- c(1, NA, 3)
x == NA
# [1] NA NA NA      <- ni TRUE ni FALSE

is.na(x)
# [1] FALSE TRUE FALSE
```

Un faltante no es igual ni distinto de nada: es desconocido. Filtrar con `== NA` devuelve una máscara de `NA` que no selecciona ninguna fila.

Parte IV. Procedimiento de diagnóstico

Cinco pasos, en orden

1. **Leer el mensaje entero** y separar las dos piezas: qué y dónde.
2. **Verificar el objeto**: si existe, qué tipo tiene, qué contiene.
3. **Reducir**: encontrar la línea mínima que reproduce el problema.
4. **Comparar** lo que hay con lo que se esperaba: tipos, cantidad de filas, faltantes.
5. **Registrar** el mensaje y la causa en la bitácora antes de seguir.

El paso 5 no es burocracia: el mismo error vuelve a aparecer tres semanas después.

Comandos de inspección

Pregunta	Python	R
¿Existe el objeto?	`"df" in globals()`	`exists("datos")`
¿Qué es?	`type(x)`	`class(x)`
¿Qué tiene adentro?	`df.info()`	`str(datos)`
¿De qué tipo es cada columna?	`df.dtypes`	`sapply(datos, class)`
¿Cuántas filas y columnas?	`df.shape`	`dim(datos)`
¿Cómo empieza?	`df.head()`	`head(datos)`
¿Cuántos faltantes hay?	`df.isna().sum()`	`colSums(is.na(datos))`
¿Qué columnas tiene?	`df.columns`	`names(datos)`

`df.info()` y `str(datos)` son los dos comandos más rentables del curso: contestan cinco preguntas a la vez.

Cómo buscar un mensaje

El mensaje contiene nombres propios que solo existen en la máquina de quien lo escribió. Buscar así no sirve:

```
KeyError: 'peso_kg_2024_final'
```

Buscar así sí:

```
pandas KeyError columna que existe en el csv
```

Regla: conservar el tipo de error y la parte genérica del mensaje; borrar los nombres de archivos, columnas y rutas.

El ejemplo mínimo reproducible

Antes de pedir ayuda —a un compañero, a un buscador o a un modelo de lenguaje— conviene reducir el problema a algo que otro pueda ejecutar:

```python
import pandas as pd
d = pd.DataFrame({"peso": ["45,0", "120,5"]})
d["peso"].mean()     # TypeError
```

Tres líneas, sin archivos externos, sin datos reales. **Muchas veces el problema aparece solo al armar este ejemplo**: la reducción es en sí misma el diagnóstico.

Parte V. Ejercicios

Ejercicio 1: opción múltiple

Una sola respuesta correcta por pregunta. Justificar en la bitácora las que la consigna indique.

Preguntas 1 a 4

1. En R, `Error in df$peso : object of type 'closure' is not subsettable` significa que:

a. El data frame `df` está vacío
b. `df` nunca fue creado y el nombre corresponde a una función de R
c. La columna `peso` no existe en `df`
d. Falta cargar el paquete que define `$`

2. En Python, `float("45,0")` se detiene con `ValueError`. En R, `as.numeric("45,0")` devuelve `NA` con una advertencia. Esto significa que:

a. R interpretó bien la coma decimal y Python no
b. Los dos fallaron en lo mismo; solo Python interrumpió el trabajo
c. R convirtió el valor y Python no pudo
d. Son dos operaciones distintas y no se pueden comparar

3. Un script de R calcula `sum(datos$pesos)` y devuelve `0`. La columna del archivo se llama `peso`. Lo que ocurrió es que:

a. Todos los pesos valen cero
b. `datos$pesos` devolvió `NULL` y la suma de nada es cero
c. R convirtió los pesos a texto
d. El archivo se leyó vacío

4. Sobre los mismos datos con un faltante, `df["peso"].mean()` devuelve `82.75` y `mean(datos$peso)` devuelve `NA`. Esto se explica porque:

a. R leyó mal el archivo
b. Python descarta los faltantes por defecto y R no

c. Los dos lenguajes calculan promedios distintos
d. Falta convertir la columna en Python

Preguntas 5 a 8

5. `ParserError: Expected 3 fields in line 3, saw 4` indica que:

a. El archivo tiene cuatro columnas y se pidieron tres
b. La fila 3 tiene un separador de más respecto del encabezado
c. El separador declarado es el equivocado
d. La fila 3 está vacía

6. `UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1` aparece porque:

a. El archivo está dañado
b. El archivo está guardado en otra codificación y contiene eñes o tildes
c. Falta instalar un paquete de idioma
d. El separador decimal es una coma

7. En R, `x <- c(10, 20, 30)` y luego `x[5]`. El resultado es:

a. Un error de índice fuera de rango
b. `NA`, sin ningún aviso
c. El último elemento, `30`
d. Un vector vacío

8. En R, `datos$pes` devuelve la columna `peso`. Esto ocurre porque:

a. R corrigió el error de tipeo
b. `$` completa nombres parciales cuando no hay ambigüedad
c. Las dos columnas existen y son iguales
d. `peso` y `pes` son el mismo nombre para R

Preguntas 9 a 12

9. Un alumno ejecuta `df.dropna()`, no aparece ningún error y `df` sigue teniendo las mismas filas. La causa es que:

a. No había faltantes en los datos
b. El resultado se calculó y no se asignó a ningún nombre
c. `dropna()` no funciona en esa versión de pandas
d. Faltaba ejecutar la celda anterior

10. `IndentationError: expected an indented block after 'if' statement on line 1` no tiene equivalente en R porque:

a. R no tiene estructuras condicionales
b. R delimita los bloques con llaves y no con espacios
c. R corrige la sangría automáticamente
d. En R la sangría también es obligatoria, pero avisa distinto

11. La consola de R muestra `+` en lugar de `>` y no responde. Lo que corresponde hacer es:

a. Reiniciar la sesión y perder los objetos creados
b. Cancelar con `Esc`: quedó una comilla o un paréntesis sin cerrar

c. Escribir `+` de nuevo para confirmar
d. Volver a instalar el paquete que se estaba usando

12. En R, `c(1, 2, 3, 4) + c(1, 2)` devuelve `2 4 4 6` sin ninguna advertencia porque:

a. R sumó solo los dos primeros elementos
b. R repitió el vector corto hasta igualar el largo, y los largos son múltiplos
c. R convirtió los dos vectores a un solo número
d. Es un error que se muestra recién al imprimir

Clave de corrección

	Correcta	El error que detecta
1	**b**	Suponer que un nombre no usado está vacío
2	**b**	Leer la advertencia de R como si fuera un éxito
3	**b**	Aceptar un cero creíble sin verificar el nombre
4	**b**	No ver que el tratamiento de faltantes es una decisión
5	b	Confundir un archivo mal formado con una mala lectura
6	b	Atribuir al archivo un problema de codificación
7	**b**	Esperar que R avise cuando se sale de rango
8	**b**	Confiar en el nombre escrito sin verificar cuál se devolvió
9	b	Suponer que toda operación modifica el objeto
10	b	No distinguir sintaxis de convención
11	b	Leer una pregunta del intérprete como un error
12	**b**	Esperar un error donde hay reciclaje silencioso

Las seis en negrita corresponden a fallas silenciosas: conviene discutirlas en clase.

Ejercicio 2: diagnóstico

Para cada síntoma, indicar **causa probable** y **el comando de una línea que la confirma**.

a. Un promedio de salarios da `NA` en R. El archivo tiene 1.200 filas y ningún salario vacío a simple vista.

b. Una tabla leída en Python tiene una sola columna, cuyo nombre es `id;fecha;monto`.

c. `df["monto"].sum()` devuelve un texto larguísimo en lugar de un número.

d. Un data frame de R se lee sin error, pero tiene una columna menos que el archivo original y los nombres de fila se ven raros.

e. El mismo script funciona en la máquina de un compañero y falla en la propia con `FileNotFoundError`.

Clave del ejercicio 2

	Causa probable	Confirmación
a	Decimales con coma: la columna es texto o quedó en `NA`	`str(datos)` o `sum(is.na(datos$salario))`
b	Separador equivocado: el archivo usa `;`	`pd.read_csv(ruta, nrows=3)` y mirar `df.columns`
c	La columna es texto y la suma concatena	`df.dtypes`
d	Alguna fila tiene un campo de más y R corrió los datos	`dim(datos)` y `head(datos)` con los nombres de fila
e	Ruta absoluta con el usuario del compañero	`getwd()` / `Path.cwd()` y `list.files()`

Ejercicio 3: para la bitácora

Provocar deliberadamente **seis errores**, tres en cada lenguaje, tomados de causas distintas del catálogo.

Para cada uno, registrar:

1. La línea que lo produjo
2. El mensaje completo, copiado tal cual
3. Las respuestas a las tres preguntas de la Parte I
4. La corrección

> **Nota**
> Un error provocado a propósito y entendido cuesta dos minutos. El mismo error encontrado durante el práctico cuesta media hora.

Resumen operativo

Si el mensaje dice…	Mirar…
`not defined` / `not found`	El orden de ejecución de las celdas
`Syntax` / `unexpected` / prompt `+`	Paréntesis y comillas sin cerrar
`non-numeric` / `TypeError`	Los tipos de las columnas
`could not convert` / `NAs introduced`	El separador decimal
`KeyError` / `NULL` / `undefined columns`	Los nombres reales de las columnas
`FileNotFoundError` / `cannot open`	El directorio de trabajo
`Expected N fields` / filas raras	El archivo crudo, en un editor de texto
Nada	Los tipos, el largo y la cantidad de faltantes

La última fila es la que exige el curso.

10. SeudoCodigo

1. Programación

- Un programa de computador es esencialmente una serie de acciones que se deben ejecutar en un orden determinado para lograr de esta forma el resultado deseado.

2. De la Lógica a la Práctica: Seudocódigo y Código

2.1 Concepto de Seudocódigo

- **Propósito:** Diseñar la lógica secuencial y el control de errores de un algoritmo antes de enfrentarse a la sintaxis rígida de un lenguaje.
- **Ventaja pedagógica:** Desconecta el razonamiento analítico de las comas, corchetes o errores sintácticos de Python o SQL.
- **Aplicación:** Sirve como mapa conceptual antes de implementar la ingesta o limpieza.

2.2 Ingesta con Control de Codificación (*Encoding*)

Seudocódigo

```
INICIO Ingesta_Datos

    IMPRIMIR "Lectura de CSV..."

    INTENTAR
        Cargar CSV en 'df_raw' (UTF-8)
    SI FALLA POR CODIFICACIÓN ENTONCES
        IMPRIMIR "[AVISO]: Error UTF-8,
                  reintentando con latin-1..."
        Cargar CSV en 'df_raw' (latin-1)
    FIN INTENTAR

FIN Ingesta_Datos
```

Codigo

```python
# 3. Ingesta de datos

import pandas as pd

print("Iniciando la lectura del archivo CSV...")

try:
    df_raw = pd.read_csv(ruta_csv, encoding="utf-8")
except UnicodeDecodeError:
    # Fallback para datos abiertos en Latin-1
    print("[AVISO]: Error con UTF-8, "
          "intentando con 'latin-1'...")
    df_raw = pd.read_csv(ruta_csv, encoding="latin-1")
```

3 Ejercicio crear código a partir de un seudo-código

Seudocódigo para crear un vector

```
INICIO Filtrar_Temperaturas

    // 1. Crear el vector con datos
    DEFINIR temperaturas = [18, 22, 15, 25, 30]

    // 2. Filtrar elementos mayores a 20
    CREAR vector_filtrado VACÍO

    PARA CADA temp EN temperaturas
        SI temp > 20 ENTONCES
            AGREGAR temp A vector_filtrado
        FIN SI
    FIN PARA

    IMPRIMIR vector_filtrado

FIN Filtrar_Temperaturas
```

Código

R

```
[1] 22 25 30
```

Python

```
[22, 25, 30]
```

11. Cargar datos: de la fuente al DataFrame

Clase teórico-práctica
Joselina Davyt-Colo

0.1 Qué cubre esta clase y qué no

El anexo de errores clasifica los mensajes que produce una lectura fallida. El práctico B ejercita el caso del CSV mal formado. Esta clase cubre lo anterior a las dos cosas: **cuántas fuentes distintas existen, qué decide cada función de lectura y qué información sobre los datos sobrevive según de dónde vengan**.

Python

Ningún bloque de este documento se ejecuta al compilar: las salidas están escritas debajo de cada bloque, como en el material de fundamentos. Para reproducirlas hay que correr una vez `generar_datos_clase.py`, que crea la carpeta `datos_clase/` con los archivos de ejemplo en los seis formatos.

> **Nota Sobre los tipos que se ven abajo**
> Las columnas de texto aparecen como `object` en pandas 2 y como `str` en pandas 3. Es el mismo tipo con dos nombres; acá se usa `object`, que es lo que trae la instalación de Anaconda de la mayoría.

R

Ningún bloque de este documento se ejecuta al compilar: las salidas están escritas debajo de cada bloque, como en el material de fundamentos. Para reproducirlas hay que correr una vez `generar_datos_clase.R`, que crea la carpeta `datos_clase/` con los archivos de ejemplo en los seis formatos.

> **Paquetes**
> ```
> install.packages(c("readr", "readxl", "haven", "DBI", "RSQLite",
>                    "jsonlite", "openxlsx"))
> ```
> `readr`, `readxl` y `haven` vienen con el tidyverse. Los otros tres se instalan por separado. `sqlite3` no hace falta: `RSQLite` trae el motor adentro.

1 Parte I. Teoría

1.1 Un archivo no es una tabla

Un archivo es una secuencia de bytes. Una tabla es una estructura con filas, columnas y un tipo por columna. Entre una cosa y la otra hay una traducción, y esa traducción no es única: el mismo archivo admite varias lecturas, todas sintácticamente válidas y solo una correcta.

Esto no es una limitación técnica que un software mejor podría resolver. Es una propiedad del problema. El archivo no contiene la instrucción de cómo leerse a sí mismo, así que la instrucción tiene que venir de otro lado: de quien lo escribió, de la documentación, o —lo más frecuente— de una hipótesis que uno formula y después

verifica.

1.2 Las cinco decisiones de toda lectura

Cada vez que se ejecuta `read_csv()` o su equivalente, la función toma cinco decisiones. Si no se declaran, las toma por defecto, en silencio y sin avisar.

#	Decisión	Si se equivoca
1	Dónde termina un campo y empieza el siguiente	Columnas de menos, o todo en una sola
2	Qué secuencias de caracteres son números	Columnas numéricas que quedan como texto
3	Qué representa un valor ausente	Faltantes que entran a los cálculos como dato
4	Cuál fila es el encabezado	Nombres de columna absurdos y una fila perdida
5	Cómo se traduce cada byte a un carácter	Eñes y tildes rotas, o un error de codificación

Python

Las cinco existen en todos los formatos, aunque en algunos vengan resueltas de fábrica. Esa es exactamente la diferencia entre formatos que veremos más adelante.

R

`readr` agrupa cuatro de las cinco en un solo objeto, `locale()`: la marca decimal, la de miles, la codificación y el formato de fecha. Esa decisión de diseño no es cosmética: dice que las convenciones de un archivo son una unidad y no cinco argumentos sueltos.

1.3 El ciclo: leer, inspeccionar, declarar, releer

De ahí se sigue el método de trabajo de esta clase, que es el mismo para cualquier fuente:

1. **Leer** con los argumentos por defecto. Esta primera lectura no sirve para analizar: sirve para ver qué supuso el programa.
2. **Inspeccionar** el resultado contra el archivo crudo: forma, tipos, primeras filas, faltantes.
3. **Declarar** explícitamente las decisiones que estaban mal.
4. **Releer** con esas decisiones escritas en el código.

La primera lectura es una hipótesis; la inspección la contrasta; la segunda lectura la corrige y la deja registrada. Lo que queda versionado en el repositorio no es el archivo: es la lectura declarada.

R

> **Importante**
> En R el paso 1 es visible. Cuando `readr` lee un archivo, imprime la especificación de columnas que adivinó. Ese mensaje no es ruido que haya que silenciar: **es la hipótesis, impresa**. Silenciarlo con `show_col_types = FALSE` antes de haberlo leído

es saltearse el paso 2.

1.4 Qué puede cargar cada formato

Los formatos se distinguen por cuánta información sobre los propios datos son capaces de transportar. Cuanto menos transporta el archivo, más hay que reconstruir a mano en la lectura.

Python

Formato	Transporta	Hay que reconstruir
CSV	Texto y nada más	Tipos, faltantes, orden de categorías, todo
Excel	Texto, algo de formato, varias hojas	Tipos reales, dónde empieza la tabla
Stata `.dta`	Tipos, etiquetas de variable y de valor	Poco
SPSS `.sav`	Tipos, etiquetas, faltantes declarados	Poco
Parquet	Tipos exactos, incluido el orden de las categóricas	Nada
Base de datos	Tipos, relaciones, restricciones	Nada, pero hay que escribir la consulta
JSON de una API	Estructura anidada	El aplanado y todos los tipos

R

Formato	Transporta	Hay que reconstruir
CSV	Texto y nada más	Tipos, faltantes, orden de los niveles, todo
Excel	Texto, algo de formato, varias hojas	Tipos reales, dónde empieza la tabla
Stata `.dta`	Tipos, etiquetas de variable y de valor	Poco
SPSS `.sav`	Tipos, etiquetas, faltantes declarados	Poco
`.rds`	El objeto de R exacto, factores incluidos	Nada
Base de datos	Tipos, relaciones, restricciones	Nada, pero hay que escribir la consulta
JSON de una API	Estructura anidada	El aplanado y todos los tipos

Importante
Un `.csv` no es un formato peor que un `.dta`: es un formato que delega. Todo lo que el `.dta` guarda adentro, el `.csv` lo deja en manos del código que lo lee. Por eso un

> `.csv` sin su diccionario de variables es un dataset incompleto.

1.5 Procedencia: de dónde vino y si se puede volver a buscar

Un dato tiene, además de contenido, un origen que hay que poder declarar. No es lo mismo un archivo que alguien bajó a mano hace seis meses que una URL escrita en el código.

Origen	Se puede reproducir	Riesgo
Archivo suelto en el escritorio	No	Nadie sabe de dónde salió ni de cuándo es
Archivo en el repositorio	Sí	Se desactualiza en silencio
URL en el código	Sí, mientras la URL viva	La fuente cambia el archivo sin avisar
API con parámetros	Sí, y con la consulta explícita	Requiere credenciales y control de versión

La regla práctica que se sigue en el curso: **descargar una vez, guardar la copia con fecha, y trabajar sobre la copia**. La descarga queda en el código para que sea reproducible; el análisis no depende de que la conexión funcione hoy.

2 Parte II. Seis fuentes

Python

Los archivos de ejemplo representan una cadena de locales comerciales con ventas mensuales por sucursal. Los crea `generar_datos_clase.py`, que hay que correr una sola vez antes de seguir.

R

Los archivos de ejemplo representan una cadena de locales comerciales con ventas mensuales por sucursal. Los crea `generar_datos_clase.R`, que hay que correr una sola vez antes de seguir.

2.1 CSV: el caso base

El CSV es el formato que más delega, y por eso el que más decisiones exige. Este archivo usa las tres convenciones habituales en datos uruguayos: punto y coma como separador, coma decimal y codificación `latin-1`.

Primera lectura: la hipótesis por defecto

Python

```python
import pandas as pd
from pathlib import Path

CARPETA = Path("datos_clase")
```

```
ingenua = pd.read_csv(CARPETA / "ventas_local.csv", encoding="latin-1")

ingenua.shape          # (6, 1)
list(ingenua.columns)  # ['id_local;sucursal;mes;ventas_uyu;empleados']
list(ingenua.index)[:2]
```

```
['101;Centro;2025-01;128500', '102;Centro;2025-02;143200']
```

Cinco columnas se convirtieron en una. Y hay algo peor que eso: la coma decimal de `128500,50` sí funcionó como separador, así que cada fila quedó partida en dos pedazos y el primero terminó usado como nombre de fila. La tabla no está incompleta, está corrida.

R

```
ingenua <- read.csv("datos_clase/ventas_local.csv")
ingenua
```

```
                           id_local.sucursal.mes.ventas_uyu.empleados
101;Centro;2025-01;128500                                       5;12
102;Centro;2025-02;143200;12
103;Pocitos;2025-01;98750                                       25;8
104;Cord\xf3n;2025-02;110430                                    75;9
105;Pocitos;2025-03;152980                                        8
106;Cord\xf3n;2025-03;121045                                     4;9
```

```
dim(ingenua)
# [1] 6 1
```

Seis líneas que muestran tres decisiones fallando a la vez:

- **El separador.** `read.csv()` supone que las columnas se separan con coma. Como el archivo usa `;`, las cinco columnas quedaron en una sola, cuyo nombre es `id_local.sucursal.mes.ventas_uyu.empleados`.
- **El decimal.** La coma de `128500,5` sí funcionó como separador, así que esa fila se partió en dos pedazos y el primero terminó de nombre de fila. Miralo bien: **la fila 2 no se partió**, porque `143200` no lleva coma. La corrupción ni siquiera es uniforme.
- **La codificación.** `Cord\xf3n` es el byte `f3` de `latin-1` mostrado en crudo, porque R esperaba UTF-8.

> **Advertencia**
>
> **Python**
> Ningún mensaje de error apareció. Una lectura con el separador equivocado es, en los dos lenguajes, una falla silenciosa: devuelve un objeto que parece un DataFrame y se puede seguir trabajando con él.

R

Ningún mensaje de error apareció. Una lectura con el separador equivocado es una falla silenciosa: devuelve un objeto que parece un data frame y se puede seguir trabajando con él. :::

Inspección

Antes de arreglar nada, mirar el archivo crudo. Son tres líneas y evitan adivinar:

Python

```
for linea in (CARPETA /
↪  "ventas_local.csv").read_text(encoding="latin-1").splitlines()[:3]:
    print(linea)
```

```
id_local;sucursal;mes;ventas_uyu;empleados
101;Centro;2025-01;128500,5;12
102;Centro;2025-02;143200,0;12
```

R

```
writeLines(head(readLines("datos_clase/ventas_local.csv", encoding = "latin1"), 3))
```

```
id_local;sucursal;mes;ventas_uyu;empleados
101;Centro;2025-01;128500,5;12
102;Centro;2025-02;143200;12
```

Ahí se leen las tres decisiones de una: el separador es `;`, el decimal es `,`, y el encabezado está en la primera fila.

Segunda lectura: la hipótesis declarada

Python

```
ventas = pd.read_csv(CARPETA / "ventas_local.csv",
                     sep=";", decimal=",", encoding="latin-1")
ventas
```

```
   id_local sucursal      mes  ventas_uyu  empleados
0       101   Centro  2025-01   128500.50         12
1       102   Centro  2025-02   143200.00         12
2       103  Pocitos  2025-01    98750.25          8
3       104   Cordón  2025-02   110430.75          9
4       105  Pocitos  2025-03   152980.00          8
5       106   Cordón  2025-03   121045.40          9
```

```
ventas.dtypes
```

```
id_local        int64
sucursal       object
mes            object
ventas_uyu    float64
empleados       int64
dtype: object
```

El mismo archivo, la misma función, tres argumentos de diferencia: seis filas por cinco columnas con `ventas_uyu` numérica, en lugar de una columna de texto.

En R:

```
library(readr)

# read_csv2() ya asume ";" como separador y "," como decimal
ventas <- read_csv2("datos_clase/ventas_local.csv",
                    locale = locale(encoding = "latin1"))

# la forma general, declarando cada decisión por separado
ventas <- read_delim("datos_clase/ventas_local.csv",
```

```
                    delim = ";",
                    locale = locale(decimal_mark = ",", encoding = "latin1"))
```

R

R base tiene una función específica para la convención europea, `read.csv2()`, que ya asume `;` y coma decimal:

```
ventas <- read.csv2("datos_clase/ventas_local.csv", fileEncoding = "latin1")
str(ventas)
```

```
'data.frame':   6 obs. of  5 variables:
 $ id_local  : int  101 102 103 104 105 106
 $ sucursal  : chr  "Centro" "Centro" "Pocitos" "Cordón" ...
 $ mes       : chr  "2025-01" "2025-02" "2025-01" "2025-02" ...
 $ ventas_uyu: num  128500 143200 98750 110431 152980 ...
 $ empleados : int  12 12 8 9 8 9
```

Con `readr` la misma lectura devuelve un **tibble**, que al imprimirse muestra el tipo de cada columna debajo del nombre. La inspección deja de ser un paso separado:

```
library(readr)

ventas <- read_csv2("datos_clase/ventas_local.csv",
                    locale = locale(encoding = "latin1"))
```

```
ℹ Using "','" as decimal and "'.'" as grouping mark.
Rows: 6 Columns: 5
── Column specification ────────────────────────────────────────
Delimiter: ";"
chr (2): sucursal, mes
dbl (3): id_local, ventas_uyu, empleados
```

Ese mensaje es la hipótesis de `readr`, escrita. Dice que adivinó dos columnas de texto y tres numéricas. Y ahí se ve el problema que queda: `mes` quedó como texto y `id_local` como número decimal.

2.1.4 La lectura declarada

R

En R la corrección no requiere un segundo paso: el tipo de cada columna se declara dentro de la misma llamada, con `col_types`.

```
ventas <- read_delim(
    "datos_clase/ventas_local.csv",
    delim  = ";",
    locale = locale(decimal_mark = ",", encoding = "latin1"),
    col_types = cols(
        id_local   = col_integer(),
        sucursal   = col_factor(levels = c("Centro", "Cordón", "Pocitos")),
        mes        = col_date(format = "%Y-%m"),
        ventas_uyu = col_double(),
        empleados  = col_integer()))
ventas
```

```
# A tibble: 6 × 5
  id_local sucursal mes        ventas_uyu empleados
     <int> <fct>    <date>          <dbl>     <int>
```

```
1      101 Centro   2025-01-01    128500.        12
2      102 Centro   2025-02-01    143200         12
3      103 Pocitos  2025-01-01     98750.         8
4      104 Cordón   2025-02-01    110431.         9
5      105 Pocitos  2025-03-01    152980          8
6      106 Cordón   2025-03-01    121045.         9
```

Una sola llamada y las cinco columnas quedaron con el tipo que corresponde: entero, factor con sus niveles en orden declarado, fecha, decimal, entero.

> **Nota**
> Comparado con Python, esta es una ventaja real de `readr`: `pandas` también acepta un argumento `dtype`, pero las categóricas ordenadas y las fechas se terminan de construir después de leer, en líneas aparte. Acá la lectura y la declaración de tipos son el mismo acto.

En Python:

```python
import pandas as pd

ventas = pd.read_csv("datos_clase/ventas_local.csv",
                     sep=";", decimal=",", encoding="latin-1")

# los tipos se terminan de declarar después de leer
ventas["sucursal"] = pd.Categorical(ventas["sucursal"],
                                    categories=["Centro", "Cordón", "Pocitos"])
ventas["mes"] = pd.PeriodIndex(ventas["mes"], freq="M")
```

2.2 Excel: dónde empieza la tabla

Una planilla no es un archivo de datos: es un documento que puede contener datos. Tiene títulos, notas al pie, celdas combinadas, varias hojas y filas en blanco de adorno. La decisión número 4 —cuál fila es el encabezado— acá es la que manda.

Python

```python
pd.read_excel(CARPETA / "ventas.xlsx").head(4)
```

```
  Reporte de ventas — uso interno Unnamed: 1 Unnamed: 2  Unnamed: 3 Unnamed: 4
0          Generado el 12/03/2025        NaN        NaN         NaN        NaN
1                             NaN        NaN        NaN         NaN        NaN
2                       id_local   sucursal        mes  ventas_uyu  empleados
3                             101     Centro    2025-01    128500.5         12
```

Las tres filas de título se leyeron como si fueran datos: los nombres de columna salieron del texto del título y quedaron `Unnamed: 1`, `Unnamed: 2`. La tabla real empieza en la cuarta fila.

R

```r
library(readxl)

read_excel("datos_clase/ventas.xlsx")
```

```
New names:
• `` -> `...2`
• `` -> `...3`
# A tibble: 9 × 5
```

```
  `Reporte de ventas — uso interno` ...2     ...3    ...4       ...5
  <chr>                             <chr>    <chr>   <chr>      <chr>
1 Generado el 12/03/2025            NA       NA      NA         NA
2 NA                                NA       NA      NA         NA
3 id_local                          sucursal mes     ventas_uyu empleados
4 101                               Centro   2025-01 128500.5   12
5 102                               Centro   2025-02 143200     12
```

Las tres filas de título se leyeron como si fueran datos: el nombre de la primera columna salió del texto del título y las otras cuatro quedaron `...2`, `...3`. Además **todas las columnas son texto**, incluidas las numéricas, porque la primera fila de datos que encontró era el encabezado real.

Un archivo de Excel además contiene varias hojas, y por defecto se lee la primera:

Python

```python
pd.ExcelFile(CARPETA / "ventas.xlsx").sheet_names
# ['ventas', 'locales']
```

R

```r
excel_sheets("datos_clase/ventas.xlsx")
# [1] "ventas"  "locales"
```

Declarando la hoja y cuántas filas saltear:

Python

```python
ventas_xl = pd.read_excel(CARPETA / "ventas.xlsx", sheet_name="ventas", skiprows=3)
ventas_xl.dtypes
```

```
id_local        int64
sucursal       object
mes            object
ventas_uyu    float64
empleados       int64
dtype: object
```

R

```r
ventas_xl <- read_excel("datos_clase/ventas.xlsx", sheet = "ventas", skip = 3)
ventas_xl
```

```
# A tibble: 6 × 5
  id_local sucursal mes     ventas_uyu empleados
     <dbl> <chr>    <chr>        <dbl>     <dbl>
1      101 Centro   2025-01    128500.        12
2      102 Centro   2025-02    143200         12
3      103 Pocitos  2025-01     98750.         8
```

> **Importante**
>
> **Python**
> Abrir un CSV con Excel para “ver cómo es” no es inocuo: Excel interpreta al abrir, convierte códigos con ceros a la izquierda en números, y fechas en otras fechas. Si después se guarda, esas conversiones quedan escritas en el archivo. Para inspeccionar un CSV se usa un editor de texto o el propio código.

En R:

```
library(readxl)

excel_sheets("datos_clase/ventas.xlsx")
# [1] "ventas"  "locales"

ventas_xl <- read_excel("datos_clase/ventas.xlsx",
                        sheet = "ventas",
                        skip  = 3)
```

R

Abrir un CSV con Excel para “ver cómo es” no es inocuo: Excel interpreta al abrir, convierte códigos con ceros a la izquierda en números, y fechas en otras fechas. Si después se guarda, esas conversiones quedan escritas en el archivo. Para inspeccionar un CSV se usa un editor de texto o el propio código. ::: En Python:

```
pd.ExcelFile("datos_clase/ventas.xlsx").sheet_names
ventas_xl = pd.read_excel("datos_clase/ventas.xlsx", sheet_name="ventas", skiprows=3)
```

2.3 Stata y SPSS: cuando el archivo trae sus etiquetas

Son los formatos de los paquetes estadísticos, y los que va a encontrar quien trabaje con microdatos de encuesta: la Encuesta Continua de Hogares del INE y la mayoría de las encuestas de institutos de investigación se publican en `.dta` o `.sav`.

Python

La diferencia con un CSV es de fondo: estos archivos guardan los tipos y además guardan **etiquetas**, que son documentación adentro del propio archivo.

```
ventas_dta = pd.read_stata(CARPETA / "ventas.dta")
ventas_dta.dtypes
```

```
id_local        int32
sucursal       object
mes            object
ventas_uyu    float64
empleados       int32
dtype: object
```

Los tipos llegaron declarados: `id_local` es entero porque en el archivo dice que es entero, no porque pandas lo haya adivinado mirando los valores.

Las etiquetas de variable se leen aparte, con el lector en modo iterador:

```
with pd.read_stata(CARPETA / "ventas.dta", iterator=True) as lector:
    etiquetas = lector.variable_labels()

etiquetas
```

```
{'id_local': '',
 'sucursal': '',
 'mes': '',
 'ventas_uyu': 'Ventas mensuales en pesos uruguayos',
 'empleados': 'Personal ocupado en el local'}
```

Eso —“Ventas mensuales en pesos uruguayos”— es exactamente lo que en el práctico A había que escribir a mano en el diccionario de variables. En un `.dta` viaja con los datos.

R

La diferencia con un CSV es de fondo: estos archivos guardan los tipos y además guardan **etiquetas**, que son documentación adentro del propio archivo. En R los dos formatos se leen con el mismo paquete.

```
library(haven)

ventas_dta <- read_dta("datos_clase/ventas.dta")
attr(ventas_dta$ventas_uyu, "label")
# [1] "Ventas mensuales en pesos uruguayos"
```

Eso —“Ventas mensuales en pesos uruguayos”— es exactamente lo que en el práctico A había que escribir a mano en el diccionario de variables. En un `.dta` viaja con los datos, y `haven` lo deja pegado a la columna como un atributo.

SPSS y las etiquetas de valor

Python

SPSS agrega un nivel más: las **etiquetas de valor**, que traducen los códigos numéricos a texto. En una encuesta la variable llega como `1`, `2`, `3` y la etiqueta dice qué significa cada número. Pandas no lee `.sav`; hace falta `pyreadstat`, que no viene con Anaconda y se instala con `pip install pyreadstat`.

```
import pyreadstat

crudo, meta = pyreadstat.read_sav(CARPETA / "ventas.sav")
crudo[["sucursal_cod", "ventas_uyu"]].head(3)
```

```
   sucursal_cod  ventas_uyu
0           1.0   128500.50
1           1.0   143200.00
2           2.0    98750.25
```

```
meta.variable_value_labels
# {'sucursal_cod': {1.0: 'Centro', 2.0: 'Pocitos', 3.0: 'Cordón'}}

legible, _ = pyreadstat.read_sav(CARPETA / "ventas.sav", apply_value_formats=True)
legible[["sucursal_cod", "ventas_uyu"]].head(3)
```

```
  sucursal_cod  ventas_uyu
0       Centro   128500.50
1       Centro   143200.00
2      Pocitos    98750.25
```

R

SPSS agrega un nivel más: las **etiquetas de valor**, que traducen los códigos numéricos a texto. En una encuesta la variable llega como `1`, `2`, `3` y la etiqueta dice qué significa cada número.

```
ventas_sav <- read_sav("datos_clase/ventas.sav")
ventas_sav[, c("sucursal_cod", "ventas_uyu")]
```

```
# A tibble: 6 × 2
  sucursal_cod ventas_uyu
  <dbl+lbl>         <dbl>
1 1 [Centro]      128500.
2 1 [Centro]      143200
3 2 [Pocitos]      98750.
4 3 [Cordón]      110431.
5 2 [Pocitos]     152980
6 3 [Cordón]      121045.
```

`<dbl+lbl>` es un tipo propio de `haven`: un número que arrastra su etiqueta. La celda muestra las dos cosas a la vez, el código y lo que significa.

```
class(ventas_sav$sucursal_cod)
# [1] "haven_labelled" "vctrs_vctr" "double"

attr(ventas_sav$sucursal_cod, "labels")
#  Centro Pocitos  Cordón
#       1       2       3
```

Para trabajar, se elige uno de los dos: `as_factor()` se queda con las etiquetas, `zap_labels()` se queda con los códigos.

```
as_factor(ventas_sav$sucursal_cod)[1:3]
# [1] Centro  Centro  Pocitos
# Levels: Centro Pocitos Cordón
```

> **Nota**
>
> **Python**
> Dos cosas para registrar. Primera: aplicar o no las etiquetas es una decisión, no un detalle de presentación —con los códigos se puede cruzar con otros archivos de la misma encuesta, con las etiquetas se puede leer una tabla— y lo que no conviene es cambiar de criterio a mitad del análisis. Segunda: los enteros volvieron como `1.0` y no como `1`, porque SPSS no distingue enteros de decimales.

En R, el paquete `haven` cubre los dos formatos y guarda las etiquetas como atributos de la columna:

```
library(haven)

ventas_dta <- read_dta("datos_clase/ventas.dta")
attr(ventas_dta$ventas_uyu, "label")
# [1] "Ventas mensuales en pesos uruguayos"

ventas_sav <- read_sav("datos_clase/ventas.sav")
ventas_sav$sucursal_cod <- as_factor(ventas_sav$sucursal_cod)   # aplica las etiquetas
```

R

Acá hay una diferencia de diseño con Python que vale la pena señalar. `haven` pega los metadatos a la columna: el dato y su documentación viajan juntos y no se pueden separar por descuido. `pyreadstat`, en Python, devuelve dos objetos —los datos por un lado y los metadatos por otro—, y mantenerlos alineados queda por cuenta de quien programa.

Lo que no cambia: aplicar o no las etiquetas es una decisión, no un detalle de

presentación. Con los códigos se puede cruzar con otros archivos de la misma encuesta; con las etiquetas se puede leer una tabla. Lo que no conviene es cambiar de criterio a mitad del análisis. ::: En Python:

```python
import pyreadstat

crudo, meta = pyreadstat.read_sav("datos_clase/ventas.sav")
meta.variable_value_labels
# {'sucursal_cod': {1.0: 'Centro', 2.0: 'Pocitos', 3.0: 'Cordón'}}

legible, _ = pyreadstat.read_sav("datos_clase/ventas.sav", apply_value_formats=True)
```

2.4 Una URL es una ruta más

Las funciones de lectura aceptan una dirección web donde aceptan una ruta. No hay nada especial que aprender:

Python

```python
url = "https://ejemplo.gub.uy/datos/serie.csv"
serie = pd.read_csv(url, sep=";", decimal=",", encoding="latin-1")
```

```r
serie <- readr::read_csv2("https://ejemplo.gub.uy/datos/serie.csv")
```

R

```r
serie <- read_csv2("https://ejemplo.gub.uy/datos/serie.csv")
```

```python
serie = pd.read_csv("https://ejemplo.gub.uy/datos/serie.csv",
                    sep=";", decimal=",", encoding="latin-1")
```

Lo que sí hay que aprender es qué no hacer con eso. Leer directo de la URL cada vez que se corre el análisis tiene tres problemas: el análisis deja de funcionar sin conexión, la fuente puede cambiar el archivo sin avisar, y dos corridas del mismo código en fechas distintas pueden dar resultados distintos sin que quede registro.

El patrón que resuelve las tres cosas es descargar una vez y fijar la copia:

Python

```python
from datetime import date
import urllib.request

url = "https://ejemplo.gub.uy/datos/serie.csv"
destino = CARPETA / f"serie_{date.today():%Y-%m-%d}.csv"

if not destino.exists():                    # solo la primera vez
    urllib.request.urlretrieve(url, destino)

serie = pd.read_csv(destino, sep=";", decimal=",", encoding="latin-1")
```

R

```r
url     <- "https://ejemplo.gub.uy/datos/serie.csv"
destino <- file.path("datos_clase", paste0("serie_", Sys.Date(), ".csv"))

if (!file.exists(destino)) {              # solo la primera vez
    download.file(url, destino, mode = "wb")
}
```

```
serie <- read_csv2(destino, locale = locale(encoding = "latin1"))
```

`mode = "wb"` no es opcional en Windows: sin eso, R abre la descarga en modo texto y puede alterar los saltos de línea de un archivo binario.

El nombre del archivo lleva la fecha de descarga; el código de descarga queda versionado; el análisis corre sobre la copia. La procedencia queda declarada y el resultado es reproducible.

2.5 Base de datos: la consulta es parte de la lectura

Python

Con un archivo se lee todo y se filtra después. Con una base de datos se decide cuánto traer, y esa decisión se escribe en SQL. El módulo `sqlite3` viene con Python, así que no hay nada que instalar.

```
import sqlite3

con = sqlite3.connect(CARPETA / "comercio.db")
pd.read_sql("SELECT name FROM sqlite_master WHERE type = 'table'", con)
```

```
      name
0   ventas
1  locales
```

R

Con un archivo se lee todo y se filtra después. Con una base de datos se decide cuánto traer, y esa decisión se escribe en SQL. `RSQLite` trae el motor de la base adentro del paquete, así que no hay que instalar nada más.

```
library(DBI)

con <- dbConnect(RSQLite::SQLite(), "datos_clase/comercio.db")
dbListTables(con)
# [1] "locales" "ventas"
```

Traer una tabla entera es lo mismo que leer un archivo. Pero también se puede pedir que el trabajo lo haga la base y que llegue el resultado ya agregado, cruzando dos tablas:

Python

```
consulta = """
    SELECT v.sucursal,
           l.region,
           SUM(v.ventas_uyu) AS total_ventas,
           COUNT(*)          AS meses
    FROM ventas v
    JOIN locales l ON v.sucursal = l.sucursal
    GROUP BY v.sucursal, l.region
    ORDER BY total_ventas DESC
"""
resumen = pd.read_sql(consulta, con)
con.close()
resumen
```

```
  sucursal  region  total_ventas  meses
0   Centro     Sur     271700.50      2
1  Pocitos     Sur     251730.25      2
2   Cordón  Centro     231476.15      2
```

R

```
resumen <- dbGetQuery(con, "
    SELECT v.sucursal,
           l.region,
           SUM(v.ventas_uyu) AS total_ventas,
           COUNT(*)          AS meses
    FROM ventas v
    JOIN locales l ON v.sucursal = l.sucursal
    GROUP BY v.sucursal, l.region
    ORDER BY total_ventas DESC")

dbDisconnect(con)
resumen
```

```
  sucursal region total_ventas meses
1   Centro    Sur     271700.5     2
2  Pocitos    Sur     251730.2     2
3   Cordón Centro     231476.1     2
```

> **Nota**
>
> **Python**
> Acá aparece una decisión que en un CSV no existe: **cuánto del análisis ocurre antes de que los datos lleguen**. Con tablas grandes la respuesta suele ser "bastante", porque traer diez millones de filas para después promediarlas es caro y lento. Pero cada agregación hecha en SQL es una decisión analítica que queda fuera del script de análisis, y hay que documentarla igual.

En R:

```
library(DBI)

con <- dbConnect(RSQLite::SQLite(), "datos_clase/comercio.db")
dbListTables(con)

resumen <- dbGetQuery(con, "
    SELECT v.sucursal, SUM(v.ventas_uyu) AS total_ventas
    FROM ventas v GROUP BY v.sucursal")

dbDisconnect(con)
```

R

Acá aparece una decisión que en un CSV no existe: **cuánto del análisis ocurre antes de que los datos lleguen**. Con tablas grandes la respuesta suele ser "bastante", porque traer diez millones de filas para después promediarlas es caro y lento. Pero cada agregación hecha en SQL es una decisión analítica que queda fuera del script de análisis, y hay que documentarla igual.

`dbplyr` permite escribir esa consulta con verbos de `dplyr` y que se traduzca sola a SQL. Es cómodo, y tiene el mismo riesgo: la agregación sigue ocurriendo del lado de la base.

::: En Python:

```
import sqlite3

con = sqlite3.connect("datos_clase/comercio.db")
resumen = pd.read_sql(consulta, con)
con.close()
```

2.6 API: del JSON anidado a la tabla

Una API devuelve JSON, que es una estructura de árbol y no una tabla. La dificultad no está en la conexión sino en el aplanado: hay que decidir qué nivel del árbol corresponde a una fila.

Así se ve una respuesta típica:

```
{
  "meta": {
    "fuente": "sistema interno",
    "actualizado": "2025-03-31"
  },
  "resultados": [
    {
      "local": {"id": 101, "sucursal": "Centro"},
      "periodo": "2025-01",
      "ventas": 128500.5
    },
    {
      "local": {"id": 103, "sucursal": "Pocitos"},
      "periodo": "2025-01",
      "ventas": 98750.25
    }
  ]
}
```

Python

Los datos no están en la raíz: están adentro de `resultados`, y cada resultado tiene a su vez un objeto `local` anidado. Convertir la lista sin más deja el diccionario metido dentro de una celda:

```
import json

crudo = json.loads((CARPETA / "respuesta_api.json").read_text(encoding="utf-8"))
pd.DataFrame(crudo["resultados"])
```

```
                              local  periodo     ventas
0   {'id': 101, 'sucursal': 'Centro'}  2025-01  128500.50
1  {'id': 103, 'sucursal': 'Pocitos'}  2025-01   98750.25
```

La columna `local` contiene objetos, no valores. `json_normalize()` aplana el árbol y arma un nombre de columna por camino:

```
pd.json_normalize(crudo["resultados"], sep="_")
```

```
   periodo     ventas  local_id local_sucursal
0  2025-01  128500.50       101         Centro
1  2025-01   98750.25       103        Pocitos
```

R

Los datos no están en la raíz: están adentro de `resultados`, y cada resultado tiene a su vez un objeto `local` anidado. `jsonlite` convierte lo que puede a data frame, pero el nivel anidado queda como un data frame adentro de otro:

```r
library(jsonlite)

crudo <- fromJSON("datos_clase/respuesta_api.json")
str(crudo, max.level = 2)
```

```
List of 2
 $ meta      :List of 2
  ..$ fuente     : chr "sistema interno"
  ..$ actualizado: chr "2025-03-31"
 $ resultados:'data.frame': 2 obs. of  3 variables:
  ..$ local  :'data.frame': 2 obs. of  2 variables:
  ..$ periodo: chr [1:2] "2025-01" "2025-01"
  ..$ ventas : num [1:2] 128500 98750
```

La columna `local` **es un data frame**, no un vector. Se imprime como si fuera normal, pero no lo es: `ncol()` devuelve 3 y no 4. Con `flatten = TRUE` el árbol se aplana de verdad:

```r
datos <- fromJSON("datos_clase/respuesta_api.json", flatten = TRUE)$resultados
datos
```

```
  periodo    ventas local.id local.sucursal
1 2025-01 128500.50      101         Centro
2 2025-01  98750.25      103        Pocitos
```

La llamada real a la API:

Python

```python
import requests

respuesta = requests.get("https://api.ejemplo.uy/ventas",
                         params={"desde": "2025-01", "hasta": "2025-03"},
                         timeout=30)
respuesta.raise_for_status()          # corta acá si el servidor devolvió un error

datos = pd.json_normalize(respuesta.json()["resultados"], sep="_")
```

```r
library(httr2)
library(jsonlite)

datos <- request("https://api.ejemplo.uy/ventas") |>
    req_url_query(desde = "2025-01", hasta = "2025-03") |>
    req_perform() |>
    resp_body_string() |>
    fromJSON(flatten = TRUE)
```

R

```r
library(httr2)

datos <- request("https://api.ejemplo.uy/ventas") |>
    req_url_query(desde = "2025-01", hasta = "2025-03") |>
```

```
    req_perform() |>
    resp_body_string() |>
    fromJSON(flatten = TRUE)
```

> **Advertencia**
>
> **Python**
>
> `raise_for_status()` no es un detalle. Sin esa línea, un error del servidor —que suele venir con su propio JSON de error— entra al análisis como si fuera un conjunto de datos vacío o distinto, sin producir ninguna excepción.

R

`req_perform()` corta con error si el servidor responde 4xx o 5xx, que es lo que uno quiere. Si en cambio se usa `fromJSON(url)` directamente, un error del servidor —que suele venir con su propio JSON de error— entra al análisis como si fuera un conjunto de datos distinto, sin producir ninguna excepción. ::: En Python:

```
import requests

respuesta = requests.get("https://api.ejemplo.uy/ventas",
                         params={"desde": "2025-01", "hasta": "2025-03"},
                         timeout=30)
respuesta.raise_for_status()          # esta línea no es opcional

datos = pd.json_normalize(respuesta.json()["resultados"], sep="_")
```

3 Parte III. La lectura declarada, encapsulada

Las lecturas corregidas de esta clase tienen algo en común: son varias líneas con argumentos que hay que acertar, y se van a repetir en cada script del proyecto. Copiarlas es garantía de que en algún momento van a quedar desalineadas.

La forma de que la lectura sea una sola decisión tomada una vez es encerrarla en una función que devuelva el dataset ya tipado:

Python

```
ORDEN_SUCURSAL = ["Centro", "Cordón", "Pocitos"]

def cargar_ventas(ruta=CARPETA / "ventas_local.csv"):
    """Lee el archivo de ventas con todas las decisiones declaradas."""
    datos = pd.read_csv(ruta, sep=";", decimal=",", encoding="latin-1")
    datos["sucursal"] = pd.Categorical(datos["sucursal"],
                                       categories=ORDEN_SUCURSAL, ordered=False)
    datos["mes"] = pd.PeriodIndex(datos["mes"], freq="M")
    return datos

ventas = cargar_ventas()
ventas.dtypes
```

```
id_local          int64
sucursal       category
mes           period[M]
ventas_uyu      float64
```

```
empleados         int64
dtype: object
```

```
ventas.groupby("sucursal", observed=True)["ventas_uyu"].sum()
```

```
sucursal
Centro     271700.50
Cordón     231476.15
Pocitos    251730.25
Name: ventas_uyu, dtype: float64
```

Cualquier script del proyecto hace `from carga import cargar_ventas` y obtiene el mismo dataset, con los mismos tipos, sin repetir un solo argumento. Si mañana la fuente cambia el separador, se corrige en un lugar.

R

```
NIVELES_SUCURSAL <- c("Centro", "Cordón", "Pocitos")

cargar_ventas <- function(ruta = "datos_clase/ventas_local.csv") {
    read_delim(
        ruta,
        delim  = ";",
        locale = locale(decimal_mark = ",", encoding = "latin1"),
        col_types = cols(
            id_local   = col_integer(),
            sucursal   = col_factor(levels = NIVELES_SUCURSAL),
            mes        = col_date(format = "%Y-%m"),
            ventas_uyu = col_double(),
            empleados  = col_integer()))
}

ventas <- cargar_ventas()
```

Cualquier script del proyecto hace `source("carga.R")` y obtiene el mismo dataset, con los mismos tipos, sin repetir un solo argumento. Si mañana la fuente cambia el separador, se corrige en un lugar.

3.1 Resumen

Python

Fuente	Python	R	Lo que hay que declarar
CSV	`pd.read_csv`	`readr::read_delim`	Separador, decimal, codificación, faltantes
Excel	`pd.read_excel`	`readxl::read_excel`	Hoja y dónde empieza la tabla
Stata	`pd.read_stata`	`haven::read_dta`	Casi nada; las etiquetas se leen aparte
SPSS	`pyreadstat.read_sav`	`haven::read_sav`	Si se aplican o no las etiquetas de valor
URL	La misma función, con la dirección	Igual	Cuándo se descargó y dónde quedó la copia
Base de datos	`pd.read_sql`	`DBI::dbGetQuery`	La consulta: qué traer y qué agregar antes

Fuente	Python	R	Lo que hay que declarar
API	`pd.json_normalize`	`jsonlite::fromJSON`	Qué nivel del árbol es una fila

R

Fuente	R	Python	Lo que hay que declarar
CSV	`readr::read_delim`	`pd.read_csv`	Separador, decimal, codificación, faltantes
Excel	`readxl::read_excel`	`pd.read_excel`	Hoja y dónde empieza la tabla
Stata	`haven::read_dta`	`pd.read_stata`	Casi nada; la etiqueta viene como atributo
SPSS	`haven::read_sav`	`pyreadstat.read_sav`	Si se aplican o no las etiquetas de valor
URL	La misma función, con la dirección	Igual	Cuándo se descargó y dónde quedó la copia
Base de datos	`DBI::dbGetQuery`	`pd.read_sql`	La consulta: qué traer y qué agregar antes
API	`jsonlite::fromJSON`	`pd.json_normalize`	Qué nivel del árbol es una fila

Las tres cosas que hay que llevarse:

Python

1. La primera lectura es una hipótesis, no un resultado. Se inspecciona antes de usarla.
2. El formato determina cuánto hay que reconstruir a mano. Un CSV no es peor que un `.dta`: es un formato que delega, y hay que hacerse cargo de lo delegado.
3. La lectura declarada se escribe una vez, en una función, y se versiona. Es parte del análisis, no un paso previo al análisis.

R

1. La primera lectura es una hipótesis, no un resultado. En R esa hipótesis se imprime sola: hay que leerla antes de silenciarla.
2. El formato determina cuánto hay que reconstruir a mano. Un CSV no es peor que un `.dta`: es un formato que delega, y hay que hacerse cargo de lo delegado.
3. La lectura declarada se escribe una vez, con `col_types`, y se versiona. Es parte del análisis, no un paso previo al análisis.

12. Carga de datos

Joselina Davyt

Espacio de trabajo y organización de proyectos de ciencia de datos

- Una buena organización de archivos es la base de cualquier proyecto profesional
- Estructura de directorios: mantener una estructura clara e independiente del sistema:

data/raw/ (datos originales e inmutables)

data/processed/ (limpios)

notebooks/ o src/ o scripts (código)

outputs/ o resultados (reportes y modelos)

Rutas

- ¿qué son las rutas?: definen en qué carpeta y archivos estoy trabajando en el proyecto de ciencia de datos
- Preferible usar rutas relativas y no absolutas, porque las rutas absolutas impiden la reproducibilidad de los proyectos.
- Ruta relativa: “data/raw/”
- Ruta absoluta: “C:/Users/…”
- Las rutas son texto (=string/=character) para los lenguajes de programación
- Existen paquetes y funciones que interpretan la ruta como tal

Trabajar con rutas

R

```
getwd() # al ejecutarlo me dice en dónde está parado el entorno ahora
```

```
[1] "C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-datos/clases/clase4"
```

```
# sirve para fijar la ruta del entorno o espacio de trabajo, genera rutas absolutas no
 ↪  relativas
setwd("C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-
 ↪  datos/clases/clase4")
 ↪
```

Python

Antes, si estas trabajando dentro de RStudio ejecuta (dentro de VS no es necesario)

```
reticulate::py_require(c("pandas", "numpy", "matplotlib"))
```

En python permite trabajar con rutas absolutas y relativas:

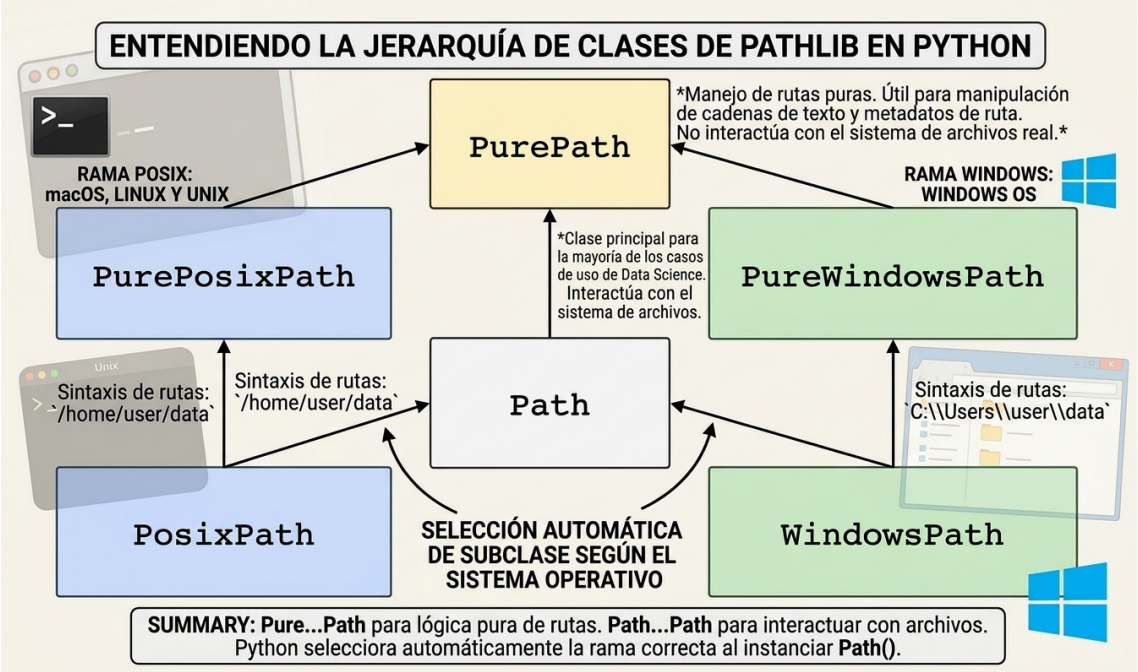
```
import os
```

```
# es el equivalente a getwd() de R
os.getcwd()
# Subir un nivel en el árbol de carpetas (equivalente a 'cd ..')
os.chdir("..")
os.getcwd()
```

```
# Entrar a una subcarpeta desde el directorio actual
os.chdir("clases/clase4")
# ruta absoluta:
os.chdir("C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-
↪  datos/clases/clase3")
```

Trabajar con rutas: el enfoque moderno

Usar con paquetes para trabajar desde la raíz del proyecto y sus subcarpetas.



R

```
# 1 - instalar paquete
#install.packages("here")
# 2- cargar paquete
library(here)
```

Python

```
# 1 - instalar paquete
#pip install pathlib
# 2 - cargar paquete
from pathlib import Path
# listado de subdirectorios
p = Path('.')
```

Carga de csv

Fuente: Catalogo de datos abiertor Uy

Leer csv

R

```
library(here)
library(readr)

# 1. Construir la ruta relativa hacia el archivo CSV
# Estructura esperada: tu_proyecto/data/raw/datos.csv
archivo <- "cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv"
ruta_csv <- here(archivo)
ruta_csv <- here("clases", "clase4", archivo)
ruta_csv
```

```
[1] "C:/Users/jodavyt/Documents/GitHub/Introduccion-a-la-ciencia-de-datos/clases/clase4
↪  /cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv"
```

```
# 2. Verificar que el archivo exista
if (!file.exists(ruta_csv)) {
  stop(paste("No se encontró el archivo en:", ruta_csv))
}

# 3. Cargar el archivo CSV
df <- read_csv(ruta_csv)

# 4. Inspección inicial de los datos
cat("--- Archivo cargado exitosamente desde:", basename(ruta_csv), "---\n")
```

```
--- Archivo cargado exitosamente desde:
↪  cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv ---
```

```
print(spec(df))  # Muestra el tipo de dato asignado a cada columna
```

```
cols(
  pesada = col_double(),
  fecha_dia = col_character(),
  fecha = col_character(),
  matricula_letra = col_character(),
  tipo_residuo = col_double(),
  tipo_residuo_desc = col_character(),
```

```
  subtipo_residuo = col_double(),
  subtipo_residuo_desc = col_character(),
  peso_neto = col_double(),
  peso_bruto = col_double(),
  tara = col_double(),
  balanza = col_double(),
  medio_transp = col_double(),
  medio_transp_desc = col_character(),
  trayecto = col_logical(),
  trayecto_desc = col_logical(),
  tipo_recorrido = col_logical(),
  tipo_recorrido_desc = col_logical(),
  tipo_volcado_desc = col_character(),
  nomenclatura_circuito = col_logical(),
  municipio = col_logical(),
  lugar_salida = col_logical(),
  lugar_salida_desc = col_logical(),
  permiso = col_logical(),
  nro_permiso = col_double(),
  permiso_id = col_double(),
  tipo_permiso = col_double(),
  tipo_permiso_desc = col_character(),
  peso_autorizado = col_logical(),
  destino_volcado = col_double(),
  destino_volcado_desc = col_character(),
  tipo_pesada = col_double(),
  tipo_pesada_desc = col_character(),
  basura_propia = col_character(),
  fmod = col_character(),
  viaje_fecha = col_character(),
  fecha_tara = col_character(),
  sitio = col_double(),
  sitio_desc = col_character(),
  tipo_movimiento = col_double(),
  tipo_movimiento_desc = col_character()
)
```

```
head(df)
```

```
# A tibble: 6 × 41
   pesada fecha_dia      fecha    matricula_letra tipo_residuo tipo_residuo_desc
    <dbl> <chr>          <chr>    <chr>                  <dbl> <chr>
1 3996241 30/11/23 00:00 30/11/2… SIM                        1 TIPO DOMICILIARIO
2 3996243 30/11/23 00:00 30/11/2… SIM                        1 TIPO DOMICILIARIO
3 3996253 30/11/23 00:00 30/11/2… SIM                        1 TIPO DOMICILIARIO
4 3996276 30/11/23 00:00 30/11/2… SIM                        1 TIPO DOMICILIARIO
5 3996298 30/11/23 00:00 30/11/2… SIM                        1 TIPO DOMICILIARIO
6 3996303 30/11/23 00:00 30/11/2… SIM                        1 TIPO DOMICILIARIO
# ℹ 35 more variables: subtipo_residuo <dbl>, subtipo_residuo_desc <chr>,
#   peso_neto <dbl>, peso_bruto <dbl>, tara <dbl>, balanza <dbl>,
#   medio_transp <dbl>, medio_transp_desc <chr>, trayecto <lgl>,
#   trayecto_desc <lgl>, tipo_recorrido <lgl>, tipo_recorrido_desc <lgl>,
#   tipo_volcado_desc <chr>, nomenclatura_circuito <lgl>, municipio <lgl>,
#   lugar_salida <lgl>, lugar_salida_desc <lgl>, permiso <lgl>,
#   nro_permiso <dbl>, permiso_id <dbl>, tipo_permiso <dbl>, …
```

Python

```python
from pathlib import Path
import pandas as pd

# 0. nombre archivo
archivo = "cantidad_de_residuos_en_la_estacion_de_transferencia_2023.csv"

# 1. Definir la ruta raíz del proyecto o del directorio de trabajo actual
BASE_DIR = Path.cwd()

# 2. Construir la ruta relativa de forma segura utilizando el operador /
# Estructura esperada: tu_proyecto/data/raw/datos.csv
ruta_csv = BASE_DIR / "clases" / "clase4" / archivo

# 3. Verificar que el archivo realmente existe antes de cargarlo
if not ruta_csv.exists():
    raise FileNotFoundError(f"No se encontró el archivo en: {ruta_csv}")

# 4. Cargar el archivo CSV
df = pd.read_csv(ruta_csv)

# 5. Inspección inicial de los datos
print(f"--- Archivo cargado exitosamente desde: {ruta_csv.name} ---")
print(df.info())
print("\nPrimeras 5 filas:")
print(df.head())
```

Data profiling/Inspección de datos/Reconocimiento de datos

Actividades esenciales:

- Evaluar el tamaño del archivo y las necesidades de procesamiento
- Evaluar si el caso a analizar está completo con los datos obtenidos: filas (unidades de observación) y columnas (características o variables)
- Confirmar el tipo de datos obtenidos
- Identificar el delimitador (separación entre columnas)
- Revisar inconsistencias en los nombres de las variables
- Esta etapa debe culminar con un código definitivo de carga de datos (tipo de conexión, tipo de archivos, tipos de datos, completitud)

Unidad de observación

- La unidad de observación refiere al objeto de estudio
- Permite identidicar las columnas IDs por las cuales se podrían unir datos
- Ejemplos video juegos:
  - Nivel Acción. **Fila:** Evento atómico (*Acción dentro del juego*), ejemplo event_id.
  - Nivel Partida. **Fila:** Partida (*Match*) o Jugador-Partida (*Player-Match*), ejemplo campos: `match_id`, `player_id`, `kills`, `deaths`, `win_bool`.
  - Nivel jugador. **Fila:** Un Jugador único (*Perfil Agregado*), por ejemplo calcular predicción de abandono,retención a 7/30 días y gasto acumulado. **Campo:** `player_id`

- Nivel título. **Fila:** Un videojuego o título (*App Level*). **Objetivo:** Estudio de catálogo, estrategia de precios y competencia por género. **Campos:** `app_id`, `title`, `genre`, `price`, `peak_concurrent_users`.

Problemas de identificación de datos e integración

- Verificar consistencia de formato de IDs, ¿por qué? porque son las variables mediante las cuales se unen distintas tablas de datos.
- Verificar duplicados
- Distintas fechas de generación de registros, por ejemplo: la solicitud de un tratamiento se realiza en diciembre de 2025 y se realiza en 2026
- Duplicados: más de un registro por IDs, ¿es necesario o se quiere mantener
- Solución: comenzar con una lista de los NO SÉ.

Data profiling: identificador

R

```r
# dimensiones y estructura de los datos
str(df) # 1222 filas y 41 columnas
```

```
spc_tbl_ [1,222 × 41] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ pesada               : num [1:1222] 4e+06 4e+06 4e+06 4e+06 4e+06 ...
 $ fecha_dia            : chr [1:1222] "30/11/23 00:00" "30/11/23 00:00" "30/11/23 00:00"
  ↪  "30/11/23 00:00" ...
 $ fecha                : chr [1:1222] "30/11/23 08:28" "30/11/23 08:30" "30/11/23 08:36"
  ↪  "30/11/23 08:56" ...
 $ matricula_letra      : chr [1:1222] "SIM" "SIM" "SIM" "SIM" ...
 $ tipo_residuo         : num [1:1222] 1 1 1 1 1 1 1 1 1 1 ...
 $ tipo_residuo_desc    : chr [1:1222] "TIPO DOMICILIARIO" "TIPO DOMICILIARIO" "TIPO
  ↪  DOMICILIARIO" "TIPO DOMICILIARIO" ...
 $ subtipo_residuo      : num [1:1222] 1 1 1 1 1 1 1 1 1 1 ...
 $ subtipo_residuo_desc : chr [1:1222] "ASIMILABLE A DOMICILIARIO" "ASIMILABLE A
  ↪  DOMICILIARIO" "ASIMILABLE A DOMICILIARIO" "ASIMILABLE A DOMICILIARIO" ...
 $ peso_neto            : num [1:1222] 11590 8660 6090 7920 9740 ...
 $ peso_bruto           : num [1:1222] 28450 26070 25020 24720 26890 ...
 $ tara                 : num [1:1222] 16860 17410 18930 16800 17150 ...
 $ balanza              : num [1:1222] 5 5 5 5 5 5 5 5 5 5 ...
 $ medio_transp         : num [1:1222] 1 1 50 1 1 50 1 1 1 50 ...
 $ medio_transp_desc    : chr [1:1222] "COMPACTADOR LEVANTE LAT." "COMPACTADOR LEVANTE
  ↪  LAT." "COMPACTADOR LAT.CAJA DESM" "COMPACTADOR LEVANTE LAT." ...
 $ trayecto             : logi [1:1222] NA NA NA NA NA NA ...
 $ trayecto_desc        : logi [1:1222] NA NA NA NA NA NA ...
 $ tipo_recorrido       : logi [1:1222] NA NA NA NA NA NA ...
 $ tipo_recorrido_desc  : logi [1:1222] NA NA NA NA NA NA ...
 $ tipo_volcado_desc    : chr [1:1222] "SIN PERMISO" "SIN PERMISO" "SIN PERMISO" "SIN
  ↪  PERMISO" ...
 $ nomenclatura_circuito: logi [1:1222] NA NA NA NA NA NA ...
 $ municipio            : logi [1:1222] NA NA NA NA NA NA ...
 $ lugar_salida         : logi [1:1222] NA NA NA NA NA NA ...
 $ lugar_salida_desc    : logi [1:1222] NA NA NA NA NA NA ...
 $ permiso              : logi [1:1222] NA NA NA NA NA NA ...
 $ nro_permiso          : num [1:1222] NA NA NA NA NA NA NA NA NA NA ...
 $ permiso_id           : num [1:1222] NA NA NA NA NA NA NA NA NA NA ...
 $ tipo_permiso         : num [1:1222] NA NA NA NA NA NA NA NA NA NA ...
```

```
$ tipo_permiso_desc    : chr [1:1222] NA NA NA NA ...
$ peso_autorizado      : logi [1:1222] NA NA NA NA NA NA ...
$ destino_volcado      : num [1:1222] 34 34 34 34 34 34 34 34 34 34 ...
$ destino_volcado_desc : chr [1:1222] "ETRA (Estacion de Transferencia)" "ETRA (Estacion
 ↪  de Transferencia)" "ETRA (Estacion de Transferencia)" "ETRA (Estacion de
 ↪  Transferencia)" ...
$ tipo_pesada          : num [1:1222] 1 1 1 1 1 1 1 1 1 1 ...
$ tipo_pesada_desc     : chr [1:1222] "AUTOMATICA" "AUTOMATICA" "AUTOMATICA"
 ↪  "AUTOMATICA" ...
$ basura_propia        : chr [1:1222] "NO" "NO" "NO" "NO" ...
$ fmod                 : chr [1:1222] "16/01/24 10:41" "16/01/24 10:41" "16/01/24 10:41"
 ↪  "16/01/24 10:41" ...
$ viaje_fecha          : chr [1:1222] "29/11/23 00:00" "29/11/23 00:00" "30/11/23 00:00"
 ↪  "30/11/23 00:00" ...
$ fecha_tara           : chr [1:1222] "30/11/23 08:39" "30/11/23 08:40" "30/11/23 08:53"
 ↪  "30/11/23 09:15" ...
$ sitio                : num [1:1222] 2 2 2 2 2 2 2 2 2 2 ...
$ sitio_desc           : chr [1:1222] "ETRA" "ETRA" "ETRA" "ETRA" ...
$ tipo_movimiento      : num [1:1222] 1 1 1 1 1 1 1 1 1 1 ...
$ tipo_movimiento_desc : chr [1:1222] "DESCARGA" "DESCARGA" "DESCARGA" "DESCARGA" ...
- attr(*, "spec")=
 .. cols(
 ..   pesada = col_double(),
 ..   fecha_dia = col_character(),
 ..   fecha = col_character(),
 ..   matricula_letra = col_character(),
 ..   tipo_residuo = col_double(),
 ..   tipo_residuo_desc = col_character(),
 ..   subtipo_residuo = col_double(),
 ..   subtipo_residuo_desc = col_character(),
 ..   peso_neto = col_double(),
 ..   peso_bruto = col_double(),
 ..   tara = col_double(),
 ..   balanza = col_double(),
 ..   medio_transp = col_double(),
 ..   medio_transp_desc = col_character(),
 ..   trayecto = col_logical(),
 ..   trayecto_desc = col_logical(),
 ..   tipo_recorrido = col_logical(),
 ..   tipo_recorrido_desc = col_logical(),
 ..   tipo_volcado_desc = col_character(),
 ..   nomenclatura_circuito = col_logical(),
 ..   municipio = col_logical(),
 ..   lugar_salida = col_logical(),
 ..   lugar_salida_desc = col_logical(),
 ..   permiso = col_logical(),
 ..   nro_permiso = col_double(),
 ..   permiso_id = col_double(),
 ..   tipo_permiso = col_double(),
 ..   tipo_permiso_desc = col_character(),
 ..   peso_autorizado = col_logical(),
 ..   destino_volcado = col_double(),
 ..   destino_volcado_desc = col_character(),
 ..   tipo_pesada = col_double(),
 ..   tipo_pesada_desc = col_character(),
 ..   basura_propia = col_character(),
```

```
  ..   fmod = col_character(),
  ..   viaje_fecha = col_character(),
  ..   fecha_tara = col_character(),
  ..   sitio = col_double(),
  ..   sitio_desc = col_character(),
  ..   tipo_movimiento = col_double(),
  ..   tipo_movimiento_desc = col_character()
  .. )
 - attr(*, "problems")=<pointer: 0x0000022df6de7700>
```

```r
# filas únicas: detectando la columna que contiene el ID
length(unique(df$matricula_letra)) #¿cuál es la columna identificador?
```

```
[1] 12
```

Python

```python
import pandas as pd

# 1. Dimensiones y estructura de los datos (Equivalente a str(df))
df.info()
# Nota: df.shape te da exactamente las dimensiones (filas, columnas) -> (1222, 41)

# 2. Filas únicas: detectando la columna que contiene el ID (Equivalente a
↪  length(unique(...)))
df["matricula_letra"].nunique()
```

Data profiling 2: valores faltantes

R

```r
colSums(is.na(df)) # Conteo de NA por columna
```

```
               pesada             fecha_dia                 fecha
                    0                     0                     0
      matricula_letra          tipo_residuo     tipo_residuo_desc
                    0                     0                     0
      subtipo_residuo  subtipo_residuo_desc             peso_neto
                    0                     0                     0
          peso_bruto                  tara               balanza
                    0                     0                     0
         medio_transp     medio_transp_desc              trayecto
                    0                     0                  1222
        trayecto_desc        tipo_recorrido   tipo_recorrido_desc
                 1222                  1222                  1222
    tipo_volcado_desc nomenclatura_circuito             municipio
                    0                  1222                  1222
         lugar_salida     lugar_salida_desc               permiso
                 1222                  1222                  1222
          nro_permiso            permiso_id          tipo_permiso
                  828                   828                   828
    tipo_permiso_desc       peso_autorizado       destino_volcado
                  828                  1222                     0
 destino_volcado_desc           tipo_pesada      tipo_pesada_desc
                    0                     0                     0
        basura_propia                  fmod           viaje_fecha
                    0                     0                   437
           fecha_tara                 sitio            sitio_desc
```

```
                   0                    0                    0
      tipo_movimiento  tipo_movimiento_desc
                   0                    0
```

```
sapply(df, function(x) sum(x == "" | x == "N/A", na.rm = TRUE)) # Falsos nulos
```

```
               pesada            fecha_dia                fecha
                    0                    0                    0
      matricula_letra         tipo_residuo    tipo_residuo_desc
                    0                    0                    0
      subtipo_residuo  subtipo_residuo_desc            peso_neto
                    0                    0                    0
          peso_bruto                 tara              balanza
                    0                    0                    0
         medio_transp    medio_transp_desc             trayecto
                    0                    0                    0
        trayecto_desc       tipo_recorrido  tipo_recorrido_desc
                    0                    0                    0
    tipo_volcado_desc nomenclatura_circuito            municipio
                    0                    0                    0
         lugar_salida    lugar_salida_desc              permiso
                    0                    0                    0
          nro_permiso           permiso_id         tipo_permiso
                    0                    0                    0
    tipo_permiso_desc      peso_autorizado      destino_volcado
                    0                    0                    0
 destino_volcado_desc          tipo_pesada     tipo_pesada_desc
                    0                    0                    0
        basura_propia                 fmod          viaje_fecha
                    0                    0                    0
           fecha_tara                sitio           sitio_desc
                    0                    0                    0
      tipo_movimiento  tipo_movimiento_desc
                    0                    0
```

Python

```
df.isnull().sum()  # Conteo de NA por columna
(df == "").sum()   # Falsos nulos (cadenas vacías)
```

Data profiling 3: duplicados

R

```
sum(duplicated(df)) # Cantidad de filas exactamente iguales
```

```
[1] 0
```

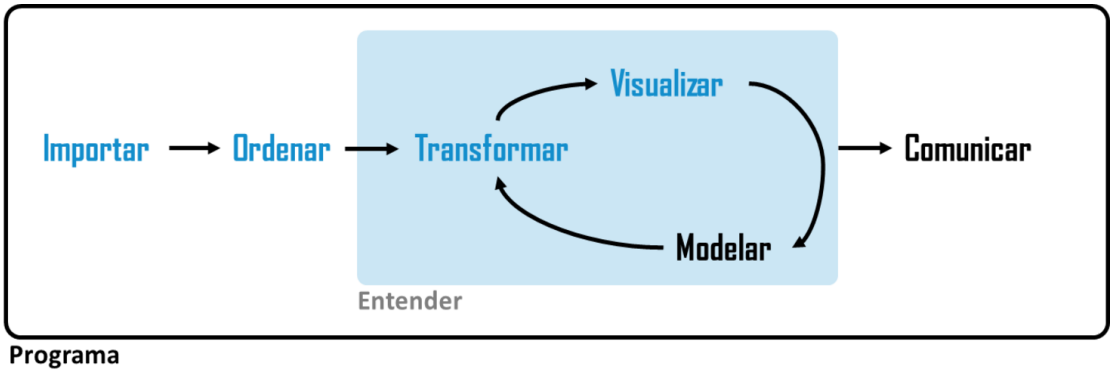
Python

```
df.duplicated().sum() # Cantidad de filas exactamente iguales
```

13. Flujo de Análisis de Datos y PostgreSQL

De la ingesta y perfilado al almacenamiento e integración de resultados
Teórico del Curso

1. El Flujo de Análisis de Datos



1.1 Ingesta / Carga

- **Definición:** Es la primera fase del ciclo de vida donde se capturan, extraen e importan los datos crudos (*raw data*) desde diversas fuentes de origen hacia el entorno de procesamiento o almacenamiento analítico.
- **Objetivo central:** Garantizar la disponibilidad y la integridad inicial de la información, registrando el origen exacto sin modificar ni alterar los datos originales durante este paso.
- **Desafíos técnicos:** Gestión de errores en la red o interrupciones en conexiones de origen, adaptabilidad ante esquemas variables y control de duplicaciones durante el transporte.

1.2 Perfilado de Datos (*Data Profiling*)

- **Definición:** Proceso sistemático de inspección y evaluación estadística sobre los datos crudos recién ingestados, realizado antes de aplicar cualquier regla de modificación o limpieza.
- **Objetivo central:** Comprender la estructura, la calidad real y la salud del conjunto de datos para diagnosticar anomalías antes de transformarlo.
- **Métricas e inspección:**
  - Conteo de nulos y cálculo de cardinalidad (cantidad de valores únicos por columna).
  - Distribuciones de frecuencia, detección de tipos de datos inconsistentes y detección de rangos fuera de límite (*outliers*).
  - Identificación de patrones de formato erróneos (ej. formatos de fecha mezclados o teléfonos mal estructurados).

1.3 Limpieza y Preparación (*Data Wrangling*)

- **Estandarización de tipos de datos:** Conversión explícita de campos de texto a tipos estructurados adecuados, como enteros (`INT`), decimales (`NUMERIC`), fechas (`DATE`) o booleanos.
- **Tratamiento de faltantes y errores:** Imputación fundamentada de valores nulos (`NULL`), corrección de inconsistencias tipográficas y eliminación guiada de registros corruptos según los hallazgos del perfilado.
- **Deduplicación y normalización:** Eliminación de registros idénticos o lógicamente repetidos, acomodando las columnas a un estándar coherente para el sistema.

1.4 Exploración (*EDA - Exploratory Data Analysis*)

- **Análisis estadístico descriptivo:** Evaluación profunda mediante métricas de tendencia central (media, mediana, moda), dispersión (desviación estándar, varianza) y percentiles.
- **Análisis de relaciones:** Identificación de correlaciones entre variables, detección de agrupamientos naturales (*clustering*) y comportamiento de las variables a lo largo del tiempo.
- **Validación con el negocio:** Comprobación de que las distribuciones y los datos resultantes respetan las reglas operativas y la lógica del dominio de aplicación.

1.5 Transformación y Modelado

- **Ingeniería de atributos (*Feature Engineering*):** Creación de nuevas variables calculadas a partir de las existentes (ej. cálculo de margen operativo, tasa de retención o antigüedad de cliente).
- **Agregación de datos:** Resumen y consolidación de la información en diferentes niveles de granularidad (métricas diarias, mensuales, por categoría o región).
- **Modelado relacional y analítico:** Organización de las variables en esquemas estructurados (como tablas de hechos y dimensiones) conectadas mediante claves primarias y foráneas.

1.6 Entrega de Resultados

- **Definición:** Etapa final donde el valor generado por la transformación de los datos se distribuye hacia sus destinos de consumo final según las necesidades de la organización.
- **Destinos de los resultados:**
  - **Tablas o Vistas SQL:** Creación de nuevas columnas, tablas físicas o vistas (`Views`) en la base de datos para que otros sistemas operativos consuman los datos limpios.
  - **Cuadros de Mando (*Dashboards*):** Publicación en herramientas de BI (Power BI, Metabase, Tableau) para visualización interactiva y monitoreo continuo.
  - **Reportes Formales:** Generación automática de documentos estáticos en PDF o HTML para auditorías y presentaciones directivas.
  - **Archivos de Hoja de Cálculo:** Exportación hacia planillas (Excel, Google Sheets) para consumo directo por parte de usuarios finales.
  - **Modelos de Machine Learning:** Alimentación de tuberías (*pipelines*) de entrenamiento automático para sistemas predictivos.

  - **Webhooks y APIs Externa:** Consumo por microservicios o aplicaciones cliente que requieren consumir la información procesada en tiempo real.

2. Métodos y Tipos de Carga de Datos

2.1 Carga Inicial (*Full Load*)

- **Mecanismo:** Lectura e importación total del volumen de datos disponible por única vez o para reiniciar el almacenamiento de forma completa.
- **Caso de uso:** Poblamiento inicial de bases de datos nuevas, migraciones completas de sistema o creación de respaldos históricos.
- **Ventajas y desventajas:**
  - *Ventaja:* Garantiza la consistencia completa de la base sin depender de estados o cargas previas.
  - *Desventaja:* Elevado consumo de tiempo, ancho de banda y capacidad de cómputo en volúmenes masivos.

2.2 Carga Incremental

- **Mecanismo:** Identificación, captura e inserción exclusiva de los registros que se han creado, modificado o actualizado desde la última ejecución.
- **Estrategias de detección:**
  - Control por marcas de tiempo (*timestamps* de actualización).
  - Secuencias incrementales numéricas.
  - Captura de cambios en la base de datos (*CDC - Change Data Capture*).
- **Ventajas y desventajas:**
  - *Ventaja:* Mínimo consumo de recursos de red y ejecución extremadamente rápida.
  - *Desventaja:* Requiere construir y mantener una lógica técnica más compleja para evitar duplicaciones o inconsistencias.

2.3 Formatos de Origen: Archivos Planos y Estructurados

- **CSV / TSV / TXT:** Formatos basados en texto con separadores definidos (comas, tabuladores). Son livianos, universales y legibles por cualquier sistema, pero no guardan tipos de datos ni reglas de integridad.
- **Excel (`.xlsx`):** Ampliamente extendido en la gestión administrativa. Admite múltiples pestañas y formato visual, pero presenta bajo rendimiento y restricciones de tamaño en volúmenes masivos.

2.4 Formatos de Origen: Semiestructurados

- **JSON:** Formato basado en pares clave-valor y arreglos. Muy flexible y estándar nativo de la Web y APIs modernas.
- **XML:** Lenguaje de marcado basado en etiquetas jerárquicas strictly estructuradas. Común en integraciones empresariales e intercambios de datos legacy.

2.5 Formatos de Origen: Masivos / Columnares

- **Apache Parquet:** Formato binario columnar comprimido. Permite leer solo las columnas necesarias para una consulta, optimizando drásticamente la analítica sobre grandes volúmenes.
- **Apache Avro:** Formato binario orientado a filas que almacena el esquema en

formato JSON. Optimizado para transmisión de eventos en tiempo real.
- **ORC:** Formato columnar altamente comprimido para acelerar lecturas complejas dentro de entornos analíticos masivos.

2.6 Formatos de Origen: Fuentes Externas

- **APIs REST:** Interconexión con servicios web externos que retornan datos bajo demanda mediante peticiones HTTP en formatos estructurados.
- **Web Scraping:** Extracción automatizada de información estructurada a partir del código fuente de sitios web.
- **Réplicas de Bases de Datos:** Conexión directa a motores transaccionales (OLTP) mediante protocolos nativos para extraer tablas del sistema.

3. Integradora: Flujo de Trabajo con PostgreSQL

3.1 Arquitectura de Integración

- **Flujo de trabajo completo:**

  Fuentes $\longrightarrow$ Ingesta $\longrightarrow$ Perfilado $\longrightarrow$ PostgreSQL $\longrightarrow$ Ciencia de Datos $\longrightarrow$ Entrega (SQL, BI, A

- **El rol de PostgreSQL:** Actúa como el centro neurálgico del procesamiento relacional y semianalítico, recibiendo las cargas y sirviendo la información limpia a los diferentes destinos.

3.2 El Rol de PostgreSQL en el Flujo

- **Tratamiento nativo multiformato:** Capacidad para almacenar e indexar datos relacionales tradicionales junto con estructuras flexibles (`JSONB`).
- **Capa analítica persistente:** Creación de *Views* y *Materialized Views* para procesar reglas de negocio y ofrecer tablas analíticas optimizadas.
- **Conectividad directa:** Compatibilidad con drivers JDBC/ODBC que permiten entregar los resultados directamente a herramientas de BI, scripts de Python o sistemas cliente.

4. Debate Técnico: ¿Es Excel una Base de Datos?

4.1 Excel es una Hoja de Cálculo

- **Diseño de interfaz:** Está pensado para la edición manual, el cálculo reactivo celda a celda y la presentación visual de datos de pequeña escala.
- **Ausencia de integridad referencial:** No fuerza relaciones estrictas entre tablas; una fila puede contener tipos de datos totalmente mixtos en la misma columna.
- **Limitaciones de concurrencia y tamaño:** No soporta acceso simultáneo masivo de escritura y está limitado a un máximo fijo de filas por hoja ($1\,048\,576$ filas).

4.2 Por qué PostgreSQL sí es una Base de Datos Relacional

- **Motor SGBD / DBMS:** Diseñado para gestionar transacciones seguras con propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).
- **Tipado e integridad estricta:** Exige esquemas donde cada columna tiene un tipo de dato único, claves primarias, claves foráneas y restricciones (*constraints*).

- **Concurrencia masiva:** Maneja miles de conexiones simultáneas de lectura y escritura sin corromper el almacenamiento ni degradar el sistema.

5 Ciclo de vida - Despliegue de modelos

5.1 Ciencia de Datos (Modelado Predictivo y Avanzado)

- **Definición:** Etapa donde se aplican técnicas avanzadas de aprendizaje automático (*Machine Learning*), estadística inferencial y algoritmos predictivos sobre los datos limpios y estructurados.
- **Objetivo central:** Evolucionar del análisis descriptivo (“qué pasó”) al análisis predictivo (“qué pasará”) y prescriptivo (“qué debemos hacer”).
- **Componentes clave:**
  - **Selección de algoritmos:** Elección de modelos según el problema (clasificación, regresión, *clustering*, series temporales, NLP).
  - **Entrenamiento y Validación:** División de los datos en conjuntos de entrenamiento (*train*) y prueba (*test*) para calibrar el modelo.
  - **Evaluación de métricas:** Medición del desempeño mediante precisión, *recall*, $F1$-score, R^2 o error cuadrático medio ($RMSE$).
  - **Optimización de hiperparámetros:** Ajuste fino de las variables internas del algoritmo para evitar el sobreajuste (*overfitting*).

5.2 Despliegue de Modelos (*MLOps*)

- **Definición:** Proceso de llevar un modelo entrenado desde el entorno de desarrollo a un entorno de producción para que realice predicciones operativas en tiempo real o por lotes.
- **Integración con el sistema:**
  - Consumo directo de datos procesados almacenados en **PostgreSQL**.
  - Generación de resultados que alimentan la fase de entrega (predicciones, puntuaciones de riesgo, segmentación de clientes).

“‘{mermaid} %%| fig-width: 6 %%| out-width: 70% graph LR subgraph Datos [“1. Pipeline de Datos”] A[(Fuentes Raw)] –> B[Ingesta] B –> C[Perfilado y Limpieza] C –> D[(PostgreSQL)] end

```
subgraph ML ["2. Pipeline de Machine Learning"]
    D --> E[Feature Engineering]
    E --> F[Entrenamiento del Modelo]
    F --> G[Evaluación y Registro]
end

subgraph Operaciones ["3. MLOps y Despliegue"]
    G --> H[API / Servicio Inferencia]
    H --> I[Monitoreo Drift / Performance]
    I -- Reentrenamiento --> F
end

subgraph Consumo ["4. Entrega"]
    H --> J[Predicciones en BD]
    H --> K[Dashboards / BI]
end

classDef db fill:#336791,stroke:#fff,stroke-width:2px,color:#fff;
```

```
classDef ml fill:#2e7d32,stroke:#fff,stroke-width:2px,color:#fff;
class D db;
class F,G,H ml;
```

“‘

14. Carga y transformacion

Paquetes

Carga de datos: ejemplo medioambiente y aire

Ozono

Datos minutales de concentración de ozono (O3) en aire ambiente.

```
ozono <- read_csv(here::here("clases/clase7/o3_01_2024_04_2024.csv"),
                        locale = locale(encoding = "latin1"))
```

```
Rows: 291926 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): estacion
dbl  (3): o3, longitud, latitud
dttm (1): fecha
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Material particulado

Material particulado menor de 2,5 micras medido en estaciones automáticas

```
pm <- read_csv(here::here("clases/clase7/pm_2_5_01_2024_04_2024.csv"),
                        locale = locale(encoding = "latin1"))
```

```
Rows: 424779 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): estacion
dbl  (3): pm2_5, longitud, latitud
dttm (1): fecha
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Conteo vehicular en las principales avenidas de Montevideo

El sistema de conteo vehicular que utiliza el Centro de Gestión de Movilidad (CGM) para la medición de los volúmenes vehiculares que circulan por las arterias que se encuentran centralizadas, utiliza analíticas de video para la generación de los datos necesarios para la elaboración de planes y coordinaciones semafóricas. Los detectores miden el total de vehículos y su clasificación por largo de los mismos, por sentido de circulación, pudiendo discriminarse por carril dentro de cada sentido.

```
auto <- read_csv(here::here("clases/clase7/autoscope_04_2024_volumen.csv"),
                        locale = locale(encoding = "latin1"))
```

```
Rows: 3207345 Columns: 11
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (3): dsc_avenida, dsc_int_anterior, dsc_int_siguiente
dbl  (6): cod_detector, id_carril, latitud, longitud, volume, volumen_hora
date (1): fecha
```

```
time (1): hora
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Data profiling: id, nulos y duplicados

Información básica del dataset

```
str(ozono)
```

```
spc_tbl_ [291,926 x 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ fecha   : POSIXct[1:291926], format: "2024-01-01 00:00:00" "2024-01-
01 00:00:00" ...
 $ o3      : num [1:291926] 6 NA 8 NA 8 NA 10 NA 10 NA ...
 $ estacion: chr [1:291926] "ColÃ³n" "Curva de MaroÃ±as" "ColÃ³n" "Curva de MaroÃ±as" ...
 $ longitud: num [1:291926] -56.2 -56.1 -56.2 -56.1 -56.2 ...
 $ latitud : num [1:291926] -34.8 -34.9 -34.8 -34.9 -34.8 ...
 - attr(*, "spec")=
  .. cols(
  ..   fecha = col_datetime(format = ""),
  ..   o3 = col_double(),
  ..   estacion = col_character(),
  ..   longitud = col_double(),
  ..   latitud = col_double()
  .. )
 - attr(*, "problems")=<pointer: 0x00000201642f5750>
```

Identificador

Para el caso del ozono los datos se toman por estación (que tiene una ubicación geográfica, es decir, es una característica de la estación) y por fecha. El identificador podría ser la fecha. ¿Por qué? Porque sirve para unir con otros datos. También podría ser una combinación de fecha y estación. 1- Tenemos problemas con los valores de las estaciones

```
ozono[1:10,3]
```

```
# A tibble: 10 x 1
   estacion
   <chr>
 1 ColÃ³n
 2 Curva de MaroÃ±as
 3 ColÃ³n
 4 Curva de MaroÃ±as
 5 ColÃ³n
 6 Curva de MaroÃ±as
 7 ColÃ³n
 8 Curva de MaroÃ±as
 9 ColÃ³n
10 Curva de MaroÃ±as
```

```
unique(ozono$estacion)
```

```
[1] "ColÃ³n"            "Curva de MaroÃ±as"
```

```
# 1. Creamos el diccionario de traducción (Busca = Reemplaza)
pauta_limpieza <- c(
  "Ã³" = "o",
```

```
  "Ã±" = "ni",
  "Ã¡" = "á",
  "Ã©" = "é",
  "Ã" = "í",
  "Ãº" = "ú",
  "Ã“" = "Ó",
  "Ã‘" = "NI",
  "Ã"  = "í" # Nota: a veces la 'í' sola se rompe así, déjala al final de la lista
)
# 2. Aplicamos la limpieza a tu columna
ozono_limpio <- ozono %>%
  mutate(
    # Cambia 'tu_columna' por el nombre real de tu columna
    estacion = str_replace_all(estacion, pauta_limpieza)
  )
```

Valores faltantes

```
colSums(is.na(ozono))
```

```
fecha       o3 estacion longitud  latitud
    0    36159        0        0        0
```

La única variable que tiene valores faltantes es la medición, por lo tanto se pueden eliminar los valores faltantes, ya que no eliminamos identificador (podría discutirse si hay que eliminar fechas sin registro). Quito los NA de todo el dataset (si hubieran más columnas con NA también quita esa fila):

```
ozono_sinna <- ozono_limpio %>%
  na.omit()
```

Quito los NA de columnas especificas:

```
ozono_sinna <- ozono_limpio %>%
  filter(is.na(o3) != TRUE)
```

Duplicados

Para saber rápidamente el número total de filas repetidas en todo tu dataset:

```
ozono_sinna %>%
  duplicated() %>%
  sum()
```

```
[1] 0
```

Mostrar solo las filas que están repetidas y ver qué tienen dentro

```
# Si quieres ver TODAS las filas involucradas en el duplicado (la primera y sus copias):
ozono_repetidas <- ozono_sinna %>%
  filter(duplicated(.) | duplicated(., fromLast = TRUE))
```

Eliminar duplicados

```
# Conserva solo la primera aparición de cada fila única
dataset_sindup <- ozono_sinna %>%
  distinct()
# Si quieres eliminar duplicados basados solo en una columna (ej. 'nombre_barrio')
```

```
# y conservar el resto de las columnas del primer registro:
dataset_sindup_columna <- ozono_sinna %>%
  distinct(estacion, .keep_all = TRUE)
```

Tipos de datos y transformación de datos

Fechas

El formato “POSIXct” “POSIXt” representan el formato estándar para manejar fechas y horas exactas (lo que en programación se conoce como Timestamp o Datetime)

```
class(ozono$fecha)
```

```
[1] "POSIXct" "POSIXt"
```

1. ¿Qué es POSIXct? (La clase real) La ct significa calendar time (tiempo de calendario).Internamente, R guarda esta variable como un número entero: la cantidad de segundos que han pasado desde el 1 de enero de 1970 hasta ese momento. Al ser un número por dentro, es un formato extremadamente rápido y eficiente para hacer cálculos (por ejemplo, restar dos fechas para saber cuántas horas o días pasaron).
2. ¿Qué es “POSIXt”? (La clase heredada). Es una “superclase” virtual de R. Solo sirve para que R sepa que esa variable es un formato de tiempo y que puede usar con ella funciones de fechas (como sumarle días, extraer el mes, etc.)

Crear variables

```
ozono_new <- ozono_sinna %>%
 mutate(
    anio = lubridate::year(fecha),
    mes  = lubridate::month(fecha),
    hora = lubridate::hour(fecha)
  )
```

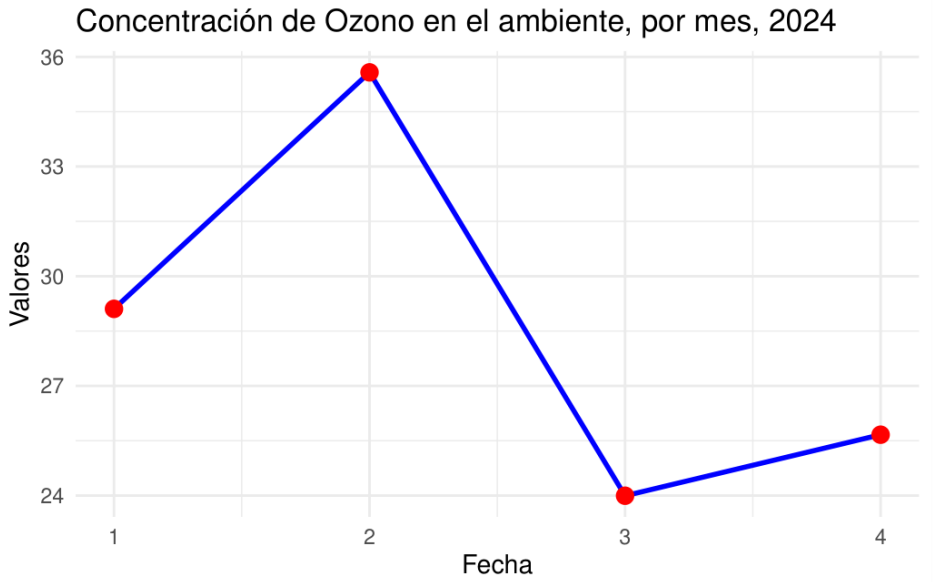
Generación de información

Agregación mensual

```
ozono_mes <- ozono_new %>%
  group_by(mes) %>%
  summarise(
    o3_medio_mes = mean(o3),
    n_registros_mes = n())
```

Visualización

```
ozono_mes %>%
ggplot(aes(x = mes, y = o3_medio_mes)) +
  geom_line(color = "blue", linewidth = 1) +  # Une los puntos con una línea
  geom_point(color = "red", size = 3) +       # Dibuja los puntos individuales
  theme_minimal() +                           # Aplica un diseño limpio
  labs(
    title = "Concentración de Ozono en el ambiente, por mes, 2024",
    x = "Fecha",
    y = "Valores"
  )
```



Partes básicas de un gráfico:

- dataset
- variable eje x
- variable eje y
- tipo de gráfico
- títulos: del gráfico y nombres de los ejes

Información completa en Cheetsheet: https://ggplot2.tidyverse.org/

Análisis profundo: Agregación temporal: minutal -> horario -> diario

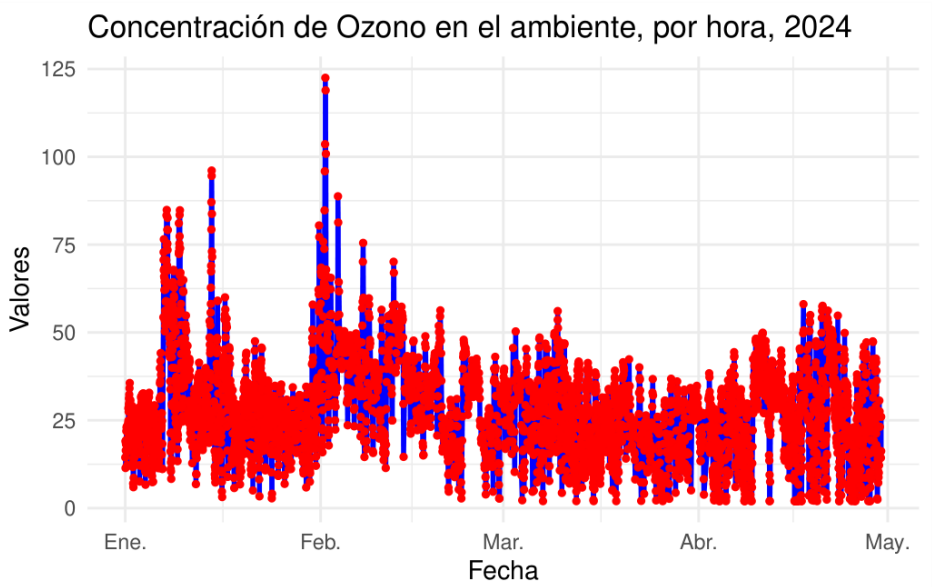
Pregunta central: al agregar, ¿qué estadístico representa el fenómeno?

- media: nivel típico
- máximo: episodios puntuales (sensible a errores del sensor)
- percentil 95: episodios altos pero robusto a picos aislados
- n: cuántos minutos respaldan el valor agregado (cobertura)

Horario

```
ozono_hora <- ozono_sinna %>%
  mutate(fecha_hora = floor_date(fecha, unit = "hour")) %>%
  group_by(estacion, fecha_hora) %>%
  summarise(
    media  = mean(o3),
    maximo = max(o3),
    p95    = quantile(o3, 0.95),
    n_min  = n(),               # de un máximo de 60
    .groups = "drop"
  )

ozono_hora %>%
ggplot(aes(x = fecha_hora, y = media)) +
  geom_line(color = "blue", linewidth = 1) +  # Une los puntos con una línea
  geom_point(color = "red", size = 1) +       # Dibuja los puntos individuales
```

```
theme_minimal() +                          # Aplica un diseño limpio
labs(
  title = "Concentración de Ozono en el ambiente, por hora, 2024",
  x = "Fecha",
  y = "Valores"
)
```



Teoría estadística

```
source(here::here("clases/clase7/media_percentil.R"))
```

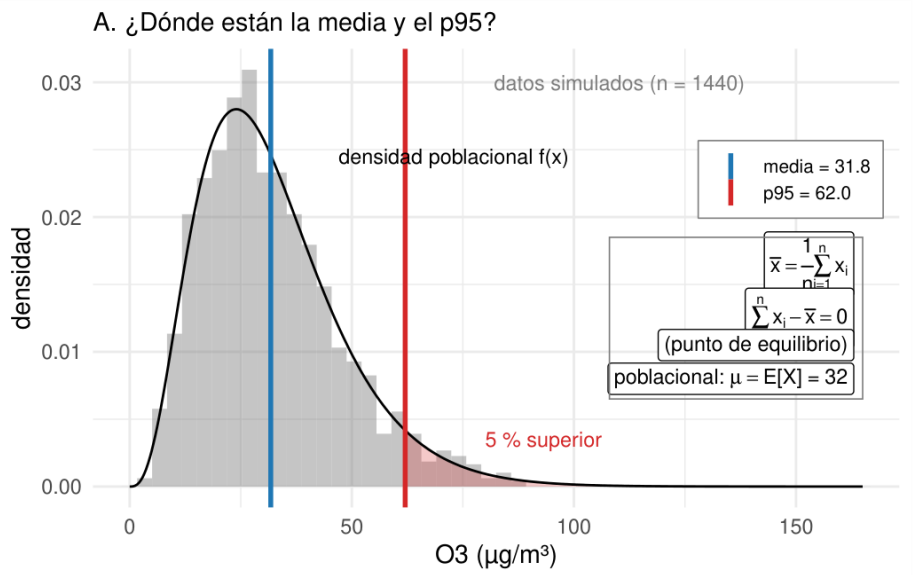
```
Warning in annotate("label", x = 163, y = c(0.0165, 0.0125, 0.0105, 0.008), :
Ignoring unknown parameters: `label.size`
```

```
Warning in annotate("label", x = 1, y = c(0.93, 0.82, 0.74, 0.685, 0.6), :
Ignoring unknown parameters: `label.size`
```

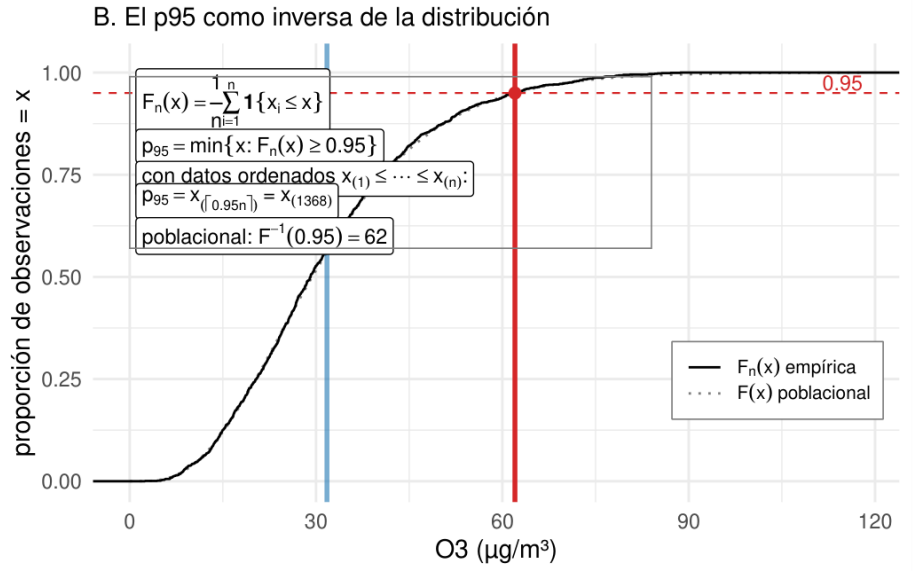
```
Warning in annotate("label", x = 300, y = 3.5, size = 3, label.size = 0.3, :
Ignoring unknown parameters: `label.size`
```

```
media=31.76  p95=62.02  |  con picos: media=34.99  p95=63.30
```

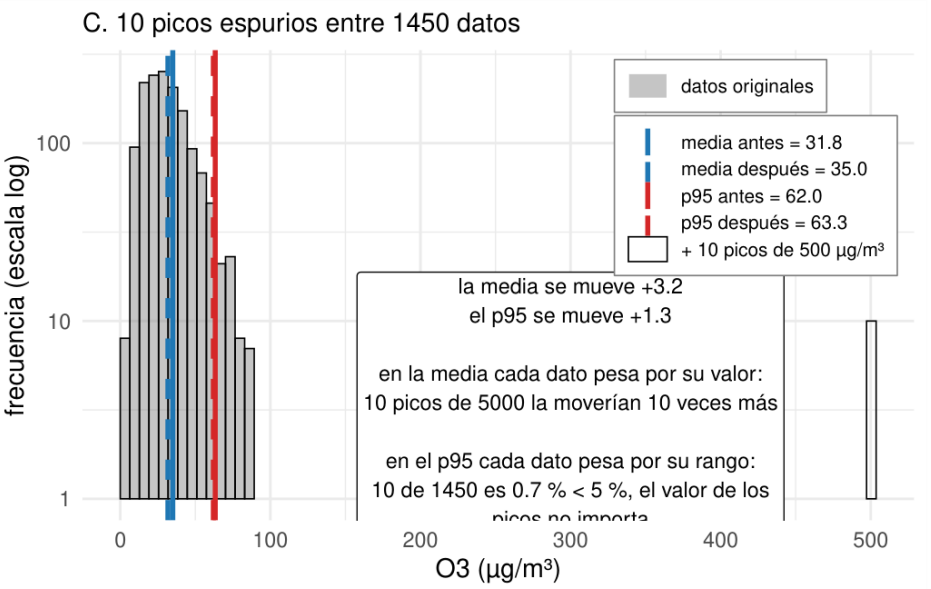
```
pA
```


A. ¿Dónde están la media y el p95?
datos simulados (n = 1440)
densidad poblacional f(x)
media = 31.8
p95 = 62.0
(punto de equilibrio)
poblacional: μ = E[X] = 32
5 % superior
densidad
O3 (µg/m³)

```
pB
```


B. El p95 como inversa de la distribución
0.95
con datos ordenados
poblacional: F⁻¹(0.95) = 62
Fn(x) empírica
F(x) poblacional
proporción de observaciones ≤ x
O3 (µg/m³)

```
pC
```

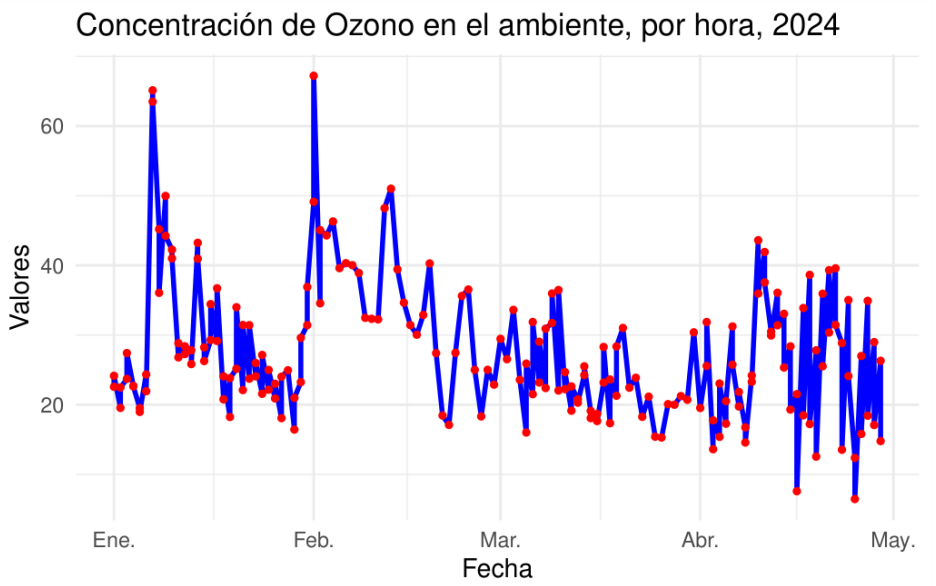


Diario

```
ozono_dia <- ozono_sinna %>%
  mutate(dia = as_date(fecha)) %>%
  group_by(estacion, dia) %>%
  summarise(
    media  = mean(o3),
    maximo = max(o3),
    p95    = quantile(o3, 0.95),
    n_min  = n(),               # de un máximo de 1440
    .groups = "drop"
  )
```

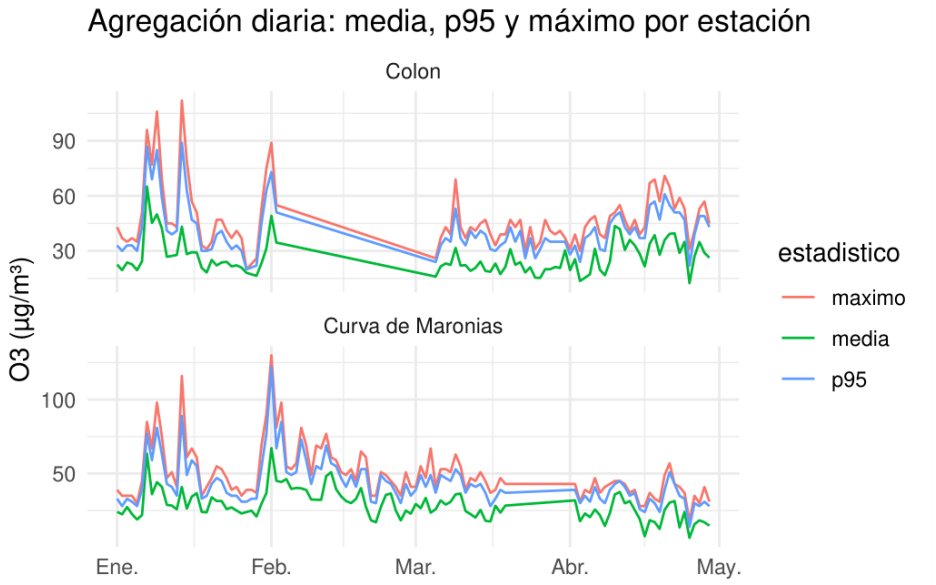
```
ozono_dia %>%
ggplot(aes(x = dia, y = media)) +
  geom_line(color = "blue", linewidth = 1) +  # Une los puntos con una línea
  geom_point(color = "red", size = 1) +       # Dibuja los puntos individuales
  theme_minimal() +                           # Aplica un diseño limpio
```

```
labs(
  title = "Concentración de Ozono en el ambiente, por hora, 2024",
  x = "Fecha",
  y = "Valores"
)
```



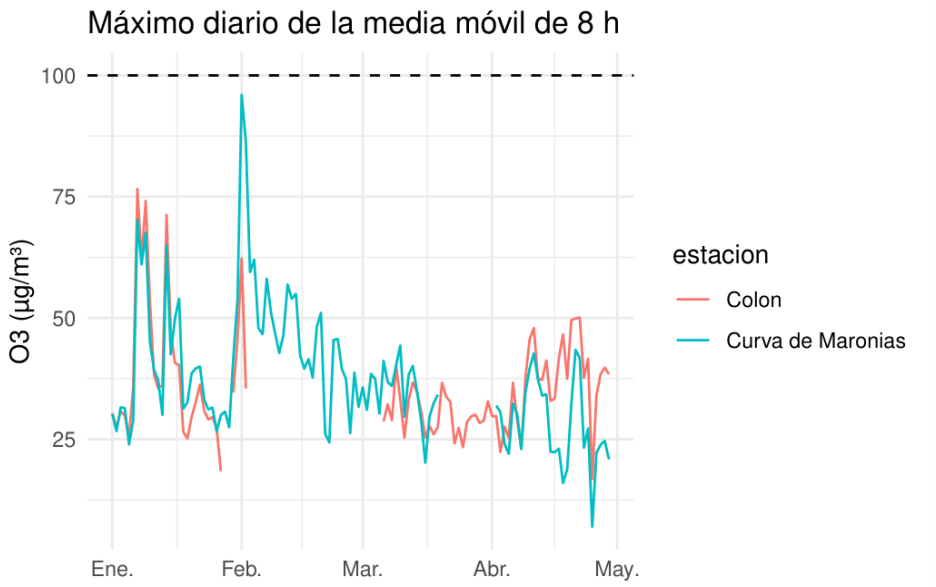
Las tres curvas cuentan historias distintas sobre el mismo día

```
ozono_dia %>%
  pivot_longer(c(media, p95, maximo), names_to = "estadistico", values_to = "o3") %>%
  ggplot(aes(x = dia, y = o3, color = estadistico)) +
  geom_line() +
  facet_wrap(~ estacion, ncol = 1, scales = "free_y") +
  theme_minimal() +
  labs(title = "Agregación diaria: media, p95 y máximo por estación",
       x = NULL, y = "O3 (µg/m³)")
```



El estadístico normativo: máximo diario de la media móvil de 8 horas Las guías OMS (2021) para O3 usan el máximo diario de la media móvil de 8 h. Una media móvil requiere una grilla horaria COMPLETA: si faltan horas, la ventana de 8 posiciones deja de ser una ventana de 8 horas.

```
grilla_horaria <- seq(min(ozono_hora$fecha_hora), max(ozono_hora$fecha_hora), by =
↪  "hour")

ozono_8h <- ozono_hora %>%
  select(estacion, fecha_hora, media) %>%
  complete(estacion, fecha_hora = grilla_horaria) %>%   # inserta NA en horas ausentes
  arrange(estacion, fecha_hora) %>%
  group_by(estacion) %>%
  mutate(
    # stats::filter, no dplyr::filter (mismo nombre, función distinta)
    media_8h = as.numeric(stats::filter(media, rep(1 / 8, 8), sides = 1))
  ) %>%
  ungroup()
max_8h_dia <- ozono_8h %>%
  mutate(dia = as_date(fecha_hora)) %>%
  group_by(estacion, dia) %>%
  summarise(
    max_8h  = if (all(is.na(media_8h))) NA_real_ else max(media_8h, na.rm = TRUE),

    horas_validas = sum(!is.na(media_8h)),
    .groups = "drop"
  )
ggplot(max_8h_dia, aes(x = dia, y = max_8h, color = estacion)) +
  geom_line() +
  geom_hline(yintercept = 100, linetype = "dashed") +   # guía OMS 2021: 100 µg/m³
  theme_minimal() +
  labs(title = "Máximo diario de la media móvil de 8 h",
       x = NULL, y = "O3 (µg/m³)")
```




```
source(here::here("clases/clase7/mapa.R"))
```

```
Warning: package 'htmlwidgets' was built under R version 4.6.1
mapa
```

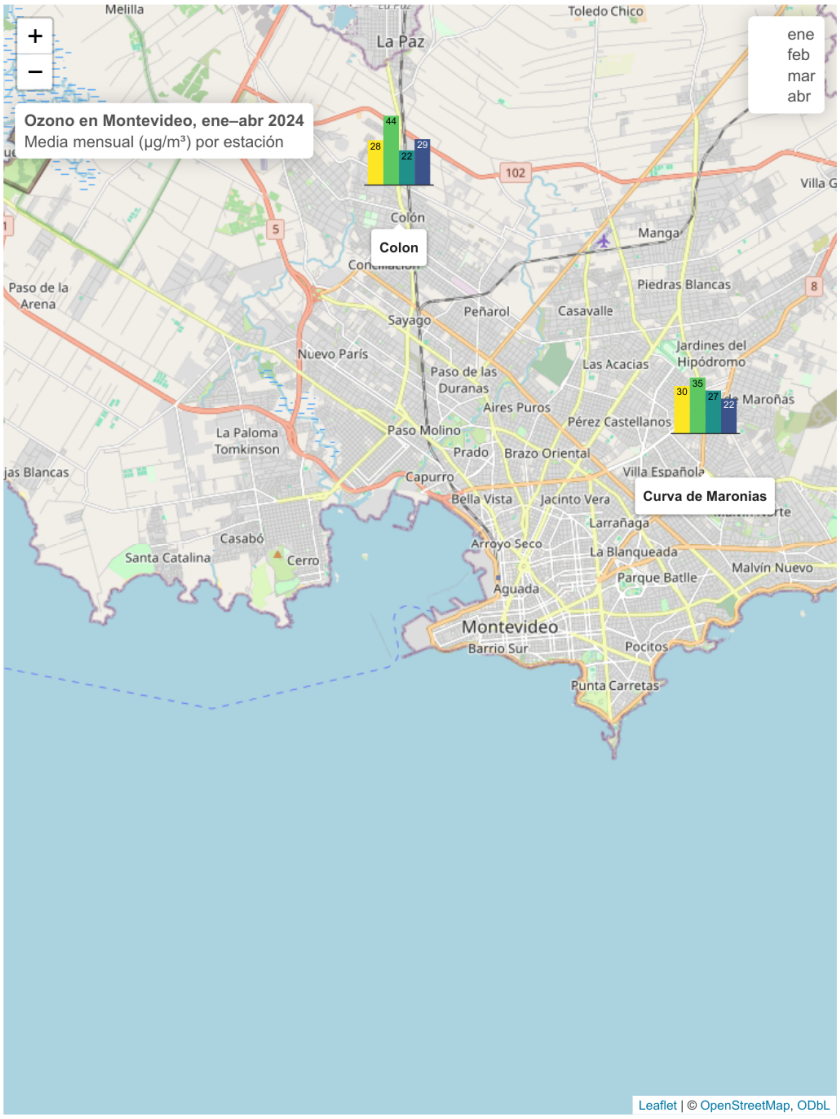


Figure 1: Ozono en Montevideo, ene–abr 2024

---

9. Patrones de faltantes

Tres tipos de faltante en una serie de sensor:

a) explícito: la fila existe, o3 es NA

b) en rachas: muchos NA consecutivos -> sensor caído o en mantenimiento

c) implícito: la fila no existe (el minuto no está en el archivo)

Eliminar NA sin mirar esto borra evidencia sobre el proceso de

medición.

9.1 Faltantes explícitos por estación

```
ozono_limpio %>%
  group_by(estacion) %>%
  summarise(
    n_filas  = n(),
    n_na     = sum(is.na(o3)),
    prop_na  = mean(is.na(o3)),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 4
  estacion          n_filas  n_na prop_na
  <chr>               <int> <int>   <dbl>
1 Colon              126378 13549   0.107
2 Curva de Maronias  165548 22610   0.137
```

```
## 9.2 ¿Cuándo faltan? Proporción de NA por día
na_dia <- ozono_limpio %>%
  mutate(dia = as_date(fecha)) %>%
  group_by(estacion, dia) %>%
  summarise(prop_na = mean(is.na(o3)), .groups = "drop")
ggplot(na_dia, aes(x = dia, y = prop_na)) +
  geom_col(fill = "grey40") +
  facet_wrap(~ estacion, ncol = 1) +
  theme_minimal() +
  labs(title = "Proporción de minutos sin dato, por día y estación",
       x = NULL, y = "Proporción de NA")
```

```
## 9.3 Rachas de NA consecutivos (rle = run length encoding)
rachas <- ozono_limpio %>%
  arrange(estacion, fecha) %>%
  group_by(estacion) %>%
  reframe({
    r <- rle(is.na(o3))
    tibble(faltante = r$values, largo = r$lengths)
  })

rachas %>%
  filter(faltante) %>%
  mutate(tipo = cut(largo, breaks = c(0, 1, 10, 60, 1440, Inf),
                    labels = c("1 min", "2-10 min", "11 min-1 h", "1 h-1 día", "> 1
                    ↪  día"))) %
  count(estacion, tipo, name = "n_rachas")
```

```
# A tibble: 8 x 3
  estacion          tipo       n_rachas
  <chr>             <fct>         <int>
1 Colon             2-10 min          3
2 Colon             11 min-1 h        3
3 Colon             1 h-1 día        33
4 Colon             > 1 día           1
5 Curva de Maronias 2-10 min          3
6 Curva de Maronias 11 min-1 h        8
7 Curva de Maronias 1 h-1 día        28
8 Curva de Maronias > 1 día           1
```

Lectura: muchas rachas de 1 min = ruido de transmisión; pocas

rachas > 1 día

= sensor fuera de servicio. Son mecanismos distintos y se tratan

distinto.

```
## 9.4 Faltantes implícitos: minutos que no están en el archivo
ozono_limpio %>%
  group_by(estacion) %>%
  summarise(
    primero   = min(fecha),
    ultimo    = max(fecha),
    esperados = as.numeric(difftime(max(fecha), min(fecha), units = "mins")) + 1,
    presentes = n_distinct(fecha),
    .groups = "drop"
  ) %>%
  mutate(implicitos = esperados - presentes,
         prop_implicitos = implicitos / esperados)
```

```
# A tibble: 2 x 7
  estacion          primero             ultimo              esperados presentes
  <chr>             <dttm>              <dttm>                  <dbl>     <int>
1 Colon             2024-01-01 00:00:00 2024-04-29 23:59:00    172800    126378
2 Curva de Maronias 2024-01-01 00:00:00 2024-04-29 23:59:00    172800    165548
```

```
# i 2 more variables: implicitos <dbl>, prop_implicitos <dbl>
```

19

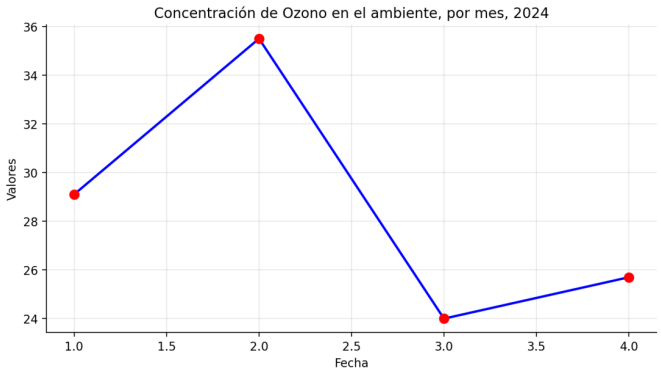
15. Las partes de un gráfico

Introducción a la Ciencia de Datos

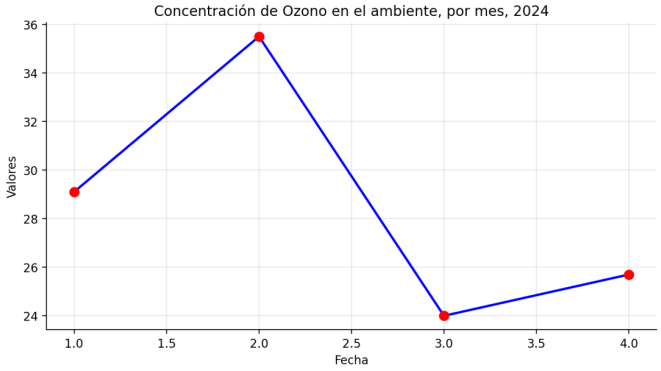
Primera parte: un gráfico, una serie

El gráfico del que partimos

Python



R



Concentración media mensual de ozono, de enero a abril. Una sola serie, cuatro observaciones.

El código que lo produce

Python

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(ozono_mes["mes"], ozono_mes["o3_medio_mes"],
        color="blue", linewidth=2)                      # línea
ax.scatter(ozono_mes["mes"], ozono_mes["o3_medio_mes"],
           color="red", s=60, zorder=3)                 # puntos
ax.set(title="Concentración de Ozono en el ambiente, por mes, 2024",
       xlabel="Fecha", ylabel="Valores")
ax.grid(alpha=0.3)
```

```
for lado in ("top", "right"):
    ax.spines[lado].set_visible(False)
plt.tight_layout()
plt.show()
```

R

```
ozono_mes %>%
  ggplot(aes(x = mes, y = o3_medio_mes)) +      # datos y mapeo
  geom_line(color = "blue", linewidth = 1) +     # línea
  geom_point(color = "red", size = 3) +          # puntos
  theme_minimal() +
                                                 # diseño limpio
  labs(
    title = "Concentración de Ozono en el ambiente, por mes, 2024",
    x = "Fecha",
    y = "Valores"
  )
```

Cada línea de este código controla una parte distinta del gráfico. Las vamos a recorrer una por una.

Un objeto que se construye por capas

R

```
p <- ggplot(ozono_mes, aes(x = mes, y = o3_medio_mes))   # todavía vací
p + geom_line()
                                                         # una capa
p + geom_line() + geom_point()
                                                         # dos capas
```

- `ggplot()` devuelve un objeto: los datos y el mapeo, sin nada dibujado.
- Cada `+` agrega una capa o una modificación. El orden importa: lo último se dibuja encima.
- El objeto se puede guardar en una variable, seguir sumándole cosas y mostrarlo cuando se quiera. No hay `fig` y `ax` por separado: el gráfico es una sola cosa que crece.

Los datos y el mapeo

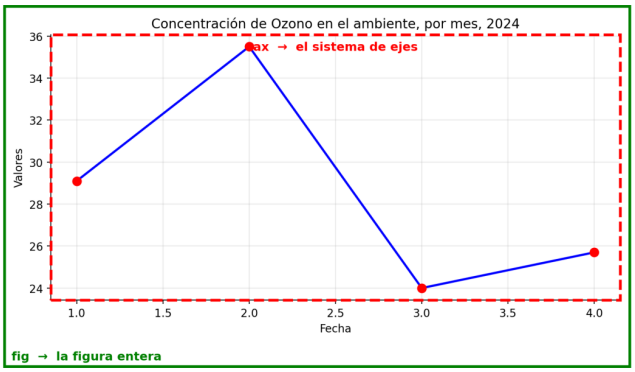
R

```
ggplot(ozono_mes, aes(x = mes, y = o3_medio_mes))
```

- Los datos son la única parte que aporta información nueva; todo lo demás sirve para leerla.
- `aes()` es el **mapeo**: conecta una columna de la tabla con una propiedad visual. Acá, `mes` con la posición horizontal y `o3_medio_mes` con la vertical.
- Lo que va dentro de `aes()` se lee de los datos, fila por fila. Cuatro pares (mes, concentración) traducidos a posiciones dentro del área de los ejes.

Dos objetos, no uno

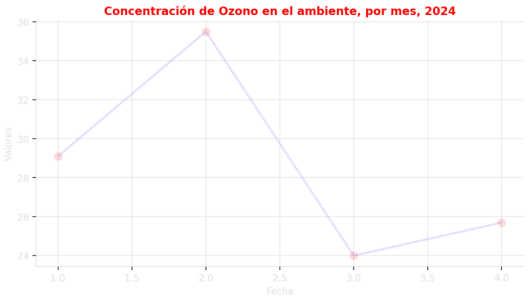
Python



- `fig` es el lienzo completo: tamaño, resolución, márgenes, archivo de salida.
- `ax` es el sistema de ejes: donde viven los datos, las escalas y las marcas.
- Una figura puede contener varios sistemas de ejes; por eso `plt.subplots()` devuelve las dos cosas.

El título

Python




```
ax.set(title="Concentración de Ozono en el ambiente, por mes, 2024")
```

- Dice **qué** se está midiendo, **de quién** y **cuándo**.
- Un título describe el contenido; también puede afirmar el hallazgo (“El ozono alcanza su máximo en verano”). La segunda opción exige estar seguro de lo que el gráfico muestra.

Las etiquetas de los ejes

Python

```
ax.set(xlabel="Fecha", ylabel="Valores")
```

- Estas dos etiquetas son el punto débil del gráfico. “Valores” no informa nada: falta la variable y falta la unidad.
- El eje vertical debería decir qué se mide y en qué unidad; el horizontal, que la unidad de tiempo es el mes.

Las marcas y sus etiquetas

Python

- El eje horizontal muestra marcas en 1.5, 2.5 y 3.5: meses que no existen. Pasa porque `mes` es numérico y matplotlib lo trata como continuo.
- Se corrige fijando las marcas a mano: `ax.set_xticks([1, 2, 3, 4])` y `ax.set_xticklabels(["Ene", ...])`.
- El eje vertical tampoco arranca en cero: admisible, pero exagera la variación.

R

- El eje horizontal muestra marcas en 1.5, 2.5 y 3.5: meses que no existen. Pasa porque `mes` es numérico y ggplot lo trata como continuo.
- Se corrige con la escala del eje:

```
scale_x_continuous(breaks = 1:4, labels = c("Ene", "Feb", "Mar", "Abr")
scale_y_continuous(limits = c(0, 40))
```

- `breaks` fija dónde van las marcas; `labels`, qué dicen. `limits` fija el rango.
- El eje vertical tampoco arranca en cero: admisible, pero exagera la variación. Con `limits = c(0, 40)` la caída de febrero a marzo se ve del tamaño que tiene.
- Las escalas (`scale_x_*`, `scale_y_*`, `scale_color_*`) son donde se decide cómo un valor se traduce a una posición o a un color.

La grilla y los bordes

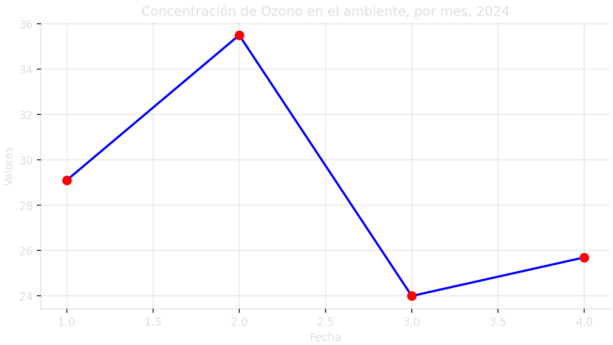
R

```
theme_minimal()
theme(panel.grid.minor = element_blank())
```

- El tema controla todo lo que no son datos: fondo, grilla, bordes, tipografía.
- `theme_minimal()` ya hace lo que uno quiere por defecto: grilla tenue, sin marco, sin fondo gris. Es la razón por la que el código de partida es corto.
- La grilla ayuda a estimar valores concretos sin recorrer el eje con la vista. La secundaria (`panel.grid.minor`) suele sobrar: marca posiciones donde no hay valores.
- Criterio general: quitar todo elemento que no ayude a leer un valor. `theme()` permite ajustar uno por uno; se usa después de `theme_minimal()`, no en lugar de.

Los datos

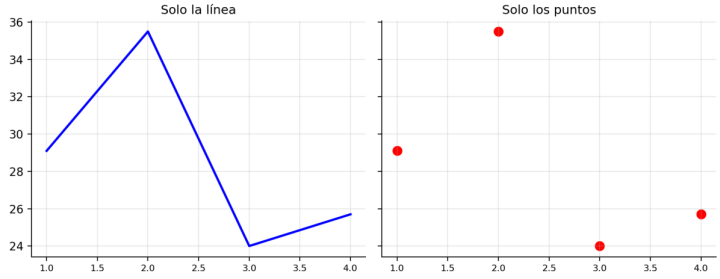
Python



- Es la única parte que aporta información nueva; todo lo demás sirve para leerla.
- Cuatro pares (mes, concentración) traducidos a posiciones dentro del área de los ejes.

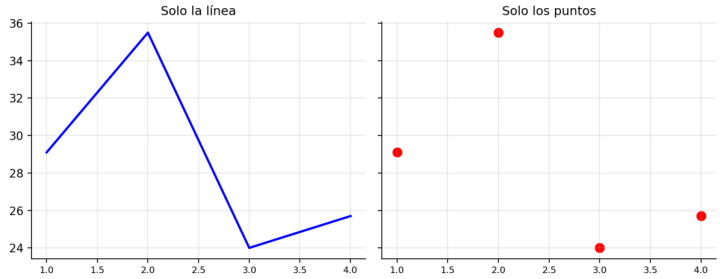
La geometría: dos capas superpuestas

Python



- La **línea** (`ax.plot`) afirma que hay continuidad entre meses consecutivos y dirige la lectura hacia la tendencia.
- Los **puntos** (`ax.scatter`) marcan dónde hay una medición efectiva y frenan la lectura de valores intermedios que nadie observó.
- Juntos: se ve la tendencia sin perder de vista que los datos son discretos.

R



- `geom_line()` afirma que hay continuidad entre meses consecutivos y dirige la lectura hacia la tendencia.
- `geom_point()` marca dónde hay una medición efectiva y frena la lectura de valores intermedios que nadie observó.

- Juntos: se ve la tendencia sin perder de vista que los datos son discretos. Las dos capas comparten el mapeo que se declaró en `ggplot()`.

Atributos estéticos: fuera o dentro de `aes()`

R

```
geom_line(color = "blue", linewidth = 1)      # fijo: fuera de aes()
geom_point(color = "red", size = 3)           # fijo: fuera de aes()
```

- `color`, `linewidth` y `size` puestos así no codifican ninguna variable: son decisiones de legibilidad, no información.
- El mismo atributo pasa a ser informativo cuando va **dentro** de `aes()` y recibe una columna: `aes(color = estacion)`. Entonces ggplot asigna un color por valor y agrega la leyenda.
- Esa distinción (fuera de `aes()`: fijo; dentro: mapeado) es la regla más importante de ggplot.

El título y las etiquetas

R

```
labs(title = "Concentración de Ozono en el ambiente, por mes, 2024",
     x = "Fecha", y = "Valores")
```

- El título dice **qué** se está midiendo, **de quién** y **cuándo**. También puede afirmar el hallazgo (“El ozono alcanza su máximo en verano”); eso exige estar seguro de lo que el gráfico muestra.
- Las etiquetas de eje son el punto débil del gráfico. “Valores” no informa nada: falta la variable y falta la unidad.
- El eje vertical debería decir qué se mide y en qué unidad; el horizontal, que la unidad de tiempo es el mes.
- `labs()` también controla el nombre de la leyenda (`color = "Estación"`) y el pie (`caption = "Fuente: ..."`).

Atributos estéticos

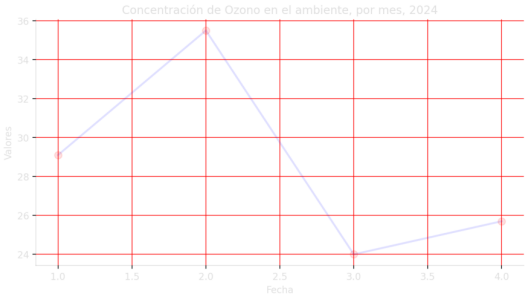
Python

```
ax.plot(..., color="blue", linewidth=2)
ax.scatter(..., color="red", s=60, zorder=3)
```

- `color`, `linewidth` y `s` no codifican ninguna variable: acá son decisiones de legibilidad, no información.
- Un atributo estético pasa a ser informativo cuando se lo asigna a una variable, por ejemplo un color por estación del año.
- `zorder=3` define qué se dibuja encima. Sin eso, la línea taparía los puntos.

La grilla

Python




```
ax.grid(alpha=0.3)
```

- Ayuda a estimar valores concretos sin recorrer el eje con la vista.
- `alpha=0.3` la deja visible pero por detrás de los datos. Una grilla opaca compite con la serie.

Los bordes

Python

```
for lado in ("top", "right"):
    ax.spines[lado].set_visible(False)
```

- Los bordes superior y derecho no aportan referencia alguna: se sacan.
- Los bordes izquierdo e inferior sostienen las marcas, así que se quedan.
- Criterio general: quitar todo elemento que no ayude a leer un valor.

Lo que este gráfico todavía no tiene

Python

- **Unidades:** sin ellas no se puede juzgar si los valores son altos o bajos.
- **Fuente de los datos:** de dónde salen las mediciones y quién las produjo.
- **Leyenda:** acá no hace falta, porque hay una sola serie. Con dos o más pasa a ser obligatoria.
- **Nota al pie:** aclaraciones sobre cobertura, faltantes o cambios de método.

R

- **Unidades:** sin ellas no se puede juzgar si los valores son altos o bajos.
- **Fuente de los datos:** de dónde salen las mediciones y quién las produjo. En ggplot va en `labs(caption = ...)`.
- **Leyenda:** acá no hace falta, porque hay una sola serie. Con dos o más pasa a ser obligatoria, y ggplot la genera sola en cuanto algo va dentro de `aes()`.
- **Nota al pie:** aclaraciones sobre cobertura, faltantes o cambios de método.

La figura es un objeto

Python

```
fig, ax = plt.subplots(figsize=(8, 5))     # fig y ax quedan disponible
ax.plot(...)
fig.savefig("ozono_2024.png", dpi=300, bbox_inches="tight")
plt.show()
                                           # siempre al final
```

- `plt.subplots()` ya devuelve el objeto: `fig` es la figura y se la puede seguir modificando después de dibujarla.
- `fig.savefig()` tiene que ir antes de `plt.show()`. Con algunos backends, mostrar la figura la vacía y el archivo sale en blanco.
- En un notebook, escribir `fig` en una celda la vuelve a mostrar sin recalcular nada. Para rehacerla con un cambio: `ax.set_ylim(0, 40)` y guardar de nuevo.
- `bbox_inches="tight"` recorta los márgenes sobrantes; `dpi=300` es lo mínimo razonable para imprimir o proyectar.

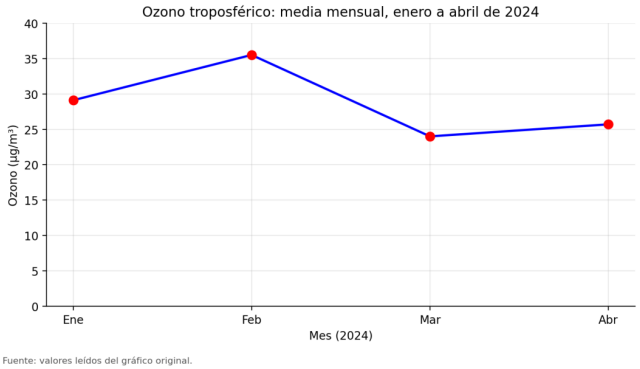
R

```
p <- ozono_mes %>%
  ggplot(aes(x = mes, y = o3_medio_mes)) +
  geom_line() + geom_point()
p
                                    # se dibuja al imprimirlo
ggsave("ozono_2024.png", p, width = 8, height = 5,
       dpi = 300, bg = "white")
```

- Un gráfico asignado a una variable no se dibuja; se dibuja al escribir su nombre o con `print(p)`.
- `ggsave()` guarda el objeto: no depende de lo que haya en pantalla. `width` y `height` en pulgadas; `dpi = 300` es lo mínimo razonable para imprimir o proyectar.
- `bg = "white"`: `theme_minimal()` deja el fondo transparente, y en algunos visores se ve negro.
- Para rehacerla con un cambio: `p + scale_y_continuous(limits = c(0, 40))` y guardar de nuevo. El original no se toca.

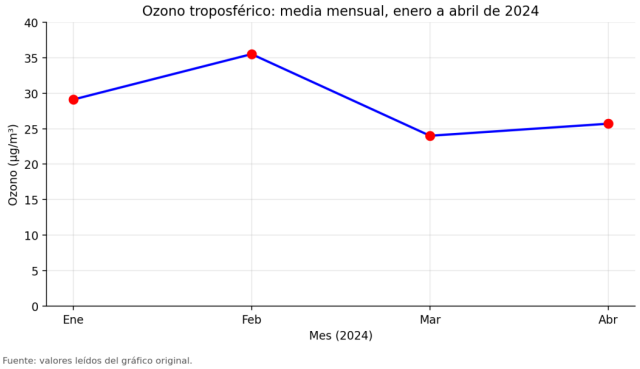
Una versión corregida

Python



Fuente: valores leídos del gráfico original.

R



Mismos datos, mismas geometrías. Cambian el título, las etiquetas, las marcas del eje horizontal, el rango del eje vertical y la fuente al pie.

Resumen de la primera parte

Python

Parte	Qué controla	Dónde se toca
Figura	lienzo y tamaño	`figsize=`
Ejes	área de datos y escalas	`ax`
Título	qué, de quién, cuándo	`title=`
Etiquetas de eje	variable y unidad	`xlabel=`, `ylabel=`
Marcas	posición a valor	`set_xticks()`, `set_ylim()`
Geometría	cómo se dibujan	`plot()`, `scatter()`
Atributos	color, grosor, orden	`color=`, `s=`, `zorder=`
Grilla	lectura de valores	`grid()`
Bordes	marco del área	`spines[...]`
Guardado	archivo de salida	`fig.savefig()`

R

Parte	Qué controla	Dónde se toca
Datos y mapeo	qué columna va a qué propiedad	`ggplot(datos, aes(...))`
Geometría	cómo se dibujan	`geom_line()`, `geom_point()`
Atributos fijos	color, grosor, tamaño	`color=`, `size=` fuera de `aes()`
Título y etiquetas	qué, de quién, cuándo; unidad	`labs()`
Marcas y rango	posición a valor	`scale_x_*(breaks, labels, limits)`
Grilla y bordes	todo lo que no son datos	`theme_minimal()`, `theme()`
Pie	fuente y notas	`labs(caption = ...)`
Guardado	archivo de salida	`ggsave()`

Segunda parte: más de una serie

Qué cambia cuando hay más de una serie

Con una serie, todas las decisiones eran de legibilidad. Con dos o más, aparecen decisiones de **codificación**: qué variable va al color, qué va a un panel aparte, en qué escala se lee cada eje.

Python

- Un color por grupo: el atributo estético pasa a ser informativo.
- Superponer o apilar: mismo eje o ejes distintos.
- Varios sistemas de ejes en una figura.
- Tres geometrías y la pregunta que responde cada una.
- Escalas: logarítmica y marcas a mano.
- Color continuo y color discreto.
- Paneles: la misma pregunta, varias veces.

R

- Un color por grupo: la variable entra en `aes()`.
- Superponer o apilar: mismo eje o paneles.
- Tres geometrías y la pregunta que responde cada una.
- Escalas: logarítmica y marcas a mano.
- Color continuo y color discreto.
- Paneles con `facet_wrap()`: la misma pregunta, varias veces.

Los ejemplos son los de la clase 9: ozono y PM2.5 en Montevideo, enero a abril de 2024, Catálogo de Datos Abiertos.

Un color por grupo

Python

En la primera parte, `color="blue"` era una decisión de legibilidad. Un atributo pasa a ser **informativo** cuando se lo asigna a una variable.

```
for est, d in o3_dia.groupby("estacion"):
    ax.plot(d["fecha"], d["media"], label=est)
ax.legend(title="Estación")
```

- `groupby` parte la tabla en una subtabla por estación.
- El `for` dibuja cada subtabla; matplotlib asigna un color distinto por llamada.
- `label=` nombra cada trazo; `legend()` los muestra.
- Con dos o más series, la leyenda deja de ser opcional.

Este patrón (`for` sobre `groupby`, `label`, `legend`) es el mismo para líneas, puntos y barras.

R

```
ggplot(o3_h, aes(x = fecha_hora, y = media, color = estacion)) +
  geom_line(linewidth = 0.3) +
  labs(color = "Estación")
```

- `color = estacion` **dentro** de `aes()`: ggplot parte los datos por estación, dibuja una línea por grupo, asigna un color a cada una y agrega la leyenda. Una línea de código.
- No hay bucle: en ggplot se declara qué variable distingue los grupos y el resto lo hace la gramática.

- `labs(color = ...)` pone el título de la leyenda. Con dos o más series, la leyenda deja de ser opcional.

Lo mismo vale para `fill` (barras, cajas), `shape` (puntos) y `linetype`.

Superponer o apilar

Python

Dos series se pueden poner **en el mismo eje** o **en ejes distintos**.

- **Mismo eje:** se comparan niveles. Solo si tienen la misma unidad y rangos parecidos. Ozono por estación: sí.
- **Ejes apilados:** se comparan formas. Cuando la unidad o el rango difieren. Ozono y PM2.5: apilados, con el eje horizontal compartido.

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 7), sharex=True)
ax1.plot(o3_h["fecha"], o3_h["o3"])
ax2.plot(pm_h["fecha"], pm_h["pm2_5"])
```

- `sharex=True` alinea el eje horizontal: el mismo valor cae en la misma vertical en los dos paneles.
- Poner ozono (0–120) y PM2.5 (3–50) en el mismo eje aplasta al PM2.5 y no muestra nada.

R

Dos series se pueden poner **en el mismo panel** o **en paneles distintos**.

- **Mismo panel:** se comparan niveles. Solo si tienen la misma unidad y rangos parecidos. Ozono por estación: sí, `aes(color = estacion)`.
- **Paneles apilados:** se comparan formas. Cuando la unidad o el rango difieren. Ozono y PM2.5: sí, apilados.

```
p_o3 / p_pm
                                                # patchwork: "/" apila, "|
facet_wrap(~ contaminante, ncol = 1, scales = "free_y")
```

- Dos caminos: `patchwork` combina dos gráficos ya hechos; `facet_wrap()` parte un solo gráfico por una variable. Para facetar, los dos contaminantes tienen que estar en la misma tabla, con una columna que diga cuál es cuál (`bind_rows`).
- `scales = "free_y"`: cada panel con su propio eje vertical. Sin eso, ggplot comparte escalas, y ozono (0–120) aplasta a PM2.5 (3–50).

Varios paneles con `facet_wrap()`

R

```
ggplot(aire, aes(x = pm2_5, y = o3)) +
  geom_point(size = 0.8, alpha = 0.4) +
  facet_wrap(~ panel)
```

- Un panel por valor de la variable. Título, ejes y grilla se repiten solos; las etiquetas de eje van una sola vez.
- Por defecto los paneles **comparten escalas**: es lo correcto cuando se van a comparar. `scales = "free"` es la excepción, no la regla; con escalas distintas la vista compara formas que no son comparables.

- Para poner un número por panel (el `r`, el `n`), se calcula en una tabla aparte y se arma la etiqueta del panel con `sprintf`: el título del panel es una columna más.

Varios sistemas de ejes

Python

`plt.subplots(filas, columnas)` devuelve un arreglo de ejes.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8),
                         sharex=True, sharey=True)
grupos = aire.groupby(franjas, observed=True)
for ax, (franja, d) in zip(axes.flat, grupos):
    r = d["o3"].corr(d["pm2_5"])
    ax.scatter(d["pm2_5"], d["o3"], s=8, alpha=0.4)
    ax.set_title(f"{franja}   r = {r:.2f}")
```

- `axes.flat` recorre la grilla de ejes como una lista.
- `zip` empareja cada eje con un grupo.
- `sharex`, `sharey`: **obligatorios** cuando los paneles se van a comparar. Con escalas distintas por panel, la vista compara formas que no son comparables.
- Las etiquetas de eje van una vez por fila y por columna, no en cada panel.

Tres geometrías, tres preguntas

Python

Geometría	Función	Pregunta que responde
Línea	`ax.plot()`	¿Cómo cambia a lo largo de un eje ordenado?
Puntos	`ax.scatter()`	¿Cómo se relacionan dos variables?
Cajas	`ax.boxplot()`	¿Cómo se distribuye, por grupo?

- La línea afirma continuidad entre observaciones consecutivas. Solo tiene sentido si el eje horizontal es ordenado: tiempo, o una variable continua.
- Los puntos no afirman nada entre observaciones: cada uno es una medición.
- La caja resume: mediana, cuartiles, extremos. Pierde el detalle, gana comparación.

Un gráfico de barras es una línea sin continuidad: categorías, no un eje ordenado.

R

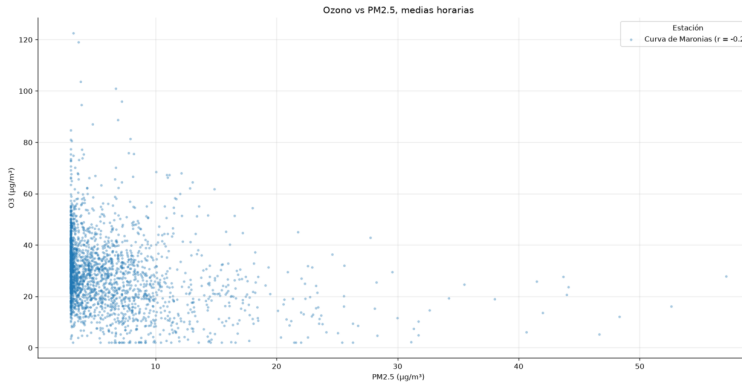
Geometría	Función	Pregunta que responde
Línea	`geom_line()`	¿Cómo cambia a lo largo de un eje ordenado?
Puntos	`geom_point()`	¿Cómo se relacionan dos variables?
Cajas	`geom_boxplot()`	¿Cómo se distribuye, por grupo?

- La línea afirma continuidad entre observaciones consecutivas. Solo tiene sentido si el eje horizontal es ordenado: tiempo, o una variable continua.
- Los puntos no afirman nada entre observaciones: cada uno es una medición.
- La caja resume: mediana, cuartiles, extremos. Pierde el detalle, gana comparación. Pide una variable discreta en `x` (`month(dia, label = TRUE)`).

`geom_col()` es una línea sin continuidad: categorías, no un eje ordenado.

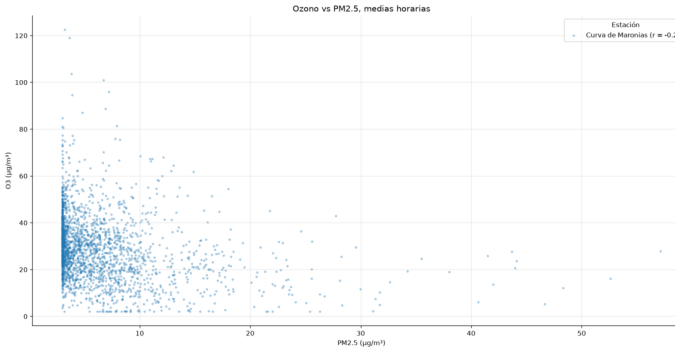
Dispersión: dos variables

Python



- Cada punto es una hora. `s=6` y `alpha=0.3` para que se vea dónde hay muchos.
- La pared vertical en PM2.5 = 3 no es atmósfera: es el piso del sensor.
- La leyenda quedó cortada: `figsize` demasiado ancho para el texto. Se ve antes de entregar, no después.

R



- Cada punto es una hora. `size = 0.8` y `alpha = 0.3` para que se vea dónde hay muchos.
- La pared vertical en PM2.5 = 3 no es atmósfera: es el piso del sensor.
- El `r` en el título viene de `cor()`, calculado antes y pegado con `sprintf`. El título no se escribe a mano si el número puede cambiar.

Densidad: cuando hay miles de puntos

Python

Con 2.800 puntos, `scatter` sin cuidado es una mancha.

- `alpha=0.3`: transparencia; donde se apilan, se oscurece.
- `s=6`: tamaño del punto en puntos al cuadrado. Chico.
- `ax.set_xlim`, `ax.set_ylim`: recortar la zona vacía, con la advertencia de que recortar esconde extremos.
- Si aun así no se ve, `ax.hexbin(x, y, gridsize=40)` cuenta puntos por celda y colorea el conteo. Es un histograma en dos dimensiones.

R

Con 2.800 puntos, `geom_point()` sin cuidado es una mancha.

- `alpha = 0.3`: transparencia; donde se apilan, se oscurece.
- `size = 0.8`: chico. El tamaño por defecto (1.5) es para decenas de puntos, no miles.
- `coord_cartesian(xlim = ..., ylim = ...)`: recortar la zona vacía **sin descartar datos**. `scale_*_continuous(limits = ...)` los descarta, y las medias o cajas calculadas después cambian.
- Si aun así no se ve, `geom_hex()` cuenta puntos por celda y colorea el conteo. Es un histograma en dos dimensiones.

El objetivo es que el lector vea **dónde está la masa** y **dónde están los pocos**. Un gráfico donde todo es igual de negro no muestra ninguna de las dos.

Escalas: cuándo usar logaritmo

PM2.5 va de 3 a 55, con casi todo entre 3 y 15. En escala lineal, el 80 % de los puntos ocupa el 20 % del ancho.

Python

```
ax.set_xscale("log")
ax.set_xticks([3, 5, 10, 20, 30, 50])
ax.set_xticklabels(["3", "5", "10", "20", "30", "50"])
ax.minorticks_off()
```

- La escala logarítmica **separa lo chico y comprime lo grande**. Sirve para variables asimétricas a la derecha: concentraciones, ingresos, tamaños.
- Sin las tres líneas siguientes, las marcas salen como 3 × 100: correctas e ilegibles. Las marcas se fijan a mano en valores redondos.
- Se usa en un eje si esa variable lo pide; no por costumbre.

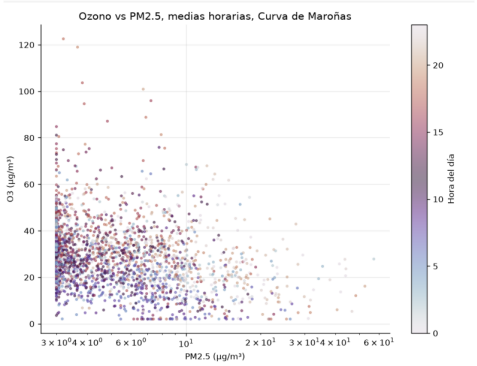
R

```
scale_x_log10(breaks = c(3, 5, 10, 20, 30, 50))
```

- La escala logarítmica **separa lo chico y comprime lo grande**. Sirve para variables asimétricas a la derecha: concentraciones, ingresos, tamaños.
- `breaks` a mano en valores redondos: sin eso, ggplot marca 3, 10, 30 y el lector no tiene el 5 ni el 20 para apoyarse.
- La escala se cambia en el eje, no en los datos: `log(pm2_5)` en `aes()` da el mismo dibujo con marcas ilegibles (1.1, 2.3, 3.4).
- Se usa en un eje si esa variable lo pide; no por costumbre.

Color continuo: una variable numérica

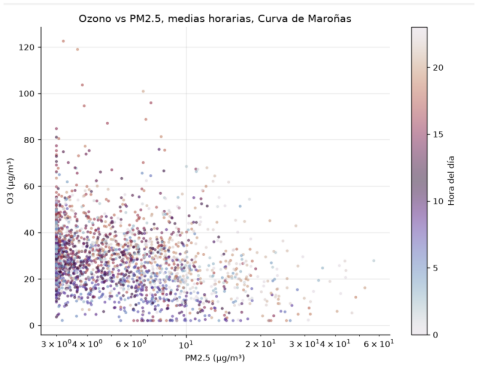
Python




```
sc = ax.scatter(
    x, y, s=6, alpha=0.5,
    c=aire["fecha"].dt.hour,
    cmap="twilight")
fig.colorbar(sc, ax=ax,
    label="Hora del día")
```

- `c=` recibe una variable; `cmap` la traduce a color; `colorbar` es su leyenda.
- `twilight` es cíclico: 0 h y 23 h tienen el mismo color. Correcto para la hora, pero difícil de leer: el rojo de las 17 se confunde con el violeta de las 12.

R

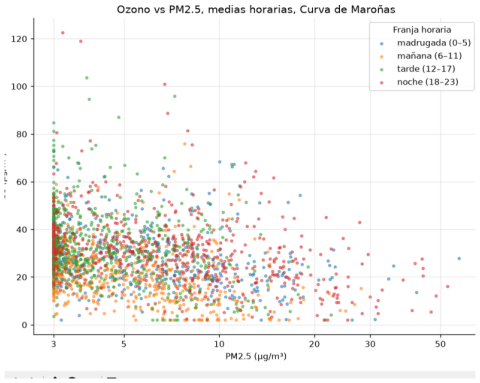



```
ggplot(aire,
  aes(x = pm2_5, y = o3,
      color = hour(fecha_hora))) +
  geom_point(size = 0.8, alpha = 0.5) +
  scale_color_viridis_c() +
  labs(color = "Hora del día")
```

- Una columna numérica en `aes(color = ...)`: ggplot elige una escala continua y dibuja la barra de color como leyenda.
- `scale_color_viridis_c()`: paleta legible y ordenada. La hora es cíclica (0 y 23 se parecen) y viridis no lo es; el borde entre 23 y 0 es un salto de color que no está en los datos.

Color discreto: cortar y agrupar

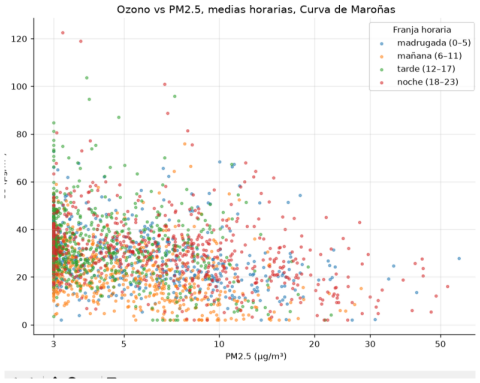
Python




```
hora = aire["fecha"].dt.hour
franjas = pd.cut(hora,
    bins=[-1, 5, 11, 17, 23],
    labels=["madrugada", "mañana",
            "tarde", "noche"])
grupos = aire.groupby(franjas,
                      observed=True)
for franja, d in grupos:
    ax.scatter(d["pm2_5"], d["o3"],
        s=8, alpha=0.5, label=franja)
```

- Cuatro colores con leyenda se leen mejor que 24 tonos con colorbar.
- **Los cortes son una decisión.** En enero el sol se pone después de las 20: la franja “noche” contiene tardes. Cambiar `bins` cambia lo que se ve.

R



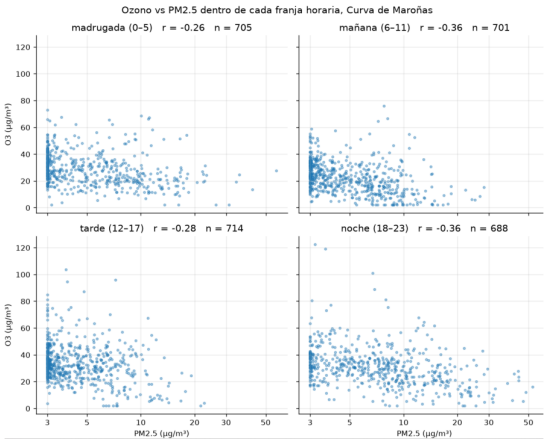

```
aire <- aire %>%
  mutate(franja = cut(
    hour(fecha_hora),
    breaks = c(-1, 5, 11, 17, 23),
    labels = c("madrugada", "mañana",
               "tarde", "noche")))

ggplot(aire,
  aes(x = pm2_5, y = o3,
      color = franja)) +
  geom_point(size = 0.8, alpha = 0.5)
```

- `cut()` convierte la hora en un factor de cuatro niveles; ggplot pasa a escala discreta y leyenda con cuatro entradas.

- **Los cortes son una decisión.** En enero el sol se pone después de las 20: la franja “noche” contiene tardes. Cambiar `breaks` cambia lo que se ve.

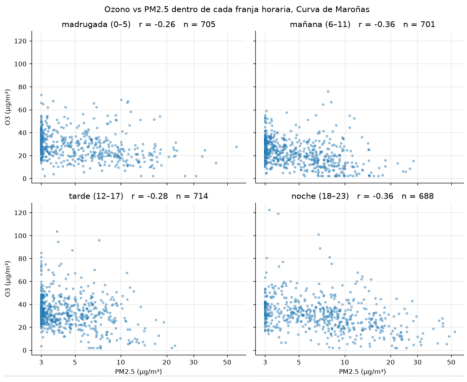
Paneles: la misma pregunta, varias veces

Python



- Un panel por franja, mismos ejes. Dentro de cada uno, r entre $-0{,}26$ y $-0{,}36$; en el gráfico sin partir, $-0{,}20$.
- Cuando el color mezcla demasiado, partir en paneles es la alternativa: cada panel compara horas parecidas entre sí.
- El `r` en el título de cada panel convierte cuatro imágenes en cuatro números comparables.

R




```
r_franja <- aire %>%
  group_by(franja) %>%
  summarise(r = cor(o3, pm2_5)) %>%
  mutate(panel = sprintf(
    "%s  r = %.2f", franja, r))

aire %>%
  left_join(r_franja, by = "franja") %>%
  ggplot(aes(x = pm2_5, y = o3)) +
  geom_point(size = 0.8, alpha = 0.4) +
  facet_wrap(~ panel)
```

- Un panel por franja, mismos ejes. Dentro de cada uno, r entre $-0{,}26$ y $-0{,}36$; sin

partir, $-0{,}20$. El `r` en la etiqueta del panel convierte cuatro imágenes en cuatro números comparables.

Tres figuras, tres lecturas del mismo dato

Python

Figura	Lo que muestra	Lo que afirma
Dispersión simple	una nube con $r = -0{,}2$	hay una asociación débil
Color por franja	los extremos son de horas distintas	la hora está mezclada
Paneles por franja	$r \approx -0{,}3$ en cada uno	la hora diluía la asociación

- Los datos son los mismos. Cambió qué variable se codificó con color y si se partió o no.
- Una figura **sostiene una afirmación** cuando la afirmación se lee sin que haya que explicarla. Las tres son correctas; solo la tercera sostiene la última fila.
- El título puede decir la afirmación (“Dentro de cada franja, más PM2.5 va con menos ozono”). Exige estar seguro.

R

Figura	Lo que muestra	Lo que afirma
Dispersión simple	una nube con $r = -0{,}2$	hay una asociación débil
Color por franja	los extremos son de horas distintas	la hora está mezclada
Paneles por franja	$r \approx -0{,}3$ en cada uno	la hora diluía la asociación

- Los datos son los mismos. Cambió qué variable entró en `aes(color = ...)` y si se facetó o no.
- Una figura **sostiene una afirmación** cuando la afirmación se lee sin que haya que explicarla. Las tres son correctas; solo la tercera sostiene la última fila.
- El título puede decir la afirmación (“Dentro de cada franja, más PM2.5 va con menos ozono”). Exige estar seguro.

Antes de entregar

Python

- **Título:** qué se mide, de quién, cuándo. O la afirmación, si está segura.
- **Unidades** en cada eje. Sin ellas no se sabe si un valor es alto o bajo.
- **Leyenda** si hay dos o más series. Legible, sin cortar.
- **Marcas** en valores que existen, con etiquetas que el lector use.
- **Fuente** al pie: de dónde salen los datos y quién los produjo.
- **Nota** sobre cobertura, faltantes o decisiones: qué estación, qué período, cómo se agregó, con qué cortes.

R

- **Título:** qué se mide, de quién, cuándo. O la afirmación, si está segura.

- **Unidades** en cada eje. Sin ellas no se sabe si un valor es alto o bajo.
- **Leyenda** con nombre (`labs(color = ...)`); la que ggplot genera sola lleva el nombre de la columna, que rara vez es el que el lector necesita.
- **Marcas** en valores que existen, con etiquetas que el lector use.
- **Fuente** al pie: `labs(caption = ...)`.
- **Nota** sobre cobertura, faltantes o decisiones: qué estación, qué período, cómo se agregó, con qué cortes.

Y una pregunta antes de guardar: ¿qué afirmación sostiene esta figura, y se lee sin que yo esté al lado para explicarla?

Resumen de la segunda parte

Python

Parte	Qué controla	Dónde se toca
Grupos	un color por categoría	`groupby` + `label=` + `legend()`
Paneles	varios ejes comparables	`subplots(..., sharex, sharey)`
Dispersión densa	la masa y los pocos	`s=`, `alpha=`, `hexbin()`
Escala	separar lo chico	`set_xscale("log")` + marcas
Color continuo	variable numérica	`c=`, `cmap=`, `colorbar()`
Color discreto	variable cortada	`pd.cut()` + `groupby`
Pie	fuente y notas	`fig.text(0.01, 0.01, ...)`

Figuras de la primera parte: valores leídos del gráfico original (enero a abril de 2024). Figuras de la segunda parte: ozono y PM2.5, Curva de Maroñas, enero a abril de 2024, Catálogo de Datos Abiertos de Uruguay.

R

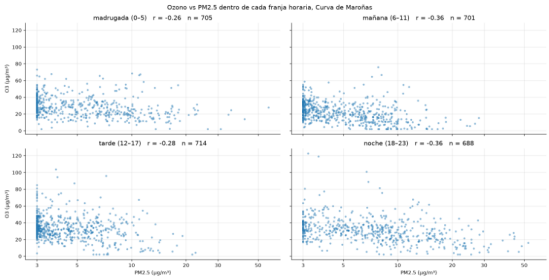
Parte	Qué controla	Dónde se toca
Grupos	un color por categoría	`aes(color = variable)`
Apilar	dos gráficos en uno	`patchwork`: `p1 / p2`
Paneles	la misma pregunta varias veces	`facet_wrap(~ variable)`
Dispersión densa	la masa y los pocos	`size=`, `alpha=`, `geom_hex()`
Escala	separar lo chico	`scale_x_log10(breaks = ...)`
Color continuo	variable numérica	`scale_color_viridis_c`
Color discreto	variable cortada	`cut()` + `aes(color = factor)`
Recorte	ver una zona sin descartar	`coord_cartesian()`

Figuras de la primera parte: valores leídos del gráfico original (enero a abril de 2024). Figuras de la segunda parte: ozono y PM2.5, Curva de Maroñas, enero a abril de 2024, Catálogo de Datos

16. Series de tiempo: una primera mirada

Introducción a la Ciencia de Datos

De dónde venimos



En la clase 9 unimos ozono y PM2.5 en Curva de Maroñas y encontramos $r = -0{,}26$. Al partir por franja horaria, r subía a $-0{,}3$ en cada franja: la hora del día estaba mezclada en la correlación. Hoy el tiempo deja de ser un estorbo y pasa a ser el objeto de estudio.

- Qué es una serie de tiempo y qué exige.
- Sus componentes: tendencia, estacionalidad, irregular.
- La memoria de una serie: el correlograma.

Cada diapositiva dice qué sección del script la hace. Tres herramientas nuevas: `resample`, `rolling`, `shift`.

Qué es una serie de tiempo

Definición

Una **serie temporal** es una sucesión de observaciones de una variable tomadas a lo largo del tiempo, de manera que los valores aparecen ordenados en el tiempo (de la Fuente).

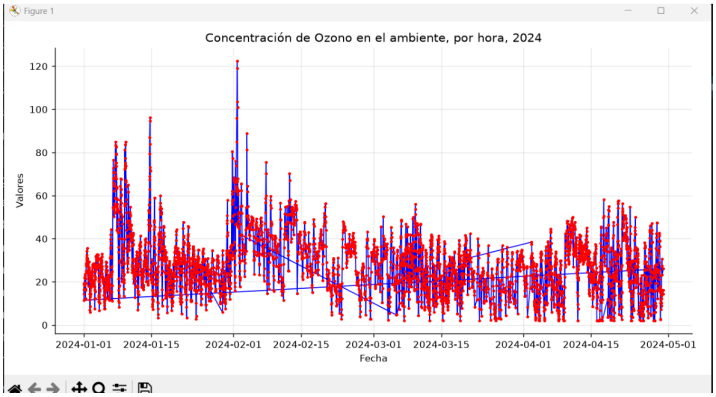
Tres condiciones para trabajarla:

1. **Ordenada:** la posición de cada dato es su fecha.
2. **Equidistante:** el intervalo entre dos observaciones consecutivas es el mismo en toda la serie.
3. **Sin huecos:** no faltan observaciones intermedias. Si faltan, se marcan como faltantes, no se saltan.

Una tabla con una columna `fecha` no es todavía una serie de tiempo: es una tabla. Se convierte en serie cuando el tiempo pasa a ser el índice y cumple las tres condiciones.

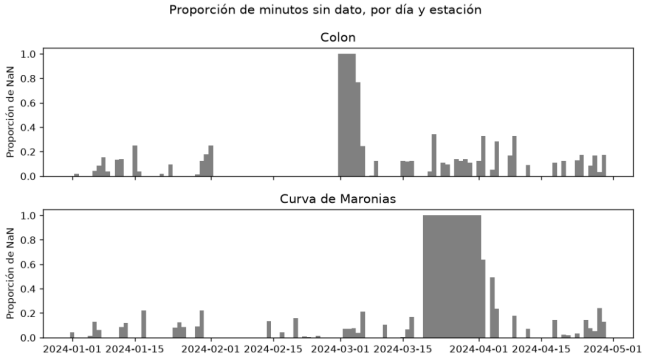
Script, sección 1: `set_index("fecha")["o3"].sort_index()`.

Una serie es de una sola cosa



- Este es el gráfico horario de la clase 7, con las dos estaciones en una sola línea. Las diagonales largas son la línea saltando de una estación a la otra.
- Una serie de tiempo es la sucesión de valores de **una** unidad observable (Mauricio): una estación, un producto, una cuenta. Dos unidades son dos series, y se dibujan como dos.
- Es el `filter(estacion == ...)` del script antes de `set_index`.

Sin huecos: se marcan, no se saltan



- Proporción de minutos sin dato por día (clase 7). Colón no midió del 1 al 8 de marzo; Curva de Maroñas, del 18 de marzo al 2 de abril.
- Si esas filas simplemente no están, “la observación anterior” al 3 de abril es la del 17 de marzo, y cualquier comparación con el pasado se rompe.
- La serie de tiempo conserva el lugar: el hueco es un `NaN` en su fecha.

Stock y flujo

De la Fuente distingue dos clases de magnitud:

- **Stock:** el valor en un instante. Concentración de ozono a las 15:00. Si se agregan minutos en una hora, se **promedia**.
- **Flujo:** lo acumulado desde la observación anterior. Vehículos que pasaron entre las 15:00 y las 16:00. Si se agregan minutos en una hora, se **suma**.

El valor de un flujo depende del intervalo elegido; el de un stock, no. Es la pregunta de la clase 7 (“¿qué estadístico representa el fenómeno?”) con nombre: antes de agregar, saber si la variable es stock o flujo.

Ozono y PM2.5 son stocks. El conteo vehicular es un flujo.

Por qué el paso tiene que ser fijo

Todo lo de hoy compara la serie con ella misma **k pasos atrás**. "k pasos atrás" significa "hace k× paso" solo si el paso es siempre el mismo y no falta ninguno.

Con una tabla que tiene huecos, "la fila anterior" a veces es el minuto anterior y a veces es tres horas antes. La comparación mezcla distancias distintas y no mide nada.

Primero, saber cuál es el paso real de cada sensor:

```
print(o3_s.index.to_series().diff().value_counts().head())
print(pm_s.index.to_series().diff().value_counts().head())
```

El valor más frecuente es el paso de muestreo; los demás son huecos o irregularidades. Script, sección 2.

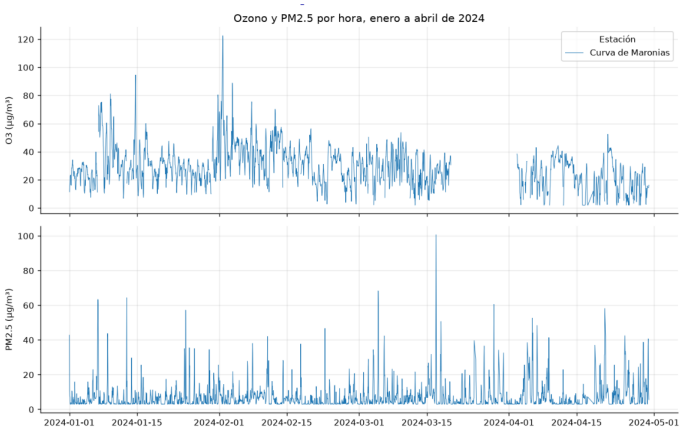
Cajas de tiempo: `resample`

```
serie = pd.DataFrame({
    "o3":   o3_s.resample(PASO).mean(),
    "pm25": pm_s.resample(PASO).mean(),
    "n_o3": o3_s.resample(PASO).count(),
    "n_pm": pm_s.resample(PASO).count(),
})
```

- Corta el tiempo en cajas de `PASO` alineadas al reloj (00:00, 00:10, …) y promedia lo que cae en cada una.
- Devuelve la grilla **completa**: caja vacía = `NaN`. Las tres condiciones de la definición quedan cumplidas.
- Las dos series quedan en las mismas cajas aunque sus sellos originales no coincidieran: es la unión de la clase 9, con el tiempo como clave.
- `count` dice cuántas mediciones respaldan cada valor (la cobertura de la clase 7).

El paso se elige con el diagnóstico anterior: el más corto que deje al menos una medición por caja en la serie más rala. Script, sección 3.

Mirar la serie completa



- Cuatro meses por hora, la estación en común, apilados. Se ve el nivel, si sube o baja, los huecos, y que el PM2.5 es de picos cortos sobre un piso.

- No se ve el ciclo diario: para eso se recorta una semana sobre el índice, `serie["2024-03-04":"2024-03-10"]`. Ninguna escala sola alcanza. Sección 4.

Componentes

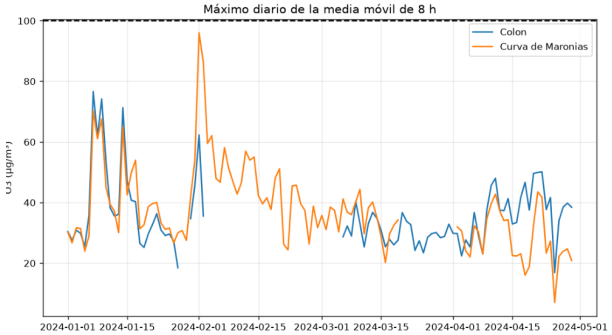
Los componentes de una serie

El análisis clásico (de la Fuente) descompone una serie en cuatro partes:

Componente	Qué es	En nuestros datos
Tendencia (T)	movimiento regular a largo plazo	el ozono baja de enero a abril
Estacional (E)	oscilación que se repite con período fijo	el ciclo de 24 h
Cíclica (C)	oscilación de medio plazo, período irregular	no se ve en 4 meses
Irregular (A)	lo esporádico e imprevisible	lo demás

Esquema **aditivo** ($Y = T + E + A$), que es el que usamos; el **multiplicativo** ($Y = T \cdot E \cdot A$) sirve cuando la oscilación crece con el nivel. En los manuales la "estación" es la del año; acá es la **hora del día** y el período es un día (144 pasos de 10 min). Misma idea, otra escala.

Tendencia: media móvil

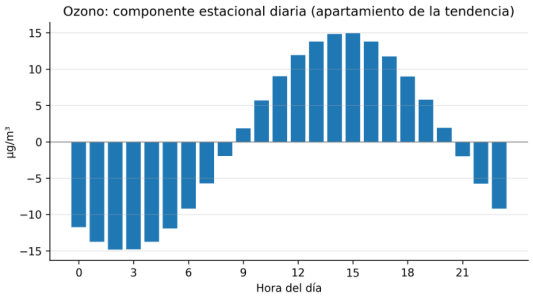


Ya vieron una media móvil: en la clase 7, la de 8 horas que usa la guía OMS. La misma herramienta, con otra ventana, estima la tendencia:

```
serie["o3_tend"] = serie["o3"].rolling(POR_DIA, center=True).mean()
```

- La ventana es el **período** (un día): al promediar un día entero, el ciclo diario se cancela y queda el nivel de fondo.
- `center=True`: cada valor va al centro de su ventana. Se pierde medio día en cada extremo. Una media móvil describe; **no predice**. Sección 4b.

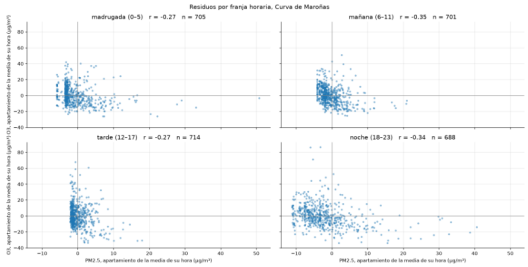
Estacionalidad: el perfil por hora




```
sin_tend = serie["o3"] - serie["o3_tend"]
perfil = sin_tend.groupby(serie.index.hour).mean()     # 24 valores
```

- Sacada la tendencia, queda estacional + irregular. Promediando por hora del día, lo irregular se cancela y queda el ciclo.
- Son los **índices estacionales** del esquema aditivo: cuánto se aparta cada hora de la tendencia. Suman aproximadamente cero.
- Es el perfil horario de la clase 9, ahora con nombre. Script, sección 4c.

Lo irregular

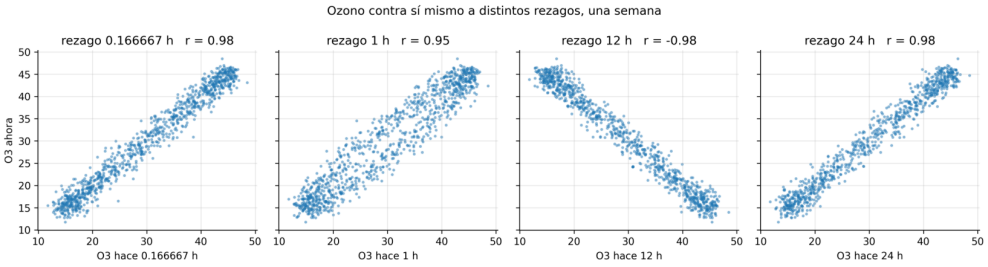



```
serie["o3_irreg"] = sin_tend - serie
```

- Lo irregular es lo que sobra: serie menos tendencia menos el índice de su hora. Cuánto ese día, a esa hora, se apartó de lo esperable.
- **Desestacionalizar** es restar el componente estacional (o dividir, en el esquema multiplicativo). La figura es eso, hecho en la clase 9 con la media de cada hora: los ejes ya son “apartamiento de lo normal para esa hora”.
- Describe la serie; no la explica. Qué produce el ciclo es una pregunta de la disciplina. Sección 4c.

Autocorrelación

La memoria de una serie: gráfico de rezago



```
ax.scatter(semana["o3"].shift(k), semana["o3"])
```

`shift(k)` corre la serie k renglones: en cada fila queda el valor de hace k pasos. Es una dispersión de la serie contra sí misma. Script, sección 5.

Autocorrelación

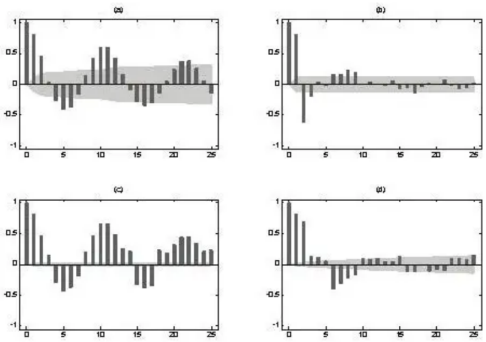
El *r* del gráfico de rezago es una correlación de Pearson entre dos tramos de la serie, cada uno con su media y su desvío. La **autocorrelación de orden k** fija la media y la varianza de toda la serie, las mismas para todo *k*:

$$r_k = \frac{\sum_{t=k+1}^{N} (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{N} (y_t - \bar{y})^2}$$

```
x = (serie["o3"] - serie["o3"].mean()).fillna(0)   # NaN -> 0
acf = [(x * x.shift(k)).sum() / (x ** 2).sum() for k in rezagos]
```

- $r0 = 1$ siempre; rk alto: lo de ahora se parece a lo de hace k pasos.
- Un faltante reemplazado por la media aporta cero: es la convención de `acf()` en R y de `plot_acf` en Python, que calculan exactamente esto.
- La sucesión $r1, r2, \ldots$ es la **función de autocorrelación** (ACF); su gráfico de barras es el **correlograma** (Mauricio 2.2.9).
- La banda $\pm 1{,}96/\sqrt{N}$ es la referencia: una barra que la supera se toma como distinta de cero. Con N grande es muy angosta. Sección 6, una barra por hora de rezago, hasta 48 h.

Leer un correlograma

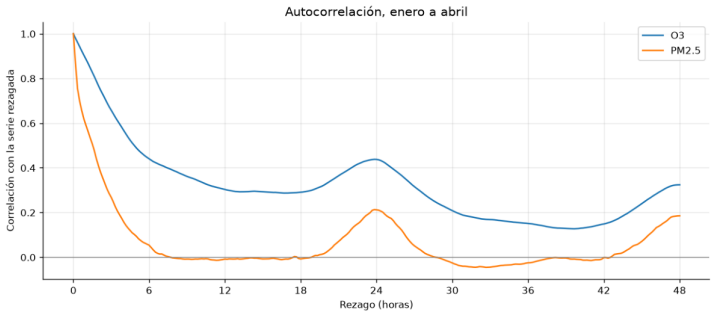


Formas típicas (Mauricio, cap. 2):

- **Cae rápido a cero** (b, d): poca memoria. Lo de hace unas horas ya no dice nada sobre ahora. En (b) solo importa el rezago 1.
- **Cae muy lento:** hay tendencia. Todo se parece a todo porque el nivel cambia despacio.
- **Sube y baja con período fijo** (a, c): hay estacionalidad. El correlograma repite el ciclo de la serie; en (c) el período es 12.

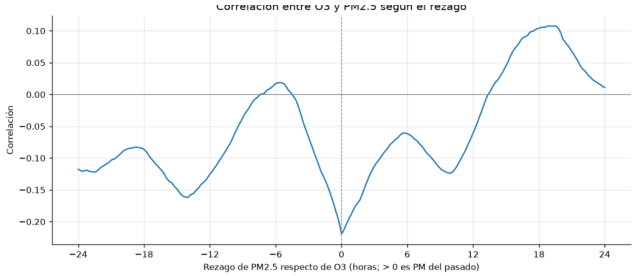
La banda gris es la referencia $\pm 1{,}96/\sqrt{N}$. Un correlograma real puede mostrar varias formas a la vez, en distintas zonas del eje.

Nuestro correlograma



- **Horas:** PM2.5 llega a cero en 7 h; el ozono a las 6 h sigue en 0,45. El ozono tiene más memoria de corto plazo.
- **24 y 48 h:** las dos tienen la joroba del ciclo diario. Modesta.
- **El piso del ozono:** entre 12 y 18 h de rezago no baja de 0,3. Los días se parecen a los días: regímenes que duran más de una jornada, y algo de la tendencia enero–abril. El PM2.5 no tiene ese piso.

Correlación cruzada



$$r_{xy}(k) = \frac{\sum_t (x_t - \bar{x})(y_{t-k} - \bar{y})}{\sqrt{\sum_t (x_t - \bar{x})^2 \, \sum_t (y_t - \bar{y})^2}}$$

- La misma construcción con dos series: ozono de ahora contra PM2.5 de hace k pasos, cada una centrada con su media. Es `ccf()` en R.
- En $k = 0$ es el r de la clase 9. El resto dice a qué distancia temporal la relación es más fuerte. Las dos series tienen ciclo diario, así que la curva también. Script, sección 7.

Cierre

Resumen

Concepto	Script	Herramienta
Serie de tiempo: valores a paso fijo, sin huecos	1	`set_index("fecha")`
Paso de muestreo	2	`diff().value_counts()`
Grilla completa por cajas	3	`resample(PASO).mean()`
Tendencia	4b	`rolling(POR_DIA, center=True)`
Estacional (perfil por hora)	4c	`groupby(index.hour).m`
Irregular (lo que sobra)	4c	una resta
Rezago	5	`shift(k)`
Autocorrelación r_k	6	la fórmula, en un `for`
Correlación cruzada $r_{xy}(k)$	7	ídem, con dos series

Bibliografía

- S. de la Fuente Fernández, Estadística: Series temporales, Universidad Autónoma de Madrid. Definición, stock y flujo, componentes, medias móviles, índices estacionales. Es la base de la primera mitad.
- J. A. Mauricio, Introducción al Análisis de Series Temporales, Universidad Complutense de Madrid, 2007. Capítulo 1 (definiciones, ejemplos de series) y sección 2.2 (ACF muestral, correlograma). Todo lo demás del libro es un curso aparte.

Datos: ozono y PM2.5, Curva de Maroñas, enero a abril de 2024, Catálogo de Datos Abiertos de Uruguay, Intendencia de Montevideo.

17. Tránsito y calidad del aire

Series de tiempo, correlación y regresión lineal
Datos abiertos de Montevideo: conteo vehicular, NO_2 y PM2.5, junio de 2024
Introducción a la Ciencia de Datos · Universidad de Montevideo

Los datos

Fuente	Qué mide	Cada cuánto	Dónde
Conteo vehicular	Vehículos que pasan, por carril	5 minutos	224 puntos en avenidas
NO_2	Dióxido de nitrógeno en el aire ($\mu g/m^3$)	1 minuto	2 estaciones
PM2.5	Material particulado fino ($\mu g/m^3$)	1 minuto	3 estaciones

Fuente: Catálogo de Datos Abiertos del Uruguay (Intendencia de Montevideo).

Período: junio de 2024, el mes en que las tres fuentes tienen datos a la vez.

Unidad de observación: una hora. Todo se lleva a valores por hora para poder compararlo.

Pregunta: si el tránsito influye en el aire, ¿qué esperarían ver en los datos? Escriban su hipótesis antes de seguir.

Qué medimos y en qué unidades

Concentración

Los contaminantes se miden en microgramos por metro cúbico ($\mu g/m^3$): cuánta masa de la sustancia hay en un metro cúbico de aire. Un microgramo es una millonésima de gramo.

NO_2, dióxido de nitrógeno

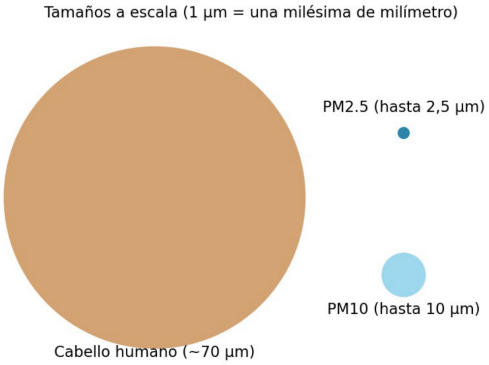
Gas que se forma en la combustión de los motores. Dura poco en el aire.

PM2.5, material particulado fino

Partículas de hasta 2,5 micrómetros, unas 30 veces más finas que un cabello. Vienen de motores, quemas, polvo y reacciones en el aire, y pueden quedar suspendidas horas o días.

Tránsito

Vehículos por hora. Al sumar varios puntos de conteo, un mismo auto puede contarse más de una vez: es un indicador de actividad, no una cantidad de autos.



Series de tiempo

Qué es

Una variable medida muchas veces, en orden. Cada valor tiene fecha y hora.

Paso de muestreo

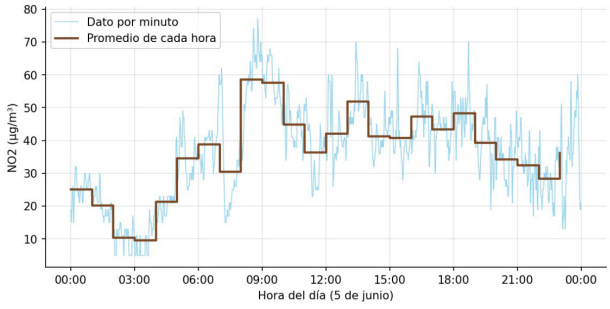
Cada cuánto hay un dato: el NO_2 cada minuto, el tránsito cada 5 minutos.

Agregar

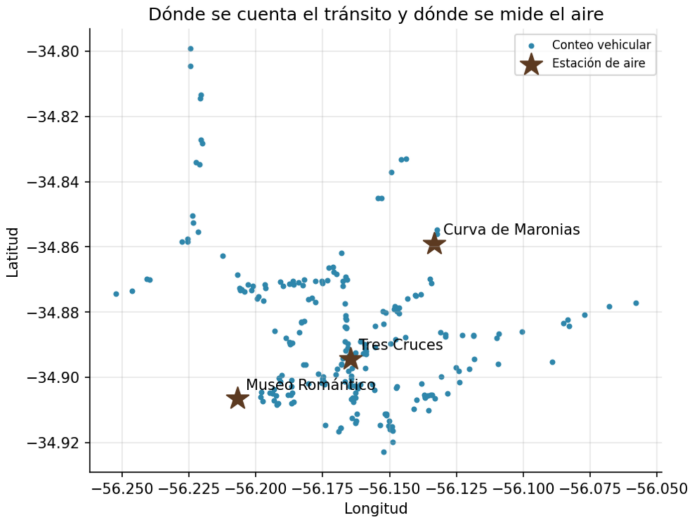
Para comparar, llevamos todo al mismo paso: el promedio de cada hora (la suma, en el caso del tránsito).

Cómo leer el gráfico

- Eje horizontal: el tiempo.
- Eje vertical: el valor.
- Celeste: dato por minuto. Marrón: promedio de cada hora.



Paso 1: dónde se mide



Cómo leer el gráfico

- Eje horizontal: longitud (oeste a este).
- Eje vertical: latitud (sur a norte).
- Puntos celestes: conteo vehicular. Estrellas: estaciones de aire.

Qué vemos

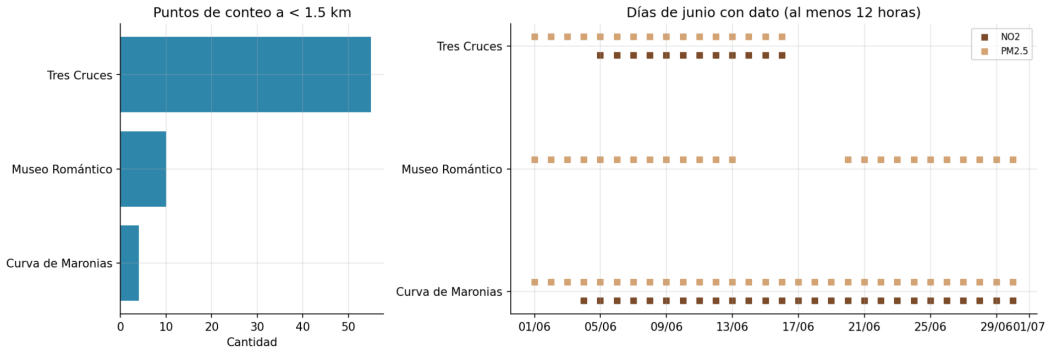
Los puntos de conteo están repartidos de forma desigual: muchos alrededor de una estación, pocos alrededor de otras.

En Python

```
puntos = auto.groupby(
  ["latitud", "longitud"]).size()
```

Pregunta: ¿en qué estación buscarían la relación con el tránsito?

Paso 2: elección de la estación



Cómo leer el gráfico

- Izquierda: puntos de conteo a menos de 1,5 km de cada estación.
- Derecha: cada cuadrado es un día con dato (marrón NO_2, arena PM2.5).

Qué vemos

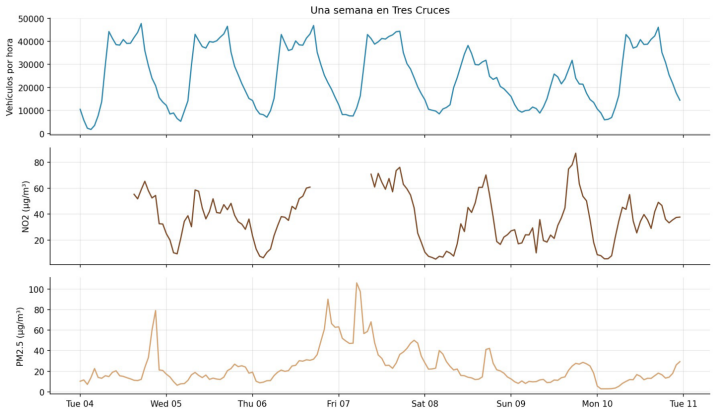
Tres Cruces tiene 55 puntos cerca pero NO_2 solo la primera mitad del mes. Curva de Maroñas tiene NO_2 todo el mes y 4 puntos. Museo Romántico no mide NO_2.

Pregunta: ¿más días de datos o la causa mejor medida?

En Python

```
d = dist_km(lat, lon,
            lat_est, lon_est)
cerca = puntos[d < 1.5]
```

Paso 3: una semana de datos



Cómo leer el gráfico

- Eje horizontal: los días de una semana.
- Eje vertical: el valor de cada serie; cada panel tiene su propia escala.
- Los cortes en la línea son horas sin dato.

Qué vemos

El tránsito repite su forma cada día y baja el fin de semana. El NO_2 sube y baja parecido, con más ruido. El PM2.5 tiene episodios.

En Python

```
semana = df["2024-06-04":"2024-06-10"]
```

Pregunta: ¿qué día es cada curva? ¿Por qué el domingo el NO_2 llega a 87?

Promedio y normalización

Perfil horario

Para cada hora del día (0 a 23) se promedian todos los días del mes: es la forma de un día típico.

Normalizar

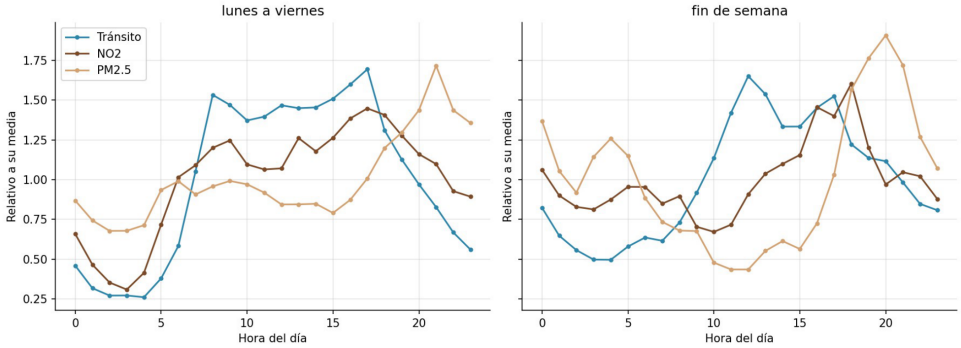
Tránsito y NO_2 tienen unidades distintas. Para compararlos se divide cada valor por el promedio de su serie: 1 es el promedio, 2 el doble, 0,5 la mitad.

Ejemplo con los datos de junio

	Promedio	3 h	Pico de la tarde
Tránsito (veh/h)	25.254	7.625	47.386
÷ promedio	1	0,30	1,88
NO_2 (µg/m³)	38,2	12,2	55,4
÷ promedio	1	0,32	1,45

A las 3 de la mañana el tránsito está en 0,30 de su promedio y el NO_2 en 0,32: los dos en alrededor de un tercio de lo normal. Con unidades distintas no se veía; normalizado se compara directamente.

Paso 4: perfil horario



Cómo leer el gráfico

- Eje horizontal: hora del día (0 a 23).
- Eje vertical: valor dividido por su promedio.
- Izquierda: lunes a viernes. Derecha: fin de semana.

Qué vemos

El NO_2 sigue al tránsito. Los picos llegan en orden: tránsito, NO_2, PM2.5 (entre semana: 17 h, 18 h y 21 h).

Pregunta: ¿por qué el PM2.5 sube cuando el tránsito ya baja?

En Python

```
perfil = df.groupby(
  [tipo_dia, df.index.hour]).mean()
perfil / perfil.mean()
```

Correlación

Qué mide

El coeficiente r mide cuánto dos variables suben y bajan juntas en línea recta. Va de −1 a 1.

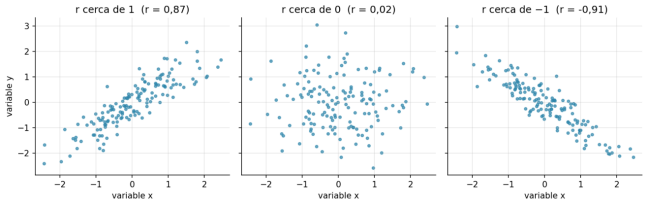
- Cerca de 1: cuando una sube, la otra también.
- Cerca de −1: cuando una sube, la otra baja.
- Cerca de 0: sin relación lineal.

Diagrama de dispersión

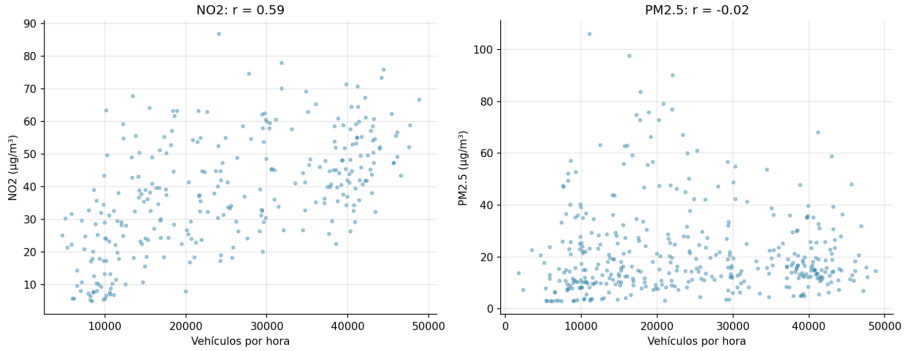
Cada punto es una observación; cada eje, una variable.

Correlación no es causa

Las ventas de helado y los golpes de calor suben juntos, porque los dos dependen del verano. Con el aire pasa algo parecido con la hora del día.



Paso 5: correlación



Cómo leer el gráfico

- Cada punto es una hora.
- Eje horizontal: vehículos por hora.
- Eje vertical: el contaminante en esa hora.

Qué vemos

NO_2: r = 0,59. PM2.5: r = −0,02. Pero los dos tienen ciclo diario, y parte de la r del NO_2 viene de ahí.

Pregunta: ¿todo lo que sube de día está relacionado?

En Python

```
r = df["transito"].corr(df["no2"])
```

Desvío respecto de lo habitual

Idea

A cada hora se le resta lo que suele pasar a esa hora: el promedio de esa hora, separando días de semana y fin de semana.

desvío = valor medido − valor habitual

Qué queda

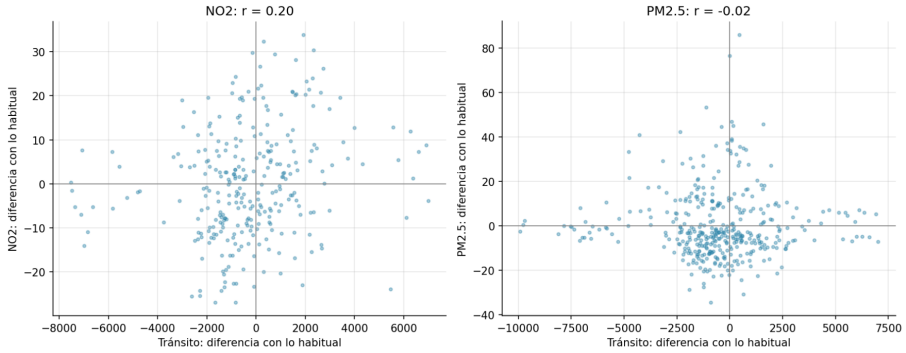
- Positivo: más de lo habitual.
- Negativo: menos de lo habitual.
- Cero: una hora normal.

Al restar lo habitual desaparece el ciclo diario, y la pregunta pasa a ser: cuando hay más tránsito que lo normal, ¿hay más contaminante que lo normal?

Viernes 14 de junio, 18 h	Tránsito (veh/h)
Medido	34.794
Habitual a las 18 h, días de semana	36.668
Desvío	−1.874

Esa hora pasaron unos 1.900 vehículos menos que en un viernes normal a las 18.

Paso 6: sin el ciclo diario



Cómo leer el gráfico

- Cada punto es una hora.
- Ejes: diferencia con lo habitual.
- El cruce de las líneas grises es una hora normal.

Qué vemos

La r del NO_2 baja de 0,59 a 0,20: la mayor parte era el ciclo diario. No llega a cero. El PM2.5 sigue en cero.

Pregunta: ¿qué significa un punto arriba a la derecha?

En Python

```
def desvio(s):
  clave = [s.index.dayofweek >= 5,
           s.index.hour]
  return s - s.groupby(clave)
           .transform("mean")
```

Rezago

Idea

El efecto puede llegar después de la causa. Rezagar es comparar el contaminante de ahora con el tránsito de k horas antes.

Cómo se hace

Se copia la columna del tránsito corrida k filas. En la tabla, k = 1.

Qué se grafica

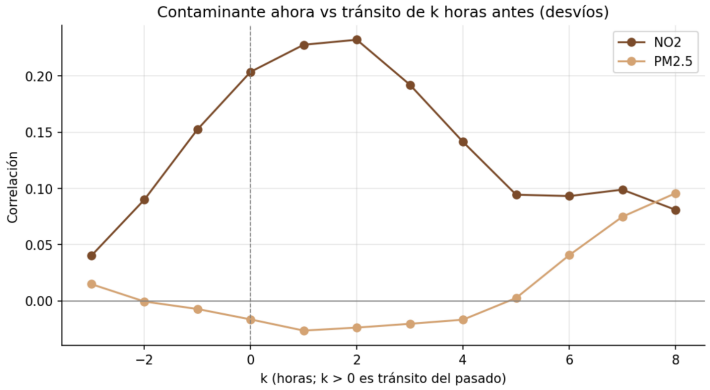
Para cada k se calcula una correlación. En el gráfico del paso 7, k va en el eje horizontal y la correlación en el vertical.

k negativo: el contaminante antes que el tránsito. Si la causa es el tránsito, ahí no debería aparecer relación.

Lunes 10/6	Tránsito	Tránsito 1 h antes
6 h	16.524	—
7 h	30.533	16.524
8 h	42.952	30.533
9 h	41.132	42.952
10 h	37.074	41.132

```
df["transito"].shift(1)
```

Paso 7: correlación con rezagos



Cómo leer el gráfico

- Eje horizontal: k, horas de rezago del tránsito.
- Eje vertical: correlación de los desvíos.
- Cada punto es una correlación como la del paso 6.

Qué vemos

El NO_2 responde en 1 a 2 horas y se apaga en unas 5. El PM2.5 no reacciona en las primeras horas y empieza a subir donde termina el gráfico.

En Python

```
[res["no2"].corr(
   res["transito"].shift(k))
 for k in range(-3, 9)]
```

Pregunta: ¿qué harían con el PM2.5?

Comparación con el azar

El problema

Una correlación de 0,2 puede salir por casualidad, sobre todo si se busca el mejor rezago entre varios.

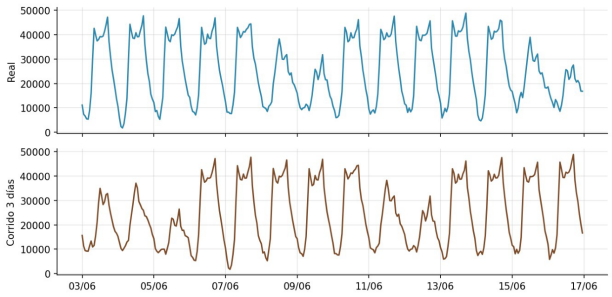
La idea

Se corre toda la serie de tránsito varios días (lo que sale por el final entra por el principio). Así se rompe cualquier relación real con el aire, pero el tránsito conserva su forma.

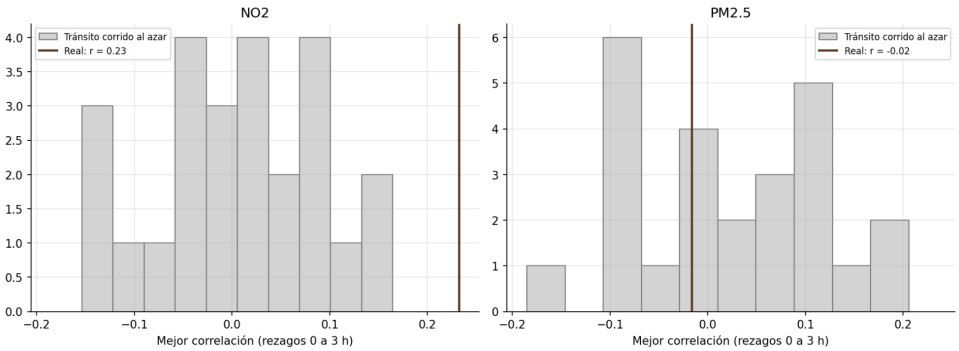
Con cada corrimiento se calcula la correlación: eso es lo que da el azar.

La regla

Si la correlación real queda por encima de casi todas las del azar, es difícil explicarla por casualidad.



Paso 8: comparación con el azar



Cómo leer el gráfico

- Barras grises: cuántas veces el azar dio cada correlación.
- Línea marrón: la correlación real.
- Eje horizontal: correlación (rezagos de 0 a 3 h).

Qué vemos

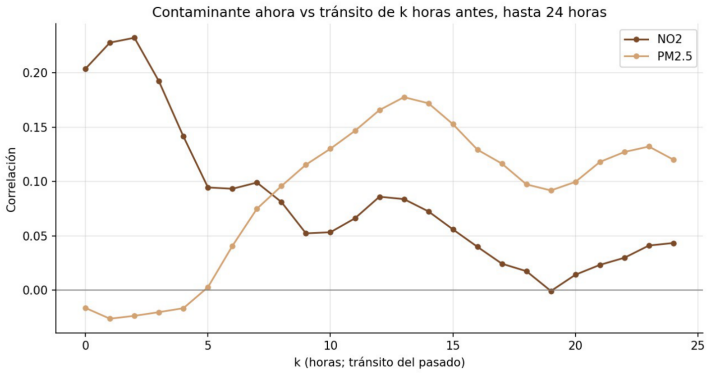
NO_2: ninguna corrida iguala la correlación real. PM2.5, con rezagos de 0 a 3 horas: queda en el medio del azar.

Pregunta: ¿cómo sabemos que un resultado no es suerte?

En Python

```
np.roll(res["transito"].to_numpy(),
        24 * d)
```

Paso 9: rezagos de hasta 24 horas



Cómo leer el gráfico

- Igual que el paso 7, con k hasta 24 horas.
- Eje horizontal: horas de rezago.
- Eje vertical: correlación.

Qué vemos

El NO_2 responde en 1 a 2 horas. El PM2.5 sube desde las 5 horas y llega a su máximo a las 13: se acumula. Tránsito de la mañana (8 h) + 13 h = 21 h, el pico del PM2.5 en el paso 4.

En Python

```
[res["pm25"].corr(
   res["transito"].shift(k))
 for k in range(0, 25)]
```

Pregunta: ¿de qué hora es el tránsito que respiramos a las 21?

Regresión lineal

La recta

y = a + b·x. La ordenada a es dónde la recta corta el eje vertical; la pendiente b es cuánto sube y cuando x sube una unidad.

Mínimos cuadrados

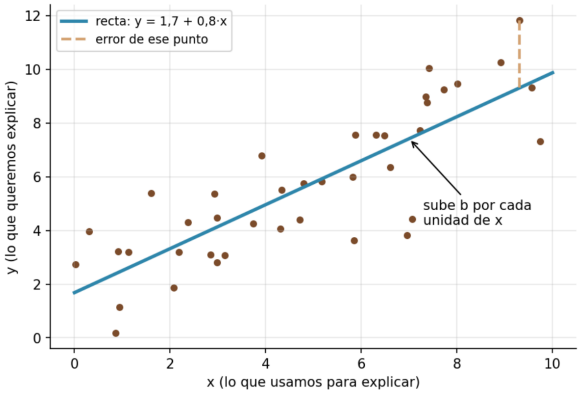
De todas las rectas posibles se elige la que hace más chica la suma de los errores al cuadrado. La línea punteada es el error de un punto.

R^2

Qué parte de la variación de y reproduce el modelo. Va de 0 a 1.

Cuidado

Una pendiente clara no implica que la recta explique mucho: los puntos pueden estar lejos.



Paso 10: modelo de regresión

lo que medimos = lo habitual de esa hora + lo que agrega el tránsito + lo que no sabemos

$y(t) = h(t) + a + b \cdot x(t - k) + \text{error}$

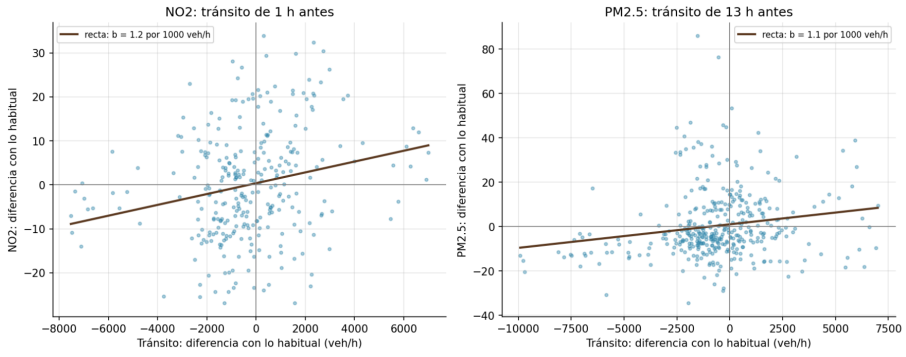
- h(t): lo habitual, el promedio de esa hora del día (pasos 4 y 6).
- x(t − k): desvío del tránsito de k horas antes; k = 1 para el NO_2 y k = 13 para el PM2.5 (pasos 7 y 9).
- a, b: la recta entre los desvíos, por mínimos cuadrados.
- error: lo que el modelo no incluye (viento, lluvia, otras fuentes).

	k	b *	R^2 solo h(t)	R^2 con tránsito
NO_2	1 h	1,2	0,47	0,50
PM2.5	13 h	1,1	0,19	0,22

* µg/m³ por cada 1.000 vehículos por hora de más

```
b, a = np.polyfit(x_desvio, y_desvio, 1)     # la recta entre los desvíos
modelo = habitual + a + b * x_desvio         # la predicción
```

Paso 10a: la recta entre los desvíos



Cómo leer el gráfico

- Cada punto es una hora.
- Eje horizontal: desvío del tránsito, k horas antes.
- Eje vertical: desvío del contaminante. La recta, en marrón.

Qué vemos

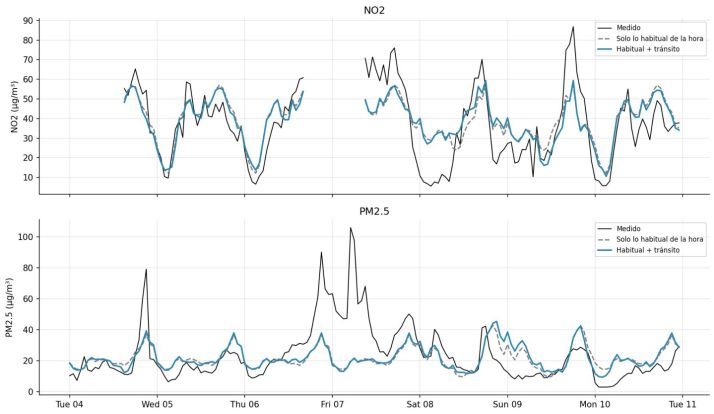
b = 1,2: con 5.000 vehículos por hora de más, el NO_2 sube unos 6 µg/m³, cerca del 15 % de su nivel. El PM2.5 responde con fuerza parecida, 13 horas después.

Pregunta: ¿cuánto bajaría el NO_2 con 5.000 autos menos por hora?

En Python

```
b, a = np.polyfit(
  d["transito_desvio"],
  d["desvio"], 1)
```

Paso 10b: modelo y valores medidos



Cómo leer el gráfico

- Eje horizontal: una semana, hora a hora.
- Negro: medido. Gris: solo lo habitual. Celeste: habitual más tránsito.

Qué vemos

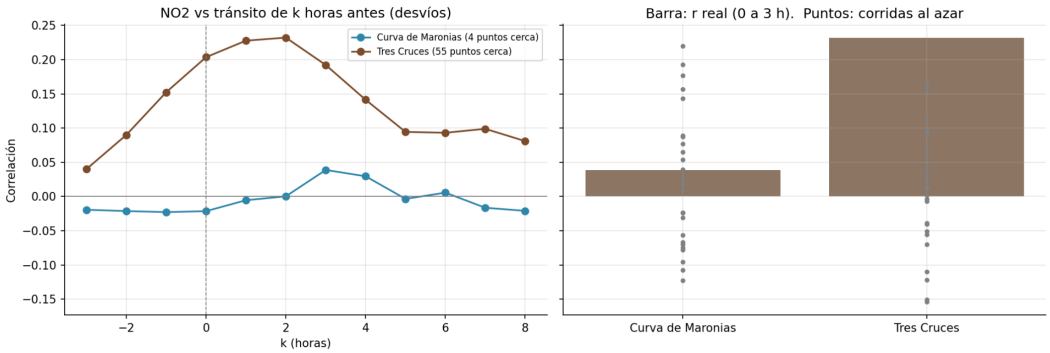
Sin ninguna otra variable, el modelo sigue la forma de cada día. Se equivoca en los episodios y en algunas noches: ahí está lo que no incluye.

En Python

```
modelo = habitual + a + b * x_desvio
```

Pregunta: ¿qué pasó la noche del viernes 7, cuando el NO_2 cayó casi a cero?

Paso 11: la misma pregunta en otra estación



Cómo leer el gráfico

- Izquierda: como el paso 7, una línea por estación.
- Derecha: barra = correlación real; puntos grises = corrimientos al azar.

Qué vemos

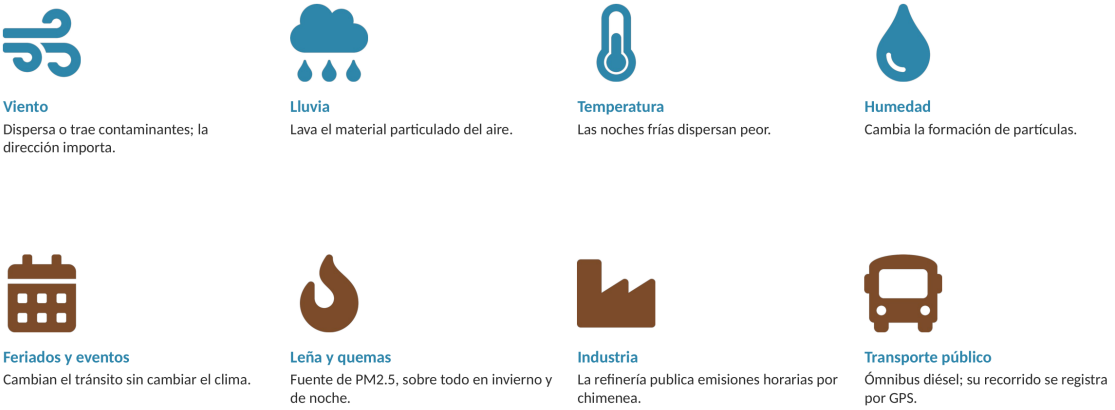
Tres Cruces (55 puntos de conteo cerca): la relación aparece y supera al azar. Curva de Maroñas (4 puntos): queda en medio del azar.

Pregunta: ¿que no se vea significa que no existe?

En Python

```
for est in estaciones_no2:
    serie, cerca = serie_estacion(est)
```

Otras variables que podrían explicar el aire



Cómo se incorporarían al análisis

$y(t) = h(t) + b \cdot \text{tránsito}(t - k) + c \cdot \text{viento}(t) + d \cdot \text{lluvia}(t) + e \cdot \text{feriado}(t) + \text{error}$

Variables numéricas

Viento, lluvia, temperatura: una columna más, cada una con su coeficiente.

Variables categóricas

Feriado sí o no, día de semana: se codifican con 0 y 1, como ya hicimos al separar semana y fin de semana.

Varias estaciones a la vez

Las mismas horas en distintos lugares: datos de panel y datos espaciales (unidad 7).

Cambios puntuales

Un corte de calle, una obra, la parada de la refinería: comparar antes y después contra un lugar que no cambió es el método de diferencias en diferencias (unidad 8).

Datos faltantes

Cada variable nueva trae sus propios huecos: hay que decidir qué hacer con las horas sin dato.

Conclusiones

- El NO_2 responde al tránsito en 1 a 2 horas y la relación se apaga en pocas horas.
- El PM2.5 responde más tarde, alrededor de 13 horas: acumula el tránsito del día y muestra su pico de noche.
- Con solo la hora del día y el tránsito, sin otras variables, el modelo reproduce la mitad de la variación del NO_2 ($R^2 = 0{,}50$) y una quinta parte de la del PM2.5 ($R^2 = 0{,}22$).
- La relación se ve donde el tránsito está bien medido; donde hay pocos puntos de conteo no se distingue del azar.

Limitaciones

- Unos 12 días de NO_2 en una estación.
- No se separa el tránsito de las calles cercanas del de toda la ciudad.
- Faltan variables de tiempo meteorológico y otras fuentes.

Pregunta: ¿cuáles de las hipótesis que escribieron al principio se sostuvieron?

18. Preguntas, gráficos y una primera mirada a las series de tiempo

Introducción a la Ciencia de Datos
20 de octubre de 2026

Primera hora: qué preguntas responden los datos

Antes de los datos, la pregunta

Una **pregunta de ciencia de datos** es una pregunta que se puede responder con datos que existen o que se pueden obtener, mediante un análisis que se puede describir de antemano.

Leek y Peng (2015) observan que muchos errores de análisis no son errores de cálculo: son errores de **tipo de pregunta**. Se calcula bien una correlación y se la presenta como una causa; se describe una muestra y se la presenta como el país entero.

Hoy vamos a leer titulares reales de diarios uruguayos y preguntarnos tres cosas:

- qué pregunta hay detrás del titular, y de qué tipo es;
- qué datos y qué análisis harían falta para responderla;
- qué gráfico le falta a la noticia.

Seis tipos de pregunta (Leek y Peng, 2015)

Tipo	Qué busca	Ejemplo en los titulares
Descriptiva	Resumir los datos que se tienen	«El desempleo baja a 7% en junio»
Exploratoria	Buscar patrones no planteados de antemano	Desempleo por edad y sexo
Inferencial	De una muestra a la población, con incertidumbre	Encuesta de 900 casos, ± 3,3%
Predictiva	Anticipar un valor futuro o de un individuo	«¿A cuánto estará el dólar?»
Causal	Qué pasa con una variable si se cambia otra	Ley de alcohol cero y siniestros
Mecanística	Cómo, paso a paso, un cambio produce otro	Por qué el NO_2 tiene dos picos diarios

El tipo no lo decide el titular: lo decide qué datos hay y cómo se obtuvieron.

Las confusiones más frecuentes

Exploratoria leída como causal. Dos variables se mueven juntas (café y longevidad, hogares unipersonales y alquileres) y el titular dice que una causa la otra. Faltan el grupo de comparación y las variables que podrían explicar las dos.

Descriptiva leída como inferencial. Una muestra autoseleccionada (responde quien

quiere, por internet) se presenta como la opinión del país.

Inferencial sin incertidumbre. «La pobreza bajó de 17,3% a 16,6%»: si la diferencia es menor que el margen de error, los datos no permiten afirmar que bajó.

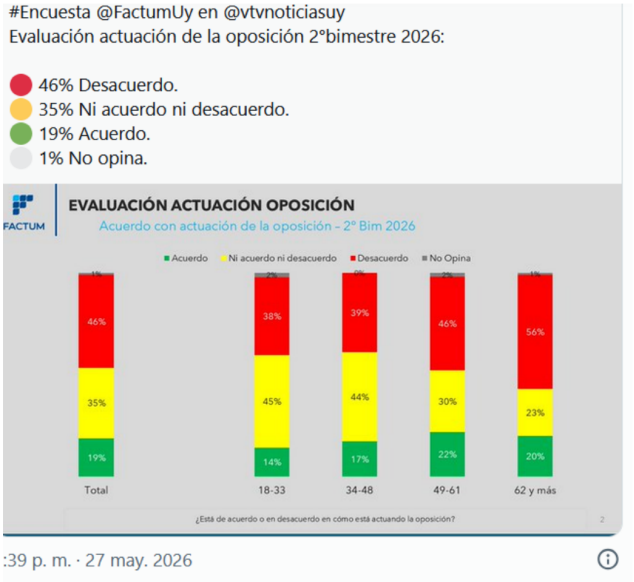
Cambio de medición leído como cambio de realidad. Una nueva metodología de pobreza, más cámaras que registran accidentes, denuncias en lugar de delitos.

Tipo de pregunta y tipo de análisis

El tipo de análisis depende de la **estructura de los datos**, que es lo que organiza el programa del curso:

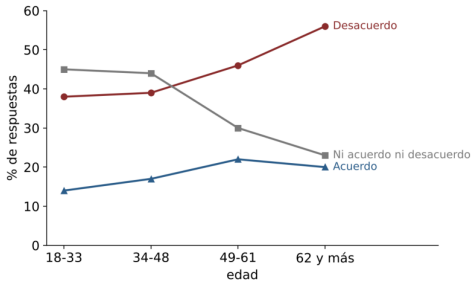
Estructura de los datos	Análisis	Titulares
Una variable medida a intervalos regulares	Serie temporal	Desempleo mensual, homicidios por semestre
Unidades clasificadas en grupos	Comparación entre grupos	Desempleo por edad, pobreza por tramo
Dos variables en las mismas unidades	Correlación, dispersión	Educación y desempleo
Unidades ubicadas en el espacio	Datos espaciales	Pobreza por municipio
Antes y después de una intervención	Antes/después	Alcohol cero, regulación del cannabis
Intervenidos y no intervenidos	Grupo de control, diferencias en diferencias	Uruguay contra Chile

Un mismo dato, tres gráficos: el gráfico que publicó Factum

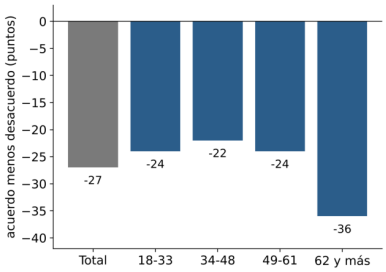


Factum, segundo bimestre de 2026, 900 casos por celular. Barras apiladas al 100% por edad. Abajo y arriba se comparan bien: comparten una base. El segmento del medio no: cada uno empieza a otra altura. El semáforo carga de juicio las respuestas y confunde a quien no distingue rojo y verde.

Un mismo dato, tres gráficos: líneas y saldo



Con la edad, lo neutral pasa a desacuerdo. El acuerdo casi no cambia.



Saldo neto = acuerdo menos desacuerdo. Casi igual bajo los 62 años.

Actividad: titulares

Cada equipo recibe tres tarjetas con titulares reales y el texto de la noticia. Para cada una:

1. Identificar la pregunta que hay detrás del titular y clasificarla.
2. Indicar el tipo de análisis que usa o necesitaría.
3. Indicar qué datos se necesitan para responderla.
4. Dibujar el gráfico que le falta a la noticia, con los datos del texto si alcanzan: tipo de gráfico, variable de cada eje, qué representa cada punto o barra.

Para ir más lejos. Decidir si el titular afirma más de lo que los datos permiten y reescribirlo para que diga solo lo que los datos sostienen.

El gráfico se dibuja a mano. Al final, cada equipo presenta una tarjeta y su gráfico.

Segunda hora: una primera mirada a las series de tiempo

Lo que los diarios hacen sin decirlo

Tres titulares de la primera hora comparan de una manera particular:

- «Desempleo inició 2026 con una suba **respecto a diciembre** de 2025»
- «Salario real aumentó en junio un 0,88% **en comparación interanual**»
- «Los delitos bajaron 3,7% en el primer semestre de 2026 **respecto a igual período** de 2025»

¿Por qué comparar junio con junio, y no junio con mayo? Porque hay variables que cambian con la época del año, y comparar meses distintos mezcla ese cambio con el que nos interesa.

Para razonar sobre esto necesitamos un objeto nuevo: la **serie de tiempo**.

Qué es una serie de tiempo

Definición. Una serie de tiempo es un registro sistemático a lo largo del tiempo de una variable de interés (Rincón y De Jesús Romo, 2024). Se escribe

$x1, x2, \dots, xn,$

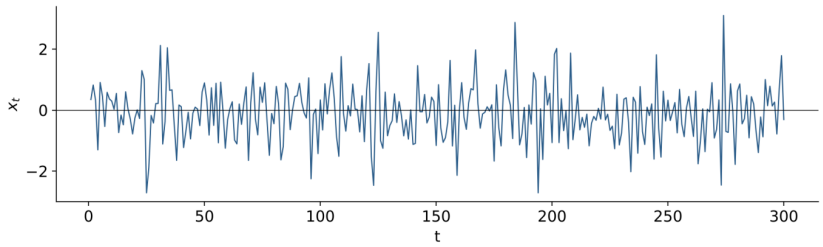
donde xt es el valor observado en el momento t.

Tres condiciones prácticas (de la Fuente; Mauricio):

- **Ordenada**: el orden de las observaciones es parte del dato.
- **A intervalos iguales**: cada hora, cada mes, cada año.
- **De una sola unidad**: una estación, un país, una empresa. Dos unidades son dos series.

Se estudian para **describir, interpretar, controlar y pronosticar** el comportamiento de la variable (Rincón y De Jesús Romo, 2024).

Primer modelo elemental: el ruido blanco



Definición. Un **ruido** es una sucesión $Z1, Z2, \dots$ de variables aleatorias independientes, todas con la misma distribución, de media μ y varianza $\sigma 2$. Si además son normales con media 0 se llama **ruido blanco**, $Zt \sim WN(0, \sigma 2)$. Lo que pasó en t no dice nada sobre lo que pasa en $t+1$: no hay memoria.

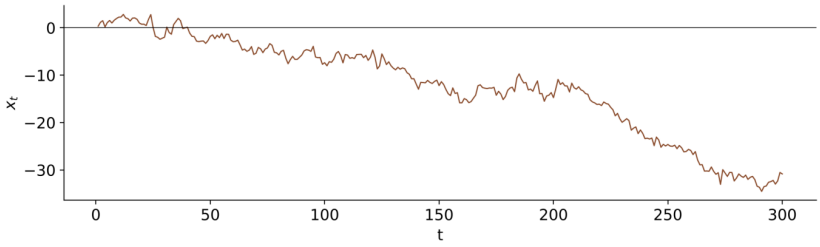
Para qué sirve el ruido blanco

Es el modelo de lo impredecible. Describe la parte de una serie que no se puede anticipar a partir de su pasado: por ejemplo, el error de medición de un sensor. Si una serie es ruido, el mejor pronóstico es siempre su media, sin importar lo que pasó antes.

Es la pieza con la que se construyen los demás modelos. La caminata aleatoria, que viene a continuación, es una suma de ruidos. Los modelos que veremos en las próximas clases también se arman a partir de un ruido.

Es la referencia para leer un correlograma. Cuando se calcule la autocorrelación de una serie, la pregunta será si se distingue de la de un ruido. Si no se distingue, en esa serie no hay memoria que aprovechar.

Segundo modelo elemental: la caminata aleatoria



Definición. Una **caminata aleatoria** acumula ruidos:

$$X_t = X_{t-1} + Z_t, \qquad X_0 = x_0.$$

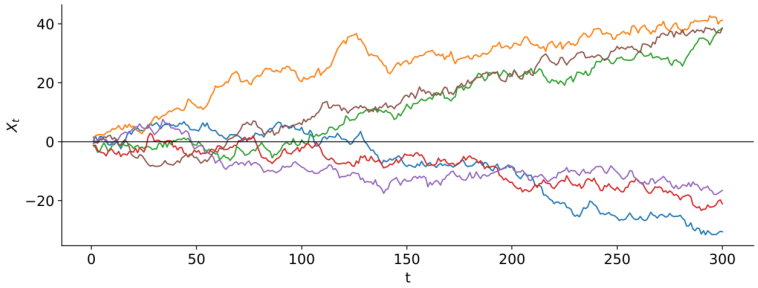
Cada valor es el anterior más un paso al azar. Desarrollando:

$$X_t = x_0 + Z_1 + Z_2 + \cdots + Z_t.$$

Para qué sirve la caminata aleatoria

Es el modelo más simple de un precio. Un tipo de cambio o el precio de una acción se parecen a una caminata: cada día parten del valor del día anterior y se mueven un poco hacia arriba o hacia abajo. **Da el pronóstico ingenuo.** Si $Xt+1 = Xt + Zt+1$ y el paso $Zt+1$ tiene media 0 (es tan probable que suba como que baje), el mejor pronóstico de $Xt+1$ es Xt: el valor de mañana se pronostica con el de hoy. Es la referencia contra la que se compara cualquier método de pronóstico más elaborado: si no le gana a "mañana igual que hoy", no sirve.

Una caminata parece tener tendencia, aunque no la tiene



Seis caminatas generadas con **el mismo mecanismo**: pasos al azar de media

0. Ninguna tiene una tendencia, porque nada en el mecanismo empuja hacia

arriba ni hacia abajo. Sin embargo, tres terminan muy arriba y tres muy abajo.

Las subidas y bajadas largas son azar acumulado. Ver una serie que sube durante un tiempo no alcanza para decir que tiene tendencia: también podría ser una caminata.

Cómo se mide la memoria: media y autocovarianza

Con una serie observada $x1, \ldots, xn$:

Media muestral

$$\bar{x} = \frac{1}{n}\sum_{t=1}^{n} x_t$$

Autocovarianza muestral al rezago τ: cuánto varían juntos los valores separados por τ pasos.

$$\text{cov}_x(\tau) = \frac{1}{n}\sum_{t=1}^{n-\tau}(x_t - \bar{x})(x_{t+\tau} - \bar{x})$$

Con $\tau = 0$ es la varianza de la serie.

La autocorrelación

Definición. La **autocorrelación muestral** al rezago τ es

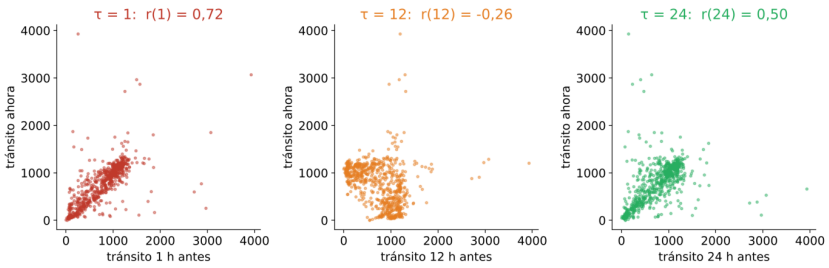
$$r(\tau) = \frac{\text{cov}_x(\tau)}{\text{cov}_x(0)}.$$

- Vale entre -1 y 1, y $r(0) = 1$.
- $r(\tau)$ cerca de 1: lo que pasó hace τ pasos anticipa bien lo de ahora.
- $r(\tau)$ cerca de 0: no hay relación lineal a esa distancia.
- $r(\tau)$ negativo: valores separados por τ tienden a estar en lados opuestos de la media.

Para el ruido blanco la autocorrelación es 0 en todo rezago distinto de 0. Para la caminata es muy alta y decae despacio.

Cada rezago, una comparación

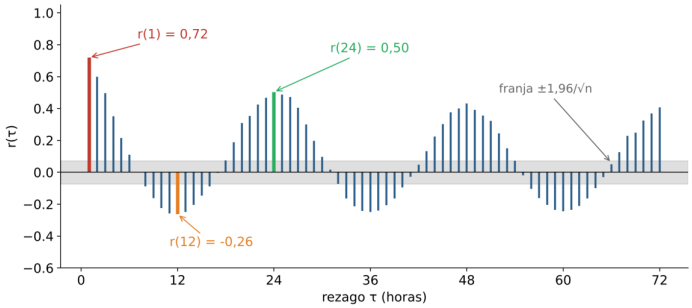
Para calcular $r(\tau)$ se compara cada valor de la serie con el de τ pasos antes. Cada gráfico tiene un punto por hora de junio: el tránsito en ese momento (eje vertical) contra el tránsito τ horas antes (eje horizontal).



- $\tau = 1$: los puntos siguen una diagonal creciente. Una hora se parece a la anterior.
- $\tau = 12$: no hay diagonal creciente. Doce horas antes, de día, el tránsito era alto si ahora, de noche, es bajo.
- $\tau = 24$: vuelve la diagonal. Cada hora se parece a la misma hora del día anterior.

El correlograma

Definición. El **correlograma** es el gráfico que tiene en el eje horizontal el rezago τ y en el eje vertical $r(\tau)$, con una barra por rezago (Rincón y De Jesús Romo, 2024).



Junta en un solo gráfico todas las comparaciones de la diapositiva anterior: las tres barras de color son los tres gráficos de dispersión. La **franja gris** marca $\pm 1{,}96/\sqrt{n}$: si la serie fuera un ruido, el 95% de las barras caería dentro. Una barra fuera de la franja indica que hay memoria a esa distancia.

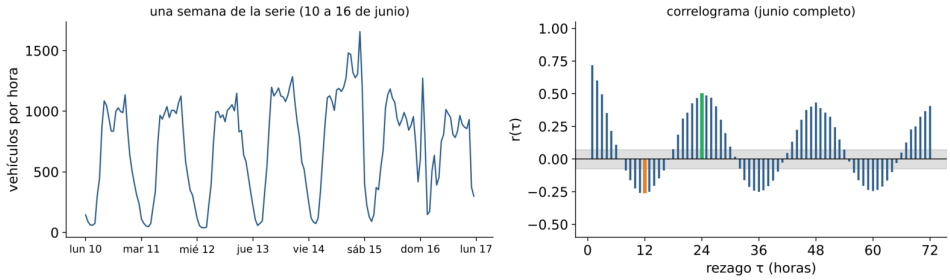
Qué se lee en un correlograma

Se mira la **forma** de las barras que salen de la franja (Mauricio, 2007). Hay tres formas básicas:

- **Cae rápido**: las primeras barras son altas y enseguida quedan dentro de la franja. La memoria es corta: lo que pasó hace unas horas ya no dice nada sobre lo de ahora.
- **Cae lento**: las barras son altas y bajan muy despacio. La memoria es larga: la serie no vuelve a un nivel medio.
- **Es periódico**: las barras suben y bajan con un ritmo. La distancia entre dos picos es el **período**: cada cuánto la serie repite su comportamiento.

En las diapositivas que siguen hay un ejemplo de cada forma, siempre con la serie a la izquierda y su correlograma a la derecha.

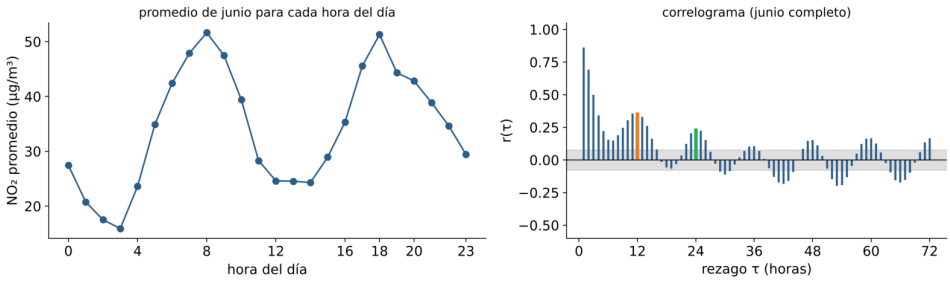
Un correlograma periódico: el tránsito



Qué se ve. La serie sube de día y baja de noche, todos los días. En el correlograma las barras suben y bajan, con picos en 24, 48 y 72 horas.

Qué significa. El período es de 24 horas: el tránsito de una hora se parece al de la misma hora del día anterior. En 12 horas la barra es negativa, $r(12) = -0{,}26$: doce horas después, el tránsito suele estar del otro lado de su media.

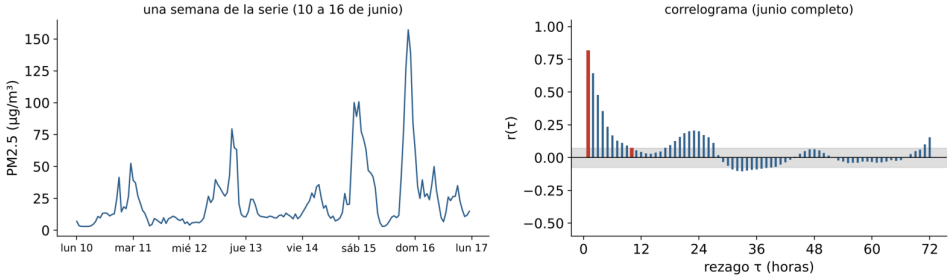
Un correlograma periódico de 12 horas: el NO2



Qué se ve. A la izquierda, para cada hora del día, el promedio del NO2 en los 30 días de junio: hay dos máximos, a las 8 y a las 18 horas. En el correlograma hay picos en 12 horas, $r(12) = 0{,}36$, y en 24 horas, $r(24) = 0{,}24$.

Qué significa. El período es de 12 horas: cada 12 horas el NO2 vuelve a estar alto.

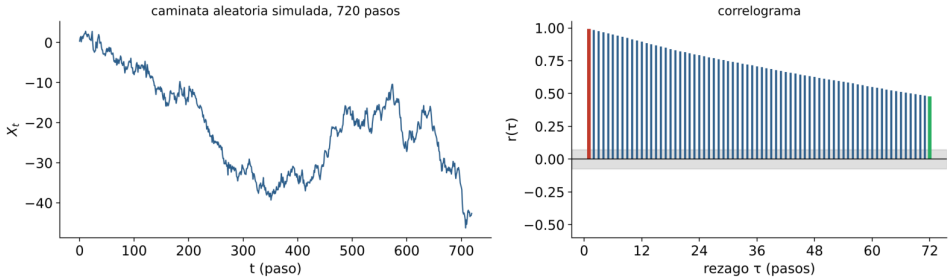
Un correlograma que cae rápido: el PM2.5



Qué se ve. Picos cortos sobre un piso bajo. En el correlograma, $r(1) = 0{,}82$ y a las 10 horas la barra ya está dentro de la franja, $r(10) = 0{,}07$. En 24 horas hay una joroba chica, $r(24) = 0{,}20$.

Qué significa. La memoria dura pocas horas: un episodio de contaminación se mantiene unas horas y se va. El ciclo diario existe, pero es débil.

Un correlograma que cae lento: la caminata aleatoria

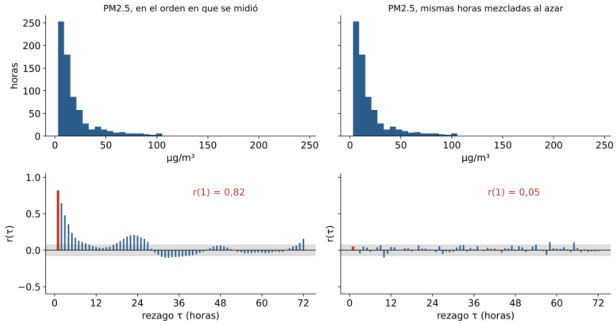


Qué se ve. La caminata se aleja y no vuelve a un nivel medio. Todas las barras son altas y bajan muy despacio: $r(1) = 0{,}99$, $r(24) = 0{,}79$, $r(72) = 0{,}48$.

Qué significa. La memoria es larga: cada valor es el anterior más un paso chico, así que lo de hace 72 pasos todavía se parece a lo de ahora.

La autocorrelación es una propiedad del orden

Se toman las 720 horas de PM2.5 y se **mezclan al azar**: los mismos valores, en otro orden.



Arriba, los histogramas son idénticos: misma media (20,06 µg/m³), misma varianza. **Abajo**, el correlograma de la serie medida cae rápido, $r(1) = 0{,}82$; el de la mezclada queda dentro de la franja, como un ruido, $r(1) = 0{,}05$. La media y el histograma no ven el orden; la autocorrelación sí.

El operador retraso

Definición. El **operador retraso** B corre la serie un paso hacia atrás:

$$B\,x_t = x_{t-1}, \qquad B^k x_t = x_{t-k}.$$

En cada fila, *Bxt* es el valor que la serie tenía una hora antes. En pandas es `shift(k)`; en R, `lag(x, k)`.

hora (1 de junio)	x_t (tránsito)	$B\,x_t$
00:00	192	—
01:00	103	192
02:00	95	103
03:00	94	95
04:00	100	94
05:00	223	100

Curva de Maroñas, vehículos por hora. La primera fila no tiene valor anterior: queda vacía.

Qué es diferenciar una serie

Definición. Diferenciar una serie es reemplazar cada valor por cuánto cambió respecto del valor anterior:

$$\nabla x_t = x_t - x_{t-1} = x_t - B\,x_t.$$

La serie de **niveles** se convierte en una serie de **cambios**.

hora	x_t	$B\,x_t$	∇x_t
00:00	192	—	—
01:00	103	192	-89
02:00	95	103	-8
03:00	94	95	-1
04:00	100	94	6
05:00	223	100	123

Por ejemplo, a la 01:00 la diferencia es 103 −192 = −89: pasaron 89 vehículos menos que a las 00:00.

Diferencias: cómo se leen y cómo se deshacen

- Una serie de n valores tiene $n-1$ diferencias: se pierde el primero.
- Positiva: la serie subió. Negativa: bajó. Cero: no cambió.
- Se puede volver atrás: xt = $x1$ + $\nabla x2$ + ⋯+ ∇xt. Sumar las diferencias desde el primer valor reconstruye la serie.

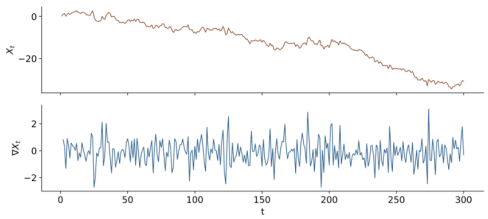
Diferencia de retraso k**:** $\nabla kxt = xt - xt-k$ compara con el valor de k pasos antes. Con datos mensuales, la **variación interanual** de los diarios es $\nabla 12\ xt$: cada mes contra el mismo mes del año anterior.

Diferenciar la caminata devuelve el ruido

La caminata se construye sumando ruidos: $X_t = X_{t-1} + Z_t$. Si se despeja Z_t:

$$\nabla X_t = X_t - X_{t-1} = Z_t.$$

Diferenciar deshace la suma: los cambios de la caminata son exactamente los pasos al azar.



Arriba, una caminata de 300 pasos; abajo, sus diferencias: el ruido con el que se construyó.

t	1	2	3	4
Z_t	0,5	$-1{,}2$	0,3	0,8
X_t $(X_0 = 0)$	0,5	$-0{,}7$	$-0{,}4$	0,4
∇X_t	—	$-1{,}2$	0,3	0,8

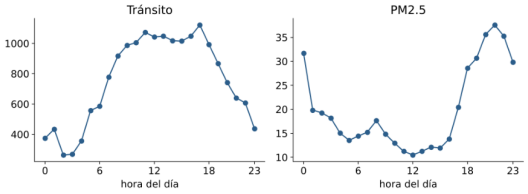
Estacionariedad, en una versión intuitiva

Definición (estacionariedad débil). Una serie es estacionaria si

1. su media no cambia con el tiempo, y
2. la covarianza entre Xt y $Xt+\tau$ depende solo de la distancia τ, no de t.

Es decir: el comportamiento de la serie es el mismo en cualquier tramo que se mire.

- El **ruido** es estacionario.
- La **caminata** no: no vuelve a un nivel medio.
- El **tránsito** tampoco: su media depende de la hora del día.



Fuentes

- J. T. Leek y R. D. Peng (2015). What is the question? Science, 347(6228), 1314–1315.
- L. Rincón y V. De Jesús Romo (2024). Una introducción a las series de tiempo. Facultad de Ciencias, UNAM. Capítulo 1: modelos elementales, funciones básicas, operadores, estacionariedad y correlograma.
- J. A. Mauricio (2007). Introducción al Análisis de Series Temporales. Universidad Complutense de Madrid. Sección 2.2: correlograma.
- A. de la Fuente Fernández. Series temporales. Universidad Autónoma de Madrid.
- Datos: tránsito, NO2 y PM2.5, Curva de Maroñas, junio de 2024. Catálogo de Datos Abiertos de Uruguay, Intendencia de Montevideo (`transito_aire_junio2024.csv`, clase 10).
- Encuesta de Factum, segundo bimestre de 2026, publicada en El Observador y VTV Noticias.

19. Inteligencia artificial para leer documentos

Tokens, probabilidades y la API de Gemini
Introducción a la Ciencia de Datos — UM, FCEE

La inteligencia artificial y el modelo

Inteligencia artificial

Inteligencia artificial (IA). Sistema computacional que realiza tareas que asociamos a la inteligencia humana — percibir, reconocer, traducir, decidir— a partir de patrones obtenidos de datos. **Modelo.** Función que transforma una entrada en una salida:

$$\hat{y} = f(x;\, \theta)$$

x: la entrada (una foto, un texto)

$\hat{y}$: la salida (el texto del ticket)

θ: los **parámetros**, números que fijan el comportamiento

de f

Un modelo de lenguaje actual tiene miles de millones de parámetros. Russell y Norvig (2021), capítulo 1.

Entrenamiento e inferencia

Entrenamiento. Proceso de ajustar los parámetros θ para que las salidas del modelo se parezcan a las salidas correctas en un conjunto grande de ejemplos. **Función de pérdida.** Número que mide cuánto se equivoca el modelo en los ejemplos. Entrenar es buscar los

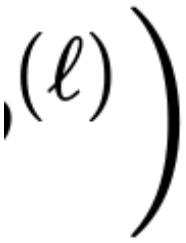
θ que la hacen chica.

Inferencia. Uso del modelo ya entrenado, con los parámetros fijos, para producir una salida ante una entrada nueva. Hoy solo hacemos inferencia: usamos un modelo que Google entrenó. Cómo se entrena se estudia en Aprendizaje Automático. Goodfellow, Bengio y Courville (2016), capítulo 5.

Red neuronal

Red neuronal. Modelo formado por **capas**. Cada capa recibe un vector de números, lo multiplica por una matriz de parámetros, le suma un vector y aplica una función no lineal:

$$h^{(\ell+1)} = \sigma\Big($$



σ W (ℓ)h (ℓ) + b

W (ℓ), b

(ℓ): parámetros de la capa ℓ

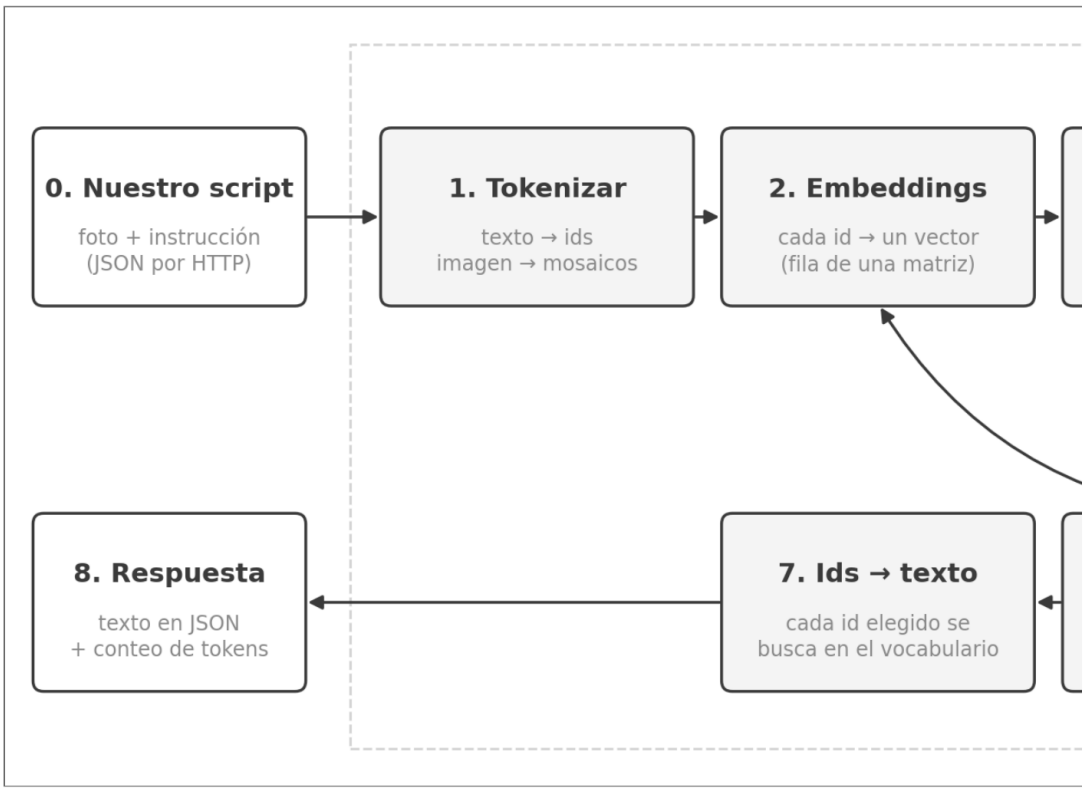
σ: función no lineal, por ejemplo σ(u) = max(0, u)

Apilando muchas capas, la composición de funciones simples puede representar relaciones muy complejas: por ejemplo, de píxeles a letras. Goodfellow, Bengio y Courville (2016), capítulo 6.

Transformer

Transformer. Tipo de red neuronal diseñada para procesar **secuencias**. Recibe una lista de vectores —uno por token— y devuelve otra lista del mismo largo. **Atención.** Operación central del Transformer: para calcular el nuevo vector de cada token, combina los vectores de los tokens anteriores, dándole más peso a los que son más relevantes. Ejemplo: para decidir qué escribir después de «TOTAL $», la atención puede darle mucho peso a los tokens de la imagen donde está impreso el total. Gemini está construido sobre Transformers de este tipo. Vaswani et al. (2017). Gemini Team (2023), sección 2: los modelos Gemini se construyen sobre decodificadores Transformer.

El recorrido completo dentro de Gemini

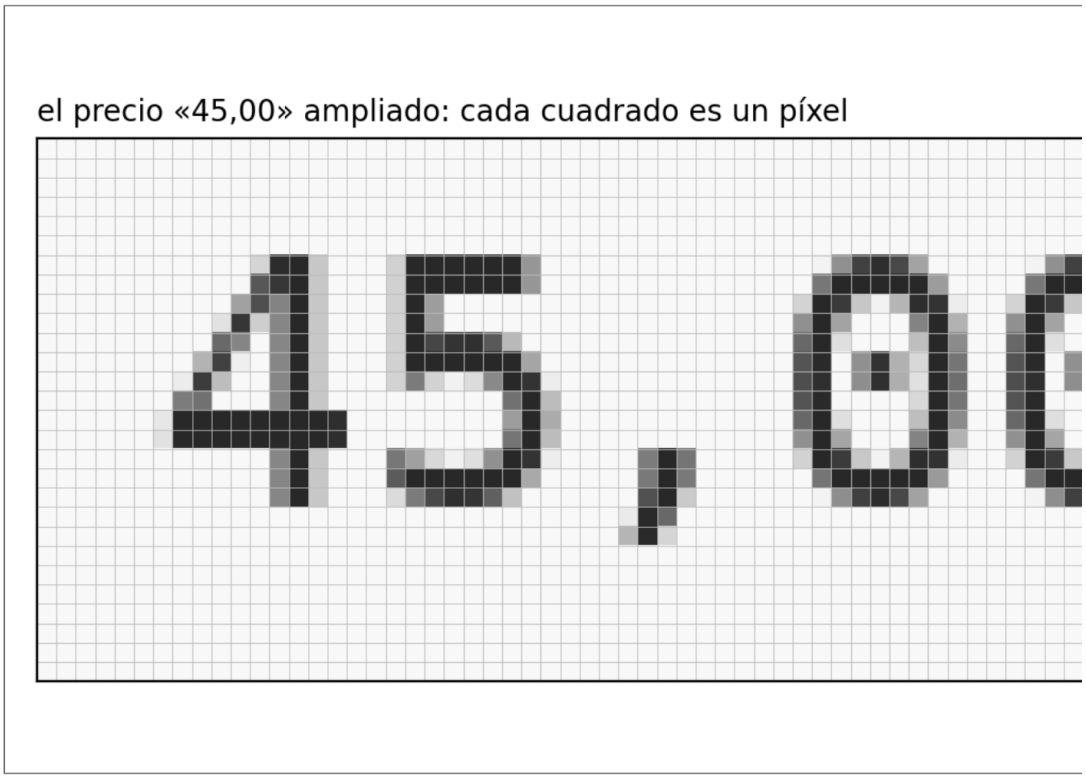


Nuestro script solo hace el paso 0 y recibe el paso 8. Los pasos 1 a 7 ocurren en el servidor de Google. Las diapositivas que siguen explican cada paso. Esquema general de un modelo de lenguaje Transformer: Jurafsky y Martin, Speech and Language Processing. Aplicación a Gemini: Gemini Team (2023).

La entrada: imágenes y texto como números

Imagen digital

Imagen digital. Matriz de números. Cada entrada es un **píxel** y su valor es la intensidad de luz: en escala de grises, de 0 (negro) a 255 (blanco).



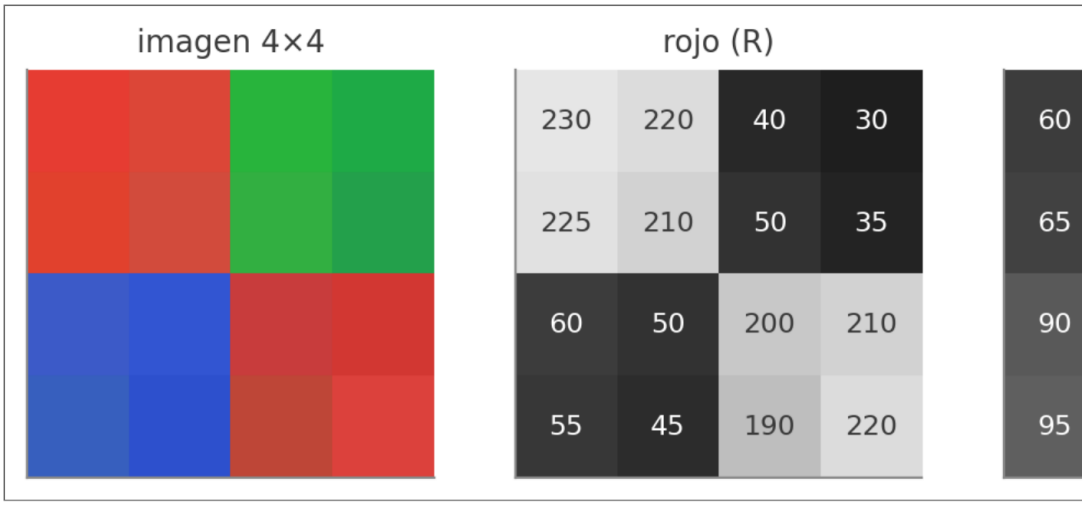
Gonzalez y Woods (2018), capítulo 2.

Imágenes en color

Imagen en color (RGB). Tres matrices del mismo tamaño, una por canal: rojo, verde y azul. Una foto de alto

es un arreglo de H × W × 3 H y ancho W

números.



Una foto de celular de 3000 × 4000 píxeles son 36 millones de números. Gonzalez y Woods (2018), capítulo 6.

Token

Token. Unidad mínima en la que un modelo de lenguaje parte su entrada y su salida. Puede ser una letra, un fragmento de palabra o una palabra entera. **Tokenización.** Procedimiento que corta un texto en tokens. Siempre corta igual: el mismo texto da siempre los mismos tokens.

El símbolo ␣ representa un espacio: el espacio forma parte del token. Los **números se cortan dígito por dígito**:

`109` son tres tokens, `1`, `0` y `9`.

Segmentación ilustrativa. Dígitos separados: Gemma Team (2024), sección sobre el tokenizador. En Gemini, un token equivale en promedio a unos 4 caracteres: Google, documentación de la API de Gemini, «Tokens».

Vocabulario

Vocabulario. Lista fija y finita de todos los tokens que el modelo puede leer o escribir.

Se construye **antes** de entrenar el modelo, con un algoritmo aplicado a una colección enorme de textos (el **corpus**).

Una vez construido **no cambia**: el modelo se entrena con ese vocabulario y queda atado a él.

El programa que corta el texto usando el vocabulario se llama **tokenizador**. Es parte del modelo, no de nuestro código. Gemini usa un tokenizador llamado **SentencePiece**. Google no publica el vocabulario de Gemini, pero sí el de **Gemma**, un modelo abierto que usa parte del mismo tokenizador: tiene 256.000 tokens. Gemini Team (2023), sección 4. Gemma Team (2024), sección sobre el tokenizador. Kudo y Richardson (2018).

Cómo se construye el vocabulario: BPE

BPE (byte pair encoding, codificación por pares).

Algoritmo que arma el vocabulario uniendo, paso a paso, los pares de tokens vecinos más frecuentes en el corpus.

1. Empezar con un vocabulario de símbolos sueltos: letras, dígitos, signos y el espacio.
2. Partir cada palabra del corpus en esos símbolos.
3. Contar cuántas veces aparece cada par de tokens vecinos.
4. Unir el par más frecuente en un token nuevo y agregarlo al vocabulario.
5. Repetir 3 y 4 hasta llegar al tamaño deseado (por ejemplo, 256.000 tokens).

Cada unión agrega exactamente un token. Las secuencias frecuentes terminan siendo

un solo token; las raras quedan partidas en pedazos. Sennrich, Haddow y Birch (2016). Jurafsky y Martin, Speech and Language Processing, capítulo sobre palabras y tokens. SentencePiece implementa BPE y una variante: Kudo y Richardson (2018).

BPE con un ejemplo

Corpus de juguete:

palabra	veces
de	12
leche	10
dulce	6
del	5
lechuga	4
queso	3

Vocabulario inicial: 12

símbolos, `a c`

`d e g h l o`

`q s u ␣`

paso	par más frecuente	veces	token nuevo	`␣leche` queda	`␣lechuga` queda
1	`␣` + `d`	23	`␣d`	`␣·l·e·c·h·e`	`␣·l·e·c·h·u·g·a`
2	`␣d` + `e`	17	`␣de`	`␣·l·e·c·h·e`	`␣·l·e·c·h·u·g·a`
3	`␣` + `l`	14	`␣l`	`␣l·e·c·h·e`	`␣l·e·c·h·u·g·a`
4	`␣l` + `e`	14	`␣le`	`␣le·c·h·e`	`␣le·c·h·u·g·a`
5	`␣le` + `c`	14	`␣lec`	`␣lec·h·e`	`␣lec·h·u·g·a`
paso	par más frecuente	veces	token nuevo	`␣leche` queda	`␣lechuga` queda
6	`␣lec` + `h`	14	`␣lech`	`␣lech·e`	`␣lech·u·g·a`
7	`␣lech` + `e`	10	`␣leche`	`␣leche`	`␣lech·u·g·a`
8	`␣d` + `u`	6	`␣du`	`␣leche`	`␣lech·u·g·a`

El punto · separa tokens. «veces» del paso 1: ␣d aparece al inicio de de (12), dulce (6) y del (5): 12 + 6 + 5 = 23. Cálculo propio con el algoritmo de Sennrich et al. (2016) (archivo `bpe_ejemplo.py`).

Resultado: palabras frecuentes enteras, raras partidas

Después de 8 pasos el vocabulario tiene 12 + 8 = 20 tokens, y:

palabra	frecuencia en el corpus	tokens	cantidad
leche	10	␣leche	1
lechuga	4	␣lech u g a	4
dulce	6	␣du l c e	4

Lo mismo pasa a gran escala. El vocabulario distingue mayúsculas de minúsculas, y en los textos «leche» aparece mucho más que «LECHE». Por eso un ticket, escrito en mayúsculas, probablemente se parte en más tokens que el mismo texto en minúsculas.

texto	tokens (ilustrativo)
dulce de leche	dulce ␣de ␣leche → 3

DULCE DE LECHE

DUL CE ␣DE ␣LE CHE → 5

La cantidad real de tokens de un texto se puede consultar con la función `countTokens` de la API de Gemini.

De dónde salen los ids

Id (identificador). Número de la posición del token en la lista del vocabulario. Es solo un número de orden, como el número de fila de una tabla.

El vocabulario del ejemplo, en el orden en que se construyó:

id	token	id	token
0	a	10	u
1	c	11	␣
2	d	12	␣d
3	e	13	␣de
4	g	14	␣l
5	h	15	␣le
6	l	16	␣lec
7	o	17	␣lech
8	q	18	␣leche
9	s	19	␣du

Tokenizar «dulce de leche»:

[19, 6, 1, 3, 13, 18]

token

␣du

l c e

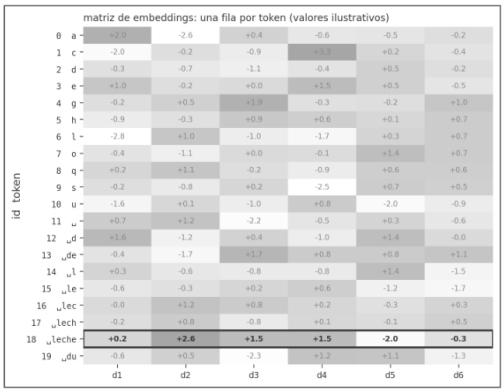
␣de ␣leche

id

19 6 1 3 13 18 El texto entra al modelo como la lista de números

El modelo **nunca ve letras**: solo estos números. En Gemma los ids van de 0 a 255.999.

Del id al embedding



Embedding (representación

vectorial). Vector de números reales asociado a cada token. **Matriz de embeddings.** Matriz con **una fila por token del vocabulario**. La

fila número es eli embedding del token con id i

. El id sirve para buscar la fila: el id 18 dice «usá la fila 18»,

que es el vector de ␣leche.

Los valores de la matriz son parámetros: se ajustan

durante el entrenamiento. En un modelo real cada fila tiene miles de números. Valores ilustrativos. Jurafsky y Martin, Speech and Language Processing, capítulos sobre embeddings y Transformers.

La imagen también se convierte en tokens

Modelo multimodal. Modelo que recibe más de un tipo de entrada — texto, imagen, audio— y los procesa juntos como una sola secuencia. La imagen no pasa por el vocabulario de palabras: se corta en pedazos y cada pedazo se convierte directamente en vectores, que ocupan lugares en la secuencia igual que los tokens de texto. En Gemini:

una imagen de hasta 384 × 384 píxeles cuenta **258 tokens**

una imagen más grande se corta en **mosaicos** de 768 × 768 píxeles, y

cada mosaico cuenta 258 tokens Esquema ilustrativo. Conteo de tokens: Google, documentación de la API de Gemini, «Image understanding» y «Tokens». Entrada de texto e imágenes intercaladas: Gemini Team (2023), sección 2.

La salida: probabilidades

Modelo de lenguaje

Modelo de lenguaje. Modelo que, dada una secuencia de tokens, asigna una probabilidad a cada token posible del vocabulario como **siguiente** token:

P(tn+1

$$= v \mid t_1, t_2, \ldots, t_n)$$

para cada v del vocabula

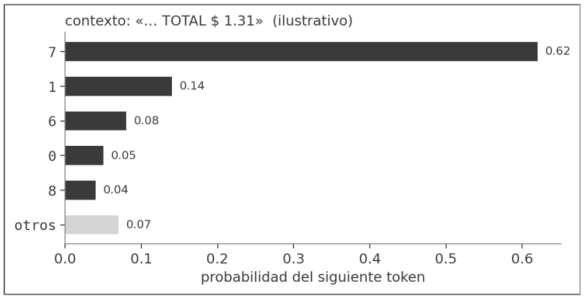
La probabilidad de un texto completo se obtiene encadenando:

$$P(t_1, \ldots, t_n) = \prod_{i=1}^{n} P(t_i$$

| t1, … , ti−1

En nuestro caso, los primeros tokens son los de la imagen y los de la instrucción; los siguientes son los que el modelo va escribiendo. Jurafsky y Martin, Speech and Language Processing, capítulo sobre modelos de lenguaje con n-gramas.

Siguiente token



Después de leer la imagen y de haber escrito «TOTAL $ 1.31», el modelo tiene una distribución de probabilidad sobre lo que sigue. Si la imagen es nítida, casi toda la probabilidad cae en el token correcto. Si está borrosa, la probabilidad se reparte entre candidatos parecidos. Valores ilustrativos.

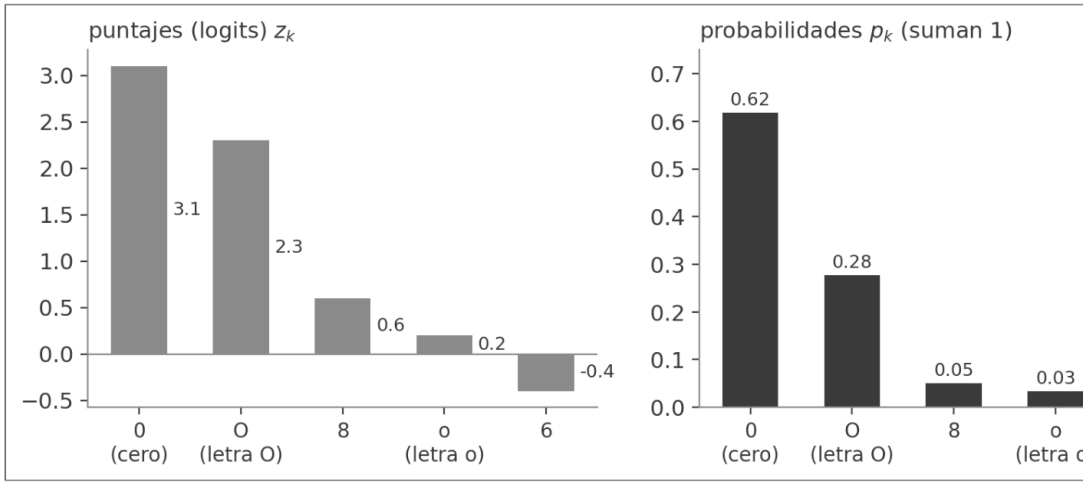
Jurafsky y Martin, Speech and Language Processing, capítulo sobre modelos de lenguaje con Transformers.

De puntajes a probabilidades: softmax

La última capa de la red da un **puntaje** (logit) zk por cada token del vocabulario. La función k **softmax** los convierte en probabilidades:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Cada pk está entre 0 y 1, y todas suman 1. El puntaje más alto recibe la probabilidad más alta.



Goodfellow, Bengio y Courville (2016), sección 6.2.2.3.

Softmax paso a paso

Como los dígitos son tokens separados, el modelo escribe «109» dígito por dígito. Puntajes para el token que sigue a «1» (valores ilustrativos):

candidato	z_k	e^{z_k}	$p_k = e^{z_k}/35{,}88$
0 (cero)	3,1	22,20	**0,62**
O (letra)	2,3	9,97	0,28
8	0,6	1,82	0,05
o (letra)	0,2	1,22	0,03
6	−0,4	0,67	0,02
suma		**35,88**	**1,00**

El cero y la letra O se parecen en la imagen: por eso la letra se lleva el 28 %. Así se producen las confusiones como leer

`109,00`, con la letra O en lugar del cero.

Del puntaje al texto: los ids a la salida

La última capa da **un puntaje por cada id del vocabulario**: en Gemma, 256.000 puntajes. El softmax los convierte en 256.000 probabilidades que suman 1.

Con el vocabulario de juguete (20 ids), después de leer ␣du l c e ␣de:

id	token	probabilidad
18	␣leche	**0,81**
17	␣lech	0,07
13	␣de	0,03
…	(los otros 17 ids)	0,09

El modelo elige un **id** (el 18), lo busca en la lista del

vocabulario y recién ahí aparece el texto: ␣leche. Este paso

final, de ids a texto, se llama **detokenización**. Probabilidades ilustrativas. Jurafsky y Martin, Speech and Language Processing, capítulo sobre modelos de lenguaje con Transformers.

Generación autorregresiva

Generación autorregresiva. El modelo escribe la respuesta de a un token por vez: calcula la distribución del siguiente token, elige uno, lo agrega a la secuencia y repite con la secuencia más larga.

aso	lo que ya está escrito	token elegido
1	*(imagen + instrucción)*	DUL
2	DUL	CE
3	DULCE	␣DE
4	DULCE DE	␣LE
…	…	…
13	DULCE DE LECHE 400G $	␣

14

```
DULCE DE LECHE 400G $    1
```

15

```
DULCE DE LECHE 400G $ 1  0
```

Termina cuando elige un token especial de **fin de secuencia** o llega al máximo de tokens permitido. Jurafsky y Martin, Speech and Language Processing, capítulo sobre modelos de lenguaje con Transformers.

Cómo se elige el token: temperatura

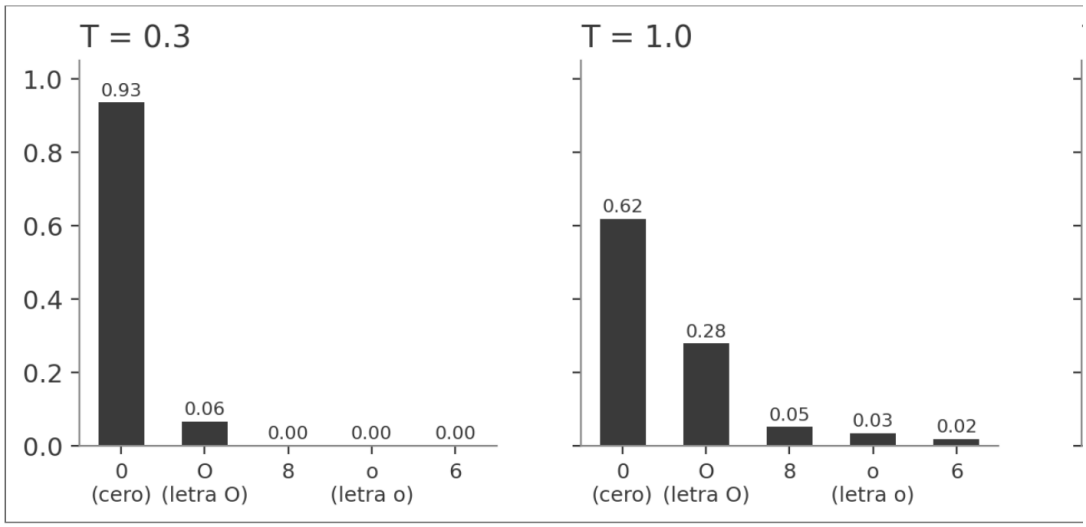
Decodificación voraz. Elegir siempre el token de mayor probabilidad. **Muestreo.** Sortear el token según las probabilidades pk .

Temperatura T

. Parámetro que divide los puntajes antes

k ∝ ezk/T.

del softmax: p



Con T

chica la distribución se concentra en el más probable. Para leer un ticket conviene temperatura baja: queremos la lectura más probable, no variedad. Jurafsky y Martin, Speech and Language Processing, sección sobre muestreo. El parámetro `temperature` de

Gemini: Google, documentación de la API, «generateContent».

Alucinación

Alucinación. Salida del modelo que es plausible pero no está respaldada por la entrada. El modelo elige tokens probables **dado todo lo que aprendió**, no solo dada la imagen. En un ticket:

completar un precio borroso con un valor típico

«corregir» un total para que coincida con la suma

agregar un ítem que suele aparecer en tickets parecidos

escribir una fecha con formato distinto al impreso Un OCR clásico, cuando no puede leer, deja un hueco o un carácter raro. Un modelo de lenguaje tiende a escribir algo que **parece** correcto. Por eso hay que controlar la salida. Ji et al. (2023).

Lo que se sabe de Gemini y lo que no

Publicado por Google:

se construye sobre **decodificadores Transformer** (Gemini Team, 2023)

usa el tokenizador **SentencePiece**, entrenado sobre una muestra grande del corpus (Gemini Team, 2023)

recibe texto intercalado con imágenes, audio y video (Gemini Team, 2023)

un token ≈ 4 caracteres; una imagen chica = 258 tokens (documentación de la API) No publicado:

el vocabulario exacto y sus ids

la cantidad de parámetros y los detalles de las versiones actuales

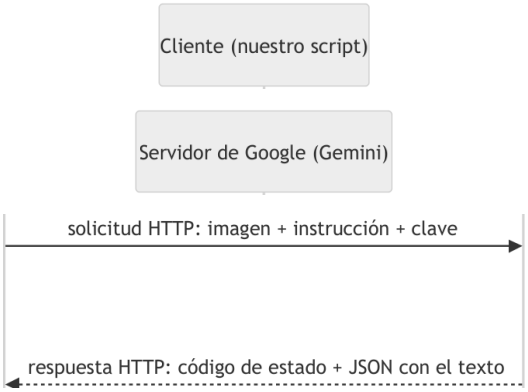
con qué textos e imágenes exactos se entrenó Por eso, en esta clase, los ids y las probabilidades concretas son **ilustrativos**: el mecanismo es el descripto, los números no son los de Gemini.

La API

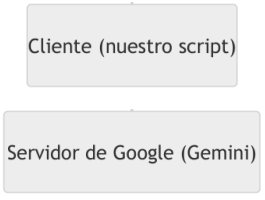
API

API (interfaz de programación de aplicaciones).

Conjunto de reglas que permite a un programa pedirle un servicio a otro programa. **Cliente.** Programa que hace el pedido: nuestro script. **Servidor.** Computadora que recibe el pedido, lo procesa y responde: la de Google, donde está el modelo.



tokeniza, calcula probabilidades, genera la respuesta token a token



HTTP

HTTP. Protocolo con el que se comunican cliente y servidor en la web. Cada intercambio es una **solicitud** y una **respuesta**. **Método.** Tipo de pedido. `GET` pide un recurso; `POST` envía datos para que el servidor los procese. Para mandar una imagen se usa `POST`. **Encabezados (headers).** Pares `nombre: valor` con información sobre la solicitud: qué formato tiene el cuerpo, quién la hace. **Cuerpo (body).** Los datos que

se envían: en nuestro caso, un JSON con la imagen y la instrucción. MDN Web Docs, «An overview of HTTP».

Anatomía de la dirección (endpoint)

Endpoint. Dirección a la que se envía la solicitud.

```
https://generativelanguage.googleapis.com/v1beta/models/g
3.8-flash:generateContent
```

esquema `https`: HTTP cifrado **servidor**: la máquina de Google que atiende la API de Gemini

versión `v1beta`: la API cambia; la versión fija qué reglas se usan

recurso: qué modelo se usa

acción `generateContent`: generar una respuesta Google, documentación de la API de Gemini, «generateContent».

La solicitud completa

```
POST /v1beta/models/gemini-3.8-flash:generateContent HTTP/1.1
Host: generativelanguage.googleapis.com
Content-Type: application/json
x-goog-api-key: AIza...clave...

{
  "contents": [{
    "parts": [
      {"text": "Transcribí los ítems de este ticket como JSON ..."},
      {"inline_data": {"mime_type": "image/jpeg",
                       "data": "/9j/4AAQSkZJRgABAQ..."}}
    ]
  }],
  "generationConfig": {"temperature": 0,
                       "responseMimeType": "application/json"}
}
```

`contents` → `parts`: la entrada es una lista de partes, texto e

imagen. `generationConfig` fija la temperatura y pide la respuesta en JSON. Google, documentación de la API de Gemini, «generateContent» e «Image understanding».

Base64: cómo viaja la imagen dentro del JSON

Base64. Codificación que escribe cualquier secuencia de bytes usando solo 64 caracteres imprimibles (`A–Z`, `a–z`, `0–9`,

`+`, `/`). Cada 3 bytes se convierten en 4 caracteres.

Hace falta porque el JSON solo admite texto, y una imagen son bytes arbitrarios.

paso	resultado
bytes del archivo (hexadecimal)	`FF D8 FF`
los mismos 24 bits	`11111111 11011000 11111111`
en grupos de 6 bits	`111111 111101 100011 111111`
valor de cada grupo (0 a 63)	63, 61, 35, 63

carácter de la tabla base64

`/ 9 j /`

Todo JPEG empieza con los bytes `FF D8 FF`: por eso en base64 empieza con `/9j/`. El texto resultante ocupa alrededor de un tercio más que el archivo. Josefsson (2006), RFC 4648, sección 4.

Clave de API y autenticación

Autenticación. Procedimiento con el que el servidor verifica quién hace la solicitud.
Clave de API. Cadena secreta que identifica el proyecto

que hace la solicitud. Se envía en el encabezado `x-goog- api-key`.

La clave se obtiene gratis en Google AI Studio con una cuenta de Google.

Quien tiene la clave usa la cuota del dueño: no se comparte ni se sube a GitHub.

En el código se lee de una **variable de entorno** o de un archivo excluido del repositorio. Google, documentación de la API de Gemini, «Using Gemini API keys».

La respuesta


```
{
  "candidates": [{
    "content": {
      "parts": [{"text": "{\"comercio\": \"CAFETERIA EL CLAUSTRO\", \"items\": [
    },
    "finishReason": "STOP"
  }],
  "usageMetadata": {
    "promptTokenCount": 1310,
    "candidatesTokenCount": 142,
    "totalTokenCount": 1452
  }
}
```


`candidates[0].content.parts[0].text`: el texto generado

`finishReason`: por qué terminó (`STOP` = fin de secuencia)

`usageMetadata`: tokens de entrada (imagen + instrucción),

de salida y total Cantidades de tokens ilustrativas. Google, documentación de la API de Gemini, «generateContent».

Códigos de estado

Código de estado. Número de tres cifras con el que el servidor indica cómo terminó la solicitud.

código	significado	qué hacer
200	todo bien	leer el JSON
400	solicitud mal armada	revisar el cuerpo: campos, base64
403	clave inválida o sin permiso	revisar la clave
429	demasiadas solicitudes	esperar y reintentar
500 / 503	error o sobrecarga del servidor	reintentar más tarde

Fielding, Nottingham y Reschke (2022), RFC 9110, sección

15. Google, documentación de la API de Gemini,

«Troubleshooting».

Límites y costo

Cuota. Cantidad máxima de solicitudes o tokens permitida por minuto y por día.

Nivel gratuito: sin costo y sin tarjeta, con cuotas por minuto y por día que dependen del modelo.

Nivel pago: se cobra por token, con precios distintos para tokens de entrada y de salida.

Tamaño máximo de una solicitud con la imagen incluida: 20 MB.

En el nivel gratuito, Google usa el contenido enviado para mejorar sus productos. En el pago, no. Por eso una foto grande «cuesta» más que una chica: tiene más mosaicos, y por lo tanto más tokens. Google, documentación de la API de Gemini, «Pricing» y «Rate limits». Límite de 20 MB: «Image understanding».

El programa cliente en Python

```python
import base64, json, os, requests

URL = ("https://generativelanguage.googleapis.com/v1beta/"
       "models/gemini-3.8-flash:generateContent")
clave = os.environ["GEMINI_API_KEY"]

with open("ticket.jpg", "rb") as f:
    imagen = base64.b64encode(f.read()).decode("ascii")

cuerpo = {
    "contents": [{"parts": [
        {"text": INSTRUCCION},
        {"inline_data": {"mime_type": "image/jpeg", "data": imagen}},
    ]}],
    "generationConfig": {"temperature": 0, "responseMimeType": "application/json
}
r = requests.post(URL, headers={"x-goog-api-key": clave}, json=cuerpo, timeout=1
r.raise_for_status()                            # corta si el código no es 200
texto = r.json()["candidates"][0]["content"]["parts"][0]["text"]
ticket = json.loads(texto)                      # de texto JSON a diccionario
```

El programa cliente en R

```r
library(httr2); library(jsonlite)

clave  <- readLines("clave_gemini.txt", warn = FALSE)[1]
url    <- "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-fl
bytes  <- readBin("ticket.jpg", what = "raw", n = file.size("ticket.jpg"))
imagen <- gsub("[\r\n]", "", base64_enc(bytes))          # foto -> base64

cuerpo <- list(contents = list(list(parts = list(
  list(text = instruccion),
  list(inline_data = list(mime_type = "image/jpeg", data = imagen))))),
  generationConfig = list(temperature = 0))

respuesta <- request(url) |>
  req_headers(`x-goog-api-key` = clave) |>                 # la clave en un enc
  req_body_json(cuerpo) |>                                 # cuerpo JSON, métod
  req_perform()

datos <- resp_body_json(respuesta)                         # JSON -> lista
texto <- datos$candidates[[1]]$content$parts[[1]]$text
```

Los mismos pasos que en Python: leer la foto, pasarla a base64, armar el JSON, enviar, leer la respuesta. Archivos completos: `leer_ticket.py` y `leer_ticket.R`. Paquete httr2: Wickham (2024), documentación en httr2.rlib.org. Google, documentación de la API de Gemini, «generateContent».

La instrucción (prompt)

Prompt. Texto que se le da al modelo junto con la entrada para indicar qué tarea hacer y en qué formato responder.

```
Transcribí este ticket de compra uruguayo. Respondé solo con JSON:
{"comercio": texto, "fecha": "dd/mm/aaaa",
 "items": [{"descripcion": texto, "monto": número}],
 "total": número}
- Copiá las descripciones como están impresas.
- Los montos usan punto de miles y coma decimal: 1.250,00 es 1250.
- Un descuento es un monto negativo.
- No incluyas SUBTOTAL, IVA, EFECTIVO, CAMBIO ni TARJETA como ítems.
- Si algo no se puede leer, poné null. No lo completes.
```

Cada regla del prompt corresponde a un error posible. La última apunta a la alucinación. Google, documentación de la API de Gemini, «Prompt design strategies».

Del texto a la tabla

Dato no estructurado y dato estructurado

Dato no estructurado. Información sin filas ni columnas definidas: texto corrido, imágenes, audio. **Dato estructurado.** Información organizada en una tabla: cada fila es una unidad de observación y cada columna una variable.

```
        CAFETERIA EL CLAUSTRO
          RUT 2174199170017
     AV. ITALIA 3120 - MONTEVIDEO
         22/09/2026   09:41

CAFE CON LECHE              120,00
MEDIALUNA DULCE              45,00
MEDIALUNA DULCE              45,00
AGUA S/GAS 500ML             90,00

SUBTOTAL                    300,00
IVA INCLUIDO                 54,10
TOTAL                       300,00
EFECTIVO                    500,00
CAMBIO                      200,00

        GRACIAS POR SU COMPRA
```

comercio	fecha	descripcion	monto
Cafetería El Claustro	22/09/2026	café con leche	120
Cafetería El Claustro	22/09/2026	medialuna dulce	45
Cafetería El Claustro	22/09/2026	medialuna dulce	45

comercio	fecha	descripcion	monto
Cafetería El	22/09/2026	agua s/gas 500ml	90

Claustro Unidad de observación: el ítem comprado.

Errores de lectura

Error de lectura. Diferencia entre lo que dice el documento y lo que devuelve el modelo.

tipo	definición	ejemplo
sustitución	un carácter se lee como otro	`109,00` → `109,00` (letra O en lugar de cero)
omisión	algo impreso no aparece	renglón desteñido
inserción	aparece algo que no está	un ítem que no existe
error de estructura	se lee bien pero va a otro campo	la cantidad `2 x 45,00` como un ítem más
alucinación	valor plausible sin respaldo en la imagen	un precio borroso completado con un valor típico

Control de consistencia

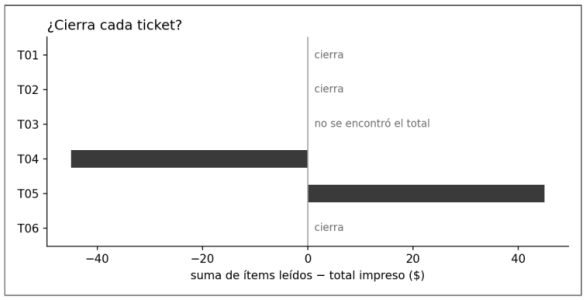
Control de consistencia.

Verificación que usa una relación que los datos deben cumplir, sin necesidad de conocer el valor correcto. En un ticket:

$$\sum_{\text{ítems}} \text{monto} = \text{TOTAL}$$

ı´tems

Si la diferencia no es cero, falta un ítem (T04) o sobra uno (T05). Ojo: un modelo de lenguaje puede «acomodar» un número para que la suma cierre. Que cierre no garantiza que esté bien.



Referencia y tasa de recuperación

Referencia. Versión de los datos que se toma como correcta: la transcripción hecha a mano. **Tasa de recuperación.** Proporción de los elementos de la referencia que aparecen en lo leído:

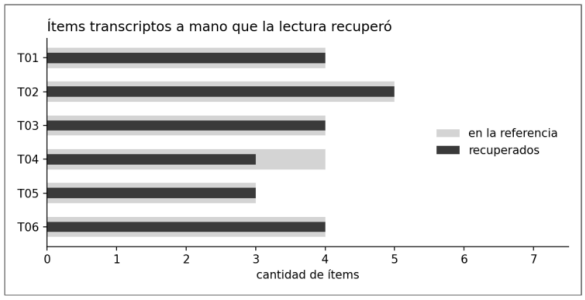
ı´tems recuperados

$$\text{tasa} =$$

ı´tems en la referencia

$$\text{To4: } 3/4 = 0{,}75$$

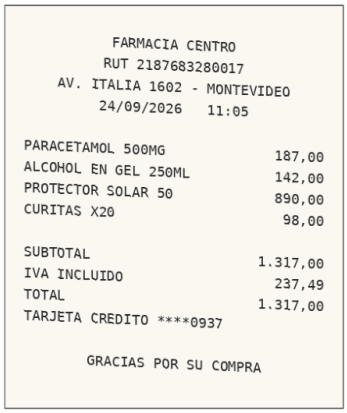
.



Datos personales

Dato personal y dato sensible

Ley 18.331, artículo 4: **Dato personal.** Información de cualquier tipo referida a personas físicas o jurídicas determinadas o determinables. **Dato sensible.** Datos personales que revelen origen racial y étnico, preferencias políticas, convicciones religiosas o morales, afiliación sindical e informaciones referentes a la salud o a la vida sexual. Subir una foto a la API es enviarla a un tercero, que en el nivel gratuito la usa para mejorar sus productos. En un ticket: nombre, cédula, dígitos de la tarjeta. Un ticket de farmacia puede revelar información de salud.



FARMACIA CENTRO
RUT 2187683280017
AV. ITALIA 1602 - MONTEVIDEO
24/09/2026 11:05

PARACETAMOL 500MG	187,00
ALCOHOL EN GEL 250ML	142,00
PROTECTOR SOLAR 50	890,00
CURITAS X20	98,00
SUBTOTAL	1.317,00
IVA INCLUIDO	237,49
TOTAL	1.317,00
TARJETA CREDITO ****0937	

GRACIAS POR SU COMPRA

Ley 18.331 (2008), artículo 4, texto publicado en IMPO. Uso de datos en el nivel gratuito: Google, «Pricing».

Para la bitácora

Con un ticket propio: transcribirlo a mano y compararlo con lo que devolvió Gemini. Clasificar cada diferencia según la tabla de errores.

¿Cuántos tokens de entrada usó la solicitud (`promptTokenCount`)? ¿Cuántos corresponden a la imagen? ¿Cierra el control de consistencia? Si cierra, ¿alcanza para confiar en la tabla?

¿Hubo alguna alucinación? ¿Qué regla del prompt debió evitarla?

¿Qué datos personales tapaste antes de subir la foto?

Referencias

Fielding, R., Nottingham, M. y Reschke, J. (2022). HTTP Semantics. RFC 9110. IETF.

Gemini Team (2023). Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805.

Gemma Team (2024). Gemma: Open Models Based on Gemini Research and Technology. arXiv:2403.08295.

Gonzalez, R. C. y Woods, R. E. (2018). Digital Image Processing (4.ª ed.). Pearson.

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Google. Documentación de la API de Gemini: «Tokens», «Image understanding», «generateContent», «Pricing», «Rate limits», «Using Gemini API keys», «Prompt design strategies». ai.google.dev (consultada el 29/09/2026).

Ji, Z. et al. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, 55(12).

Josefsson, S. (2006). The Base16, Base32, and Base64 Data Encodings. RFC 4648. IETF.

Jurafsky, D. y Martin, J. H. Speech and Language Processing (3.ª ed., borrador en línea). web.stanford.edu/~jurafsky/slp3

Kudo, T. y Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. EMNLP 2018: System

Demonstrations.

Ley N.º 18.331 (2008). Protección de datos personales y acción de habeas data. IMPO.

MDN Web Docs. An overview of HTTP. developer.mozilla.org

Russell, S. y Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4.ª ed.). Pearson.

Sennrich, R., Haddow, B. y Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL 2016.

Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.

Wickham, H. (2024). httr2: Perform HTTP Requests and Process the Responses. Paquete de R. httr2.r-lib.org

Speaker notes